

PrimeSim™ XA User Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>.

All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

New in This Release	14
Related Products, Publications, and Trademarks	14
Conventions	15
Customer Support	16

1. Introducing the PrimeSim XA Tool	17
Understanding the PrimeSim XA Flow	18

2. Getting Started	19
Setting Up the Simulator	19
Configuring the PrimeSim XA License	20
Setting Up the Path to the Executable	24
Configuring a Temporary Directory Location	24
Using an Initialization File	25
Controlling the Interval of Simulation Status Update	25
Running the Simulator	26
Specifying Netlist Format	27
Using Command Line Options	27
Running Simulator Version II	34
Setting Simulation Levels	35
Simulating SRAM or Flash Designs	36
Minimum Resistor Threshold	36
Using Commands and Options	36
Support for Compressed (.gzip) Files	38
Suspending or Resuming License	38
Saving and Restoring Simulations	39
Saving a Simulation	40
Restoring a Saved Simulation	41
Output Files	43
Output Waveform Files	44
Supported Waveform File Formats	44

Setting Waveform Options	45
3. PrimeSim HSPICE Netlist Compatibility	50
Running PrimeSim XA With the PrimeSim HSPICE Netlist Format	51
Syntax and Behavior Variations	51
Using "Curly" Brackets	51
Specifying Equations or Expressions	51
PrimeSim HSPICE-Compatible Netlist Commands	51
PrimeSim HSPICE-Compatible Control Options	54
Built-In Functions	58
Elements	59
Simulating With S-Parameter	60
Device Models	61
Transient Analysis	65
Single Point Analysis	65
Single Parameter Sweep Analysis	66
Multiple Parameter Sweep Analysis	67
Using a Parameter Step in Multiple Parameter Sweep Analysis	69
Parameter Step Examples	70
Specifying .DATAVAR Permutations	73
Output Files	73
Time-Window-Based Analysis	74
Monte Carlo Analysis in PrimeSim XA	75
Violation Checks Using HSPICE .BIASCHK Statements	75
Syntax	76
Using the set_biaschk Command	84
Using the noise Option	85
Supported Netlist Options	86
Output Formats and Examples	86
Bisection Methodology and Behavior Variations	90
PrimeSim HSPICE-Encrypted Netlist Format	92
Post-Layout Simulation Galaxy Parasitic Database (GPD)	92
4. Vector Stimulus Files	94
Using the Digital Vector File as Stimulus	94

Reading the PrimeSim HSPICE Vector File	96
Tabular Data Section	96
Vector Pattern Definition Section	97
Waveform Characteristics Section	105
Controlling Simulation End Time Using <code>autostop</code> Argument	108
Specifying Masking Patterns	109
Setting Time-Dependent Logic Voltage Thresholds	111
Using Parameterized Vector Files	113
Using a Value Change Dump File	114
Reading a VCD File	115
Specifying a VCD Control File	116
Defining Bus Notation Using the <code>#format</code> Statement	116
Defining Vectors and Types Using the <code>#input</code> , <code>#output</code> and <code>#bidirectional</code> Statements	117
Mapping Variables in VCD File to Nodes in the Schematic Using <code>#alias</code> Statement	120
Defining Attributes for Vectors	122
Defining the "Don't Care" Windows Using <code>#ignorewindow</code>	124
Using the Extended Value Change Dump File	125
Reading an EVCD File	126
EVCD Port Direction Rule	126
Defining the Port Direction	131
5. Probing and Measuring	132
Probing Statements and Commands	132
External Table Parameter File Interface	133
Using the <code>.PROBE</code> or <code>.PLOT</code> Statements	134
Using <code>.PRINT</code> Statements	140
Using the <code>.LPROBE</code> and <code>.LPRINT</code> Statements	141
Using the PrimeSim XA Probing Commands	142
Measuring Statements and Commands	143
PARAM Functions	144
AVG, MAX, MIN, PP, RMS and INTEG Functions	144
FIND, DERIVATIVE, and AT Functions	145
FIND, DERIVATIVE, and WHEN Functions	145
WHEN Function	146
TRIG-TARG Function	147
Using VAL, VLG and VHG to Ignore Glitch Measurement	149
TRAN_CONT Function	150

Output Files	151
PrimeSim XA Enhancements to the .MEASURE Syntax	152
External Table Parameter File Interface	153
6. Verilog-A Support	156
Verilog-A Language Support	157
Netlist Syntax for Verilog-A in the PrimeSim XA Tool	157
Using Verilog-A With the PrimeSim HSPICE Netlists	157
Generating Verilog-A Libraries for Reuse in PrimeSim XA Simulation	158
Module- and Instance-Based Partitioning: Switching Between Verilog-A and SPICE Definitions	160
Port Mapping	161
Passing Parameters	162
Passing the M-Factor	163
Defining Verilog-A Macros With -va,define Command Line Option	163
Verilog-A Output in the PrimeSim XA Tool	164
Reusing Compiled Verilog-A Files	164
Verilog-A Features Not Supported in the PrimeSim XA Tool	165
7. Postlayout Simulation Flows	167
Back-Annotation Flow	167
Understanding the Back-Annotation Flow	168
Understanding the PrimeSim XA-Supported Parasitic Netlist Format	170
Standard Parasitic Format (SPF)	171
Standard Parasitic Exchange Format (SPEF)	172
Device Property Format (DPF)	174
Running the Back-Annotation Flow	174
Rules for Selecting a Pin for Each Net	174
Mismatch Warning in Back-Annotation Flow	175
Net Does Not Exist in the Design Netlist or Is Skipped	178
Missing Instance for the Non-Bulk Net	179
Instance Does Not Exist in the Design Netlist	181
Instance Does Not Connect to This Net in the Design Netlist	183
Terminal Name Is Not Recognizable	185
Using the Analysis Statement in the Post-Layout Flow	187
Extended Back-Annotation Flow	190

Running the Extended Back-Annotation Flow	191
Limitations of the Extended Back-Annotation Flow	191
Selective Net Back-Annotation Flow	191
Running the Selective Net Back-Annotation Flow	192
Reusing the *.active_nets.rcxt File	193
Selective Net Extraction Flow	193
Running the Selective Net Extraction Flow	194
8. Sigma Amplification	196
Running Sigma Amplification	196
Configuring Inputs for Sigma Amplification	198
Determining the Amplification Factor for Sigma Amplification	200
Variation Analysis With Design-Specific Variation Corners	200
9. Monte Carlo Analysis	202
Configuring Options for Monte Carlo Analysis	203
Running Monte Carlo Analysis in PrimeSim XA	204
Supported HSPICE Options for Monte Carlo Simulation	207
Output Files From Monte Carlo Analysis	207
Running Incremental Monte Carlo Simulations	209
Setting Instance-Based Seeds	210
External Sampling During Monte Carlo Analysis	211
Random Variables	212
Data Mining Flow	212
Auto-Replacing Monte Carlo Samples When Measurements Fail	213
Controlling the Read-in of an External File	214
Running Monte Carlo Analysis With 3DIC	215
Variation Blocks in Monte Carlo Flow	215
Support for the Variation Block Within a Conditional Statement	217
Output Files	218
Differences Between PrimeSim XA and PrimeSim HSPICE	218
Distributed Processing	218
Using Relative Paths in Distributed Processing	218
Mixed-Signal Simulation	219

	Running ICSWEEP	219
	Licensing to Run Monte Carlo Analysis	219
	Limitations	219
10.	PrimeSim GPU and Multicore Simulation	221
	PrimeSim GPU Simulation	221
	Multicore Simulation	222
	Introducing Multicore Simulation	222
	Running a Multicore Simulation	222
	Determining the Multicore Benefit	223
	Factors that Affect a Multicore Simulation	223
11.	Analog Mixed-Signal Simulation	226
	Simulation Accuracy and Speed for Analog Mixed-Signal Designs	226
	Setting Simulation Levels for Analog and Mixed Signal Simulation	228
	Foundry Model Library Simulation	229
	FinFET Transceiver Simulations	229
	Setting Up the FinFET Transceiver Simulation	230
	Setting RF Options	230
	Setting Simulation Levels	231
	Setting Model Levels	231
	Setting Cores for Multicore Simulation	231
	Setting DC Options	232
	Examples	232
12.	SRAM Design Simulation	234
	SRAM Characterization Flow	234
	Accuracy and Performance Settings	235
	Commands to Replace for an SRAM Simulation	236
	Timing Methodology and Simulation Settings	236
	Dynamic Power Methodology and Simulation Settings	237
	Static and Leakage Power Methodology and Simulation Settings	238
	Debugging Tips	238
	Initializing Latch Circuitry	238
	Identifying Floating Nodes	239

Probing Sub-Nodes in Post-Layout Simulation	239
13. Flash Core Cell Models	241
Modeling Flash Core Cells	241
Using Flash Level 1	242
Supported MOS Models	242
Defining Flash Model Parameters	242
Programming and Erasing Conditions	244
Vth Change Behavior	245
Instantiating Flash Core Cells	246
Initializing Flash Core Cells	246
Probing Flash Core Cells	247
Flash Core Cell Level 1 Example	247
Using Flash Level 3	248
Supported MOS Models	248
Defining Flash Model Parameters	248
Programming and Erasing Conditions	250
Vth Change Behavior	251
Instantiating Flash Core Cells	251
Initializing Flash Core Cells	252
Probing Flash Core Cells	252
Flash Core Cell Level 3 Example	252
14. MRAM Core Cell Models	254
Modeling MRAM Core Cells	254
Spin-Torque-Transfer (STT) MRAM Core Cell Model (MRES0)	255
MRES0: STT MRAM Core Cell Definition	256
MRES0: Supported Parameter Descriptions	256
MRES0: Instantiation Example	257
MRES0: STT MRAM Core Cell Functionality	257
Limitations and Assumptions for the MRES0 Core Model	257
Dual-Active Layer (DAL) MRAM (MRES1)	257
MRES1: DAL MRAM Core Cell Definition	259
MRES1: Supported Parameter Description	259
MRES1: Instantiation Example	260
MRES1: DAL MRAM Core Cell Functionality	260

Limitations and Assumptions for the MRES1 Core Model	260
Toggle MRAM Core Cell Model (MRES2)	261
MRES2: Toggle MRAM Core Cell Definition	262
MRES2: Supported Parameter Description	262
MRES2: Instantiation Example	263
MRES2: Toggle MRAM Core Cell Functionality	263
15. Phase Change Memory (PCM) Models	264
PCM Model	264
Model Customization	265
Instantiation of PCM Cells	266
Programming Conditions	266
RESET Conditions	266
SET Conditions	266
INTERMEDIATE Conditions	267
MELTING Conditions	267
Probing PCM Cells	267
Probing Using the .probe Statement	267
Using the probe_waveform_pcm Command	268
Printing States of PCM Cells	268
Printing States of PCM Cells in Interactive Mode	268
16. Three-Dimensional Integrated Circuit (3DIC) Modularization	269
Overview of 3DIC Simulation Netlist	269
3DIC Netlist Construct and Usage	269
Instantiating Hierarchical Modules	270
Full Circuit Example	270
Module-Based PrimeSim XA Commands	272
3DIC Back-Annotation Support	274
Flat IC Module DSPFs With a Separate DSPF for the Silicon Interposer	274
DSPF Back-Annotation	276
Full 3DIC Flat Extracted DSPF	276
3DIC Support for TMI and Custom CMI Models	277
External Sampling for 3DIC Netlists	277

17. PrimeSim MOSRA Support in PrimeSim XA	280
Supported PrimeSim MOSRA Features	280
Unsupported PrimeSim MOSRA Features	281
Running PrimeSim MOSRA with Monte Carlo Analysis	282
Running MOSRA in a Fresh Simulation	282
Running MOSRA in a Post-Stress Simulation	282

A. Spectre Netlist Compatibility	284
Running PrimeSim XA With Spectre Netlist	284
Supported Spectre Simulator Elements	285
Supported Spectre Simulator Device Models	286
Supported Spectre Simulator Statements	288
Supported Spectre Simulator Functions	289
Supported Spectre Simulator Option Statement Parameters	289
Supported Spectre Simulator Tran Statement Parameters	290
Running Monte Carlo Analysis With Spectre Netlists	291
Enabling Monte Carlo Simulation With Spectre Netlists	291
Supported Parameters	291
Specifying Statistical Variation	292
Specifying Distribution	293
Using .ALTER Statements in Spectre Monte Carlo Simulation	293
Using Verilog-A With the Spectre Netlists in the PrimeSim XA Tool	296
Using Spectre Syntax in VCD Files	297
Supporting Sweeping Parameters During Transient Analysis	298
Support of the Spectre Save Statement	299
Using SPECTRE Syntax in 3DIC Netlists	299
Specifying Initial Conditions	301

B. Eldo Netlist Compatibility	303
Running PrimeSim XA With Eldo Netlist Format	303
Supported Eldo Simulator Commands	304
Supported Eldo Simulator Built-In Functions	305
Supported Eldo Simulator Options	306

Contents

Supported Eldo Simulator Elements	306
Supported Eldo Simulator Device Models	309
Running Monte Carlo Analysis With Eldo Netlists	311
Using Verilog-A With Eldo Netlists in the PrimeSim XA Tool	313
Supported Eldo Features in MOSRA Analysis	314
Specifying the Required Data	314
Supported Options of the .age Command	315
Required Arguments and Defaults	315
Known Limitations	317
Using SPECTRE Syntax in 3DIC Netlists	318
Eldo Syntax and Behavior Variations	320
Preprocessor Directives	320
Expression Syntax	321
The .EXTRACT Command	321
Specifying Initial Conditions	322
C. Debugging Tools	323
The -diff_log Command Line Option	323
Debugging Scripts	325
D. PrimeSim XA Tutorials	326
Running a Setup Timing Check	326
Running the Setup Simulation	329
Running the Back-Annotation Flow	330
Running Back-Annotation	332
Interpreting Back-Annotation Reports	332
Viewing the Simulation Output	335
Reviewing Violations	336
Generating Power Reports	340
Running the Power Reporting Simulation	344
Interpreting Power Reports	344
Using Interactive Mode	348
Using Interactive Commands	349
Using a Vector File in a Simulation	352
Running the Vector File Simulation With the PrimeSim XA Command	354

Contents

Running the Vector File Simulation Using the period Syntax	355
Running the Vector File Simulation Using the stop_at_error Syntax	356
Using a VCD File in a Simulation	357
Running a MOSRA Simulation	358

About the PrimeSim™ XA User Guide

The *PrimeSim™ XA User Guide* describes how to run transistor-level transient simulation on your circuit design using Synopsys PrimeSim XA tool.

This preface includes the following sections:

- [New in This Release](#)
 - [Related Products, Publications, and Trademarks](#)
 - [Conventions](#)
 - [Customer Support](#)
-

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in PrimeSim XA: What's New or the *PrimeSim XA Release Notes* on the [SolvNetPlus](#) site.

To see the *PrimeSim XA Release Notes*:

1. Go to the SolvNet Download Center located at the following address:
<https://solvnet.synopsys.com/DownloadCenter>
2. Select “PrimeSim XA,” and then select a release in the list that appears.

Note:

You must be logged in to SolvNetPlus for the link to connect directly to the release notes site. If you are prompted to log in to SolvNetPlus upon clicking the link, log in, then click the link again to reach the article.

Related Products, Publications, and Trademarks

For additional information about the PrimeSim XA tool, see the documentation on the Synopsys SolvNetPlus support site at the following address:

<https://solvnetplus.synopsys.com>

You might also want to see the documentation for the following related Synopsys and third-party products:

- Cadence® Spectre® Circuit Simulator
- Cadence® Virtuoso® Analog Design Environment
- Eldo®
- PrimeSim™ HSPICE®
- IC Validator™
- OASIS® (Open Artwork System Interchange Standard)
- StarRC™
- VCS®

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
Courier	Indicates syntax, such as <code>write_file</code> .
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>
Courier bold	Indicates user input—text you type verbatim—in examples, such as <code>prompt> write_file top</code>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN</code> .
	Indicates a choice among alternatives, such as <code>low medium high</code>
\	Indicates a continuation of a command line.

Convention	Description
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

1

Introducing the PrimeSim XA Tool

This chapter describes the PrimeSim XA tool and basic simulation setup information.

The PrimeSim XA tool is a transistor-level circuit simulator for transient simulation. It is designed to read in native SPICE netlists, model libraries, and commands, and supports netlist syntax in the PrimeSim HSPICE, Eldo, and Spectre formats.

Note:

This user guide describes how to simulate and analyze your circuit design with HSPICE netlists. For more information on how to run PrimeSim XA simulation with Spectre and Eldo netlists, see Appendix A, [“Spectre Netlist Compatibility”](#) and Appendix B, [“Eldo Netlist Compatibility”](#).

The PrimeSim XA tool can accurately simulate many different circuit types, including highly sensitive analog designs, and supports traditional SPICE algorithms such as:

- Newton-Raphson iteration
- Second-order integration
- Local truncation error control
- Charge conservation

The PrimeSim XA tool supports several technologies, such as specialized sparse solvers, pre-processing of linear systems, partitioning, multi-rate time step control, and table models to improve the simulation performance, while maintaining accuracy. The net effect of these technologies is that the CPU time required (per element) decreases as netlists become larger, so that the PrimeSim XA tool is an order of magnitude faster than a SPICE tool for small netlists (less than one thousand elements) and many orders of magnitude faster for large netlists (more than one million elements).

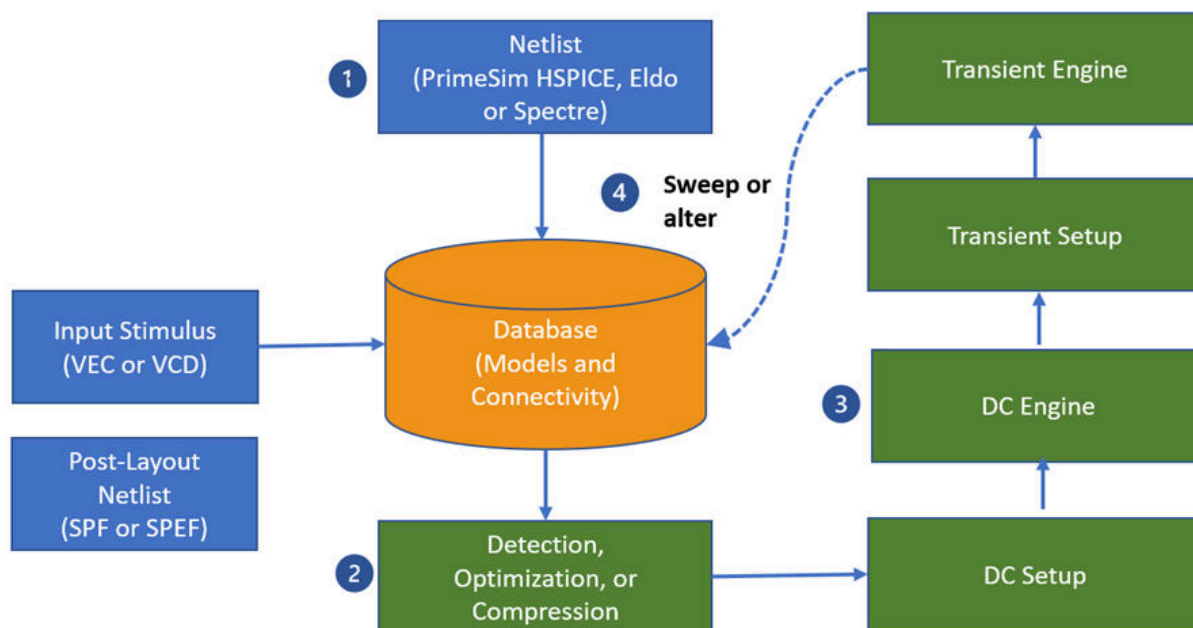
The PrimeSim XA tool also takes advantage of hierarchy in the netlist to improve capacity. Array elements, such as SRAM and DRAM cells, are automatically stored in specialized data structures to improve capacity. These data structures are designed so that the simulation memory requirements do not grow as more cells are accessed in an array.

Understanding the PrimeSim XA Flow

The PrimeSim XA flow shown in [Figure 1](#) has the following steps:

1. A top-level native-SPICE netlist is parsed from the command line. The netlist can include additional netlists and model libraries. The parsed data is stored in the database where connectivity, parameters, models, and commands are resolved. Input stimulus such as PrimeSim HSPICE Vector File (VEC) and Value Change Dump File (VCD) and a post-layout netlist, such as an SPF or SPEF file, are parsed and added to the database.
2. The PrimeSim XA tool then applies optimizations and compresses the data. This step takes advantage of the input netlist hierarchy to improve capacity and performance during the setup phase. It does not require the netlist to be hierarchical to provide enhanced performance.
3. The optimized data is passed to the DC engine to find the steady-state voltages for the beginning of the transient simulation. The DC data is passed to the transient engine for transient simulation. Both engines have a small setup phase where the data is further optimized.
4. If more than one simulation occurs due to sweeps or alters, the PrimeSim XA tool returns to the database. It modifies the data and loops to perform another simulation.

Figure 1 PrimeSim XA Simulation Flow



2

Getting Started

This chapter provides the basic information you need to start using the PrimeSim XA simulator.

This chapter contains the following topics:

- [Setting Up the Simulator](#)
- [Running the Simulator](#)
- [Using Commands and Options](#)
- [Support for Compressed \(.gzip\) Files](#)
- [Suspending or Resuming License](#)
- [Saving and Restoring Simulations](#)
- [Output Files](#)
- [Output Waveform Files](#)

Setting Up the Simulator

Before running the PrimeSim XA tool, you must complete the following setup:

- [Configuring the PrimeSim XA License](#)
- [Setting Up the Path to the Executable](#)
- [Configuring a Temporary Directory Location](#)
- [Using an Initialization File](#)
- [Controlling the Interval of Simulation Status Update](#)

Configuring the PrimeSim XA License

You must set up one of the following environment variables to point to the license paths. When specifying the pointer to the license paths, use the full path of the license file, the *port@host* syntax, or a combination of the two syntax types separated by colon.

- `SNPSLMD_LICENSE_FILE` - A pointer to the Synopsys license paths
- `LM_LICENSE_FILE` - A pointer to all license paths.

Note:

If you specify both environment variables, `SNPSLMD_LICENSE_FILE` takes precedence over `LM_LICENSE_FILE`.

In C-Shell:

```
setenv SNPSLMD_LICENSE_FILE pointer_to_license_paths
```

In Bourne, Korn, or Bash shells:

```
export SNPSLMD_LICENSE_FILE=pointer_to_license_paths
```

PrimeSim XA supports two license modes as described in the following sections:

- [Configuring the PrimeSim Continuum Token-Based License Mode](#)
- [Configuring the Standalone License Mode](#)

Configuring the PrimeSim Continuum Token-Based License Mode

To enable the PrimeSim Continuum token-based license mode, use one of the following methods:

- Define the shell environment variable `PRIMESIM` to 1.

In C-Shell:

```
setenv PRIMESIM 1
```

In Bash shell:

```
export PRIMESIM=1
```

- Use the `-primesim 1` command line option.

If the shell environment variable is set and the `-primesim` command line option is used, the command line option takes precedence and overrides the shell environment variable setting.

For example,

```
% setenv PRIMESIM 1
% xa netlist.sp
```

or

```
% xa netlist.sp -primesim 1
```

Table 1 lists the shell environment variables that control the token-based license mode and license queuing.

Table 1 Variables for Controlling Token-Based License Mode and License Queuing

Environment Variable	Description
PRIMESIM 0 1 2	This variable enables PrimeSim Continuum token-based license mode. Valid values: • 0: Disables token-based license mode. • 1: Enables token-based license mode. • 2: (Default) Enables standalone and token-based license mode. By default, the standalone keys are attempted first (see PRIMESIM_ORDER). This variable can also be enabled via the <code>-primesim</code> command line option.
PRIMESIM_ORDER 0 1	This variable controls whether standalone keys or token-based keys are attempted first when the <code>PRIMESIM</code> variable is set to 2. • 0: (Default) Standalone keys are attempted first. • 1: Token-based keys are attempted first.
PRIMESIM_WAIT_LICENSE 1 0	This variable enables and disables license queuing. • 0: Disables license queuing. • 1: (Default) Enables license queuing.
PRIMESIM_WAIT_LICENSE_TIMEOUT value	This variable controls the timeout period for license queuing. The value is in minutes and cannot exceed 43,200 minutes (30 days). The range is [1, 43200] in minutes. The default period is 2 minutes. Non-positive integer values will result in an error. The following example sets the queue timeout period to 30 minutes: <pre>% setenv PRIMESIM_WAIT_LICENSE_TIMEOUT 30</pre>

Table 1 *Variables for Controlling Token-Based License Mode and License Queuing*
(Continued)

Environment Variable	Description
<code>PRIMESIM_WAIT_LICENSE_INTERVAL</code> <i>value</i>	This variable controls the timeout period of sleeping during license queuing. The value is in seconds and cannot exceed 60 seconds and cannot be less than 1 second. The default period is 30 seconds. Integer values beyond [0, 60] will result in a warning message with resetting into the default 30 seconds. The following example sets the queue timeout period to 2 seconds: <pre>% setenv PRIMESIM_WAIT_LICENSE_INTERVAL 2</pre> Note: Setting a short interval time like 2 seconds causes frequent communication between license server and running clients, significantly increasing the load on the license server. This causes overloading of license server when there are thousands of concurrent running clients.
<code>PRIMESIM_WAIT_LICENSE_MODE</code> <i>QUEUE ROUND</i>	Controls the behavior of PrimeSim XA jobs when they encounter a license peak. QUEUE: (Default) When the license peak is reached, PrimeSim XA waiting jobs are held in a queue on the license servers. The interval time for these waiting jobs is adjusted using the <code>PRIMESIM_WAIT_LICENSE_INTERVAL</code> variable, set to 30 seconds by default. ROUND: When the license peak is reached, PrimeSim XA waiting jobs enter a periodic loop to check for available licenses with the interval time defined by the <code>PRIMESIM_WAIT_LICENSE_INTERVAL</code> setting. Consequently, disables incremental license checkout or check-in, similar to the setting of <code>PRIMESIM_ONCE_CHECKOUT_LIC=1</code>. Note: If the license server shuts down, the periodic interval time for waiting jobs is extended by 20 times the value of <code>PRIMESIM_WAIT_LICENSE_INTERVAL</code> setting to reduce CPU load. It is only effective for PrimeSim XA when the <code>PRIMESIM=1 2</code> license modes.

Configuring the Standalone License Mode

This is the default license mode if the PrimeSim Continuum token-based license mode is not enabled.

If the required license keys for the PrimeSim XA tool are not available when it is invoked, the tool aborts with an error. You can instruct the PrimeSim XA tool to queue for a license by setting the `XA_WAIT_LICENSE` environment variable. When `XA_WAIT_LICENSE` is specified and the license keys are not available, the PrimeSim XA tool remains alive and idle until the license keys become available. For example:

- In C-Shell:

```
setenv XA_WAIT_LICENSE 1
```

- In Bourne, Korn, or Bash shells:

```
export XA_WAIT_LICENSE=1
```

When the wait license feature is enabled, the PrimeSim XA tool queues indefinitely for an available license keys. To time out the license wait feature, set the `XA_WAIT_LICENSE_TIMEOUT` environment variable. The specified value is in minutes. For example:

- In C-Shell:

```
setenv XA_WAIT_LICENSE_TIMEOUT timeout_in_minutes
```

- In Bourne, Korn, or Bash shells:

```
export XA_WAIT_LICENSE_TIMEOUT timeout_in_minutes
```

Examples

The following example sets the license path to `/usr/cad/license.dat:27000@cdl1` and puts the PrimeSim XA tool on queue for the next available keys when license keys are not available.

```
setenv SNPSLMD_LICENSE_FILE /usr/cad/license.dat:27000@cdl1  
setenv XA_WAIT_LICENSE 1
```

or

```
export SNPSLMD_LICENSE_FILE=/usr/cad/license.dat:27000@cdl1  
export XA_WAIT_LICENSE=1
```

The following example enables the wait license feature and sets a timeout period to 120 minutes (2 hours). When license keys are not available after 2 hours, the wait license feature is disabled.

```
setenv SNPSLMD_LICENSE_FILE /usr/cad/license.dat:27000@cdl1  
setenv XA_WAIT_LICENSE 1  
setenv XA_WAIT_LICENSE_TIMEOUT 120
```

or

```
export SNPSLMD_LICENSE_FILE=/usr/cad/license.dat:27000@cdl1  
export XA_WAIT_LICENSE=1  
export XA_WAIT_LICENSE_TIMEOUT=120
```

Setting Up the Path to the Executable

The PrimeSim XA executable is called by the launch script path. The launch script is in the `XA_INSTALL_DIR/bin` directory. You need to set up the shell path to include the path to the PrimeSim XA launch script. For example:

C-Shell:

```
set path = (XA_INSTALL_DIR/bin $path)
```

Bourne, Korn, or Bash shells:

```
export PATH=XA_INSTALL_DIR/bin:$path
```

For example:

```
set path = ( /tools/synopsys/XA-version/bin $path )
```

or

```
export PATH=/tools/synopsys/XA-version/bin:$PATH
```

Sets the path to the launch script at the `/tools/synopsys/XA.06/bin` directory.

Configuring a Temporary Directory Location

During a PrimeSim XA simulation, the PrimeSim XA tool by default creates a temporary file named `filename.rawimage.*` under the current simulation output directory. This file sometimes creates a disk space problem due to its large size, which depends on the input netlist size. In order to avoid a potential disk space problem, you can specify the following environment variable to redirect the file to another directory.

This temporary file is removed once the simulation completes. This step is optional if disk space is not a problem.

In C-Shell:

```
setenv XA_TMP_DIR user_defined_tmpdir
```


In Bourne, Korn, or Bash shells:

```
export XA_TMP_DIR=user_defined_tmpdir
```

For example:

```
setenv XA_TMP_DIR /tmp
```

or

```
export XA_TMP_DIR=/tmp
```

Redirects the temporary file to /tmp directory.

Using an Initialization File

You can specify an initialization file, `xa.ini`, that contains commands to read in before any netlist files. Any other statements in the file are ignored.

The initialization command precedent rule is that the last command applied overrides any previous commands. The initialization file takes effect as if it were included at the beginning of the netlist file. The PrimeSim XA tool searches the initialization file in the following paths in precedence order from high to low:

1. Current simulation directory
2. \$HOME (user home directory)
3. XA_installation_dir/etc/...

Controlling the Interval of Simulation Status Update

During a PrimeSim XA simulation, the PrimeSim XA tool prints simulation status to standard error (stderr). You can use the `XA_STATUS` environment variable to control the time interval of the simulation status update to stderr. For example:

In C-Shell:

```
setenv XA_STATUS 0|time_interval_in_seconds
```

In Bourne, Korn, or Bash shells:

```
export XA_STATUS=0|time_interval_in_seconds
```

When set to 0, it disables the PrimeSim XA status update to stderr. When set to a positive integer, the PrimeSim XA tool updates the simulation status to stderr at the specified time interval in seconds. The default is 10.

The following example instructs the PrimeSim XA tool to update the simulation status every 20 seconds.

```
setenv XA_STATUS 20
```

or

```
export XA_STATUS=20
```

Running the Simulator

To start the PrimeSim XA tool, enter the following command at the UNIX command prompt:

```
xa [-format] netlist_file [command_line_options]
```

Specify the name of the top-level netlist file and the command line options. The PrimeSim XA tool invokes the simulator to perform transient simulation. For a detailed description about the supported netlist formats and command line options, see the following [Specifying Netlist Format](#) and [Using Command Line Options](#) sections.

The PrimeSim XA tool provides a different version of simulator (called *ss/2* in this document) for designs with different requirements on performance and accuracy, such as post-layout designs in advanced process technology. For more information about how to run PrimeSim XA simulator version II, see [Running Simulator Version II](#).

The PrimeSim XA tool provides commands to control simulation performance for better simulation speed and accuracy and model complexity tradeoffs. For example, you use the `set_sim_level` command to control simulation speed and model complexity by setting a simulation level. See [Setting Simulation Levels](#) and [Simulation Accuracy and Speed for Analog Mixed-Signal Designs](#) for more details.

Example

The following example reads the `input.sp` netlist file, specifies the directory name for saving output files, and uses the prefix of the input netlist file name as the prefix for the output files.

```
xa input.sp -o ./OUTPUT_DIR -outfilefmt hspice
```

When simulation is complete, the tool saves the following output files in the `/OUTPUT_DIR` directory:

- `OUTPUT_DIR/input.fsdb`
- `OUTPUT_DIR/input.log`
- `OUTPUT_DIR/input.meas`

Note:

This user guide describes how to simulate and analyze your circuit design with PrimeSim HSPICE netlists. For more information on how to run PrimeSim XA simulation with Spectre and Eldo netlists, see Appendix A, “[Spectre Netlist Compatibility](#)” and Appendix B, “[Eldo Netlist Compatibility](#)”.

This section contains the following topics:

- [Specifying Netlist Format](#)
- [Using Command Line Options](#)
- [Running Simulator Version II](#)
- [Setting Simulation Levels](#)
- [Simulating SRAM or Flash Designs](#)
- [Minimum Resistor Threshold](#)

Specifying Netlist Format

The PrimeSim XA tool supports PrimeSim HSPICE, Eldo and Spectre formats. When no netlist file format is specified, the PrimeSim HSPICE format is used. Only one netlist file format can be used for each simulation, and you can specify only one netlist file on the command line. You can include the remaining files with the appropriate simulator statements within the named netlist file.

The following table lists the command line options for each supported format.

Table 2 Supported Netlist File Format and Command Line Options

Netlist Format	Command Line Options
PrimeSim HSPICE	-hspice
Eldo	-eldo
Spectre	-spectre

To control the case sensitivity of the specified netlist, set the `-case upper|lower|sensitive` command line option. The `-case` command line option is equivalent to the `set_sim_case` option.

Using Command Line Options

The following tables describe the command line options and their usage.

- [Table 3](#) for general command line options
- [Table 4](#) for Monte Carlo command line options
- [Table 5](#) for electromigration and IR drop (EMIR) analysis command line options

Table 3 *General Command Line Options*

Command Line Option	Description
<code>-active_net_flow</code>	Enables the back-annotation active net flow with the default settings. For more information about the active net flow, see the <code>set_active_net_flow</code> command. Note that this command line option takes precedence over the <code>set_active_net_flow</code> command.
<code>-arch COMPATIBLE</code>	Switches to one more compatible math lib used instead of default Intel math lib.
<code>-c command_file</code>	Specifies the name of the command file. You can specify the PrimeSim XA commands in the command file and use the <code>-c</code> command line option to include this file for simulation. You can specify multiple command files with multiple <code>-c</code> command line options.
<code>-container</code>	Runs the simulator in container.
<code>-D<x></code>	Defines string x and runs the C preprocessor. This command line option is only valid for Eldo and Spectre netlist formats.
<code>-D<x=y></code>	Defines string x as y, and runs the C preprocessor. This command line option is only valid for Eldo and Spectre netlist formats.
<code>-dcintr [iteration]</code>	Invokes DC interactive mode when the DC engine reaches the specified <i>iteration</i> . If iteration is not specified, use Ctrl-C to enter interactive mode in DC.
<code>-gz</code>	Enables GZIP file compression of the following output files: • Result files from the <code>.OP</code> statements or initial condition files • Result files from the <code>.MEASURE</code> statements • Result files from the <code>.PRINT</code> statements • Waveform files • Result files from the dynamic CCK commands • *.mc0 files from Monte Carlo Analysis See the Running the Simulator section for more information.
<code>-I [dir]</code>	Searches the <i>dir</i> directory for included files. If <i>dir</i> is not specified, the current run directory is set.
<code>-incr</code>	Same as <code>-l</code> . Specifies a path to search for files.
<code>-intr [time[unit]]</code>	Invokes the interactive mode during transient at the specified time. If you do not specify a time value, you must use Ctrl-C to enter interactive mode in transient.

Table 3 General Command Line Options (Continued)

Command Line Option	Description
<code>-mt num_of_core max</code>	Enables multicore simulation with the specified number of cores. Use the <code>max</code> keyword to specify the maximum number of cores in the machine to run multicore simulation. This command line option takes precedence over the <code>set_multi_core</code> command. For more information about running a multicore simulation, see PrimeSim GPU and Multicore Simulation .
<code>-sim_mode 0 1</code>	Enables high performance simulation algorithms for analog and mixed signal designs. This command line option is equivalent to the <code>set_sim_mode</code> command. For more information about running a simulation on analog mixed-signal designs, see Analog Mixed-Signal Simulation .
<code>-ssl2</code>	Invokes simulator version 2 for simulation. For information about how to use this command, see the Running Simulator Version II section.
<code>-stop</code>	Same as <code>-t time[unit]</code> .
<code>-tcl</code>	Enables Tcl programming for PrimeSim XA commands. See the Enabling Tcl Mode section in <i>PrimeSim™ XA Command Reference</i> .
<code>-t time[unit]</code>	Overwrites the transient analysis time specified in the netlist. The unit can be ms, us, ps or ns. If unit is not specified, the default is ns.
<code>-time time[unit]</code>	Same as <code>-t time[unit]</code> .
<code>-top subckt_name</code>	Specifies the name of the subcircuit to be simulated as a top-level subcircuit. Here are the rules: • Do not specify a nested subcircuit. • Do not specify a Verilog-A module. • The PrimeSim XA tool only elaborates the <code>subckt_name</code> as the top-level subcircuit. • The PrimeSim XA tool elaborates pseudo top VSRC/ISRC/vector/stimulus elements. See the <code>keep_top_element</code> command for information about specifying top-level instance to be simulated.
<code>-U<x></code>	Undefines string x and runs the C preprocessor. This option is only valid in the Eldo and Spectre modes.
<code>-v [ersion]</code>	Prints binary information, including tool name, binary name, platform version, build time and date, and build identification number.

Licensing

Table 3 General Command Line Options (Continued)

Command Line Option	Description
<code>-primesim [0 1 2]</code>	Enables the PrimeSim Continuum token-based licensing scheme. Valid values: • 0: Disables token-based license mode. • 1: Enables token-based license mode. • 2: (Default) Enables standalone and token-based license mode. By default, the standalone keys are attempted first (see <code>PRIMESIM_ORDER</code>).
Saving/Restoring Simulation	
<code>-auto_restart</code>	Automatically restarts a simulation from the most recent snapshot if available. If no snapshot is available, PrimeSim XA proceeds the simulation from the first time point (time 0) with a warning message. If the restarted run completes successfully, PrimeSim XA automatically deletes the snapshot being used or the one that is generated by the <code>set_save_state</code> command. The auto-deletion functionality applies only to the image-based application.
<code>-auto_restore</code>	Same as <code>-auto_restart</code>. Automatically restores a saved simulation based on the percent value you specify with the <code>set_save_state</code> command.
<code>-enable_isnapshot_feature</code>	Enables the image-based snapshot capturing feature.
<code>-restart</code>	Restarts a saved simulation from a <code>saved_sim_file</code> that is generated by the <code>set_save_state</code> command.
<code>-restart_type type</code>	Specifies the type of saved state to use with the <code>-auto_restart</code> option. Supported types: • <code>OP</code>: (Default) Uses a snapshot that contains operating point (OP) information. • <code>image</code>: Uses an image-based snapshot. By default, the tool checks for an OP-based snapshot to restart a simulation. When used with the <code>-enable_isnapshot_feature</code> option, the tool restarts a simulation based on an image-based snapshot. If no snapshot is available, PrimeSim XA proceeds the simulation from the first time point (time 0) with a warning message.
<code>-restore saved_sim_file</code>	Same as <code>-restart</code>. Restores a saved simulation from a <code>saved_sim_file</code> that is generated by the <code>set_save_state</code> command.

Table 3 General Command Line Options (Continued)

Command Line Option	Description
<code>-restore_type type</code>	Same as <code>-restart_type</code>. Specifies the type of saved state to use with the <code>-auto_restore</code> option. Supported types: • <code>OP</code>: (Default) Uses a snapshot that contains operating point (OP) information. • <code>image</code>: Uses an image-based snapshot. By default, the tool checks for an OP-based snapshot to restore a simulation. When used with the <code>-enable_isnapshot_feature</code> option, the tool restores a simulation based on an image-based snapshot. If no snapshot is available, the PrimeSim XA tool proceeds the simulation from the first time point (time 0) with a warning message.
Output	
<code>-disable_rawimage</code>	Disables generation of a <code>.rawimage</code> file. For more information, see the <code>set_disable_rawimage</code> command.
<code>-format</code>	Same as <code>-wavefmt</code> .
<code>-fsdb_info</code>	Prints the supported versions of FSDB files.
<code>-o [outpath/]outfile</code>	Specifies the name of the directory and prefix name of the output files. <i>outpath</i> is the name to the output directory path and <i>outfile</i> is the prefix name of the output files. If <i>outpath</i> is not specified, the default is the current run directory. If <i>outfile</i> is not specified, the default is controlled by the <code>-outfilefmt</code> command line option.
<code>-outfilefmt hspice xa</code>	Specifies the output file prefix name if the prefix name of the output files is not specified with the <code>-o</code> command line option. If it is set to <code>hspice</code> , the prefix is named after the prefix of the input file name. If it is set to <code>xa</code> , the prefix is <code>xa</code> . The default is <code>xa</code> .
<code>-wavefmt format</code>	Specifies the format of the output waveform file. See the Supported Waveform File Formats section for more information.
<code>-wavedir directory_name</code>	Dumps waveforms in the specified directory different from the output directory defined by the <code>-o</code> option.
<code>-wavefile waveform_file</code>	Lets you specify new measurements (such as new or edited <code>.MEASURE</code> statements) and rerun a simulation based on the results of an existing <code>fsdb</code> or <code>wdf</code> file. This option can improve performance by skipping the simulation steps not related to the new measurements. See the <code>meas_post</code> command description for more information.
Help	
<code>-webhelp</code>	Opens a full online help collection in a web browser. Click How to Use Help in the lower left corner of the Help window for details about how to use Help features such as the Help icons, Search, and Bookmarks.

Table 3 General Command Line Options (Continued)

Command Line Option	Description
<code>-h[elp]</code>	Prints the list of command line options with brief descriptions.

Table 4 Monte Carlo Command Line Options

Command Line Option	Description
<code>-dp [#workers]</code>	Invokes distributed processing (DP) and specifies the number of workers. The workers can be distributed on one multiple-core machine or multiple machines across a network. If <code>#workers</code> is not specified, the PrimeSim XA tool assigns the workers to all available cores in the master machine. When running <code>-dp</code> with <code>-dpconfig</code> on multiple machines across the network, you must specify <code>#workers</code>. To stop and end a simulation after you have started a simulation with <code>-dp</code>, press Ctrl-C. The PrimeSim XA tool supports the following features for distributed processing: • Monte Carlo • .ALTER blocks • .DATA blocks • .TEMP sweep • Eldo .STEP parametric sweep
<code>-dpconfig config_file</code>	Specifies the configuration file for DP. If you do not specify this option, DP distributes the processes of all workers to only the master machine.
<code>-dplocation TMP NFS</code>	Specifies the location for all the results files generated by the DP simulation: • <code>TMP</code> (default): Results go to the <code>/tmp</code> folder, and are then moved to the <code>-o</code> PrimeSim XA output working directory after the whole DP simulation from all the workers is completed. • <code>NFS</code>: Results files go directly to the <code>-o</code> PrimeSim XA output working directory once they are generated.

Table 4 Monte Carlo Command Line Options (Continued)

Command Line Option	Description
<code>-dpworkerchk</code> <code>none mem load disk all</code>	Specifies the DP worker monitor level. Default is <code>all</code>. This option applies to <code>-dp -dpconfig</code> flow. <code>none</code>: No feature is monitored. <code>mem</code>: Memory monitor. Master drops the worker if the worker hosts memory consumption is too high. The lower threshold of available physical memory is controlled by <code>.OPTION DP_MEMFREE</code>. <code>load</code>: CPU load monitor. Master drops the worker if the worker host's CPU load is high. The upper threshold of the load average of one worker CPU core is controlled by <code>.OPTION DP_LOADAVG</code>. <code>disk</code>: Disk monitor. Master drops the worker if the worker's CDPL scratch disk usage is too high. Master stops the whole DP simulation if the DP master output disk usage is too high. The lower threshold of the available working (scratch) disk space is controlled by <code>.OPTION DP_DISKFREE</code>. <code>all</code>: (default) Memory, load, and disk usage are all monitored. Note: Multiple monitoring types for memory, load, and disk options are supported by combining two options with "+" syntax. For example: <pre>-dpworkercheck mem+load</pre>
<code>-dscale output.dynamic</code>	Supports dynamic adjustment of the number of DP workers. During DP processing, the simulator creates a file <code>output.dynamic</code> in the DP output directory. Now, you can launch another PrimeSim XA simulation with the following command: <pre>XA -dscale output.dynamic -dp dp_num</pre> to change or adjust the current number of DP workers to another value. In this manner, you can decrease and increase the DP worker numbers, as many times as required. For details and usage example, see the Dynamic Scaling topic in the <i>PrimeSim User Guide: Pro and SPICE</i>.
<code>-merge</code>	Merges all separated distributed processing output log files (*.log) into one file, similar to a single core run. Note that this process is time consuming if log file size is large. When you use this argument in the PrimeSim HSPICE tool, it merges the log file as well as the <code>tr0</code> waveform file. The PrimeSim XA tool only merges the log file. But the PrimeSim XA tool can generate a wrapper file for the wave viewer to load all the separate waveform files at once.

Table 5 PrimeSim EMIR Analysis Command Line Options

Command Line Option	Description
<code>-r spres_command_file</code>	Invokes the Static Power Net Resistance (SPRES) flow. <i>spres_command_file</i> contains the setup commands for resistance calculations. See the <i>PrimeSim™ XA Command Reference</i> section for more information.
<code>-ra argument</code>	Invokes Phase II simulation of EMIR analysis. See the Performing Power Net EMIR Analysis - Analysis Stage section in the <i>PrimeSim EMIR Reference Manual</i> .
<code>-radp processes</code>	Specifies the number of processes that are used for processing nets in parallel on phase II EMIR simulation. For more information, see the EMIR Pipeline Processing section in the <i>PrimeSim EMIR Reference Manual</i> and the <code>set_ra_option</code> command description.
<code>-radp_config cpdl_config_file_name</code>	Specifies the CDPL hosts configuration file. For more information, see the EMIR Pipeline Processing section in the <i>PrimeSim EMIR Reference Manual</i> and the <code>set_ra_option</code> command description. See <i>Common DP Library User Guide</i> for the <code>cpdl_config_file</code> syntax.
<code>-ralayout argument</code>	Generates the violation map for EMIR analysis. See the Power Net EMIR Analysis section in the <i>PrimeSim EMIR Reference Manual</i> .
<code>-rp argument</code>	Specifies report generation for static power net resistance analysis. See the Static Resistance Check section in the <i>PrimeSim EMIR Reference Manual</i> .

Running Simulator Version II

In addition to the default simulator (called *ss1* in this document), the PrimeSim XA tool provides the `-ss12` command line option to invoke a different version of simulator (referred to as *ss2* in this document) to improve performance and accuracy when simulating certain types of designs, such as post-layout designs in advanced process technology. Running simulation with the simulator version 2 requires less commands and simpler setup than the default *ss1* simulator to achieve the balance between result accuracy and performance as needed.

Any simulation that uses *ss1*-related commands can be run with *ss2*-related commands. However, if you run a *ss2* simulation with a setup that contains *ss1*-related commands, the simulation results might not be improved as expected. If you want to use an existing setup for *ss2* simulation, remove all *ss1*-related commands from the setup before using it for the *ss2* simulation. It is recommended that you run the *ss2* simulation with a single, top-level setup that contains only *ss2*-related commands.

You can invoke ssl2 simulation with the following ways:

- Run the simulation with the `-ssl2` command line option.

```
xa netlist_file -c cmd_file -o output_prefix -ssl2
```

If you do not specify the `-ssl2` command line option, the PrimeSim XA tool runs ssl1 simulation. The .log file indicates whether the simulation is run with the ssl1 or ssl2 simulator.

- Run the simulation with the `XA_SSL2` UNIX environment variable. If this variable is set, then all PrimeSim XA simulations use ssl2 by default. For example:

```
setenv XA_SSL2 1  
xa netlist_file -c cmd_file -o output_prefix
```

Note:

There is no equivalent ssl1 command line option. If you enable the `XA_SSL2` variable, the only way to run ssl1 simulation is to disable the variable.

Setting Simulation Levels

When running a simulation in the PrimeSim XA tool, use the `set_sim_level` command to control simulation speed and model complexity by setting a simulation level. The higher the level, the more accurate the simulation result with longer simulation time. This command can be applied to the entire netlist, subcircuits or instances. The default level is 3.

Syntax

Here is the syntax for the `set_sim_level` command:

```
set_sim_level -level level [-inst instance_spec]
```

- `-level level`

Specifies the pair of time and level to enable accuracy changes in the middle of a transient simulation.

- 1 for a functional verification of digital circuits. Level 1 is not supported if PrimeSim XA is run with the `-ssl2` option.
- 2 for a functional verification of digital circuits, more conservative than level 1. Level 2 is not supported if PrimeSim XA is run with the `-ssl2` option.
- 3 for pure digital designs (functional, timing, and power simulations)
- 4 for analog/mixed-signal verification mode

- 5 for analog/mixed-signal accuracy mode
- 6 for precision analog (FFT, jitter, eye diagram, and so on, which might be mixed with digital simulation)
- 7 for or model and netlist testing
- `-inst instance_spec`

Specifies the instance information. See the Common Syntax Definitions section for details about the *instance_spec* argument.

Simulating SRAM or Flash Designs

For standalone SRAM and flash simulations, it is recommended that you use the respective circuit-specific `set_sram_characterization` and `set_circuit_flash` commands. These commands are able to make assumptions based on the specific circuit type being simulated that the general purpose `ssl1` and `ssl2` commands do not make.

For more information, see the [SRAM Design Simulation](#) and [Flash Core Cell Models](#) sections.

Minimum Resistor Threshold

The minimum resistor threshold is set to 0.1 ohms with the `ssl2` command, compared to the minimum resistor snap value of 1e-5 for `ssl1` simulation. (A snap value means any resistor below 1e-5 is snapped up to 1e-5). The minimum resistor threshold of 0.1 ohm is in line with other FastSPICE simulators. It is not expected that the higher minimum resistor threshold setting affects accuracy, but it might have an impact on netlist parsing.

When a resistor value is less than 0.1 ohm, the resistor is shorted and the connecting nodes are "merged". In some cases, this can cause a netlist parsing error because the merging of these connecting nodes may short ideal voltage sources. In many cases, the multiple voltage source separated by a resistor of less than 0.1 ohm is a netlist error that should be corrected in the netlist. You can use the `set_resistor_option -rule 1 -min value` command if you prefer keeping these small resistors.

Using Commands and Options

The PrimeSim XA tool provides commands and command line options to control simulation accuracy and performance. You can enter the commands in different ways: in a script file, within a netlist or in the interactive debugging mode. Command line options are used on the `xa` command line for various kinds of controls, such as licensing, feature enabling, outputs, and so on.

Some of the PrimeSim XA commands provide equivalent options to set with the `.option` command in the netlist, such as `.option skip_circuit_block`. Some PrimeSim XA commands, such as `set_dc_option`, can be used only in the command script file.

You can find details of the PrimeSim XA commands in the *PrimeSim™ XA Command Reference* as well as information on command syntax definitions and so on. For a list of command line options supported in the PrimeSim XA tool, see [Using Command Line Options](#).

See Also

- [Using XA Commands in a Command Script File](#)
- [Using XA Commands Within the Netlist](#)
- [Using XA Commands During Interactive Debugging Mode](#)

Case Sensitivity and Wildcards

All commands and their arguments are case-sensitive—even those commands placed within PrimeSim HSPICE and Eldo netlists. Command arguments that refer to netlist identifiers are treated as case-insensitive in PrimeSim HSPICE and Eldo format netlists, and as case-sensitive in Spectre format netlists.

The PrimeSim XA commands and netlist statements support wildcards to match instance and node names during simulation. Each of the commands and options might have different wildcard usages. The case sensitivity of the wildcard matching is set according to case sensitivity of the netlist where the names are defined. See the individual command for options that support a wildcard for more information.

For examples of running PrimeSim XA commands with wildcards, see the Using Wildcards section in *PrimeSim XA Command Reference*.

The behavior of a wildcard in the PrimeSim XA tool might be different from the PrimeSim HSPICE, Eldo, and Spectre tools. The use of wildcards in the PrimeSim XA tool is an enhanced feature for FastSPICE applications.

For more information about PrimeSim XA compatible Spectre and Eldo commands and options, see Appendix A, “[Spectre Netlist Compatibility](#)” and Appendix B, “[Eldo Netlist Compatibility](#)”.

Support for Compressed (.gzip) Files

To save disk space and to improve performance, PrimeSim XA supports the .gzip file format for the following files:

File type	File extension	Gzip read	Gzip write
Parser input files (HSPICE/SPICE/ELDO)	Any	Yes	NA
SPF/DSPF	Any	Yes	NA
Vector file	.vec	Yes	NA
IC file	.ic	Yes	No
.MC0 file for external sampling	.mc0	Yes	Yes

When both `design.sp` and `design.sp.gz` exist, the PrimeSim XA tool always selects the exact match file first, if it exists. For example:

- If `.include design.sp` is in the netlist, then the PrimeSim tool selects `design.sp`.
- If `.include design.sp.gz` is in the netlist, then the PrimeSim tool selects `design.sp.gz`.
- If `.include design.sp` is in the netlist and only `design.sp.gz` exists, then the PrimeSim tool selects `design.sp.gz`.

Note:

The PrimeSim XA tool does not support the S-parameter and RLGC file input types in the .gzip file format.

Suspending or Resuming License

To temporarily suspend a simulation, use `kill -10 $PID`, where `$PID` is the process ID of the simulation job. After suspending with `kill -10`, the tool releases the license until you resume the simulation using `fg` or `kill -18 $PID`. Use this feature when your license pools are fully utilized and you want to free up some licenses for another critical simulation.

Note:

This feature is not recommended for multi-CPU simulations. You may free up the licenses, but the system resources may not be freed up, nor can the simulation be resumed.

To suspend a simulation job:

```
%> kill -10 $PID
```

To resume a simulation job:

```
%> kill -18 $PID
```

Saving and Restoring Simulations

Use the `set_save_state` command to capture a snapshot of the current simulation state to be restarted later with a restore command line option, such as `-auto_restore` or `-restore` and so on. The `set_save_state` command and the restore command line options together provide a solution that allows you to have simulation iterations more effectively and maximize cloud bandwidth utilization by seamlessly restarting an aborted PrimeSim XA simulation on the cloud.

For a detailed description about the commands and command line options, see the related command pages in *PrimeSim XA Command Reference* and the [Using Command Line Options](#) section, respectively.

You can save a snapshot in either OP-based or image-based.

- **OP-based snapshot:** Saves the operating point information of the current state. Use this snapshot type when you want to use the saved snapshot as the initial guess and then modify the netlist or simulation setup in the restarted run. The OP-based snapshot provides an operating point solution to be reused in a subsequent simulation run.

For example, you can reuse an OP-based snapshot to

- Change stimuli.
 - Change the accuracy level (`set_sim_level`, `set_ccap_level`, `set_tolerance_level`, and `set_model_level`).
 - Change simple capacitance and resistance values.
 - Add or remove current or voltage probes.
 - Change the value of `.tran`.
 - Set a new `set_save_state` command in the configuration file.
 - Support the `force_node_voltage` command, but not to inherit the `force_node_voltage` results from a save-to-restore operation.
- **Image-based snapshot:** Captures the current state of the simulation. Use this snapshot type when you want to resume simulations where the snapshot was taken. This allows you to fast-forward to an interesting portion of a simulation or record

checkpoints in cases where the simulation runtime is long, thus maximizing the bandwidth utilization by seamlessly restarting the aborted simulation.

PrimeSim XA resumes simulation as though the run was completed by one single continuous job. You cannot modify the netlist or simulation setup in the restarted run. Other limitations in the compute environment may apply.

To reuse an image-based snapshot, you must specify the `-enable_isnapshot_feature` command line option together with a restore command line option, such as `-restore`, `-auto_restore` and so on.

This section provides information about how to capture a snapshot of the current state of the simulation and rerun the simulation with the saved snapshot, as described in the following topics:

- [Saving a Simulation](#)
- [Restoring a Saved Simulation](#)

Saving a Simulation

To capture the current state of the simulation, run the `set_save_state` command and specify the necessary options, such as the type of snapshot to save, the time or time period to record the checkpoints, the name of the saved snapshot, and so on.

For example:

```
set_save_state -period 25% -save_on_kill 1 -type image
```

This example saves an image-based snapshot periodically at every 25% completion interval when a UNIX `kill -15` command is run. The latest generated snapshot always overrides the previous one, so only one snapshot is kept.

You can specify multiple values for `-time`. Only one value is allowed for the other options.

You cannot specify multiple `-time`, `-period` and `-period_wall_time` options on a single command line or multiple `set_save_state` commands with different `-period` or `-period_wall_time` values. If multiple commands with multiple `-period` or `-period_wall_time` are specified, the last period or wall_time is applied.

For a detailed description about the command options, see the `set_save_state` command page in *PrimeSim XA Command Reference*.

Output Files

The `set_save_state` command generates two files, one with a `.time.ic` extension and one with a `.time.ic.sup` extension for each saved time you specify with the `-time` or `-period` option. You can then use the `-restore` or `-restart` command line option to start a simulation using the saved state.

For example, if you specify the following:

```
set_save_state -time 100n 1400n
```

The PrimeSim XA tool creates two files that contain the saved data: `XA/xa.1e-07.ic` and `XA/xa.1.4e-06.ic`.

To specify the name of the saved snapshot, use the `-file` option of the `set_save_state` command.

Restoring a Saved Simulation

To restore a simulation with a snapshot that is saved by the `set_save_state` command, use the following command line options:

- `-enable_isnapshot_feature`

Enables the image-based snapshot capturing feature. Use this command line option when you restore a simulation with an image-based snapshot.

- `-auto_restart` OR `-auto_restore`

Automatically restarts a simulation from the most recent snapshot if available. If no snapshot is available, PrimeSim XA proceeds the simulation from the first time point (time 0) with a warning message.

If the restarted run completes successfully, PrimeSim XA automatically deletes the snapshot being used or the one that is generated by the `set_save_state` command. The auto-deletion functionality applies only to the image-based application.

- `-restart_type op|image` OR `-restore_type op|image`

Specifies the type of saved snapshot to use with the `-auto_restart` or `-auto_restore` option.

- `OP` (Default): Uses a snapshot that contains operating point (OP) information.
- `image`: Uses an image-based snapshot.

By default, the tool checks for an OP-based snapshot to restart a simulation. When used with the `-enable_isnapshot_feature` option, the tool restarts a simulation based on an image-based snapshot. If no snapshot is available, PrimeSim XA proceeds the simulation from the first time point (time 0) with a warning message.

If no snapshot file is available, PrimeSim XA issues an error.

- `-restart saved_sim_file` OR `-restore saved_sim_file`

Restarts a saved simulation from a `saved_sim_file` that is generated by the `set_save_state` command.

For a detailed description about these command line options, see the [Using Command Line Options](#) section.

Example 1

The following example resumes a simulation from the specified image-based *snapshot_file*. If the file does not exist, PrimeSim XA issues an error message.

```
xa netlist.sp -enable_isnapshot_feature -restore snapshot_file  
-restart_type image
```

Example 2

In the following example, PrimeSim XA automatically resumes a simulation from the most recent image-based snapshot that it can find. If the restarted simulation completes successfully, PrimeSim XA removes the snapshot being used or the one that is generated by the `set_save_state` command. If a snapshot does not exist, PrimeSim XA proceeds normally from time 0 with a warning message.

```
xa netlist.sp -enable_isnapshot_feature -auto_restart -restart_type  
image
```

Example 3

The following example automatically resumes a simulation from the most recent OP-based snapshot that it can find. If an OP-based snapshot does not exist but an image-based one does, PrimeSim XA proceeds with the latter with a warning message. When the restored simulation completes successfully, PrimeSim XA removes the snapshot being used or the one that is generated by the `set_save_state` command.

```
xa netlist.sp -enable_isnapshot_feature -auto_restart
```

If neither snapshot exists, PrimeSim XA proceeds normally from time 0 with a warning message.

Example 4

When restoring a simulation with an OP-based snapshot, you can rerun the simulation with a new configuration file and specify the prefix for the output file. To do this, run the following command:

```
xa netlist.sp -restore XA/xa.1e-07.ic -c restore_cfg -o XA/xa_restore
```

If you use the same output prefix as for the saved run, the prefix is appended with the saved time:

```
xa netlist.sp -restore XA/xa.1e-07.ic -c restore_cfg -o XA/xa
```

The output files are named `XA/xa.1e-07.*`.

Output Files

The PrimeSim XA tool generates the following output files when simulation process is complete. Different analysis flows generates different types of log files. All output files are updated for each simulation. Use the `-ga` command line option to enable gzip file compression of the output files.

The following table lists the netlist file formats that are supported in the PrimeSim XA tool.

Table 6 Supported Netlist File Format and Command Line Options

File Type	Description
*.errt	Contains the results of the timing check commands, such as <code>check_node_excess_rf</code> , <code>check_node_quick_rf</code> , <code>check_timing_edge</code> , <code>check_timing_hold</code> , <code>check_timing_pulse_width</code> , and <code>check_timing_setup</code> .
*.errz	Contains the results of the <code>check_node_zstate</code> command.
*.hotspot	Contains the results of the <code>check_node_hotspot</code> command.
*.log	Contains the simulation status, including the statistics of the netlist, simulation runtime, and memory usage information at every stage, and warning and error messages.
*.meas	Contains the results of the <code>.MEASURE</code> statements. To gzip compress output results, use the <code>-gz</code> option.
*.mc, *.mc.csv, *.mc_params	Output files from Monte Carlo Analysis when you use the <code>set_monte_carlo_option</code> command. See the <i>PrimeSim XA Command Reference</i> for the details of this command and the output files it creates. To gzip compress output results, use the <code>-gz</code> option.
*.mt	Contains the results of the <code>.MEASURE</code> statements if you specify <code>set_meas_option -format hspice</code> . This format of this file is PrimeSim HSPICE-compatible. To gzip compress output results, use the <code>-gz</code> option.
*.power	Contains the results of the <code>report_power</code> command.
*.rcxt	Contains the result of the <code>set_active_net_flow</code> command or <code>-active_net_flow</code> command line option. It is an ASCII file containing the active net information.
*.time.ic	Contains the result of the <code>.op</code> statement or <code>set_active_net_flow</code> command. The time is the transient time at which the operating point is specified. To gzip compress output results, use the <code>-gz</code> option.

Table 6 Supported Netlist File Format and Command Line Options (Continued)

File Type	Description
*.time.ic.sup	Contains the result of the <code>set_save_state</code> command. The time is the transient time at which the simulation state is saved. This file is needed when restoring a saved simulation.

Output Waveform Files

When simulation is complete, the PrimeSim XA tool generates waveform files for examining simulation results in a graphical view. The tool supports multiple waveform file formats, such as FSDB, OUT, WDF, or PSF. Use the `set_waveform_option` command to control the behavior when generating the output waveform files, such as file size, file splitting, current or signal compression, and so on.

This section includes the following topics:

- [Supported Waveform File Formats](#)
- [Setting Waveform Options](#)

Supported Waveform File Formats

[Table 7](#) lists the output waveform files supported in the PrimeSim XA tool. To specify the output waveform format, use any of the following methods with a precedence order from high to low.

For more information about each of the output waveform formats, see the `set_waveform_option` command.

1. The `-wavefmt` command line option

```
xa ... -wavefmt wdf ...
```

2. The `set_waveform_option` command

You can specify the `set_waveform_option` command in the command script file or inside the netlist with the `xa_cmd` netlist option.

```
set_waveform_option -format wdf
```

or

```
.option xa_cmd="set_waveform_option -format wdf"
```

3. The native SPICE post netlist option

The PrimeSim XA tool supports the post native-SPICE netlist option to specify the waveform format in PrimeSim HSPICE and Eldo.

```
.option post=out
```

Table 7 Supported Waveform File Formats

File Type	Description
*.fsdb	A binary waveform file. This is the default waveform format generated by the PrimeSim XA tool. Use WaveView, Cosmos Scope, or nWave to display the contents of this file.
*.out	An ASCII waveform file. Use nWave to display the content of this file.
*.wdf	A binary waveform file. Use WaveView to display the content of this file.
*.psf	A binary waveform file. Use WaveView to display the content of this file.
*.tr0	A binary waveform file. Use WaveView or DVE to display the content of this file.

Setting Waveform Options

Use the `set_waveform_option` command to control how the waveform files are written when simulation is complete. You can specify output waveform options for waveform file format, file-split size, voltage resolution, current resolution, or the percentage to write the buffered simulation progress to the log file.

Following lists the features that the `set_waveform_option` command provides and the corresponding options. For a detailed description about each of the options, see the `set_waveform_option` command in *PrimeSim™ XA Command Reference*.

- Specifying Waveform File Format

Use the `-format` option to set the waveform output file format.

- Specifying Current Resolution

Use the `-compress_v` and `-compress_i` options to specify the resolution values for the slope-based lossy compression. Any data points that deviate from a straight line by less than this lossy compression value are dropped. The default voltage resolution is 1 uV, and the default current resolution is 1 pA.

Use the `-grid_v` and `-grid_i` options to specify the resolution values for rounding lossy compression. Any data points that deviate from resolution grid are rounded

off to the closest grid. The default voltage resolution is 1 uV, and the default current resolution is 1 pA.

- Writing Buffered Simulation Progress to the Output File

Use the `-flush` option to write the buffered data to an output file during simulation. The default setting is 10%, meaning that the tool writes the simulation progress to the output file every 10% of the simulation transient end time.

The following example updates the waveform file every 100 ns of simulation time.

```
set_waveform_option -flush 100ns
```

The following example updates the waveform file every 10% of the simulation time. If the transient end time is 2us, in this example the PrimeSim XA tool updates the waveform file after at least every 200 ns.

```
set_waveform_option -flush 10%
```

- Specifying Disk Full Messages

The `-disk_full` option alerts you that the output disk is nearing the capacity limit. If the disk becomes completely full, unpredictable behavior might occur. Use the `-disk_full` option to raise warnings when the disk is nearing capacity (that is, two times the specified value or default value) and to stop the tool when the disk is nearly full (that is, space is less than the specified value). This helps to preserve the integrity of output files, but their integrity cannot be guaranteed.

The default value of disk full is 100 MB. If the `-disk_full` is not explicitly set, the PrimeSim XA tool issues warnings when the disk has less than 200 MB of space, but it does not stop with an error. The following warning message is issued instead:

```
Warning: The available disk space 175MB is less than 200MB. Ctrl-C can
suspend a simulation. Use the set_waveform_option command to adjust
the disk full threshold.
```

When the `-disk_full` option is specified, the PrimeSim XA tool stops with a disk full error if the disk space is detected to be less than the specified threshold.

In the following example, the PrimeSim XA tool issues a warning when the disk space is less than two GB and stops when it is less than one GB.

```
set_waveform_option -disk_full 1000
```

- Splitting Waveform Files

The PrimeSim XA tool provides different methods to split waveform data into multiple waveform files, based on file size, signals in the file, and time (including probe window, transient time, and wall time). Any combination of waveform partitioning based on file size, signals and time is allowed. However, if you apply more than one way of

waveform partitioning based on time (such as, probe window, transient time or wall time), an error message is issued.

For example, if you specify the following commands in one simulation run:

```
set_waveform_option -tran_time_period 100  
set_probe_window -split -window 100n 200n 300n 400n
```

The tool issues the following error message:

Error: More than one time-based waveform partitioning commands are specified. Please use only one of the following commands and run simulation again:

```
set_waveform_option -tran_time_period 100  
set_probe_window -split -window 100n 200n 300n 400n
```

In PrimeSim XA, split waveform data using one of the following methods:

- Waveform splitting by file size

To split a waveform file when its size reaches a given value, use the following command:

```
set_waveform_option -size max_file_size_MB
```

- Waveform splitting by signals

To split a waveform file into multiple files using parallel dumping, use the following command:

```
set_waveform_option -parallel_dumping 0|1|n
```

- Waveform splitting by probe window

To split a waveform file into multiple files by the specified probe window, use the following command:

```
set_probe_window -split window_spec
```

- Waveform splitting by transient time

- To write waveform data into multiple files based on the specified period of transient time and save the simulation state, use the following command:

```
set_save_state -tran_time_period -dump_waveform 1
```

- To write waveform data into multiple files without saving the simulation state, use the following command:

```
set_waveform_option -tran_time_period time
```

If you run both `set_save_state` and `set_waveform_option` commands and no other time-based waveform partitioning command in one simulation run, the PrimeSim XA tool honors the first command with the following warning message:

```
Warning: Waveform dumping in the command set_waveform_option
-tran_time_period is ignored. PrimeSim XA will only dump waveforms
specified in the command set_save_state
```

- Waveform splitting by wall time

- To write waveform data into multiple files based on specified period of wall time and save the simulation state, use the following command:

```
set_save_state -wall_time_period period_in_hour -dump_waveform 1
```

- To write waveform data into multiple files without saving simulation state, use the following command:

```
set_waveform_option -wall_time_period period_in_hour
```

If you run both `set_save_state` and `set_waveform_option` commands and no other time-based waveform partitioning command in one simulation run, the PrimeSim XA tool honors the first command with the following warning message:

```
Warning: Waveform dumping in the command set_waveform_option
-wall_time_period is ignored. PrimeSim XA will only dump waveforms
specified in the set_save_state command.
```

- Generating a Virtual FSDB File or Group File

Use the `-group 1` option to generate a virtual FSDB or group file which includes all the waveform files that are generated during waveform splitting. The PrimeSim XA tool generates a virtual file with a `.vf.fsdb` extension for waveform files in FSDB format. For WDF format, a group file is generated with a `.grp` extension.

Example 1

The following example partitions waveforms based on the maximum file size of 500 MB and wall time period of 24 hours:

```
set_waveform_option -size 500 -wall_time_period 24 -group 1
```

When simulation is complete, the following files are created:

- Waveform files: *prefix*.wt1d.fsdb, *prefix*.fsdb.2, *prefix*.wt2d.fsdb ...
- Virtual file: *prefix*.vf.fsdb

Example 2

The following example partitions waveforms based on the period of transient time of 4us using distributed processing with two cores:

```
set_waveform_option -parallel_dump 2 -tran_time_period 4u
```

When simulation is complete, the following files are created:

- Waveform files: *prefix.4e-06.mw1.fsdb*, *prefix.4e-6.mw2.fsdb*, *prefix.8e-06.mw1.fsdb*, ...
- Virtual file: *prefix.vf.fsdb*

3

PrimeSim HSPICE Netlist Compatibility

The PrimeSim XA tool is fully compatible with the PrimeSim HSPICE syntax, elements, device models, simulator commands, and netlist options; however, some exhibit different default behavior in the PrimeSim XA tool. Several additions and enhancements have been implemented in the PrimeSim XA tool to support many native PrimeSim HSPICE features, options, models, and netlist elements.

This chapter details the enhancements, limitations, and variations compared to the default PrimeSim HSPICE syntax, behavior, and options, as described in the following topics:

- [Running PrimeSim XA With the PrimeSim HSPICE Netlist Format](#)
- [Syntax and Behavior Variations](#)
- [PrimeSim HSPICE-Compatible Netlist Commands](#)
- [PrimeSim HSPICE-Compatible Control Options](#)
- [Built-In Functions](#)
- [Elements](#)
- [Device Models](#)
- [Transient Analysis](#)
- [Violation Checks Using HSPICE .BIASCHK Statements](#)
- [Bisection Methodology and Behavior Variations](#)
- [PrimeSim HSPICE-Encrypted Netlist Format](#)
- [Post-Layout Simulation Galaxy Parasitic Database \(GPD\)](#)

Running PrimeSim XA With the PrimeSim HSPICE Netlist Format

You run the PrimeSim XA tool with the following command at the UNIX command prompt:

```
$> xa [-hspice] netlist_file [command_line_options]
```

See the [Running the Simulator](#) section for details about the other command line options.

Syntax and Behavior Variations

The PrimeSim XA tool is fully compatible with PrimeSim HSPICE syntax and behavior with some variations. This section describes some important syntax and behavior differences.

Using "Curly" Brackets

The PrimeSim HSPICE tool converts curly brackets ({}) to square brackets ([]). The PrimeSim XA tool treats curly brackets and square brackets as separate characters. In the following example, the PrimeSim HSPICE tool connects V1 and R1 together by converting {} to []. The PrimeSim XA tool treats word[1] and word{1} as two separate nodes.

```
V1 word[1] 0 dc=1  
R1 word{1} 0 r=100
```

Specifying Equations or Expressions

An expression is an equation consisting of a number, parameter, waveform, or built-in function, contained within matching single or double quotes. The quotes may be omitted if the expression starts with an alphabetic character and does not contain white space. It is highly recommended to contain the expression in quotes if the expression contains white space to avoid ambiguity.

The following example shows two valid expressions:

```
.param expr1 = "3 + abs(a)"  
.param expr2 = 2+pow(a,b)
```

PrimeSim HSPICE-Compatible Netlist Commands

Note:

PrimeSim HSPICE-compatible netlist commands are documented in the PrimeSim XA Simulation Command Reference chapter of the *PrimeSim™ Continuum Reference Manual: Commands and Control Options*.

The PrimeSim XA tool interprets the first word of every line that begins with a period character (.) as a statement. Unrecognized netlist statements are ignored, and a warning is issued. The PrimeSim XA tool supports the following HSPICE netlist statements:

- .APPENDMODEL
- .ALTER
- .CFL_PROTOTYPE
- .CONNECT
- .DATAENDDATA
- .DEL LIB
- .DOUT
- .ELSE
- .ELSEIF
- .END
- .ENDDATA
- .ENDIF
- .ENDS
- .ENDL
- .EOM
- .GPD_INC
- .GLOBAL
- .IC
- .IF
- .INCLUDE / INC / INCL
- .INSTVALID
- .IVTH
- .LIB
- .MACRO
- .MALIAS

Chapter 3: PrimeSim HSPICE Netlist Compatibility

PrimeSim HSPICE-Compatible Netlist Commands

- .MEASURE / MEAS
- .MEASURE (AVG, EM_AVG, INTEG, MIN, MAX, PP, and RMS)
- .MEASURE (Continuous Results)
- .MEASURE (Derivative Function)
- .MEASURE (Equation Evaluation/Arithmetic Expression)
- .MEASURE (FIND and WHEN)
- .MEASURE (Integral Function)
- .MEASURE (Rise, Fall, Delay, and Power Measurements)
- .MODULEENDMODULE
- .MODEL
- .MOSRA
- .NODESET
- .OP
- .PAT
- .OPTION / OPTIONS
- .PARAM / PARAMETER / PARAMETERS
- .PRINT
- .PROBE
- .PROTECT / PROT
- .SAVE
- .SUBCKT
- .TEMP / TEMPERATURE
- .TITLE
- .TRAN / TR
- .UNPROTECT / UNPROT
- .VEC

[Table 8](#) lists the supported and enhanced PrimeSim HSPICE simulation statements in the PrimeSim XA tool but have different syntaxes or behaviors with the PrimeSim HSPICE tool.

Table 8 *Supported PrimeSim HSPICE Netlist Statements With Different Behaviors in PrimeSim XA*

Statement	Description
<code>.ALTER</code>	Both tools process the <code>.ALTER</code> statements to re-simulate a single input netlist using different parameter values, models, library components, temperatures, and so on. The differences come from the output files generated. In the PrimeSim XA tool, output files are suffixed with the <code>.a#</code>, where # is an incremental integer starting from 0, representing the simulation iteration count when using the <code>.ALTER</code> statement. For example - The PrimeSim XA tool generates <code>xa.a0.fsdb</code>, <code>xa.a1.fsdb</code>, ... - The PrimeSim HSPICE tool generates <code>hspice.tr0</code>, <code>hspice.tr1</code>, ...
<code>.BIASCHK</code>	Both the PrimeSim XA and PrimeSim HSPICE tools process the <code>.BIASCHK</code> command to monitor voltage bias, current, and expression during transient analysis. Not all features from the PrimeSim HSPICE tool are supported, and the PrimeSim XA tool supports additional features for the FastSPICE application. See the Violation Checks Using HSPICE .BIASCHK Statements section.
<code>.PRINT</code>	The PrimeSim XA tool processes all the <code>.PRINT</code> statements in the netlist as if they were the equivalent <code>.PROBE</code> statements. If you want the actual text output as in the PrimeSim HSPICE tool, use the <code>enable_print_statement</code> command.
<code>.PROBE</code>	Both tools output the specified signals to the specified waveform format. Enhancements have been made in the PrimeSim XA tool to support more functionality. See the Using the .PROBE or .PLOT Statements section for details.

PrimeSim HSPICE-Compatible Control Options

Note:

PrimeSim HSPICE-compatible control options are documented in the PrimeSim XA Simulation Control Options Reference chapter of the *PrimeSim™ Continuum Reference Manual: Commands and Control Options*.

The PrimeSim XA tool supports the following PrimeSim HSPICE Compatible Control Options:

- .OPTION AUTOSTOP

(The alias AUTOST is not supported in PrimeSim XA)

- .OPTION ASPEC
- .OPTION BSIM4PDS
- .OPTION CAPTAB
- .OPTION CFLFLAG
- .OPTION CMIFLAG
- .OPTION CMIUSRFLAG
- .OPTION CMIPATH
- .OPTION CSHUNT
- .OPTION DCAP
- .OPTION DEFAD
- .OPTION DEFAS
- .OPTION DEFL
- .OPTION DEFNRD
- .OPTION DEFNRS
- .OPTION DEFPD
- .OPTION DEFPS
- .OPTION DEFW
- .OPTION DUMPCFL
- .OPTION DP_DISKFREE
- .OPTION DP_MEMFREE
- .OPTION DP_LOADAVG
- .OPTION DP_MASTER_WAITTIME
- .OPTION DP_TASK_MAXATTEMPT
- .OPTION ETMIUSRINPUT

Chapter 3: PrimeSim HSPICE Netlist Compatibility

PrimeSim HSPICE-Compatible Control Options

- `.OPTION EXPLI`
 - `.OPTION GENK`
 - `.OPTION GEN_CUR_POL`
 - `.OPTION GEOSHRINK`
 - `.OPTION GMAX`
 - `.OPTION GMIN`
 - `.OPTION GMINDC`
 - `.OPTION IGNORE_MALIAS`
 - `.OPTION IVTH`
 - `.OPTION IVTH_MODEL`
 - `.OPTION KLIM`
 - `.OPTION MACMOD`
 - `.OPTION MEASDGT`
 - `.OPTION MEASFAIL`
 - `.OPTION MEASFILE`
 - `.OPTION MEASFORM`
 - `.OPTION MIXED_NUM_FORMAT`
 - `.OPTION MODMONTE`
 - `.OPTION MONTECON`
 - `.OPTION MODPARCHK`
 - `.OPTION OPTS`
 - `.OPTION NUMDGT`
 - `.OPTION PARHIER`
- (The alias `PARHIE` is not supported in PrimeSim XA)
- `.OPTION PROBE`
 - `.OPTION POST`
 - `.OPTION POST_VERSION`

Chapter 3: PrimeSim HSPICE Netlist Compatibility

PrimeSim HSPICE-Compatible Control Options

- .OPTION PUTMEAS
- .OPTION S_RATIONAL_FUNC
- .OPTION SAMPLING_METHOD
- .OPTION SEARCH
- .OPTION SCALE
- .OPTION SEED
- .OPTION SCALM
- .OPTION SLOPETOL
- .OPTION SHRINK
- .OPTION SX_FACTOR
- .OPTION SPMODEL
- .OPTION TMIAGE
- .OPTION TMIINPUT
- .OPTION TMIFLAG
- .OPTION TMISAVE
- .OPTION VAMODEL
- .OPTION TNOM
- .OPTION WACC
- .OPTION WARNLIMIT
- .OPTION WL
- .OPTION XMULT_IN_EXP / M_IN_EXP
- .OPTION XDTEMP

Table 9 lists the supported PrimeSim HSPICE simulation control options that exhibit different syntaxes or behaviors with the PrimeSim HSPICE tool.

Table 9 Supported PrimeSim HSPICE Control Options With Different Behaviors

Simulation Control Option	Description
<code>cshunt=value</code>	The default value for the PrimeSim XA tool is 1e-20 farads, which differs from the default value of 0 in the PrimeSim HSPICE tool.
<code>gmin=value</code>	The PrimeSim XA tool uses this option only during the DC operating point stage to get initial conditions. In the PrimeSim HSPICE tool this option affects the transient simulation.
<code>measfail=1 2</code>	The PrimeSim XA tool supports only 1 and 2, while the PrimeSim HSPICE tool supports 0, 1 and 2.
<code>measform=0 1 2 3</code>	The PrimeSim XA tool supports: • 0, which specifies the PrimeSim HSPICE *.mt file format • 1, which specifies the PrimeSim HSPICE*.mt file format with all values in one single line • 2, which specifies the PrimeSim XA *.meas file format • 3, which specifies the *.mt.csv Comma Separated Value format See the <code>set_meas_option</code> command for information about how to control the output format of the measurement file.
<code>post [=token]</code>	The PrimeSim HSPICE and PrimeSim XA tools support different waveform formats. See the Supported Waveform File Formats section for details about the supported waveform formats in PrimeSim XA. When not specified in PrimeSim HSPICE, the default is 0. When not specified in PrimeSim XA, the default is <code>FSDb</code> .
<code>probe=0 1</code>	The default setting of the PrimeSim XA tool is 1 while the default setting of the PrimeSim HSPICE tool is 0.
<code>seed=value</code>	The random numbers generated in the PrimeSim HSPICE and PrimeSim XA tools are different. The <code>random</code> keyword is not supported in the PrimeSim XA tool.

Built-In Functions

An expression is an equation consisting of a number, parameter, waveform, built-in functions, or arithmetic operations (+, -, *, /), contained within matching single or double quotes. The quotes may be omitted if the expression starts with an alphabetic character and does not contain white space. [Table 10](#) lists the PrimeSim HSPICE built-in functions that you cannot redefine. See the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis* for more information.

Table 10 Supported PrimeSim HSPICE Built-In Functions

ABS()	ACOS()
AGAUSS()	ASIN()
ATAN()	AUNIF()
CEIL()	COS()
DB()	COSH()
EXP()	FMOD
GAUSS()	FLOOR()
INT()	LN()
LOG()	LOG10()
LIMIT()	MAX()
NINT()	MIN()
POW()	PWR()
ROUND()	SGN()
SIN()	SIGN()
SINH()	TAN()
SORT()	TANH()
UNIF()	

Elements

The syntax for each element is compatible with PrimeSim HSPICE. For details about each element, see related topics in *PrimeSim™ HSPICE® User Guide: Basic Simulation and Analysis*.

[Table 11](#) lists the PrimeSim HSPICE elements that are supported in the PrimeSim XA tool.

Table 11 *Supported PrimeSim HSPICE Elements*

Instantiation Character	Element
C	Capacitor
D	Diode
E	Voltage-Controlled Voltage Source (VCVS)
F	Current-Controlled Current Source (CCCS)
G	Voltage-Controlled Current Source (VCCS)
H	Current-Controlled Voltage Source (CCVS)
I	Current Source
J	JFET
K	Mutual Inductor
L	Linear Inductor
M	MOSFET
Q	BJT
R	Resistor
S	S-Parameter (see Simulating With S-Parameter)
T	Ideal Transmission Line
V	Voltage Source
W	Transmission Line
X	Subcircuit Instance

Simulating With S-Parameter

The PrimeSim XA tool supports S-parameter that is compatible with the PrimeSim HSPICE tool. It is recommended to use at least `set_sim_level -level 5` for circuit containing S-parameter models.

By default, the PrimeSim XA tool uses `S_RATIONAL_FUNC=1`:

```
.OPTION S_RATIONAL_FUNC=0|1|2
```

The model description is:

```
.model name S ... RATIONAL_FUNC=0|1|2
```

The instance parameter is:

```
S_RATIONAL_FUNC=0|1|2
```

The precedence order is that the instance statement overwrites the model and option statements. The model statement overwrites the `.OPTION` statement.

Device Models

The following tables list the model APIs and device models that are supported in the PrimeSim XA tool. These APIs are fully compatible with the PrimeSim HSPICE tool. For a detailed description on these models, see the related topics in *PrimeSim™ Continuum Reference Manual: MOSFET Models* and *PrimeSim™ Continuum Reference Manual: Device Models*.

These APIs and device models are:

- Custom Common Model Interface (CMI)
- MOSFET Reliability Analysis (MOSRA)
- TSMC Model Interface (TMI)
- Si2 Compact Model Coalition Model Interface - Open Model Interface (OMI)

Table 12 *Supported HSPICE Diode Models*

Level	Diode Model
1	Non-geometric Junction Diode
2	Fowler-Nordheim Diode
3	Geometric Junction Diode
4	JUNCAP Model
5	Diode Level 500 Model
6	JUNCAP2 Model, versions 200.0, 200.1, 200.2, 200.3, 200.3.3, 200.5, 200.6, and 200.62; JUNCAP2 Express Model
7	DIODE_CMC Model, Version 2

[Table 13](#) lists the supported HSPICE BJT models in the PrimeSim XA tool. Refer to the BJT Models section in *PrimeSim Continuum Reference Manual: Device Models* for details.

Table 13 Supported HSPICE BJT Models

Level	BJT Model
1	Gummel-Poon Model
2	Quasi-saturation Model
4	VBIC95 Model, version 1.1.5
6	MEXTRAM Model, versions 504.6, 504.6.1, 504.7, 504.8, 504.10, 504.11, 504.12, 504.13, 505, 505.1, 505.2, 505.3, and 505.4
8	HICUM L2 Model, versions 2.21, 2.22, 2.23, 2.24, 2.3, 2.31, 2.32, 2.33, 2.34, 2.4, and 3.0
9	VBIC99 Model, Version 1.2
13	HICUM L0 Model, versions 1.11, 1.12, 1.2, 1.3, 1.31, 1.32, 2.0.0, and 2.1.0

[Table 14](#) lists the supported HSPICE JFET models in the PrimeSim XA tool. For more information about JFET models, see the JFET and MESFET Models section in *PrimeSim Continuum Reference Manual: Device Models*.

Table 14 Supported HSPICE JFET Models

Level	JFET Model
1	JFET Level 1 Model

[Table 15](#) lists the PrimeSim XA-supported HSPICE MOSFET models. Refer to the *PrimeSim™ Continuum Reference Manual: MOSFET Models* for details.

Table 15 Supported HSPICE MOSFET Models

Level	MOSFET Model
1	MOS Level 1
2	MOS Level 2
3	MOS Level 3
49	HSPICE BSIM3v3 Model, version 3.3

Table 15 *Supported HSPICE MOSFET Models (Continued)*

Level	MOSFET Model
50	MOS Model 9, Level 903
53	UC Berkeley BSIM3v3 Model, version 3.3
54	BSIM4 Model, versions 4.6.0, 4.6.1, 4.6.2, 4.6.2, 4.6.3, 4.6.4, 4.6.5, 4.6.6, 4.7.0, 4.8.0, 4.8.1 and 4.82
55	EKV Model, version 2.6
57	BSIM3 SOI Model, version 3.2.0
62	TFT Polysilicon Model
63	MOS Model 11, Level 1102
66	HVMOS (Synopsys® Level 66) 1.4, 1.5, 2.0, 2.1
68	HiSIM2 Model, versions 2.4.0, 2.4.1, 2.4.2, 2.4.3, 2.5.0, 2.5.1, 2.6.0, 2.6.1, 2.7.0, 2.8.0, 2.9.0, 2.9.1, 3.0.0, 3.1.0, 3.1.1, and 3.2.0
69	PSP Model, versions 102.0, 102.1, 102.2, 102.2.1, 102.3, 102.3.2, 102.3.3, 102.3.4, 103.0, 103.1, 103.1.1, 103.1.2, 103.2, 103.3, 103.4, 103.5, 103.6, 103.7, 103.8, 103.8.1, 103.8.2, and 104.0
70	BSIMSOI4 Model, versions 4.0, 4.1, 4.2, 4.3, 4.3.1, 4.4, 4.5, 4.6, 4.6.1, 100.0.1, and 100.1
72	BSIM-CMG Model, versions 105, 105.03, 105.031, 105.04, 106, 106.1, 107, 108, 109, 110, 111, 111.10, 111.2.0, and 111.2.1
73	HiSIM_HV Model, versions 1.0.2, 1.1.0, 1.1.1, 1.2.0, 1.2.1, 1.2.2, 1.2.3, 1.2.4, 2.0.0, 2.0.1, 2.1.0, 2.2.0, 2.3.0, 2.3.1, 2.3.2, 2.3.3, 2.3.4, 2.4.0, 2.4.1, 2.4.2, 2.4.3, 2.5.0, and 2.5.1
74	MOS Model 20, Level 2002
76	UTSOI Model, versions 1.1.4, 2.0.0, 2.1.0, 2.1.1, 2.2.0, 2.3.0, 102.4, 102.6, and 102.7
77	BSIM6 Model versions 6.0, 6.1, and 6.1.1; BSIM-BULK Model versions 106.2.0, 107 and 107.10
78	BSIM-IMG Model, versions 102.6.1, 102.7, 102.8, 102.9, 102.9.1, 102.9.2, 102.9.3, 102.9.4, 102.9.6 (include aging defect screening model), and 103

Table 15 *Supported HSPICE MOSFET Models (Continued)*

Level	MOSFET Model
79	EKV Model, version 302
80	HiSIMSOI 1.4 and 1.5
81	ASMHEMT 101.0, 101.1, 101.2, 101.3, and 101.4
82	MVSG_CMC 1.1.0 and 3.2.0
83	HiSIMHMG 2.0
84	HiSIM_SOTB 1.2
85	ASM ESD 101.0.0

[Table 16](#) lists the supported HSPICE capacitor models in the PrimeSim XA tool.

Table 16 *Supported Capacitor Models*

Level	Capacitor Model
7	CMC MOS Varactor Model 1.0, 1.1, 1.2, 1.3

[Table 17](#) lists the supported HSPICE resistor models in the PrimeSim XA tool.

Table 17 *Supported HSPICE Resistor Models*

Level	Resistor Model
2	CMC R2 Model, versions 1.0.0 and 1.0.1
5	CMC R3 Model, versions 1.0.0, 1.1.0, 1.1.1, and 1.1.2

[Table 18](#) lists the supported HSPICE MTJ models in the in the PrimeSim XA tool.

Table 18 *Supported HSPICE MTJ Models*

Level	MTJ Model
1	BSIM-MTJ-BDMC 1.0

Transient Analysis

The PrimeSim XA tool supports the following transient analyses:

- [Single Point Analysis](#)
- [Single Parameter Sweep Analysis](#)
- [Multiple Parameter Sweep Analysis](#)
- [Using a Parameter Step in Multiple Parameter Sweep Analysis](#)
- [Time-Window-Based Analysis](#)
- [Monte Carlo Analysis in PrimeSim XA](#)

For a detailed description about transient analysis, see the related chapters in *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

Single Point Analysis

The PrimeSim XA tool supports single point transient analysis. The syntax is:

```
.TRAN tstep tstop [UIC]
```

Argument	Description
<i>tstep</i>	This argument is used when the <code>.PRINT</code> statement and <code>enable_print_statement</code> command are specified to determine the time point printed in an ASCII output. It is not used to determine time step of the transient simulation.
<i>tstop</i>	Specifies the time when a transient analysis stops.

Argument	Description
<code>UIC</code>	A keyword that specifies not to compute the initial operating point during DC, but directly enters transient analysis. When you use <code>UIC</code> in the <code>.TRAN</code> statement, the node values (at time=0) are determined by searching for the first value found in the following order: 1.A value specified by an <code>.IC</code> statement applied to the node 2.A device IC instance parameter on an element (capacitors and inductors only) 3.A value specified by a <code>.NODESET</code> statement applied to a node 4.If none of the above is set, the tool uses a voltage of zero (0) Note that forcing a node value of the DC operating point with the <code>.IC</code> statements might not satisfy KVL and KCL rules. Forcing a node voltage applies a fixed equivalent voltage source during DC analysis; and transient analysis removes the voltage sources to calculate the second and later time points. Therefore, to correct DC convergence problems, use the <code>.NODESET</code> statements (without <code>UIC</code> in the <code>.TRAN</code> statement) liberally when a good guess can be provided, and use the <code>.IC</code> statements sparingly when the exact node voltage is known.

Single Parameter Sweep Analysis

The PrimeSim XA tool supports sweeping a single parameter for transient analysis. When you express the values of a parameter by using decade, octave, linear, or point-of-interest variation, you can use the `SWEEP` keyword to enable sweep analysis.

```
.TRAN tstep tstop [UIC] [SWEEP var type np pstart pstop]
```

The following syntax is an alternative for linear sweeps:

```
.TRAN tstep tstop [UIC] [SWEEP var pstart pstop incr]
```

Argument	Description
<code>SWEEP</code>	Enables sweep analysis.
<code>var</code>	Specifies the parameter for sweep analysis. You can use the <code>TEMP</code> keyword for temperature sweeping.
<code>type</code>	Specifies any of the following keywords: • <code>DEC</code> specifies a decade variation • <code>OCT</code> specifies octave variation (the value of the designated variable is eight times its previous value) • <code>LIN</code> specifies linear variation • <code>POI</code> specifies a list of points

Argument	Description
<i>np</i>	Specifies the number of points for <code>LIN</code> or <code>POI</code> , or the number of points per <code>DEC</code> or <code>OCT</code> .
<i>pstart</i>	Specifies the starting point of a parameter value.
<i>psstop</i>	Specifies the stopping point of a parameter value.
<i>incr</i>	Specifies the incremental value of a parameter sweeping.

Multiple Parameter Sweep Analysis

The PrimeSim XA tool supports sweeping multiple parameters for data-driven transient analysis. The data-driven analysis lets you modify any number of parameters and use the new parameter values to perform multiple transient simulation iterations using a single netlist.

You can specify an array of parameter values in the netlist using the `.DATA` statement that ends with the `.ENDDATA` statement.

Note:

The PrimeSim XA tool only supports the inline form in which the parameter must be specified in the netlist. The external file form is not supported.

You enable multiple parameters sweep analysis with the following syntax:

```
.TRAN tstep tstop [UIC] [SWEEP DATA=dataname]
```

Argument	Description
<i>tstep</i>	This argument is used when the <code>.PRINT</code> statement and <code>enable_print_statement</code> command are specified to determine the time point printed in an ASCII output. It is not used to determine time step of the sweep simulation.
<i>tstop</i>	Specifies the time when a sweep analysis stops.

Argument	Description
UIC	A keyword that specifies not to compute the initial operating point during DC, but directly enters sweep analysis. When you use <code>UIC</code> in the <code>.TRAN</code> statement, the node values (at time=0) are determined by searching for the first value found in the following order: 1.A value specified by an <code>.IC</code> statement applied to the node 2.A device IC instance parameter on an element (capacitors and inductors only) 3.A value specified by a <code>.NODESET</code> statement applied to a node 4.If none of the above is set, it uses a voltage of zero (0) Note that forcing a node value of the DC operating point with the <code>.IC</code> statements might not satisfy KVL and KCL rules. Forcing a node voltage applies a fixed equivalent voltage source during DC analysis; and transient analysis removes the voltage sources to calculate the second and later time points. Therefore, to correct DC convergence problems, use the <code>.NODESET</code> statements (without <code>UIC</code> in the <code>.TRAN</code> statement) liberally when a good guess can be provided, and use the <code>.IC</code> statements sparingly when the exact node voltage is known.
DATA=dataname	Specifies the data name, referenced in the <code>.TRAN</code> statement from a <code>.DATA</code> statement.

Output analysis files are suffixed with the `.s#` extension, where # is an incremental integer, starting from 0, representing the simulation iteration count when using the `.DATA` statement. The PrimeSim HSPICE tool differs by combining the analysis results of all iterations into one single file. The syntax for specifying an array is:

```
.DA[T[A]] dataname pname1[ ... pnameN]
pval1 [ ... pvalN]
...
pvalM [... pvalMN]
.ENDD[A[T[A]]] [dataname]
```

The following example defines an array named `data1` with two parameters, `v_val` and `td_val`, for multiple parameter sweep analysis.

```
.PARAM v_val=3.1v td_val=0ns
.DATA data1 v_val td_val

3.1v 0ns
3.1v 1ns
2.5v 2ns
1.1v 3ns
.ENDDATA data1

vDATA data 0 PULSE(0 v_val td_val 0.1ns 0.1ns 10ns 20ns)
rDATA data 0 10
```

```
.OPT POST PROBE
.PROBE v(data)
.TRAN 0.1ns 80ns SWEEP DATA=data1
.END
```

The following output files, suffixed with `.s#`, are generated.

- `xa.log`
- `xa.s0.fsdb xa.s0.meas`
- `xa.s1.fsdb xa.s1.meas`
- `xa.s2.fsdb xa.s2.meas`
- `xa.s3.fsdb xa.s3.meas`

Using a Parameter Step in Multiple Parameter Sweep Analysis

The PrimeSim XA tool supports sweeping multiple parameters within a data block in steps from a given start to stop range. You use the `.DATAVAR` block to define the parameter sweeping step size. The syntax is:

```
.DATAVAR datavarname N|L|T=k
.setparam [INST=instance_name|SUBCKT=subckt_name] \
    param_name1 = (start, stop) param_name2 = (start, stop)...
.ENDDATAVAR datavarname
```

Note:

You can abbreviate `.DATAVAR` and `.ENDDATAVAR` with `.DAV` and `.ENDDAV`.

Argument	Description
<i>datavarname</i>	Defines the name of the <code>.DATAVAR</code> block, which can be one of the following values: • <code>N n</code> specifies a linear (equidistant) mode. • <code>L l</code> specifies a logarithmic (constant ratio between two consecutive values) mode. • <code>T t</code> specifies a tabular mode. • <code>k</code> specifies the number of steps. This must be an integer value.
<code>.setparam</code>	Specifies the parameter definitions. You can use the <code>INST</code> and <code>SUBCKT</code> keywords to apply the parameter definitions to instances and subcircuits, respectively. Otherwise the parameter definitions apply globally.

Parameter Step Examples

The following example has three variations.

```
vDATA data 0 PULSE (0 v_val td_val 0.1ns 0.1ns 10ns 20ns)
.DATAVAR data1 N=3
.setparam v_val = (1.1v, 3.1v)
.setparam td_val = (0n, 2n)
.ENDDATAVAR data1
.tran 1n 100p sweep datavar=(data1)
```

Mult	v_val	td_val
1	1.1v	0u
2	2.1v	1u
3	3.1v	2u

In the following example, the width of the M0 MOS in x1 instance is swept and takes two values: 1u and 3u.

```
.DATAVAR mos_width T=2
.setparam inst=X1.M0 W = (1u, 3u).ENDDATAVAR mos_width
.tran 1n 100p sweep datavar=(mos_width)
```

The following example has six variations.

```
.DATAVAR Mult N=6
.setparam inst=X2.X1.M0 W = (2u, 3u)
.setparam inst=X3.R0 r = (1k, 2k)
.setparam inst=X3.C0 c = (0.5n, 1.0n)
.ENDDATAVAR Mult
.tran 1n 100p sweep datavar=(Mult)
```

Mult	W	r value
1	2u	1k
2	2.2u	1.2k
3	2.4u	1.4k
4	2.6u	1.6k
5	2.8u	1.8k
6	3u	2k

In the following example, the voltage source is equivalent to:

```
Vname n1 n2 PWL (0, 0, 1e-6, 0.6, 2e-6, 1.2)
Vname n1 n2 PWL (T, V)
.datavar data1 N = 3
.setparam T = (0, 2e-6) V = (0, 1.2)
.enddatavar data1
.tran 1n 100p sweep datavar=(data1)
```

In the following example, the `rpoly_rtol` subcircuit parameter is swept and take two values: -0.2 and +0.2.

```
.subckt rpoly ( p n ) value = val rpoly_rtol = 0 \
    rmain p n ' value * ( 1 + rpoly_rtol ) '
.ends
.datavar rtol T=2
.setparam subckt=rpoly rpoly_rtol = ( -0.2, +0.2 )
.enddatavar rtol
.tran 1n 100p sweep datavar=(rtol)
```

In the following example, the four variations of the resistor value are: 1, 10, 100, and 1000.

```
.DATAVAR Mult L=4
.setparam inst=X3.R0 r = (1, 1000)
.ENDDATAVAR Mult
.tran 1n 100p sweep datavar=(Mult)
```

The mathematical expression to calculate the constant ratio for the logarithmic step is:

$$\text{Constant Ratio} = \sqrt[L-1]{\frac{\text{stop}}{\text{start}}} = \sqrt[4-1]{\frac{1000}{1}} = 10$$

The following example shows a duplicate parameter defined in the same `.datavar` block.

```
.datavar var1 T=2
.setparam p1=(1, 2) p2=(1, 2)
.setparam p1=(5, 6) p3=( 1, 2)
.setparam ins=x1.x2 r1=(1, 2) r2=(1, 2)
.setparam ins=x1.x3 r2=(1, 2) r2=(1, 2)
.setparam ins=x1.x2 r1=(5, 6) r3=(1, 2)
.enddatavar
.tran 1n 100p sweep datavar=(var1)
```

After checking for duplicate parameters, here is the equivalent `.datavar` block.

```
.datavar var1 T=2 .setparam p1=(5, 6) p2=(1, 2) p3=(1, 2)
.setparam ins=x1.x2 r1=(5, 6) r2=(1, 2) r3=(1, 2)
.setparam ins=x1.x3 r2=(1, 2) r2 =(1, 2)
.enddatavar
.tran 1n 100p sweep datavar=(var1)
```

The following example shows the duplicate parameter defined in two `.datavar` blocks.

```
.datavar var1 T = 2
.setparam p1=(0, 1)
.setparam ins=x1.x2 r1=(0, 1)
.setparam subckt=sub1 q1=(0, 1)
.enddatavar

.datavar var2 T = 2
.setparam p1=(0, 1) p2=(0, 1)
.setparam ins=x1.x2 r1=(0, 1) r2=(0, 1)
.setparam ins=x1.x3 r2=(0, 1) r3=(0, 1)
.setparam subckt=sub1 q1=(0, 1) q2=(0, 1)
.setparam subckt=sub2 q1=(0, 1) q2=(0, 1)
.enddatavar

.tran 1n 100p sweep datavar=(var1, var2)
```

var1:

top	x1.x2	sub1
p1	r1	q1
0	0	0
1	1	1

var2:

top		x1.x2		x1.x3		sub1		sub2	
p1	p2	r1	r2	r2	r3	q1	q2	q1	q2
0	9	0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1	1	1

var1 * var2:

In the following table, the values in bold are chosen after checking for duplicates.

top	x1.x2	x1.x3	sub1	sub2
p1	p1 p2	r1 r1 r2	q1 q2	q1 q1 q2
0	0 0	0 0 0	0 0	0 0
0	1 1	0 1 1	1 1	1 1
1	0 0	1 0 0	0 0	0 0

top	x1.x2			x1.x3			sub1		sub2	
1	1	1	1	1	1	1	1	1	1	1

Specifying .DATAVAR Permutations

You can specify permutations of the .DATAVAR blocks with the following syntax.

```
.TRAN tstep tstop [sweep DATAVAR=(datavarnamel,... datavarnameN)]
```

This syntax is used to rerun the simulation for all the possible combination of parameters in one or more .DATAVAR blocks. The sweep syntax can refer to both the .DATAVAR label and existing .DATA labels.

In the following example, the first `data1` block changes the `X0.R1` resistor value from 0.5k to 1.5k ohms in 10 steps ($n = 10$). For each `X0.R1` value, the `data2` block changes the `X0.C1` capacitance value from 0.4nF to 0.8nF in 4 steps ($n=4$).

```
.datavar data1 n=10
.setparam inst=X0.R1 r= (0.5k, 1.5k)
.enddav data1

.datavar data2 n=4
.setparam inst=X0.C1 c= (0.4n, 0.8n)
.enddav data2

.tran 1n 100p sweep datavar=(data1, data2)
```

Output Files

A `.s#` extension is added for each combination of the .datavar blocks.

The following example:

```
.datavar data1 n=3
.setparam inst=X0.R1 r= (0.5k, 1.5k)
.enddv data1

.datavar data2 n=2
.setparam inst=X0.C1 c= (0.4n, 0.8n)
.enddv data2

.tran 1n 100p sweep datavar=(data1, data2)
```

The following output files are generated:

```
X0.R1=0.5k, X0.C1=0.4n xa.s0.meas, xa.s0.s0.fsdb
X0.R1=0.5k, X0.C1=0.8n xa.s1.meas, xa.s0.s1.fsdb
X0.R1=1k, X0.C1=0.4n xa.s0.meas, xa.s1.s0.fsdb
X0.R1=1k, X0.C1=0.8n xa.s1.s1.meas, xa.s1.s1.fsdb
```

```
X0.R1=1.5k, X0.C1=0.4n xa.s0.meas, xa.s2.s0.fsdb  
X0.R1=1.5k, X0.C1=0.8n xa.s1.meas, xa.s2.s1.fsdb
```

The parameter combination of each simulation is written to the log file together with the sweep number (#) and information about the output file (meas/waveform) extension. The following information is reported in the *.log file for each simulation:

```
.SWEEP DATAVAR (#0) (output extension is .s0.s0) X0.R1=0.5k X0.C1=0.4n  
.SWEEP DATAVAR (#1) (output extension is .s0.s1) X0.R1=0.5k X0.C1=0.8n  
.SWEEP DATAVAR (#2) (output extension is .s1.s0) X0.R1=1k X0.C1=0.4n  
.SWEEP DATAVAR (#3) (output extension is .s1.s1) X0.R1=1k X0.C1=0.8n  
.SWEEP DATAVAR (#4) (output extension is .s2.s0) X0.R1=1.5k X0.C1=0.4n  
.SWEEP DATAVAR (#5) (output extension is .s2.s1) X0.R1=0.5k X0.C1=0.8n
```

Time-Window-Based Analysis

PrimeSim XA supports the HSPICE `tempvec` option to change temperature values during transient analysis, based on time windows.

To configure window-based temperature values in the PrimeSim XA environment, run the `tempvec` option in the following syntax:

```
.TRAN tstep tstop tempvec=(t1 Temp1 t2 Temp2 t3 Temp3 ...)
```

However, PrimeSim XA does not support the HSPICE `tempstep` option, which means the temperature is constant within the same time window. For example, if you specify:

```
.tran 1n 200n tempvec=(0n 25 50n 50 100n 100)
```

The temperature is 25C between 0n and 50n, 50C between 50n and 100n, and 100C from 100n to the end of transient simulation.

Note:

The support of the `tempvec` option in PrimeSim XA has been implemented using the PrimeSim XA save and restore feature. That means the following save-and-restore limitations also apply when you specify the `tempvec` option in the PrimeSim XA environment:

- The `tempstep` option is not supported. When used, a warning is issued.
- The waveforms are split and soft links are created so that the <prefix>.fsdb file can be loaded and merged. For example, for xa.fsdb, the following files are created:
 - xa.fsdb -> xa.temp1.fsdb
 - xa.fsdb.1 -> xa.temp2.fsdb
 - xa.fsdb.2 -> xa.temp3.fsdb

When the xa.fsdb file is loaded in WaveView, all 3 files (including xa.fsdb, xa.fsdb.1 and xa.fsdb.2) are opened.

- Measurement output files are split. There is one measurement file per time interval (per temperature), using the following naming style:

<prefix>.temp1.meas

<prefix>.temp2.meas

<prefix>.temp3.meas

...

- If a measurement overlaps 2 or more time/temperature intervals, then the measurement is reported as failed. Only the measurements that can complete within the time/temperature interval are reported.

Monte Carlo Analysis in PrimeSim XA

The PrimeSim XA tool supports traditional Monte Carlo analysis for transient simulations with the PrimeSim HSPICE netlist format. For more information about running Monte Carlo analysis in the PrimeSim XA tool, see [Monte Carlo Analysis](#).

Violation Checks Using HSPICE .BIASCHK Statements

The PrimeSim XA tool supports the .BIASCHK statement from PrimeSim HSPICE netlist format to monitor voltage bias, currents, and expressions during transient analysis, and the BIASFMT option to control the format of the output violation report.

The PrimeSim XA tool supports the .BIASCHK statement with the following commands:

- `biaschk_enable = 1|0`: Sets to 1 to enable the generation of the violation report in HSPICE-compatible format based on the setting of the .BIASCHK statements. To disable violation reporting, use `biaschk_enable=0` or `.option biasfmt=off`. Default is 1.
- `set_biaschk`: Controls the support of the specified .BIASCHK statement list. For more information, see [Using the set_biaschk Command](#).

When both `biaschk_enable` and `set_biaschk` are used, the `biaschk_enable` option takes precedence over the `set_biaschk` command. The following table describes how

`biaschk_enable` affects the `set_biaschk` command behavior in the PrimeSim XA tool:

When	The PrimeSim XA Tool Does
<code>biaschk_enable</code> not set	Ignores the <code>set_biaschk</code> command setting
<code>biaschk_enable = 0</code>	Ignores the <code>set_biaschk</code> command setting
<code>biaschk_enable = 1</code>	Honors the <code>set_biaschk</code> command setting

Note:

- The `biaschk_enable=0` command is supported only when `.option biasfmt=2` is specified.
- Not all the features of the `.BIASCHK` statement from PrimeSim HSPICE are supported. The PrimeSim XA tool also supports additional features for the FastSPICE application.
- The `.BIASCHK` statement is not supported in Eldo and Spectre netlist formats.

The output violation report includes violation details, like the element (instance) name, time, terminals, bias that exceeds the limit, or the number of times the bias exceeds the limit for an element.

This section contains the following topics;

- [Syntax](#)
- [Using the `set\_biaschk` Command](#)
- [Using the noise Option](#)
- [Supported Netlist Options](#)
- [Output Formats and Examples](#)

Syntax

This section lists the syntaxes that are supported when you use the `.BIASCHK` statement in PrimeSim XA.

- The `.BIASCHK` type Syntax

The following syntax monitors the bias voltage and current for the specified device type.

Chapter 3: PrimeSim HSPICE Netlist Compatibility

Violation Checks Using HSPICE .BIASCHK Statements

```
.BIASCHK type terminal1=t1 [terminal2=t2]
+ [max=max] [min=min] [noise=val]
+ [simulation=tr] [monitor=v|i]
+ [name=name1,name2,...]
+ [mname=mname1,mname2,...]
+ [tstart=time1] [tstop=time2] [autostop]
+ [except=ename1,ename2,...]
+ [sname=sname1,sname2,...]
+ [interval=tval] [biasname=label]
```

The following syntax monitors the width and length of specified device type.

```
.BIASCHK type
+ [max=max] [min=min]
+ [simulation=tr] [monitor=l|w]
+ [name=name1,name2,...]
+ [mname=mname1,mname2,...]
+ [tstart=time1] [tstop=time2] [autostop]
+ [except=ename1,ename2,...]
+ [sname=sname1,sname2,...]
+ [interval=tval] [biasname=label]
```

The following syntax monitors some common instance conditions of the specified device type.

```
.BIASCHK type
+ [max=max] [min=min]
+ [simulation=tr] [monitor=param]
+ [name=name1,name2,...]
+ [mname=mname1,mname2,...]
+ [tstart=time1] [tstop=time2] [autostop]
+ [except=ename1,ename2,...]
+ [sname=sname1,sname2,...]
+ [interval=tval] [biasname=label]
```

The following syntax monitors an expression.

```
.BIASCHK type expr='expr_w_asterisk'
+ [condition='cond_logic_expr']
+ [max=max] [min=min]
+ [simulation=tr]
+ [name=name1,name2,...]
+ [mname=mname1,mname2,...]
+ [tstart=time1] [tstop=time2] [autostop]
+ [except=ename1,ename2,...]
+ [sname=sname1,sname2,...]
+ [interval=tval] [biasname=label]

.BIASCHK 'expression'
+ [condition=cond_logic_expr]
+ [max=max] [min=min]
```

```
+ [simulation=tr] + [tstart=time1] [tstop=time2] [autostop]
+ [sname=sname1,sname2,...]
+ [interval=tval] [biasname=label]
```

- The .BIASCHK subckt syntax

The following syntax writes the values of l, w, ad and as of subckt-instance xm2 into the .BIASCHK violation output file.

```
.BIASCHK subckt expr='abs(v(n1,n3))' mname=na45_g5b_mac max=1e-6
+ tstart=biaschk_start tstop=biaschk_stop interval=biaschk_interval
+ biasname='P_na45_g5b_mac' biasparam=l, w, ad, as

.subckt na45_g5b_mac n1 n2 n3 n4 sub l=length w=width ad=ad as=as nf=1
scale=1 sigma=1
...
.ends
xm2 nt1 nt1 0 0 0 na45_g5b_mac l=1.5u w=0.0004 ad=2.81253n as=0.128n
nf=10 sigma=1
```

The following syntax monitors an expression for bias checking on a subcircuit.

```
.BIASCHK subckt simulation=all
+ mname='XRPP'
+ expr='abs(v(pos,neg)*isub(pos))/(mx*(w+0.64u)*1/1.0e-12)'
+ max='180u' interval='0'
+ message='OOOPS'
```

- The .biaschk scoping arguments

```
+ [name=name1, name2, ...]
+ [mname=modname_1, modname_2, ...]
+ [except=name_1, name_2, ...]
+ [sname=subckt_name1, subckt_name2, ...]
```

- The .biaschk common arguments

```
+ [simulation= op|ac|dc|tr|all]
+ [tstart=time1] [tstop=time2]
+ [interval=time] [message="string"]
+ [biasname=bname] [biasparam=bparam]
+ [tstart=time1] [tstop=time2]
```

Table 19 *The .biascheck Statement Arguments*

Argument	Description
boolean 0 1 true false	Chooses a report style for Boolean violations. • 1 or true (default): The violation is printed when the expression changes from false to true. • 0 or false: The violation is printed when the expression changes from true to false.

Table 19 *The .biascheck Statement Arguments (Continued)*

Argument	Description
<code>message</code>	Defines the warning message to be dumped in the output file.
<code>biasparam</code>	Dumps values of specified parameters to the output file. - For <code>.biaschk subckt</code>, the parameters are defined by <code>.param</code> or in <code>.subckt</code> and <code>X*</code> lines. - This argument is not supported for <code>.biaschk <mos pmos nmos bipolar bjt fet diode r c></code> and <code>.biaschk 'expression'</code>
<code>type</code>	Specifies the device type to check: - MOS - PMOS - NMOS - BIPOLAR - BJT - JFET - DIODE - R - C - SUBCKT
<code>terminal1=t1</code> <code>terminal2=t2</code>	Specifies the device terminals: - MOS level 57 can be set to <code>nd, ng, ns, ne, np, n6</code> - MOS level 58 can be set to <code>nd, ngf, ns, ngb</code> - MOS level 59 can be set to <code>nd, ng, ns, ne, np</code> - Other MOS levels can be set to <code>nd, ng, ns, nb</code> - R can be set to <code>n1, n2</code> - C can be set to <code>n1, n2</code> - DIODE can be set to <code>np, nn</code> - BIPOLAR or BJT can be set to <code>nc, nb, ne, ns</code> - JFET can be set to <code>nd, ng, ns, nb</code> - SUBCKT can be set to the terminal names that are the pins defined by the subcircuit definition of <code>mname</code>.
<code>max=max</code>	Specifies the maximum value or upper threshold value.
<code>min=min</code>	Specifies the minimum value or lower threshold value.
<code>device</code>	Specifies additional conditions when using the bias check method, such as min/max for the MOS monitor.
<code>simulation</code>	Specifies the simulation type you want to monitor. The supported analysis types are: <code>op, dc, ac, tr</code> (transient), and <code>all</code> (for <code>op, dc, ac, and tr</code>). Default is <code>tr</code>. NOTE: The DC, OP and AC types are supported only in PrimeSim HSPICE and PrimeSim tools.

Table 19 *The .biascheck Statement Arguments (Continued)*

Argument	Description
<code>monitor=v i</code>	Monitors the voltage and current of the specified device.
<code>monitor=w l</code>	Monitors the width and length of the specified device.
<code>monitor=param</code>	Monitors the specified instance condition. See Table 20 for a list of supported functions for accessing device condition.
<code>name=name1,name2</code>	Scopes the device name for bias check. More than one device name can be specified with a comma (,) as a delimiter. If <code>type=SUBCKT</code>, the following rules apply: • <code>name</code> is the instance name and <code>mname</code> is the subcircuit name. • When both <code>name</code> and <code>mname</code> are defined and if a <code>name</code> is also an instance name of <code>mname</code>, then only those names are checked. Otherwise, this statement is ignored. You can specify <code>name</code> and <code>mname</code> in the same statement, but use them cautiously.
<code>mname=mname1,mname2</code>	Scopes to devices of the specified model name. If it is a macro model, you can specify it to the subcircuit name of a macro model. You can specify more than one model name with the comma (,) as delimiter. A wildcard character is supported. If <code>type=SUBCKT</code>, the following rules apply: • <code>name</code> is the instance name and <code>mname</code> is the subcircuit name. • Multiple <code>mname</code> arguments are not allowed. You can specify <code>name</code> and <code>mname</code> in the same statement, but use them cautiously.
<code>sname=sname1,sname2</code>	Scopes to the instances of the named subcircuits. Multiple subcircuit names can be specified with a comma (,) as a delimiter.
<code>tstart=time1</code> <code>tstop=time2</code>	Specifies the time window to perform bias check. <code>time1</code> is start time of a time window and the <code>time2</code> is the end time of a time window. The default value for <code>time1</code> is 0 and <code>time2</code> is the end of transient time.
<code>autostop</code>	Enables the PrimeSim XA tool to stop the simulation immediately when the related bias check fails.
<code>except=enamel,ename2</code>	Specifies the elements or instances to be excluded from bias check.
<code>interval=tval</code>	Prevents reporting intervals that are less than or equal to the time specified. Default as zero.

Table 19 The .biascheck Statement Arguments (Continued)

Argument	Description
<pre>expr = 'non_logic_expr' 'logic_expr' 'nested_exprs'</pre>	Specifies the expression to be checked for the Device: non_logic_expr: Specifies the expression to be checked. When specified, either <code>min</code> or <code>max</code> argument is required. logic_expr: Specifies the conditional expression, which contains relational operators, like '<', '>', '==', '<=', '>=', '!=', or '? :'. The <code>min</code> or <code>max</code> arguments are ignored. nested_exprs: <code>expr = "A=expr1; B=expr2; C=expr3"; condition_expr.</code> For example: <code>v_ds=v(d,s); len=1; v_ds<3 1>0.4u</code>
<pre>condition='cond_logic_expr'</pre>	A conditional logical expression. If the logical expression evaluates to TRUE, it enables the .BIASCHK statement. It supports a wildcard character if it is used with <code>expr='expr_w_asterisk'</code> and it does not if it is used with <code>'expression'</code>. If no condition is set, the .biaschk statement is always enabled.
<pre>biasname=label</pre>	Specifies the label name of the .BIASCHK statement. When not specified, the PrimeSim XA tool gives an index as the label. Use this argument to organize multiple .biaschk commands and their outputs in final bias check result files, for viewing violation details in GUI applications.
<pre>noise=val</pre>	Specifies the noise for bias voltage check to filter out some of the violations for reporting. It does not support bias current check. The default is 0v. This option filters some of the violations that exceed the upper threshold value defined by <code>max</code>. See the Using the noise Option section.

Table 20 Keywords Supported in monitor Option for Accessing Device Condition

Parameter	Description
ic	Collector current of a BJT.
ie	Emitter current of a BJT.
id	Drain current of a MOSFET or JFET.
ig	Gate current of a MOSFET or JFET.
is	Source current of a MOSFET, BJT or JFET.
ib	Bulk current of a MOSFET or base current of a BJT.
vs	Source voltage of a MOSFET, JFET or BJT.

Table 20 Keywords Supported in monitor Option for Accessing Device Condition (Continued)

Parameter	Description
vd	Drain voltage of a MOSFET or JFET.
vb	Bulk voltage of a MOSFET, JFET or BJT.
vg	Gate voltage of a MOSFET or JFET.
vc	Collector voltage of a BJT.
ve	Emitter voltage of a BJT.
vneg	Cathode voltage of a diode or low-voltage terminal voltage of a resistor or capacitor.
vpos	Anode voltage of a diode or high-voltage terminal voltage of a resistor or capacitor.
vbe	Base/emitter voltage difference of a BJT ($V_b - V_e$).
veb	Emitter/base voltage difference of a BJT ($V_e - V_b$).
vbc	Base/collector voltage difference of a BJT ($V_b - V_c$).
vcb	Collector/base voltage difference of a BJT ($V_c - V_b$).
ves	Emitter/source voltage difference of a BJT ($V_e - V_s$).
vse	Source/collector voltage difference of a BJT ($V_s - V_c$).
vcs	Collector/source voltage difference of a BJT ($V_c - V_s$).
vsc	Source/collector voltage difference of a BJT ($V_s - V_c$).
vce	Collector/emitter voltage difference of a BJT ($V_c - V_e$).
vce	Collector/emitter voltage difference of a BJT ($V_c - V_e$).
vec	Emitter/collector voltage difference of a BJT ($V_e - V_c$).
vbd	Bulk/drain voltage difference of a MOSFET and JFET ($V_b - V_d$).
vdb	Drain/bulk voltage difference of a MOSFET or JFET ($V_d - V_b$).
vds	Drain/source voltage difference of a MOSFET or JFET ($V_d - V_s$).

Table 20 Keywords Supported in monitor Option for Accessing Device Condition (Continued)

Parameter	Description
vsd	Source/drain voltage difference of a MOSFET and JFET (Vs-Vd).
vgb	Bulk/gate voltage difference of a MOSFET or JFET (Vb-Vg).
vbg	Bulk/gate voltage difference of a MOSFET or JFET (Vb-Vg).
vgd	Gate/drain voltage difference of a MOSFET or JFET (Vg-Vd).
vdg	Drain/gate voltage difference of a MOSFET or JFET (Vd-Vg).
vgs	Gate/source voltage difference of a MOSFET or JFET (Vg-Vs).
vsg	Source/gate voltage difference of a MOSFET or JFET (Vs-Vg).
vbs	Bulk/source voltage difference of a MOSFET or JFET (Vb-Vs).
vsb	Source/bulk voltage difference of a MOSFET or JFET (Vs-Vb).

Table 21 Supported Mathematical Functions

Function	Description
re(<signal>)	Real portion of a complex number. This is for AC analysis only and is currently not supported by the PrimeSim XA tool.
trim(<signal>, <from>, <to>)	Portion between two points along X-axis
avg(<signal>)	Average value waveform on the portion between two points along X-axis
rms(<signal>)	Root-mean-square value waveform on the portion between two points along X-axis

Table 21 Supported Mathematical Functions (Continued)

Function	Description
<code>min(<signal>)</code>	Minimal value waveform on the portion between two points along X-axis. An one-argument <code>min()</code> returns the minimum value during analysis, while a multiple-argument <code>min()</code> returns the minimum value among the arguments.
<code>max(<signal>)</code>	Maximal value waveform on the portion between two points along X-axis. An one-argument <code>max()</code> returns the max value during analysis, while a multiple-argument <code>max()</code> returns max value among the arguments.

Examples

This example reports the violation if the voltage on terminal 1 of a capacitor named `c10` exceeds the value of 0.1V.

```
.biaschk c terminal1=n1 max=0.1 name='c10'
```

This example reports the violation if the voltage difference of `net27` and `net25` is outside of the range of 1e-2V and 1V.

```
.biaschk 'v(net27)-v(net25)' min=1e-2 max=1
```

This example checks if the drain current of MOSFET is greater than 150uA, the `.BIASCHK` statement is enabled to check its `vgs` bias of all MOSFET. If `vgs` bias is less than 4, it reports a violation.

```
.biaschk mos expr='vgs(*)' condition='id(*) > 150u' min=4
```

This example checks if `v(clock)` is greater than 3, the `.BIASCHK` statement is enabled to check the value of voltage difference of all the drain terminal of all MOSFET and node `net25`. If the value is less than 1, it reports a violation.

```
.biaschk mos expr='vd(*)-v(net25)' condition='v(clock) > 3' min=1
```

Using the set_biaschk Command

Use the `set_biaschk` command to control the support of `.biaschk` statements for violation reporting. Each `set_biaschk` command can only either enable or disable a list of `.biaschk` statements. When multiple `set_biaschk` commands select the same `.biaschk` statement, the last `.cmd set_biaschk` takes precedence.

The `set_biaschk` command also supports the third-party `assert` statement for violation reporting.

Syntax

```
.cmd set_biaschk
+ biasname = "biaschk1" ["biaschk2", ...]
+ enable = 0 | 1
```

Argument	Description
biasname = "biaschk1" ["biaschk2", ...]	Specifies a list of .biaschk statements to be controlled by this set_biaschk by giving the biasname.
enable = 0 1	Controls the enabling flag for the specified .biaschk statements.

Example

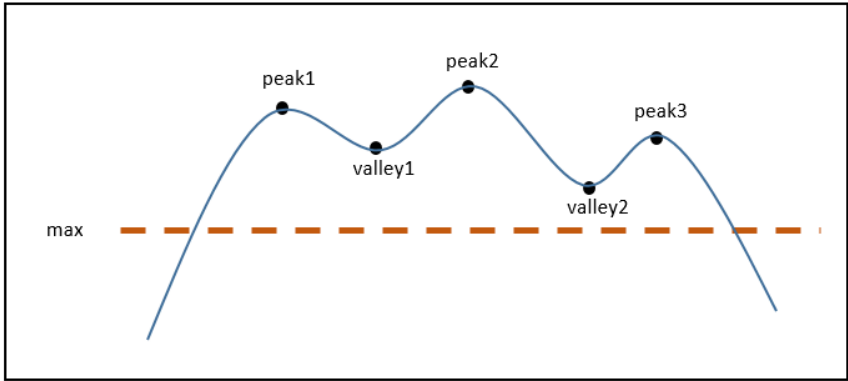
In the following example, the bias_mos_1, bias_mos_2, bias_bjt_1, bias_bjt_2 are selected to be enabled. Then, the set_biaschk command is set twice to disable bias_mos_1 and bias_bjt_1.

```
.biaschk biasname=bias_mos_1
.biaschk biasname=bias_mos_2
.biaschk biasname=bias_bjt_1
.biaschk biasname=bias_bjt_2
.cmd set_biaschk biasname = bias* enable = 1
.cmd set_biaschk biasname = bias*_1 enable = 0
```

Using the noise Option

When the bias check is outside of the safe operating range specified by max and min, the PrimeSim XA tool reports all the violations. The noise option can be used to filter out some of the violations. For example:

Figure 2 An Example of Using the noise Option



If you do not specify `noise`, the PrimeSim XA tool reports `peak1`, `peak2` and `peak3` as violations in the report. If `noise` is specified, the PrimeSim XA tool filters the violations with the following rules:

- If $(V_{peak_n} > \max) \ \& \ (V_{peak_n} - V_{valley_n} < \text{noise})$, then `reported_violation=Vpeakn`
- If $(\text{reported_violation} < V_{peak_{n+1}}) \ \& \ (V_{peak_{n+1}} - V_{valley_{n+1}} < \text{noise})$, then `reported_violation=Vpeakn+1`

In [Figure 2](#), if `Vpeak1=15.5v`, `Vpeak2=15.7v`, `Vpeak3=15.4v`, `Vvalley1=15.4v`, `Vvalley2=15.2v` and `noise=0.8`, then only `peak2` is reported as a violation.

Supported Netlist Options

The PrimeSim XA tool supports the following netlist options syntax related to the `.BIASCHK` statement:

```
.OPTION BIASFILE='fileName'  
.OPTION BIASFLUSH=auto  
.OPTION BIASFMT=0|1|2|cck|off
```

Argument	Description
<code>BIASFILE='filename'</code>	Specifies the file name of the violation report. If this option is not specified, the output file is suffixed with <code>*.biaschk</code> when <code>BIASFMT=0 2</code> or <code>*.biaschk.ccksoa</code> when <code>BIASFMT=cck</code> .
<code>BIASFLUSH=auto</code>	Enables the violations to be flushed in the output file as soon as it occurs.
<code>BIASFMT=0 1 2 cck off</code>	Specifies the output format of the violation report. It can be set to : • <code>0</code> to generate violation reports in PrimeSim HSPICE-compatible format. • <code>1</code> to generate violation reports in XA-compatible format. • <code>2</code> to generate violation reports in common SOA format. This is the common output format for Synopsys simulators, including PrimeSim XA, PrimeSim HSPICE and PrimeSim. • <code>cck</code> to generate violation reports in legacy CCK format. • <code>off</code> to disable violation reporting. See the Output Formats and Examples section for examples.

Output Formats and Examples

By default, the PrimeSim XA tool generates the `.biaschk` violation report in the same format as the HSPICE tool. To generate the violation report in other formats, use the `BIASFMT` control option. The supported formats are: HSPICE, PrimeSim XA, common SOA format, and legacy CCK.

To disable violation reporting, use `biaschk_enable=0` or `.option biasfmt=off`.

Note:

You can disable violation report with `biaschk_enable=0` only when `.option biasfmt=2` is specified.

Writing Outputs in HSPICE Format

Use `.option biasfmt=0` or `.option biasfmt=hspice` to write the `.biaschk` violation checking report in the HSPICE format.

Following is an example of the output report in HSPICE format:

*** Biaschk output during transient simulation ***

type	terminal	time	Val_ chk	method	model- name	element- name	subckt- name	Biaschk_ name
c	v(n1)	0.0000	5.0000	max	None	c10		0
expression	v(net27)-v(net25)	1.2726n	-5.0000	min	---	---	---	1
expression	v(net27)-v(net25)	3.0212n	-5.0000	min	---	---	---	1

Elements that have biaschk out of limit during the transient simulation:

type	terminal	Number Counted	model- name	element- name	subckt- name	Biaschk_ name
c	v(n1)	1	None	c10		0
expression	v(net27)-v(net25)	2	---	---	---	1

*** Biaschk end for transient simulation ***

Writing Outputs in PrimeSim XA Format

Run `.option biasfmt=1` to generate the `.BIASCHK` violation report in the PrimeSim XA format.

SOA INFORMATION

label	condition	instance
(none)	((v(g,s) < 0.31 + vt(m1) && temp(m1)>200)):TRUE	x1
(none)	((v(g,s) < 0.31 + vt(m1) && temp(m1)>200)):TRUE	x1
(none)	((v(g,s) < 0.31 + vt(m1) && temp(m1)>200)):TRUE	x1
(none)	((v(g,s) < 0.31 + vt(m1) && temp(m1)>200)):TRUE	x1
(none)	((v(g,s) < 0.31 + vt(m1) && temp(m1)>200)):TRUE	x1

expression	X window	error
v(d,s)	[0.000000e+00,1.207249e-07]	Value superior to 3.210000e+00 (up to 5.329890e+00)
v(d,s)	[1.404174e-07,1.457249e-07]	Value superior to 3.210000e+00 (up to 5.326656e+00)
v(d,s)	[1.654171e-07,1.707249e-07]	Value superior to 3.210000e+00 (up to 5.326656e+00)
v(d,s)	[1.904171e-07,1.957249e-07]	Value superior to 3.210000e+00 (up to 5.326656e+00)
v(d,s)	[2.154171e-07,8.000000e-07]	Value superior to 3.210000e+00 (up to 5.000001e+00)

Writing Outputs in Common SOA Format

Run `.option biasfmt=2` to generate the `.BIASCHK` violation report in the common SOA format with the `*.biaschk` suffix. The common SOA format is supported in Synopsys simulators, including PrimeSim XA, PrimeSim HSPICE and PrimeSim.

Following is an example of the output report that contains `.BIASCHK` violations in the common SOA output format.

```
* CCK Command: biaschk
* Tag: chk_1
* Scopes:
*   instance name=x1.x2
*   subckt name=nmos_mac
* Check Windows:
*   start=10ns stop=20ns
* message=check vgs with 3.5 threshold

Input netlist:
.biaschk subckt expr='abs(v(n1,n3))' mname=inv ...

xtop1 ... add
xtop2 ... add

.subckt add
x1 ... inv
.ends
.subckt inv
m1 ...
.ends

Report:
Dev 1: xtop1.x1.m1
Subinfo: add inv

Dev 2: xtop2.x1.m2
```


Chapter 3: PrimeSim HSPICE Netlist Compatibility

Violation Checks Using HSPICE .BIASCHK Statements

```
Subinfo: add inv

Dev <#n>: x1.ml
  Subinfo: inv
  Attributes: w=2u, l=3u

Data:
  Constraint: ((5-vd) * 10, 3, 6, 0n), TR
  Duration: seq=1, t1=1ns, t2=0.2us, span=0.199us
  Start: expr=0.163V, t=1ns
  End: expr=0.197V, t=0.2us
  Min(expr): expr=0.163V, t=1ns
  Max(expr): expr=0.693V, t=0.1802us
  Total Violation Time: 0.199us
```

Writing Outputs in Legacy CCK Format

Run `BIASFMT=cck` to write out the analysis results in the legacy CCK format.

Following is an example of the violation report in legacy CCK format:

```
*CCK Command: Biaschk*
*
Tag: 0
Scopes:
* insta=c10
TYPE:CConstraint=(V1, *, .1) numvd=all)
*
Dev #1: c10
Attributes: model=No model

Data:
  Constraint: (V1, *, .1)
  Duration: seq=1, t1=0s, t2=8ns, span=8ns
  Start: expr=5V, t=0s
  End: expr=5V, t=8ns
  Min(expr)= expr=5V, t=0s
  Min(expr)= expr=5V, t=0s
  Total Violation Time: 8ns
*
* CCK Command Footer
*
* CCK Command: biaschk
* Tag= 0
* Total Number of devices: 1
*
* CCK Command: biaschk
* Tag= 1
*Scopes:
* Inst=v(net27)-v(net25)
*Constraints: (v(net27)-v(net25), .1e-10, 1) numvd=all
*
Node #1: net27
```

```
Duration: seq=1, t1=0s, t2=3.8012ns, span=3.8012ns
Start: expr=-5, t=0s
End: expr=0.01n, t=3.8012ns
Min(expr)= expr=-5, t=1.2726ns
Min(expr)= expr=0.01n, t=3.8012ns
Duration: seq=2, t1=3.8377ns, t2=8ns, span=4.1623ns
Start: expr=-1, t=3.8377ns
End: expr=5, t=8s
Min(expr)= expr=1, t=3.8377ns
Min(expr)= expr=5, t=8ns
Total Violation Time: 7.9636ns
*
CCK Command Footer
*
CCK Command: biaschk
*Tag:1
Total number of devices: 1
*
***Total number of violation: 3
```

Bisection Methodology and Behavior Variations

The PrimeSim XA tool supports the PrimeSim HSPICE Bisection Methodology to analyze circuit timing violations. See *PrimeSim™ HSPICE® User Guide: Basic Simulation and Analysis* for details.

The PrimeSim XA tool has enhanced the Bisection Methodology. If the bisection parameter has no impact on the DC state of the circuit (only impacts timing of pulse/pwl sources), the PrimeSim XA tool can optimize the bisection process by skipping repeated device model processing and DC operating point stages. If the bisection parameter does impact the DC solution, then the PrimeSim XA tool repeats most of the netlist parsing and model processing before each bisection iteration. If the PrimeSim XA tool is unable to optimize the bisection process, a warning is issued in the PrimeSim XA log file explaining why the process could not be optimized.

There are some differences between the PrimeSim HSPICE and PrimeSim XA tools.

- Both the PrimeSim HSPICE and PrimeSim XA tools need to specify goal argument in the `.MEASURE` statement, but:
 - In the PrimeSim HSPICE tool, only `goal=value` in the `.MEASURE` statement is supported. The `goal=value` argument is interpreted as goal is greater than value.
 - The PrimeSim XA tool enhances to support several new operands (`=`, `>`, `<`, `>=`, `<=`), and all are interpreted explicitly. To have the same behavior as the PrimeSim HSPICE tool, replace `goal=value` with `goal>value` in the `.MEASURE` statement.

- Both the PrimeSim HSPICE and PrimeSim XA tools require iteration #1 to pass and iteration #2 to fail or iteration #1 to fail and iteration #2 to pass. The difference in behavior comes from the definition of "fail" as follows:
 - In the PrimeSim HSPICE tool, "fail" means the measurement did not occur, or when a voltage is not crossing the threshold set by goal at any time during the simulation. The word "failed" is written to the `.mt#` file.
 - In the PrimeSim XA tool, "fail" has the same meaning as in the PrimeSim HSPICE tool. It also means that the measurement did return a value, but that value did not satisfy the goal. This convention is useful when you use the greater than or less than operands.

The PrimeSim XA log file contains the following bisection information:

- Number of bisection iterations run
- Iteration selected as the final result
- Value of the bisection parameter for each bisection iteration
- Value of the bisection measurement for each bisection iteration
- Information related to meeting RELIN and RELOUT options

By default, the PrimeSim XA tool writes out the measurement data from the selected iteration only to the `*.meas` file. You can use `set_meas_option -bisect_meas all` to create an additional output file with `*.bisect_meas` suffix that contains the measurement results for all the bisection iterations.

The following example shows how to set up a bisection simulation with the `.TRAN` statement, `DelayTime` parameter, and `.MEASURE` statements.

```
* DFF_top Bisection Search for Setup Time* PWL Stimulus
v28 data gnd PWL
+ 0s 5v
+ 1n 5v
+ 2n 0v
+ Td = "DelayTime" $ Offsets Data from time by DelayTime

v27 clock gnd PWL
+ 0s 0v
+ 3n 0v
+ 4n 5v

* Specify DelayTime as the search parameter and provide
* the lower and upper limits.
.PARAM DelayTime= Opt1 ( 0.0n, 0.0n, 5.0n )

* Transient simulation with Bisection Optimization
.TRAN 0.1n 8n Sweep Optimize = Opt1
+ Result = MaxVout $ Look at measure
```

```
+ Model = OptMod
* This measure finds the transition if it exists
.MEASURE Tran MaxVout Max v(D_Output) Goal = 'v(Vdd) '

* This measure calculates the setup time value
.MEASURE Tran SetupTime
+ Trig v(Data)Val = 'v(Vdd)/2' Fall = 1
+ Targ v(Clock)Val = 'v(Vdd)/2' Rise = 1

* Optimization Model
.MODEL OptMod Opt Method = Bisection
```

PrimeSim HSPICE-Encrypted Netlist Format

The PrimeSim XA tool supports PrimeSim HSPICE netlists that are encrypted by the metaencrypt utility. For details about how to use the metaencrypt utility to protect your intellectual property, see the related topic in [PrimeSim HSPICE User Guides](#).

Most of PrimeSim XA features support encrypted-PrimeSim HSPICE netlist, except the following:

- MOS reliability analysis (MOSRA)
- CircuitCheck analysis
- The iset_save_state interactive mode command

Post-Layout Simulation Galaxy Parasitic Database (GPD)

The PrimeSim XA tool supports the HSPICE `.GPD_INC` command and the PrimeSim XA `load_gpd_data` command to perform a post-layout simulation on a corner used in the Galaxy Parasitic Database (GPD). The syntax for the `.GPD_INC` command is:

```
.GPD_INC gpd_path [corner_name] [-dump_spf 0|1]
```

Argument	Description
<i>gpd_path</i>	Specifies the path to the design GPD.
<i>corner_name</i>	Specifies the name of the corner to simulate. The default is the first corner inside the GPD.
<code>-dump_spf 0 1</code>	Specifies whether to dump an SPF format file: • 0(default) does not dump an SPF file. • 1 specifies to dump an SPF file and exit the simulation.

For example, the following command performs post-layout simulation with the FF corner provided in the GPD.

```
.gpd_inc dir/path_to_gpd FF
```

For more information about the `load_gpd_data` command, see *PrimeSim™ XA Command Reference*.

4

Vector Stimulus Files

This chapter describes the stimulus files in the PrimeSim XA tool.

This chapter contains the following topics:

- [Using the Digital Vector File as Stimulus](#)
- [Using a Value Change Dump File](#)
- [Using the Extended Value Change Dump File](#)

Note:

The PrimeSim XA tool uses the `set_vector_option` command to provide flexibility to control the stimulus file.

Using the Digital Vector File as Stimulus

A digital vector (VEC) file is a tabular digital description of the nodal behavior that is used to create drivers and expected outputs. The PrimeSim XA tool supports the PrimeSim HSPICE vector file format (see [Reading the PrimeSim HSPICE Vector File](#)).

The different forms of syntax are identical and compatible in the PrimeSim XA tool, with the respective tool based on the vector file format.

Note:

For the PrimeSim HSPICE VEC file, the PrimeSim XA tool supports additional enhancements for FastSPICE applications.

The following syntax rules apply to all VEC files:

- A comment begins with a semicolon character (;). Comments can start at any point along the line.
- A continuation line begins with a plus character (+). The plus symbol (+) signifies a continuation from the previous line.

- A value expression can contain numbers, netlist parameters, or a PrimeSim HSPICE equation expression. The expression must be single-quoted or double-quoted, or must not contain any white-space characters.
- All characters are case-insensitive.

A VEC file consists of the following three sections:

- [Tabular Data Section](#)
- [Vector Pattern Definition Section](#)
- [Waveform Characteristics Section](#)

Here is an example of the PrimeSim HSPICE VEC file.

Figure 3 PrimeSim HSPICE VEC File

```
; Vector Patterns Definition Section
radix      111111111111111
vname      B[[3:0]] A[[3:0]] CIN COUT S[[3:0]]
io         iiiiiiiiooooo
tunit ns

; Waveform Characteristics Section
vih 1.8
vil 0.0
voh 1.5
vol 0.5
slope 0.01

; Tabular Data Section
0 000000000LLLLL
10 000000000LLLLL
12 000000000LLLLL
20 000100000LLLLL
22 000100000LLLLH
30 001000000LLLLH
32 001000000LLHL
40 001100000LLHL
42 001100000LLHH
50 010000000LLHH
52 010000000LLHL
60 010100000LLHL
62 010100000LLHLH
70 011000000LLHLH
72 011000000LLHL
80 011100000LLHL
82 011100000LLHH
90 100000000LLHH
92 100000000LLHL
100 100100000LLHL
```

See Also

- [Reading the PrimeSim HSPICE Vector File](#)

Reading the PrimeSim HSPICE Vector File

To read in a PrimeSim HSPICE VEC file, use the `load_vector_file` command. You can specify more than one VEC file with multiple `load_vector_file` commands. The following example loads in a VEC file, named `adder.vec`.

```
xa> load_vector_file -file adder.vec
```

The PrimeSim XA tool also supports SPICE-specific statements to read in a VEC file. To incorporate a VEC file into the netlist, use the `.VEC` statement in either PrimeSim HSPICE or Eldo netlist format, and the `vec_include` or `vcd_include` statements for 3rd-party netlist format.

A VEC file consists of the following parts:

- [Tabular Data Section](#)
- [Vector Pattern Definition Section](#)
- [Waveform Characteristics Section](#)

Tabular Data Section

Although the tabular data section generally appears last in a vector file, it is described here first to introduce you to the definition of a vector.

The tabular data section defines the signal logic states at specified times. The first column lists the absolute time. The second and subsequent columns list the signal logic states at the absolute time. The syntax is:

```
[time1] signal1_value1 signal2_value1 ... signalm_value1  
[time2] signal1_value2 signal2_value2 ... signalm_value2  
...  
[timen] signal1_valuen signal2_valuen ... signalm_valuen
```

Argument	Description
<i>timen</i>	Defines the absolute time.
<i>signalm_valuen</i>	Defines the logic state of <i>signalm</i> at <i>timen</i> .

The set of values for a particular signal is a vector, which appears as a vertical column in the tabular data section. Rows in this section must appear in chronological order, because row replacement carries sequential timing information. If the tabular data section is not

specified in a vector file, the PrimeSim XA tool terminates the simulation with an error message.

You can use Verilog-sized format to specify multiple bits at once. The syntax is:

```
[size] '[base] [number]
```

Argument	Description
<i>size</i>	Specifies the number of bits.
<i>base</i>	The <i>base</i> format. The <i>base</i> can be B for binary, O for octal, and H for hexadecimal.
<i>number</i>	Can be any integer from 0 to 9, and any character from A to F . Only a subset of these characters is valid, depending on the base format. U or X represents an unknown value, and Z represents high impedance. Each character represents 1, 3 or 4 bits, depending on the base format. If the most significant bit is 0 , X , U or Z , the number is automatically extended if required to fill all the remaining bits.

The following example shows further clarification.

```
4'b1111 ; same as bit-wise 1111
12'hABx ; same as bit-wise 1010 1011 xxxx
16'bZ   ; same as bit-wise zzzz zzzz zzzz zzzz
8'h1    ; same as bit-wise 0000 0001
```

Vector Pattern Definition Section

The vector pattern definition section must occur first in a VEC file. This section describes how to define vectors names, sizes, direction, sequence, and order for each vector stimulus. The statements within this section (except the `radix`, `vname` and `mask` statements) can appear in any order. If the same statement (except the `radix` statement) of a vector is specified more than once in a VEC file, the last statement overrides the previous ones and a warning message is issued.

The following statements are described in this section:

- [radix](#)
- [vname](#)
- [io](#)
- [enable](#)
- [option CBC](#)

- [tunit](#)
- [vref](#)

For FastSPICE applications, the PrimeSim XA tool supports the following statements that are not supported in the PrimeSim HSPICE tool:

- [period and tskip](#)
- [mcheck_windows](#)
- [vchk_ignore](#)
- [stop_at_error](#)

radix

The `radix` statement must be the first non-comment line. It specifies the number of bits associated with each table vector column. The integers may be concatenated or separated by white space for clarity. The file must contain exactly one `radix` statement.

The syntax is:

```
radix #bit1 ... #bitn
```

When more than one bit is used in a column, the logic values can be represented by a hexadecimal number (see [Table 22](#)). The bit order is: most-significant to least-significant.

Table 22 *Range of radix Value*

Number of bits	Range
1	0-1
2	0-3
3	0-7
4	0-9, A-F

The following example illustrates two 1-bit vectors, followed by one 2-bit vector, and one 4-bit vector.

```
radix 11 2 4
```

vname

The `vname` statement defines the name of each vector. Each vector drives the node with the same name in the schematic netlist. If the number of bits is more than one in the `radix` statement, you can use `[starting_index:ending_index]` to specify a range of

vectors or a bus. The `vname` statement must be specified in the PrimeSim XA tool, but is optional in the PrimeSim HSPICE tool.

The syntax is:

```
vname name1 ... name_n
```

The following example defines names `va vb` for two 1-bit vectors, `vc0 vc1` for one 2-bit vector, and `vd[0] vd[1] vd[2] vd[3]` for one 4-bit vector.

```
radix 11 2 4
vname va vb vc[0:1] vd[[0:3]]
```

The following example uses `< >` as bus delimiters and `[ : ]` to define 4-bit vector, `vin<6> vin<7> vin<8> vin<9>`.

```
radix 4
vname vin<[6:9]>
```

io

The `io` statement defines the type of each vector. `I` defines input; `O` defines output; `B` defines bi-direction; and `U` defines an unused vector. There must be one definition for each column in the `radix` statement. The keywords may be concatenated or separated by white space for clarity.

The syntax is:

```
io [I|O|B|U]1 ... [I|O|B|U]n
```

All vectors default to inputs if no `io` statement is specified.

Each input vector is converted to a PWL voltage source in series with a resistor. The rise and fall times are used to smoothly transition from one state to another. See the `trise`, `tfall`, or `slope` statements in the [Waveform Characteristics Section](#) on how to define the rise time and fall time. The valid states for an input signal are shown in [Table 23](#).

Table 23 Valid States for Input Vector

State	Voltage Value	Resistance
0, U	Set to value specified in <code>VIL</code> .	0
x	Default to value specified in <code>VIL</code> . See <code>set_vector_option</code> for information on how to change the value.	
1	Set to value specified in <code>VIH</code> .	0
z	Floating.	Set to value specified in <code>TRIZ</code> .

Table 23 Valid States for Input Vector (Continued)

State	Voltage Value	Resistance
L	Set to value specified in <code>VIL</code> .	Set to value specified in <code>OUTZ</code> or <code>OUT</code> .
H	Set to value specified in <code>VIH</code> .	Set to value specified in <code>OUTZ</code> or <code>OUT</code> .

Each output vector is converted to an expected output checking statement. During simulation, the actual voltages are converted to a logic state and compared to the expected output checking statement at the specified times. A warning is generated if the logic states differ. See the `VTH`, `VOL`, or `VOH` statements in the [Waveform Characteristics Section](#) for details about how to define voltage threshold for logic-high and logic-low for output vector. The valid states for expected outputs are shown in [Table 24](#).

Table 24 Valid States for Output Vector

State	Description
0, L	Expected simulation output voltage less than the value specified in <code>VOL</code> . Otherwise, a warning is issued.
1, H	Expected simulation output voltage greater than the value specified in <code>VOH</code> . Otherwise, a warning is issued.
x	"Don't care" or ignored.
U, Z	Default to "don't care" or Ignored. The behavior for output checking can be changed with <code>set_vector_option</code> command.

Every bidirectional vector needs the `enable` or `option CBC` statement to control the direction of the vector. These statements specify the condition that controls the bidirectional vector to behave as input or output for the specified condition. See the `enable` statement or the `option CBC` statement for details about how to define the direction of the bidirectional vector. If you do not specify an `enable` or `option CBC` statement, the PrimeSim XA tool sets all bidirectional vectors as input vectors, while the PrimeSim HSPICE tool errors out.

The following example specifies `va` and `vc[0:1]` as inputs, `vb` as bidirectional, and `vd[[0:3]]` as outputs.

```
radix 11 2 4
vname va vb vc[0:1] vd[[0:3]]
io i b i o
```

enable

Every bidirectional vector needs a control signal. The `enable` statement specifies the control signal for a bidirectional vector. The control signal ensures the bidirectional vector behaves as an input or output for the specified condition.

The syntax is:

```
enable control_signal [mask]
```

Argument	Description
<code>control_signal</code>	Must be an input vector with a radix of 1. The bidirectional vector becomes output when the <code>control_signal</code> is at logic state 1 or H. To reverse the default control logic, start the <code>control_signal</code> with a tilde (~) character.
<code>mask</code>	Applies the <code>enable</code> statement only for selected signals. The <code>control_signal</code> applies to all bidirectional vectors. For more information about this option, see the Specifying Masking Patterns section.

The following example indicates that the bidirectional vector `vb` becomes output when `va` is high and the bidirectional vector `vc[0:1]` becomes output when `va` is low. The `enable` statement does not apply to the input or output vector.

```
radix 11 2 4
vname va vb vc[0:1] vd[[0:3]]
io i b b o
enable va 0 1 0 0
enable ~va 0 0 3 0
```

option CBC

The context-based control (CBC) option specifies the direction of bidirectional vectors based on the characters used in the tabular data section. If the value is 0, 1, or Z, a bidirectional vector is an input; if the value is H, L, U, or X, a bidirectional vector is an output. The syntax is:

```
option cbc
```

tunit

The `tunit` statement defines the time and unit for a time-related statement and absolute time in the tabular data section of the PrimeSim HSPICE VEC file. See [Table 25](#) for the keywords that specify the units.

The syntax is:

```
tunit time unit
```

If *time* is not specified, the default is 1. If *unit* is not specified, the default is s. If the *tunit* statement is not specified, the default is 1ns.

Table 25 Supported Units for the tunit Statement

Unit	Description
fs	Femtosecond
ps	Picosecond
ns	Nanosecond
us	Microsecond
ms	Millisecond

vref

The *vref* statement specifies the reference voltage name for each input or bidirectional vector when in input mode.

The syntax is:

```
vref ref_vname [mask_argument]
```

Argument	Description
<i>ref_vname</i>	Name of the reference voltage for each input signal. The default is GND, GROUND or 0.
<i>mask_argument</i>	Enables the <i>vref</i> only for selected vectors. For detailed syntax descriptions, see the Specifying Masking Patterns section.

The following example specifies *vss* as the reference node for *va* and *vb*, and *vc*[0:1].

```
radix 11 2 4
vname va vb vc[0:1] vd[[0:3]]
io i b b o
vref vss
```

period and tskip

The `period` statement specifies the time interval for the tabular data section, so each individual time does not need to be specified at the beginning of each row. If you use the `period` statement without the `tskip` statement, the tabular data section only contains the vector states and not the absolute times.

The time interval specified by the `period` statement is multiplied by the value specified by the `tunit` statement. If the tabular data section contains the absolute times, you can use `tskip` statement to ignore the absolute times specified at the beginning of each row in the tabular data section.

The `period` syntax is:

```
period t_interval
```

The `tskip` syntax is:

```
tskip
```

The following example shows that the first row of tabular data (11 3 F) is at time 0 ns, the second row (10 1 8) is at 10 ns, and the third row (01 2 9) is at 20 ns.

```
radix 11 2 4
vname va vb vc[0:1] vd[[0:3]]
io i b b o
period 10
11 3 F
10 1 8
01 2 9
```

In the following example, the first column of the tabular data is ignored because the `tskip` statement is specified, so the `period` statement is used to specify the time interval for the tabular data section.

```
radix 11 2 4
vname va vb vc[0:1] vd[[0:3]]
io i b b o
period 10
tskip
0.0 11 3 F
11.0 10 1 8 15.0 01 2 9
```

mcheck_windows

The `mcheck_windows` statement defines the time windows for output vector checking. You can specify multiple output vector checking windows in one command or multiple commands in a VEC file. This command compares the actual simulation outputs and vector outputs only at the specified time windows.

The syntax is:

```
mcheck_windows start_offset=start_offset_value
  stop_offset=stop_offset_value steady=0|1 [period_time=period_value]
[twindow= start_time {,stop_time ,start_time} [,stop_time]]
[mask_argument]
```

Argument	Description
start_offset=start_offset_value stop_offset=stop_offset_value	Specifies the offset time for the starting time and stopping time for output vector checking. These two arguments define a time window as $[(t - \text{start_offset_val}), (t + \text{stop_offset_val})]$ in which t is the vector strobe time for output vector checking. These arguments are the mandatory. The default time unit is ns (nanosecond).
steady=0 1	Specifies the criteria for output vector checking to pass. If <code>steady=0</code> , the output vector checking passes as long as the output logic state reaches the expected state at any time within the time window. If <code>steady=1</code> , the output vector checking passes if the output logic state matches the expected state throughout the time window. It is also a mandatory argument.
period_time=period_value	Specifies the strobe time for output vector checking. The default if this argument is the output vector checking performed at the strobe time specified in the PrimeSim HSPICE VEC file. When <code>period_val</code> is specified, the starting time is based on the <code>start_t</code> defined in <code>twindow</code> argument.
twindow= start_time {,stop_time ,start_time} [,stop_time]	Specifies the time window for output vector checking. You can specify multiple <code>start_t</code> and <code>stop_t</code> to define multiple time windows. The default if the <code>twindow</code> argument is not specified is the <code>start_t</code> and <code>stop_t</code> is 0ns and the end of the transient time. If <code>twindow</code> argument is specified, at least one <code>start_t</code> is needed.
mask_argument	Enables <code>mcheck_windows</code> only for selected vectors. For detailed syntax descriptions, see the Specifying Masking Patterns section.

The following example shows how the `mcheck_windows` statement is used for output checking. The `period=10` argument specifies the strobe time to be 10ns starting at 120ns specified by `twindow=120` argument. The time windows for output checking of `vd[1]` is from 118 ns to 122 ns, 128 ns to 132 ns, and so on. The `steady=1` argument indicates that the output checking passes only if the output state matches the expected state throughout the time window.


```
radix 11 2 4
vname va vb vc[0:1] vd[[0:3]]
io i i i o
mcheck_windows start_offset=2 stop_offset=2
+ steady=1 period=10 twindow=120
0.0 0 0 0 X
100 0 1 3 F
110 1 1 1 0
120 0 1 0 0
...
```

vchk_ignore

The `vchk_ignore` statement ignores the output vector checking at a time window.

```
vchk_ignore start_t stop_t [mask_argument]
```

Argument	Description
<code>start_t</code>	Specifies the starting time of a time window to ignore output vector checking.
<code>stop_t</code>	Specifies the stopping time of a time window to ignore output vector checking.
<code>mask_argument</code>	Enables the <code>vchk_ignore</code> command only for selected vectors. For detailed syntax descriptions, see the Specifying Masking Patterns section.

stop_at_error

The `stop_at_error` statement stops the simulation whenever output vector checking fails. It means the real simulation outputs and vector outputs during the defined time windows for masking vectors do not match.

The syntax is:

```
stop_at_error
```

Waveform Characteristics Section

The waveform characteristics section defines various attributes for vectors. [Table 26](#) lists all the PrimeSim XA-supported attribute statements. If more than one type of attribute statement is applied to a particular vector, the last statement takes precedence. A warning message is generated if there is a conflict.

The syntax for all attribute statements is:

```
KEYWORD value [mask_argument]
```

Argument	Description
<i>KEYWORD</i>	Can be any keyword listed in Table 26 .
<i>value</i>	Is the value expression applied to the <i>KEYWORD</i> . The specified value expression is multiplied by time and unit specified in the <i>tunit</i> statement if it is a time-related statement.
<i>mask_argument</i>	Enables <i>KEYWORD</i> only for selected vectors. If not specified, <i>value</i> applies to all signals. For more information about the syntax for this option, see the Specifying Masking Patterns section.

Table 26 Supported Attribute Statements for Vector

Keyword	Description
<i>idelay, idelay_r, idelay_f</i>	Specifies the delay time only for input vectors, relative to the absolute time of each row in the tabular data section. The delay time can be positive or negative. The default is set to the value specified in the <i>TDELAY</i> statement. The <i>IDELAY_R</i> and <i>IDELAY_F</i> commands specify the rising and falling inputs, respectively, for any given input vectors.
<i>odelay</i>	Specifies the delay time only for output vectors, relative to the absolute time of each row in the tabular data section. The delay time can be positive or negative. The default is set to the value specified in the <i>TDELAY</i> statement.
<i>out outz</i>	Specifies the resistance of the voltage source when the input or bidirectional vector when in input mode is driving in an <i>L</i> or <i>H</i> state. The value must be greater than or equal to zero. The default value is 0.
<i>rin</i>	Specifies the resistance of a given input signal in a 0 or 1 state. This resistance has no effect for an input signal in an HiZ state.
<i>slope</i>	Defines the rise and fall times for the input vectors to transition from logic-low voltage (specified in <i>vil</i>) to logic-high voltage (specified in <i>vih</i>) or from logic-high voltage (specified in <i>vih</i>) to logic-low voltage (specified in <i>vil</i>). The value must be greater than zero. The default is 0.1.

Table 26 *Supported Attribute Statements for Vector (Continued)*

Keyword	Description
<code>tdelay</code>	Specifies the delay time for input and output vectors relative to the absolute time of each row in the tabular data section. The delay time can be positive or negative. The default is 0.
<code>tfall</code>	Specifies the fall time for input vectors to transition from high-voltage (specified in <code>vil</code>) to low-voltage (specified in <code>vih</code>). The value must be greater than zero, and less than one-half of the smallest time period (which is the time interval of each row in the tabular data section). The fall time is set to one-half of the smallest time period, if it is larger than this value. The default is set to the value specified in the <code>SLOPE</code> statement.
<code>trise</code>	Specifies the rise time for input vectors to transition from low-voltage (specified in <code>vil</code>) to high-voltage (specified in <code>vih</code>). The value must be greater than zero, and less than one-half of the smallest time period (which is the time interval of each row in the tabular data section). The rise time is set to one-half of the smallest time period, if larger than this value. The default is set to the value specified in the <code>SLOPE</code> statement.
<code>triz</code>	Specifies the resistance of the voltage source when an input or bidirectional vector in input mode is floating in a high impedance state (<code>z</code>). The value must be greater than or equal to zero. The default value is set to 1e9 ohms.
<code>vih</code>	Specifies the logic-high voltage for input or bidirectional vectors when in input mode. The default value is set to 3.3 volts.
<code>vil</code>	Specifies the logic-low voltage for input or bidirectional vectors when in input mode. The default value is set to 0.0 volts.
<code>vth</code>	Specifies the voltage threshold of logic-high state and logic-low state for output or bidirectional vectors when in output mode. It is only used for output vector checking. A voltage value is considered a logic-high state if it is greater than or equal to the specified voltage threshold. Otherwise, it is a logic-low state. The default value is set to 1.65 volts.
<code>voh</code>	Specifies the voltage threshold for logic-high state for output or bidirectional vectors when in output mode. Used for output checking. The default value is set to the value specified in the <code>vth</code> statement.

Table 26 Supported Attribute Statements for Vector (Continued)

Keyword	Description
<code>vol</code>	Specified the voltage threshold for logic-low state for output or bidirectional vectors when in output mode. Used for output checking. The default value is set to the value specified in the <code>vth</code> statement.

The next example specifies the following attributes:

- The first `tdelay` statement indicates that all signals have the same delay time of 1ns; however, the second `tdelay` statement overrides the delay time for `va` as -1.2 ns.
- The default logic-high voltage for all input vectors is 3.3 volts. The `vih` statement overrides the logic-high voltage for `vc0` and `vc1` to 3.0 volts.
- The logic-low voltage for all input vectors, `va`, `vc[0]`, and `vc[1]` and the bidirectional vector, `vb` when in input mode is 0.0 volts.
- The input `va` signal has a resistance of 0.8 ohm.

```
radix 1 1 2 4
vname va vb vc[0:1] vd[[0:3]]
io i b i o
tdelay 1.0
tdelay -1.2 mask_by_bitmap=1 0 0 0
vih 3.0 mask_by_bitmap=0 0 3 0
vil 0.0
rin 0.8 mask_by_name=va
```

Controlling Simulation End Time Using autostop Argument

You can use the `autostop` argument in the `.VEC` statement to control the end time of the simulation when using the vector file as an input. This argument allows you to choose between using the simulation end time from the `.tran` statement or the last specified time point in the vector file as the end time.

To use the `autostop` argument in the `.VEC` statement, use this syntax:

```
.vec "vector_filename" [autostop=true|false]
```

To use the `autostop` argument in the `vec_include` or `vcd_include` statements, use this syntax:

```
vec_include "vector_filename" [autostop=yes|no]
```

or

```
vcd_include "vector_filename" [autostop=yes|no]
```

By default (when set to `false|no`), the PrimeSim XA uses the end time specified in the `.tran` statement. If you set it `true|yes`, the tool uses the last specified time point in the vector file as the end time.

When multiple `.vec` files are specified and any of them contain `autostop=true`, the simulator selects the longest time point available in the `.vec` files and uses it as the end time.

Note:

The `autostop` argument can also be used when loading `.vcd` and `.evcd` files.

Argument	Description
<code>autostop true false yes no</code>	Controls the end time of the simulation when using the vector file as an input. <code>true yes</code>: Uses the last specified time point in the vector file as the end time. <code>false no</code> (default): Uses the end time specified in the <code>.tran</code> statement.

Specifying Masking Patterns

Masking helps to change the globally specified values or checks of certain vectors to locally specified values or checks in the specified statements. You can select the vector through bitmaps and vector names. You can also predefine the masking pattern to be used in the specified statements using the `mask` statement for masking by:

- Bitmaps (compatible with the PrimeSim HSPICE tool)
- Vector names
- Using the `mask` statement

Note:

The PrimeSim HSPICE tool only supports masking by bitmaps.

Masking by Bitmaps

The syntax for `mask_argument` using masking by bitmaps is:

```
[mask_by_bitmap=]vector_bitmap
```

where `vector_bitmap` is a vector array. A value of 1 means the command is applied to the vector in corresponding column and value 0 means it is not. You can also specify the `vector_bitmap` in hexadecimal format.

Note:

The PrimeSim HSPICE tool does not support the `mask_by_bitmap` keyword.

The following example defines the logic-high voltage for:

- The `vina` vector: 3.0 volts (globally defined value)
- The `vinb` vector: 2.0 volts
- The `vinc0` and `vinc1` vectors: 1.5 volts
- The `vind[0]`, `vind[1]`, `vind[2]` and `vind[3]` vectors: 1.0 volts

```
io i i i i
vname vina vinb vinc[0:1] vind[[0:3]]
vih 3.0
vih 2.0 0 1 0 0
vih 1.5 0 0 3 0
vih 1.0 mask_by_bitmap=0 0 0 F
```

Masking by Vector Names

The syntax for *mask_argument* using masking by vector names is:

```
mask_by_name=vector_name1[... , vector_namen]
```

The following example defines the rise time for:

- The `vind[1]`, `vind[2]` and `vind[3]` vectors: 1 ns (globally defined value)
- The `vina`, `vinc0` and `vinc1` vectors: 2 ns
- The `vinb` and `vind[0]` vectors: 3 ns

```
io i i i i
vname vina vinb vinc[0:1] vind[[0:3]]
tunit 1ns
trise 1
trise 2 mask_by_name=vina, vinc[0:1]
trise 3 mask_by_name=vinb, vind[0]
```

Masking by the Mask Statements

You can predefine a masking pattern, either by bitmaps or vector names, using the `mask` statement. The `mask` statement must be specified before it is called by other statements, so the order is important.

The syntax for the `mask` statement when masking by bitmaps is:

```
mask mask_name bitmap_pattern
```

The syntax for the `mask` statement when masking by vector names is:

```
mask mask_name vector_name_pattern
```

The predefined masking pattern can be applied to the *mask_argument* as:

```
mask_by_name=mask_name1[... , mask_name]
```

The following example defines the logic-high voltage:

- The `vind[0]`, `vind[1]`, `vind[2]` and `vind[3]` vectors: 3.0 volts (globally defined value)
- The `vina` vector: 2.0 volts
- The `vinb`, `vinc0` and `vinc1` vectors: 1 volt

```
io i i i i
vname vina vinb vinc[0:1] vind[[0:3]]
mask m1 1 1 0 0
mask m2 vinb vinc[0:1]
vih 3.0
vih 2.0 mask_by_name=m1
vih 1.0 mask_by_name=m2
```

Setting Time-Dependent Logic Voltage Thresholds

In most cases, logic voltages or threshold voltages, such as `VIH`, `VIL`, `VOH` and `VOL`, are set constant from the start of the simulation to the end in a VEC file. To define a time-dependent voltage threshold for logics, specify the `tvec` argument in the following parameter format:

```
tvec=<(time_point,threshold_value)>
```

You can use the `tvec` argument to specify time-dependent voltage threshold with either of the following commands:

- `VIH`: Specifies logic-high voltage for each input signal
- `VIL`: Specifies logic-low voltage for each input signal
- `VOH`: Specifies logic-high threshold voltage for each output signal
- `VOL`: Specifies logic-low threshold voltage for each output signal
- `VTH`: Specifies logic threshold voltage for each output signal

For a detailed description about the `VIH`, `VIL`, `VOH`, `VOL` and `VTH` commands, see their command pages in the *PrimeSim™ Continuum Reference Manual: Commands and Control Options*.

For input thresholds, the affected signals change according to the given time points. When a new threshold value applied, signal transition follows `SLOPE` or `TRISE`/`TFALL` defined for

the signal. The definition of `TDELAY` is also applied to the threshold modification with `tvec` argument.

For output signals, voltages are digitized and checked against the expected values in the vector file. The time-dependent output logic thresholds have the effect on digitization and violation check in the time windows specified by the `tvec` argument.

Syntax Example

The following syntax shows how to specify time-dependent logic-high voltages for input signals using the `VIH` command:

```
VIH <voltage> [<mask>] [tvec=(<t1,v1> [,<t2,v2>, ..., <tn,vn>])]
```

where

- `<voltage>` specifies the default logic-high voltage. When the `tvec` argument is specified, this default value is used for the duration that is not defined by the `tvec` argument.
- `<mask>` specifies the name of a signal to which the `VIH` command applies. When not specified, the `VIH` command applies to all input signals.
- `tvec` defines time-dependent logic threshold in the format of (time_point, threshold_value). When defining a long sequence of values, separate time points and threshold values by a space or a comma (","). The pair of time point and threshold value must be placed inside the parenthesis.
 - `<t1, ..., tn>`: Time points, in the time unit defined in the vector file. When specified, time points must be listed in ascending order and be aligned with the time points in the tabular data section in the vector file. Otherwise, an error message will be issued.
 - `<v1, ..., vn>`: Threshold values at the specified time points.

Note:

- The time points must not exceed the end time of simulation (`tstop`) in the `.tran` statement in the netlist.
- The command `set_vector_char` in Tcl configuration file overwrites the corresponding logic voltage thresholds in the vector file. A warning message is issued when an overriding happens.

Examples

Example 1:

The following `VIH` statement defines the logic-high voltage that changes with time. The voltage value is 2.5 at the time point 0, and 2.8 at time point 12000, and so on. In time point 72000, the voltage is 1.8 and stays at 1.8 to the end of the simulation.


```
VIH 3.0 tvec=(0,2.5,12000,2.8,13000,3.0,15000,3.3,72000,1.8)
```

Example 2:

The following `VIH` statement uses the `tvec` argument to define the time-dependent voltages starting from time point 12000. The `VIH` command sets logic-high voltage value to 3.0 which is the default value to be used from time point 0. The logic-high voltage is 3.0 at the time point 0, and 2.8 at time point 12000, and so on.

```
VIH 3.0 tvec=(12000,2.8,13000,3.0,15000,3.3,72000,1.8)
```

Example 3:

In the following example, the first `VIH` command sets logic-high voltage to 3.0 for signals that match the mask 11110. The second `VIH` command sets the same time-dependent voltages as in Example 2, but only applies on the signals that match the mask 00001. Other signals are not time-dependent in the `VIH` statement.

```
VIH 3.0 11110  
VIH 3.0 00001 tvec=(12000,2.8,13000,3.0,15000,3.3,72000,1.8)
```

Example 4:

The following `VIH` statement shows that the time point defined in the `tvec` argument is not aligned with the tabular data. The time point of 14000 in the `tvec` argument does not appear in tabular data. The tool issues an error message before simulation starts.

```
VIH 3.0 tvec=(12000,2.8,14000,3.0,15000,3.3)
```

```
; Vector tabular data section  
0 000000000LLLLL  
12000 001000000LLLLL  
13000 001100000LLLLL  
15000 001110000LLLLL  
; end of tabular data section
```

Using Parameterized Vector Files

The PrimeSim XA tool supports the use of parameterized vector file for circuit simulation. The parameterized data in the vector file can be evaluated from the top-level parameters in the netlist files. It enables the customization for the vector file with parameters, such as configuration or sweeping the tabular data. The expression of parameterized data in the vector file should be correctly quoted with single or double quotes.

Only the data of scalar type can be parameterized. The signal pattern or bit mask cannot be parameterized.

Following is an example of parameterized vector file:

```
radix 1111111111111111
vname B[[3:0]] A[[3:0]] CIN COUT S[[3:00]]
io iiiiiiiiiiiooooo

; tdelay refers to ".param vec_offset=2" in the netlist
tdelay "vec_offset"
vih 1.8 111111111111100
vil 0.0
voh 1.5
vol 0.5
slope 0.01
tunit ns

; Vector tabular data section
; refer to ".param param_time=10" in the netlist
0      0000000000LLLLL
"param_time" 0000000000LLLLL
"param_time+2" 0000000000LLLLL
"param_time*2" 0001000000LLLLL
"param_time*2+2" 0001000000LLLLH
"param_time*3" 0010000000LLLLH
"param_time*3+2" 0010000000LLLHL
```

In the above example, `tdelay` in the vector file is evaluated as 2 from the top-level parameter `vec_offset`. The time values of tabular data are evaluated as 0, 10, 12, 20, 22, 30, 32 and so on.

Using a Value Change Dump File

A Value Change Dump (VCD) file is an open industry-standard ASCII format file. A VCD file is generated by a test pattern generator, RTL simulator, or other high-level simulators.

A VCD file contains three sections:

- The *header information* section describes the date, the version number of the simulator, and the timescale used.
- The *variable definitions* section contains the scope of the hierarchy, and type of variables. A variable can be a scalar or a bus. Each variable is represented by a unique identifier character.
- The *value changes* section contains the actual value changes for all variables specified at each simulation time increment. Only the variables, which change during a time increment, are listed. Each variable with a set of values forms a vector over time.

The following shows an example VCD file.

```

$date
  <date>
$end
$version
  VERILOG.XL 1.6b
$end
$timescale
  1ns
$end

$scope module      adder      $end
$var register      1 !      CIN      $end
$var register      4 "      A  [3:0]  $end
$var register      4 #      B  [3:0]  $end
$var wire           1 $      COUT     $end
$var wire           4 %      S  [3:0]  $end
$upscope           $end

$enddefinitions    $end
$dumpvars
0!
b0000      *
b0000      #
0$
b000X      %
$end
#10
b0000      #
b0000      *
0!
#12
b000X      %
0$
#20
b0001      #
#22
b000X      %
#30
b0010      #
#32
b001X      %

```

Reading a VCD File

The PrimeSim XA tool reads in the VCD file using the `load_vector_file` command. In order to use the variables as vectors, you must provide a VCD control file. A VCD control file maps the variable names in the VCD file to the node names in the schematic netlist. All variable names in the VCD control file are case-sensitive, as in the VCD file. You can also define various attributes for the vectors. If more than one attribute statement of a

certain type is applied to a particular vector, the last one takes precedence. A warning is generated if a conflict arises.

The following example uses the `load_vector_file` command to load a VCD file to the PrimeSim XA tool, named `adder.vcd`, and a VCD control file, named `ctl.adder.sig`.

```
load_vector_file -file adder.vcd -format vcd -ctl ctl.adder.sig
```

Specifying a VCD Control File

Use the following procedure to create a VCD control file that maps the variables in the VCD file to the node names in the schematic netlist.

1. Define the bus notation used in the schematic netlist using the `#format` statement.

For more information, see [Defining Bus Notation Using the #format Statement](#).

2. Define the variables to be used as vectors and the type of vector for each variable. The `#input` statement defines the input vectors; the `#output` statement defines the output vectors; and the `#bidirectional` statement defines the bidirectional vectors.

For more information, see [Defining Vectors and Types Using the #input, #output and #bidirectional Statements](#).

3. Map the variable names defined in the VCD control file to the node names in the schematic netlist using the `#alias` statement.

For more information, see [Mapping Variables in VCD File to Nodes in the Schematic Using #alias Statement](#).

4. Define various attributes for vectors.

For more information, see [Defining Attributes for Vectors](#).

5. Define the "don't care" window, where the output vector checking is not required.

For more information, see [Defining the "Don't Care" Windows Using #ignorewindow](#).

Defining Bus Notation Using the #format Statement

The `#format` statement defines the bus notation used in the schematic netlist. Only one `#format` statement is allowed in a VCD control file. If there are multiple bus notations used in the schematic netlist, you need to create multiple VCD control files with one `#format` statement for each file. The syntax is:

```
#format|#fo bus_notation
```

The default is `%[#]`.

Table 27 Character Descriptions for #format Statement

Character	Description
[]	Is an example of bus delimiter. Other bus delimiters are also supported. You can use different characters to represent the bus delimiter, such as <> or {}. It needs to match the bus notation used in the schematic netlist.
%	Represents the signal name.
#	Represents the bus indexes.

Table 28 shows an example of the mapping of variable name `tA` using the `#format` statement to various bus notation in the schematic netlist.

Table 28 Variable Mapping Example

Statement	Name Mapping in the Schematic Netlist
<code>#format %[#]</code>	<code>tA[0] tA[1] tA[2] ...</code>
<code>#format %<#></code>	<code>tA<0> tA<1> tA<2> ...</code>
<code>#format %.#</code>	<code>tA.0 tA.1 tA.2 ...</code>

Defining Vectors and Types Using the #input, #output and #bidirectional Statements

The `#input`, `#output` and `#bidirectional` statements define the variables in the VCD file as input vectors, output vectors, or bidirectional vectors, respectively. Other variables in the VCD file not defined with these statements are ignored.

You must specify a full hierarchical name in the VCD control file to map with the variables in the VCD file. If all variables in the VCD reside in one scope, use the `#scope` statement to specify the scope name in the VCD file, so a full hierarchical name is not needed. For more information, see [Defining the Hierarchical Scope Using #scope](#).

The PrimeSim XA tool uses input vectors specified with the `#input` as stimulus for the simulation. Each input vector is converted to a PWL voltage source, in series with a resistor. Table 29 shows the four valid states for input vector supported in the VCD file.

Table 29 Valid States

State	Input Vector	Resistance
0	Set to value specified in <code>#vil</code> .	0

Table 29 Valid States (Continued)

State	Input Vector	Resistance
1	Set to value specified in #vih.	0
x	Floating to High Impedance.	Set to value specified in #triz.
z	Set to value specified in #vil. See set_vector_option for information on how to change the value.	Set to value specified in #outz.

The syntax is:

```
#input|#in input_name1 [... input_namen]
```

Every output vector is converted to the expected output checking statement. During simulation, the actual simulation outputs are converted to digital states and compared to the expected output checking statements. A warning is generated if the states differ. Table 30 shows the four valid states for output vectors supported in the VCD file.

Table 30 Valid States for Output Vectors

State	Description
0	Expect simulation output voltage less than the value specified in #vol. Otherwise, a warning is generated.
1	Expect simulation output voltage greater than the value specified in #voh. Otherwise, a warning is generated.
x	"Don't care" or ignored.
z	Default to "Don't care" or Ignored. The behavior can be changed with the set_vector_option command.

The syntax is:

```
#output|#out output_name1 [... output_namen]
```

Every bidirectional vector requires the control signal to determine the direction of the bidirectional vector as input vector or output vector. The syntax is:

```
#bidirectional|#bi bi_name1 [... bi_namen] ctrl_argument
```

Argument	Description
input_name1 ... input_namen	Specifies the input vector names.

Argument	Description
<code>output_name1 ... output_namen</code>	Specifies the output vector names.
<code>bi_name1 ... bi_namen</code>	Specifies the bidirectional vector names.
<code>ctrl_argument</code>	Is the control condition for bidirectional vectors. See the Specifying the Control Condition for Bidirectional Vectors section.

Specifying the Control Condition for Bidirectional Vectors

The control condition determines the direction of the bidirectional vectors. The syntax of `ctrl_argument` is:

```
in out|condition
```

The `in` defines as input vector and the `out` defines as output vector when the condition is evaluated `true`. The condition can be a logic expression of one or multiple input vectors, output vectors, or internal signals. You can also use the operators in [Table 31](#) to define the logic expression for the condition; wildcard characters are not supported. Although the logic expression has no length limit, it is recommended not to split the logic expression across lines.

Table 31 Supported Operators for Logic Expression

Expression Operator	Description
()	Braces
~, !	NOT
&, &&	AND
^	Exclusive OR
,	Inclusive OR

Precedence is in the order shown in [Table 31](#), from highest to lowest. When specifying the logic expression, you must use a white space to separate operators and vector names. The PrimeSim XA tool may interpret `(a|b)` as a vector name, whereas `(a|b)` is interpreted as a logic expression. The following example specifies that the input vectors are `adder`, `en`, and `m_data` bus:

```
#in adder en m_data*
```

The following example specifies `adder`, `en` and `m_data` bus as input vectors; `c` as output vector; and `dq` as bidirectional vector. Bidirectional vector `dq` becomes input when both `en`

and `c` are logic-low state; otherwise, it is output. It is recommended to use a white space to separate the operators and the vector names when specifying the control condition for bidirectional vectors.

```
#input adder en m_data*
#output c
#bidirectional dq (in ~ (en & c))
```

Defining the Hierarchical Scope Using #scope

By default, you must specify a full hierarchical name in the VCD control file to map to the variables in the VCD file. If all the variables used for vectors reside in one scope in the VCD file, use the `#scope` statement to specify the scope name and avoid specifying the full hierarchical name for the variables. Only one `#scope` statement is allowed in one VCD control file. If multiple scopes are used in the VCD file, you must create multiple VCD control files with only one `#scope` statement for each file. The syntax is:

```
#scope scope_name
```

The `scope_name` is the hierarchical scope name to the variables used in the VCD file.

The following example shows how you can use the `#scope` statement in the VCD control file to map the variable names in the VCD file. It shows the section of VCD file with the `$scope` statement:

```
$scope module adder                                $end
$var register 1 !      CIN                          $end
$var register 4 "      A [3:0]                      $end
$var register 4 #      B [3:0]                      $end
$var wire      1 $      COUT                        $end
$var wire      4 %      S [3:0]                    $end
$upscope
```

To map the variables in the VCD file to the schematic netlist using the full hierarchical names, specify the following in the VCD control file:

```
#input adder.CIN
#output adder.S[[3:0]]
```

To map the variables in the VCD file to the schematic netlist with the `#scope` statement, specify the following in the VCD control file:

```
#scope adder
#input CIN
#output S[[3:0]]
```

Mapping Variables in VCD File to Nodes in the Schematic Using #alias Statement

The `#alias` statement maps the variable names in the VCD files to the node names used in the schematic netlist. Usually the `#alias` statement is not needed if the variable names

in the VCD file match the node names in the schematic netlist. You must use the `#alias` statement when:

- The variable names in the VCD file do not match the node names in the schematic netlist. Note that VCD file and VCD control file are case-sensitive.
- There are different hierarchy levels between the variable name in the VCD file and the node names in the schematic netlist.

The syntax is

```
#alias VCD_variable_name schematic_netlist_node_name
```

The *VCD_variable_name* is the variable name in the VCD file. You must specify the full hierarchical variable name unless you use the `#scope` statement. The *schematic_netlist_node_name* is the node name in the schematic netlist. If the node name is not a top-level node name, a full hierarchical node name is needed. You can use the `%` character to represent the variable name.

In the following example, the variables are under the `adder` scope. Here is a section of the VCD file with the `#scope` statement:

```
$scope module adder
$var register 1 ! CIN $end
$var register 4 " AI [3:0] $end
$var register 4 # BI [3:0] $end
$var wire 1 $ COUT $end
$var wire 4 % SO [3:0] $end
$upscope $end
```

Assuming the following match is needed:

Variable Name in VCD	Node Name in Schematic Netlist
CIN	CIN_INPUT
AI	AINPUT
BI	BINPUT
COUT	COUT_OUTPUT
SO	SOUTPUT

To map the variable names in the VCD file to the node names in the schematic netlist, specify the following:

```
#input CIN
#input AI*
#input BI*
```

```
#output COUT
#output SO*
#alias adder.CIN CIN_INPUT
#alias adder.AI[0] AINPUT[0]
#alias adder.AI[1] AINPUT[1]
#alias adder.AI[2] AINPUT[2]
#alias adder.AI[3] AINPUT[3]
#alias adder.BI[0] BINPUT[0]
#alias adder.BI[1] BINPUT[1]
#alias adder.BI[2] BINPUT[2]
#alias adder.BI[3] BINPUT[3]
#alias adder.COUT COUT_OUTPUT
#alias adder.SO[0] SOUTPUT[0]
#alias adder.SO[1] SOUTPUT[1]
#alias adder.SO[2] SOUTPUT[2]
#alias adder.SO[3] SOUTPUT[3]
```

Defining Attributes for Vectors

The PrimeSim XA tool supports many statements to define various attributes for vectors in the VCD control file. Multiple specifications of each statement are allowed in the VCD control file. If more than one individual statement is specified to a vector, the last statement overrides the previous ones. The syntax is:

KEYWORD value [vname1 ... vnamen]

Argument	Description
<i>KEYWORD</i>	Any keyword listed in Table 32 .
<i>value</i>	Value at which the <i>KEYWORD</i> is applied.
<i>vname1 ... vnamen</i>	Vector names at which the <i>KEYWORD</i> is applied. If <i>vnamen</i> is not specified, it is applied to all vectors.

[Table 32](#) lists supported attribute statements to define for vectors in the VCD file. In cases where VIH, VIL, VOH, and VOL can be parameters, you need to define the parameter values in the netlist. For example, when you define the following parameters in the netlist,

```
.param par_input_high=1.7
.param par_output_high=1.6
.param par_input_low=0.1
.param par_output_low=0.3
```

In the signal file, you can see

```
#vih 'par_input_high'
#vil 'par_input_low'
#voh 'par_output_high'
#vol 'par_output_low'
```

Table 32 Supported Attribute Statements

Keyword	Description
#idelay	Specifies the delay time only for the input vectors. The delay can be a positive or negative number. The default is 0.
#odelay	Specifies the delay time only for the output vectors. The delay can be a positive or negative number. The default is 0.
#outz	Specifies the resistance of the voltage source when an input or bidirectional vector is driving in a high impedance state. The value must be greater than or equal to zero. The default is 10 ohms.
#tfall	Specifies the fall time for the input vector to transition from logic-high state (specified in #vih) to logic-low state (specified in #vil). The value must be greater than zero, and a unit must be specified. The default is set to $(0.5 * \$timescale)$ where \$timescale is specified in the header information section of the VCD file.
#trise	Specifies the rise time for input vectors to transition from logic-low state (specified in #vil) to logic-high state (specified in #vih). The value must be greater than zero, and a unit must be specified. The default is set to $(0.5 * \$timescale)$ where \$timescale is specified in the header information section of the VCD file.
#triz	Specifies the resistance of the voltage source when an input or a bidirectional vector is in a high impedance state or when a bidirectional vector is in an output state. The value must be greater than or equal to zero. The default value is 1e6 ohms.
#vih	Specifies the logic-high voltage for the input vectors or bidirectional vectors when in input mode. The default is 3.3 volts.
#vil	Specifies the logic-low voltage for input vectors or bidirectional vectors when in input mode. The default is 0 volts.
#voh	Specifies the voltage threshold for logic-high state for output vectors or bidirectional vectors when in output mode. Used for output checking. A voltage value is considered a logic-high state, if it is greater than or equal to the specified voltage threshold. The default is $(\#vil + ((\#vih - \#vil) * 0.75))$.

Table 32 Supported Attribute Statements (Continued)

Keyword	Description
#vol	Specifies the voltage threshold for logic-low state for output vectors or bidirectional vectors when in output mode. Used for output checking. A voltage value is considered a logic-low state, if it is less than or equal to the specified voltage threshold. The default is $(\#vil + (\#vih - \#vil) * 0.25))$.
#vref	Specifies the reference voltage name for each input or bidirectional vector when in input mode.

The following example declares the following attributes:

- Logic-high voltage for all input vectors is 1.5 volts.
- Logic-low voltage for all input vectors is 0 volt.
- Voltage threshold for logic-high state of all output vectors is 1.4 volts.
- Voltage threshold for logic-low state of all output vectors is 0.1 volts.
- Input delay for vector c is 0.05ns. Fall time for all input vectors (except for bus data) is 0.1 ns and bus data is 0.2 n.

```
#vih 1.5
#vil 0
#voh 1.4
#vol 0.1
#idelay 0.05ns c
#tfall 0.1ns
#tfall 0.2ns data*
```

Defining the "Don't Care" Windows Using #ignorewindow

The PrimeSim XA tool reports any mismatches between the actual simulation output and the logic state in the VCD file for the output vectors or bidirectional vectors in the output mode. Output vector checking is continuous when using a VCD file. There can be many false mismatch errors, especially for a slow transition period. The #ignorewindow statement can be used to specify a "Don't Care" window before and after an expected transition to avoid false mismatch warnings. The syntax is:

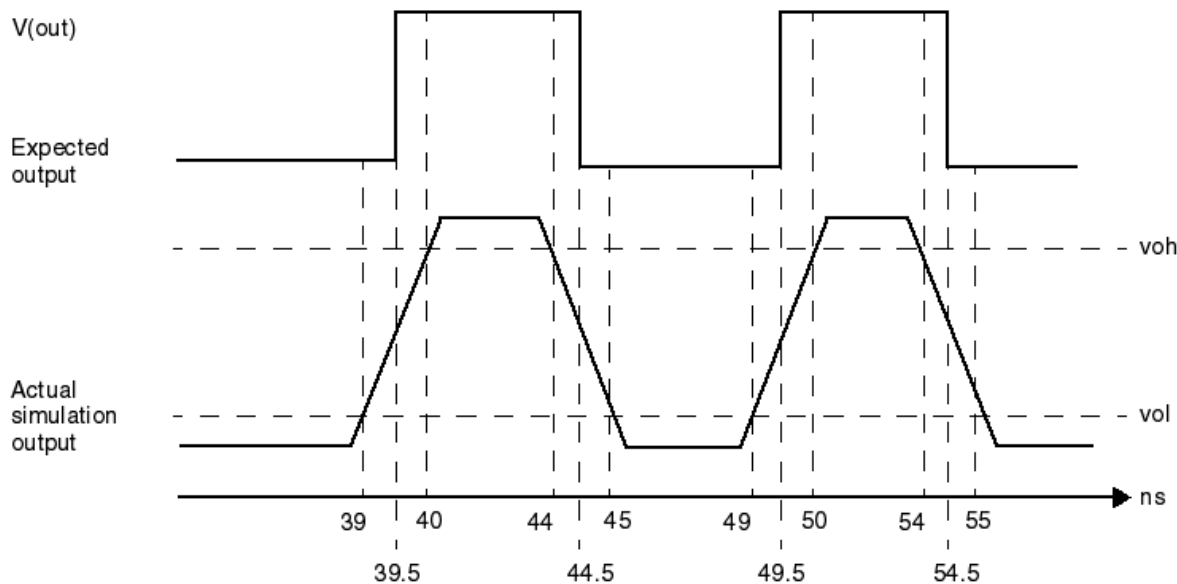
```
#ignorewindow| #iw t_bf_tran t_af_tran [vname1 ... vnamen]
```

Argument	Description
t_bf_tran	Specifies the time before the expected transition for a "Don't Care" window.

Argument	Description
<i>t_af_tran</i>	Specifies the time after the expected transition for a "Don't Care" window.
<i>vname1 ... vnamen</i>	Specifies the vector names for which the #ignorewindow is applied.

The following example shows the expected output and actual simulation output waveform.

Figure 4 *V(out) Output*



Here, if no `#ignorewindow` statement is specified in the VCD control file, the PrimeSim XA tool reports the false mismatches from 39 ns to 40 ns, 44 ns to 45 ns, 49 ns to 50 ns, and 54 ns to 55 ns. To avoid reporting these false mismatches, specify the `#ignorewindow` statement to ignore 0.5 ns before the expected transition, and 0.5 ns after the expected transition is as follows:

```
#ignorewindow 0.5ns 0.5ns out
```

Using the Extended Value Change Dump File

An extended Value Change Dump (EVCD) file is a VCD file containing directions and strengths information. The PrimeSim XA tool processes only the direction information to

determine the direction of the vector based on the state character. For more information, see the [EVCD Port Direction Rule](#) section.

Although it is optional to specify the direction information when reading in an EVCD file, you must provide waveform attribute information in the EVCD control file. The command and syntax of the EVCD control file are identical to the VCD control file. See the [Specifying a VCD Control File](#) section for details. The strength information is not used by the PrimeSim XA tool.

Reading an EVCD File

The PrimeSim XA tool reads in the EVCD file using the `load_vector_file` command. You can provide the direction information (optional) and waveform attribute information in the EVCD control file. All variable names in the EVCD control file are case-sensitive, as in the EVCD file. If more than one waveform attribute statement of a certain type is applied to a particular vector, the last one takes precedence. A warning is generated if a conflict arises.

The following example shows the command in the PrimeSim XA tool to load in an EVCD file, named `adder.evcd`, and an EVCD control file, named `ctl.adder.sig`.

```
load_vector_file -file adder.evcd -format evcd -ctl ctl.adder.sig
```

EVCD Port Direction Rule

If you do not provide the direction information, the PrimeSim XA solution follows the EVCD direction rule: the unknown direction is ignored. The default direction of each state character and its logic state are shown in [Table 33](#).

Table 33 *Default Directions of State Characters*

State Character	Default Direction	Logic State Value
D	Input	0 (Set to the value specified in #v1l)
U	Input	1 (Set to the value specified in #v1h)
N	Input	X (Default to the value specified in #v1l. See <code>set_vector_option</code> for information on how to change the value.)
z	Input	Z (Floating to High Impedance)
d	Input	0 (Set to the value specified in #v1l)
u	Input	1 (Set to the value specified in #v1h)

Table 33 *Default Directions of State Characters (Continued)*

State Character	Default Direction	Logic State Value
L	Output	1 (Check simulation outputs against value specified in #vol)
H	Output	H (Check simulation outputs against value specified in #voh)
X	Output	X (Don't care)
T	Output	Z (Default to Don't care. See <code>set_vector_option</code> for more control.)
l	Output	L (Check simulation output against value specified in #vol)
h	Output	H (Check simulation outputs against value specified in #voh)

If you provide the direction information in the EVCD control file (using the `#input`, `#output` or `#bidirectional` statement), it overrides the EVCD direction rule. See [Table 34](#) for its logic state for each valid state character when it is defined as input vector or output vector. When a bidirectional vector behaves as input, it follows the input state of each state character; when bidirectional vector behaves as output, it follows the output state of each state character.

Table 34 *Logic States for Each Valid Character*

State Character	Input Logic State	Output Logic State
D	0 (Set to value specified in #vil)	X (Don't care)
U	1 (Set to value specified in #vih)	X (Don't care)
N	X (Default to value specified in #vil. See <code>set_vector_option</code> for information on how to change the value.)	X (Don't care)
z	Z (Floating to High Impedance)	Z (Default to Don't care. See <code>set_vector_option</code> for more control.)
d	0 (Set to value specified in #vil)	X (Don't care)

Table 34 *Logic States for Each Valid Character (Continued)*

State Character	Input Logic State	Output Logic State
u	1 (Set to value specified in #vih)	X (Don't care)
L	0 (Set to value specified in #vil)	L (Check simulation output against value specified in #vol)
H	1 (Set to value specified in #vih)	H (Check simulation output against value specified in #voh)
X	X (Default to value specified in #vil See set_vector_option for more control)	X (Don't care)
T	Z (Floating to high impedance)	Z (Default to "don't care". See set_vector_option for more control.)
L	0 (Set to value specified in #vil)	L (Check simulation output against value specified in #vol)
H	1	H (Check simulation output against value specified in #voh)
O	0	L (Check simulation output against value specified in #vol)
1	1	H (Check simulation output against value specified in #voh)
?	X	X (Don't care)
F	Z	X (Don't care)
f	Z	Z (Default to "don't care". See set_vector_option for more control.)
A	0	H (Check simulation output against value specified in #voh)
a	0	X (Don't care)
B	1	L (Check simulation output against value specified in #vol)
b	1	X (Don't care)
C	X	L (Check simulation output against value specified in #vol)

Table 34 Logic States for Each Valid Character (Continued)

State Character	Input Logic State	Output Logic State
c	X	H (Check simulation output against value specified in #voh)

Following is an example of the EVCD file.

Chapter 4: Vector Stimulus Files

Using the Extended Value Change Dump File

```

$date
Mon Apr 4 14:03:50 2006
$end
$version
Chronologic Simulation VCS Release 7.1.2
$dumpports(adder "dumpports.vcd")
$end
$timescale
10ps
$end
$comment Csum: 1 bb5bd2bdf9b304b7 $end
$scope module adder $end
$var port 1 <0 ck $end
$var port [3:0] <1 a $end
$var port [3:0] <2 b $end
$var port 1 <3 cin $end
$var port [3:0] <4 s $end
$var port 1 <5 cout $end
$var port1 <6 d $end
$var port1 <7 e $end
$upscope $end
$enddefinitions $end
#0
$dumpports
pU 0 6 <0
pXXXX6666 6666 <1
pXXXX6666 6666 <2
pX 6 6 <3
pNNNN6666 6666 <4
pN 6 6 <5
pU 6 6 <6
pU 6 6 <7
$end
#500
pD 6 0 <0
pLLLL6666 0000 <1
pLLH6666 0006 <2
pL 6 0 <3
pU 6 6 <6
pD 6 6 <7
#1000
pU 0 6 <0
pLLH6666 0006 <4
pD 6 0 <5
pU 6 6 <6
pU 6 6 <7
#1500
pD 6 0 <0
pH 0 6 <3
pU 6 6 <6
pD 6 6 <7
#2000
pU 0 6 <0
pLLHL6666 0060 <4
pU 6 6 <6
pD 6 6 <7
$vcfclose #2000 $end

```

Defining the Port Direction

If you provide an EVCD control file, it overrides the EVCD direction rule. If you want to use an EVCD control file and keep the EVCD direction rules, use the `#defdir` command in the EVCD control file. The syntax is:

```
#defdir signal_list
```

The signals following `#defdir` use the EVCD direction rule. The signal list can use wildcards.

5

Probing and Measuring

The PrimeSim XA tool supports many netlist statements and simulation commands to generate both ASCII and binary waveform files and process measurements from the simulation results.

All netlist statements are supported in the PrimeSim HSPICE netlist format. Certain statements or arguments might not be supported in the Eldo netlist format. These netlist statements can also be used in the Spectre netlist format if the simulation is specified with the SPICE mode using the `simulator lang=spice` command. When you specify `simulator lang=spice`, the PrimeSim XA tool parses the statements as if they are in the PrimeSim HSPICE netlist format.

This chapter contains the following topics:

- [Probing Statements and Commands](#)
- [Measuring Statements and Commands](#)
- [PrimeSim XA Enhancements to the .MEASURE Syntax](#)
- [External Table Parameter File Interface](#)

Probing Statements and Commands

The `.PROBE`, `.PLOT` and `.PRINT` statements are native to SPICE simulators (that is, the PrimeSim HSPICE and Eldo tools). Use these statements when running PrimeSim HSPICE or Eldo netlist formats in the PrimeSim XA simulation.

The PrimeSim XA tool provides probing commands control the probing feature for all supported netlist formats. For example, you can use the `set_probe_option` command to specify criteria for probe filtering or ignore the postlayout nodes for wildcard probing. Other commands, like `probe_waveform_current` or `probe_waveform_logic`, are available to output the waveform expressions into the specified waveform format file. For details, see [Using the PrimeSim XA Probing Commands](#).

There are many different methods to probe the waveform expressions in the specified waveform format file. For more information about which method to use for specifying the output waveform format, see [Supported Waveform File Formats](#).

This section covers the following topics.

- [External Table Parameter File Interface](#)
- [Using the .PROBE or .PLOT Statements](#)
- [Using .PRINT Statements](#)
- [Using the .LPROBE and .LPRINT Statements](#)
- [Using the PrimeSim XA Probing Commands](#)

External Table Parameter File Interface

You can use the `table_param()` built-in function to specify a file that contains a look-up table containing parameter values and the PrimeSim XA tool returns the linear interpolated value from that table.

Syntax

```
.param a =  
  
table_param("table_file_name", #disc, nd1, nd2, ..., #cont, nc1, nc2...,  
value_col)
```

Note:

If #disc is out of range of given table, will be error out.

To comment out a line, add the # symbol at the beginning of the line in table file.

Argument	Description
<code>str('table_file_name')</code>	Name of file containing a parameter values table. Note: Specify the file name using the <code>str()</code> function.
<code>#disc</code>	Total number of discrete index columns.
<code>nd1, nd2, ...</code>	Discrete index column numbers.
<code>#cont</code>	Total number of continuous value columns.
<code>nc1, nc2, ...</code>	Continuous values.
<code>value_col</code>	Continuous column number used in linear interpolation.

The following examples use two-dimensional table data files, `cap.table` and `res.table`, defined as follows:

The `cap.table` file:

```
#corner layer width space Cbot Ctop Ccpl
1 3 0.0450 0.030 0.078901 0.06789 0.03456
1 3 0.0450 0.032 0.089012 0.05678 0.02109
1 3 0.0450 0.037 0.090123 0.04567 0.01098
... ..
```

The `res.table` file:

```
#corner layer width space Rtotal
1 3 0.0450 0.030 78.90123
1 3 0.0450 0.032 67.89012
1 3 0.0450 0.037 56.78901
... ..

.param cbot = table_param("./cap.table",2,corner,layer,2,width,space,1)
.param ctop = table_param("./cap.table",2,corner,layer,2,width,space,2)
.param rnom = table_param("./res.table",2,corner,layer,2,width,space,1)
.param cbot = table_param("./cap.table",2,1,1,2,0.002,0.0252,1)
```

In these examples, the resolved variables are `vddval1=0.65`, `vddval2=65`, `mytbl1=0.825`, and `mytbl2=57.5`, respectively.

Using the .PROBE or .PLOT Statements

The `.PROBE` or `.PLOT` statement is a native SPICE simulator statement for the PrimeSim HSPICE and Eldo tools. These statements write out the waveform expression in the specified output format file. Enhancements have been made to improve the usability of this netlist statement in the PrimeSim XA simulation.

The syntax is:

```
.PROBE|.PLOT
[label1 =] expression1 [ ... [labeln =] expressionn]
[subckt=subckt_name]
[except|filter=pattern]
[adonly=0|1]
```

```
[matchport=0|1]
limit=limit_value]
[level=level_value]
```

Or

```
.PROBE|.PLOT
[label1 =] PAR(expression1) [ ... [labeln =] PAR(expressionn)]
[subckt=subckt_name]
[except|filter=pattern]
[adonly=adonly_flag]
[matchport=port_flag]
[limit=limit_value]
[level=level_value]
```

Argument	Description
label1 ... labeln	Name given to the <i>expression</i> .
expression1 ... expressionn	Waveform expression, which must be a node name. See Table 35 . Note: The PrimeSim HSPICE tool requires the use of <code>PAR</code> in equation expressions, while the PrimeSim XA tool does not require the use of <code>PAR</code> . You should specify the waveform expression in the quote.
subckt=subckt_name	Performs the operation on all specified nodes or instance of the specified subcircuit. If you specify this argument, the nodes or instances must be in the subcircuit instance hierarchy level. Wildcard characters are not supported in subcircuit names.
except filter=pattern	Performs the operation on all the nodes or instances, except those of the specified pattern. You can use wildcard characters to match the node names and instances to exclude from probing.
adonly=0 1	When you specify 1, only the nodes that connect to at least one active device are printed. The default is 0. Note that this option is only effective when you specify a wildcard in voltage probes.
matchport=0 1	When you specify 1, the wildcard matching extends to match the port nodes of the subcircuit instance. The default is 0. Note that this option is only effective when you specify a wildcard character.
limit=limit_value	Specifies the absolute hierarchy level down to which the nodes or instances are specified. This option is only effective when you specify a wildcard character. The default is an infinite number of levels.

Argument	Description
<code>level=level_value</code>	Specifies the relative hierarchy level down to which the nodes or instances are specified. This option is only effective when you specify a wildcard character. The default is an infinite number of levels.

Table 35 lists the signal-access functions.

Table 35 Signal-Access Functions

Signal-Access Function	Description
<code>TIME</code>	Returns the simulation time.
<code>V(node)</code>	Returns the voltage of <i>node</i> . The <i>node</i> supports wildcard characters.
<code>Vn(instance)</code>	Returns the voltage of terminal <i>n</i> where <i>n</i> is an integer greater than or equal to 1; or <i>d</i> , <i>g</i> , <i>s</i> , <i>b</i> for MOSFET and <i>c</i> , <i>b</i> , <i>e</i> , <i>s</i> for BJT. The PrimeSim HSPICE format does not support this function. The <i>instance</i> supports wildcard characters.
<code>V(node1,node2)</code>	Returns the difference of two nodes, $V(node1) - V(node2)$. Wildcard characters are not supported in this function.
<code>I(instance)</code>	Returns the current flowing into the first terminal of a device instance. Positive values indicate the current is flowing into the terminal. Negative values indicate the current is flowing out of the terminal. The <i>instance</i> supports wildcard characters.
<code>In(instance)</code>	Returns the current flowing into an <i>instance</i> at terminal <i>n</i> where <i>n</i> is an integer greater than or equal to 1; or <i>d</i> , <i>g</i> , <i>s</i> , <i>b</i> for MOSFET and <i>c</i> , <i>b</i> , <i>e</i> , <i>s</i> for BJT. Positive values indicate the current is flowing into the terminal. Negative values indicate the current is flowing out of the terminal. This convention is not identical to PrimeSim HSPICE format, which negates the sign of some terminals of some model instances. The <i>instance</i> supports wildcard characters.

Table 35 Signal-Access Functions (Continued)

Signal-Access Function	Description
<code>ISUB(subckt_instance.port)</code> or <code>X(subckt_instance.port)</code>	Returns the total current flowing into the subcircuit port. The <code>subckt_instance</code> is a subcircuit instance name, and <code>port</code> is a name of the subcircuit port. Positive values indicate the current is flowing into the subcircuit port. Negative values indicate the current is flowing out of the subcircuit port. Both <code>subckt_instance</code> and <code>port</code> support wildcard characters. See Figure 5 for details about this function. Note that <code>x()</code> is not a supported format for Eldo netlists.
<code>IXBA(ipattern:node)</code>	Returns the sum of all the device terminal currents of the specified subcircuit instance touching the specified node when running the XBA flow. See the <code>probe_waveform_ixba</code> command for details.
<code>XN(subckt_instance,node)</code>	Returns the total current flowing into a subcircuit instance from a node that connects to the instance at one or multiple ports, or by global connectivity. The <code>subckt_instance</code> is the subcircuit instance name and <code>node</code> is the node name. Positive values indicate the current is flowing into the subcircuit. Negative values indicate the current is flowing out of the subcircuit. The <code>subckt_instance</code> supports wildcard characters, but not the <code>node</code> . See Figure 6 for an example. Note that this feature is not supported in Eldo netlist format.

The following example shows probing of the gate terminal voltage of `X1.mn1` with the `Vg()` or `V2()` signal-access function and assigns it to the `gate_voltage` label.

```
.PROBE gate_voltage = Vg(X1.mn1)
```

or

```
.PROBE gate_voltage = V2(X1.mn1)
```

The following example shows the usage of the `limit` and `level` arguments. Both statements probe the voltage of all top hierarchy nodes only. The `limit` argument applies to the absolute hierarchy level, and the `level` argument applies to the relative hierarchy level where the asterisk symbol (*) is specified.

```
.PROBE V(*) limit=0
```

or

```
.PROBE V(*) level=1
```

The following example probes the voltage of all node at the top and first hierarchy levels.

```
.PROBE V(*) limit=1
```

or

```
.PROBE V(*) level=2
```

The following example probes the voltage of the nodes under x1.x2 hierarchy level.

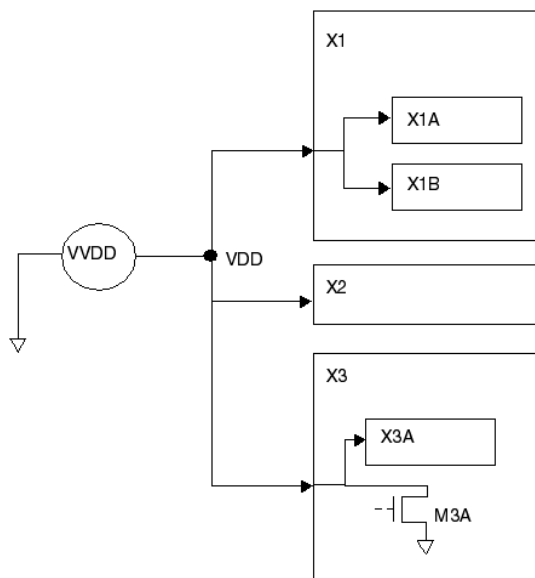
```
.PROBE V(x1.x2.*) limit 2
```

or

```
.PROBE V(x1.x2.*) level 1
```

Figure 5 shows the ISUB() or X() function in a graphical representation.

Figure 5 The ISUB() or X() Example



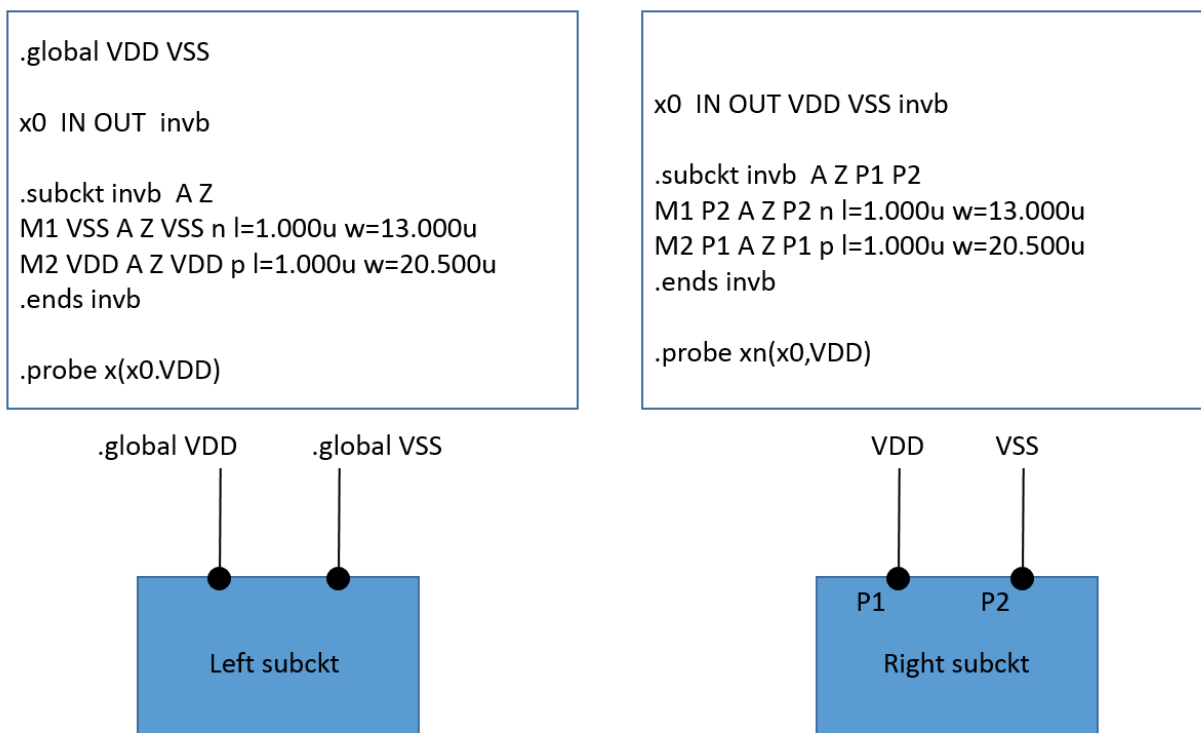
Shown in Figure 5, the total current that flows into X1.VDD is equivalent to the sum of the current flowing into X1.X1A.VDD and X1.X1B.VDD. More specifically, the statement returns the sum of ISUB(X1.X1A.VDD)+ISUB(X1.X1B.VDD) using the following statement:

```
.PROBE ISUB(X1.VDD)
```

The total current that flows into `X3.VDD` is equivalent to the sum of the current flowing into `X3.X3A.VDD` and the drain of `X3.M3A`. More specifically, the statement returns the sum of `ISUB(X3.X3A.VDD) + I1(X3.M3A)` using the following statement:

```
.PROBE ISUB(X3.VDD)
```

Figure 6 The `X()` and `XN()` Example



The examples in [Figure 6](#) show the usage of `X()` and `XN()`.

For the left subcircuit, `VDD` and `VSS` are defined as global nodes. The node names are the same in all hierarchies. In this case, you could use `X()` or `XN()` functions and the results are the same:

```
.PROBE X(x0.VDD) = .PROBE XN(x0,VDD)
```

For the right subcircuit, the top-level nodes, `VDD` and `VSS`, are connected to ports `P1` and `P2`, so the nodes name have been changed. In this case, you can only use the `XN()` function:

```
.PROBE XN(x0,VDD)
```

Using .PRINT Statements

By default, the PrimeSim XA tool treats the `.PRINT` statement as the `.PROBE` statement. You can use the `enable_print_statement` command to override the default behavior by printing all the waveform expressions specified in the `.PRINT` statements to an ASCII file suffixed with `*.print`.

The ASCII file begins with the name of the expressions to be printed, separated by a white space at the top of the line. Each additional line is the value of each expression. The first column of the ASCII file is an expression of time. The times are interpolated to, and start at, the times specified by the `.TRAN` statement.

The syntax of the ASCII file is:

```
time expname1[ ... expnamen]
time_val1 expname1_val1[ ... expnamen_val1]
...
[time_valm] [expname1_valm] [... expnamen_valm]
```

Here is an example of the `*.print` file.

```
time v(1) i1(x_inv1.xil.mp)
0.000000e+00 5.000000e+00 5.630412e-12
1.000000e-09 5.000000e+00 4.841490e-13
2.000000e-09 5.000000e+00 4.890546e-133.000000e-09 5.000000e+00
4.939602e-13
4.000000e-09 5.000000e+00 4.976397e-13
5.000000e-09 5.000000e+00 4.977350e-13
6.000000e-09 5.000000e+00 4.978302e-13
7.000000e-09 5.000000e+00 4.979254e-13
8.000000e-09 5.000000e+00 4.980206e-13
9.000000e-09 5.000000e+00 4.981158e-13
1.000000e-08 5.000000e+00 4.982111e-13
1.100000e-08 5.000000e+00 4.983063e-13
1.200000e-08 5.000000e+00 4.984015e-13
1.300000e-08 5.000000e+00 4.984967e-13
1.400000e-08 5.000000e+00 4.985920e-13
1.500000e-08 5.000000e+00 4.986872e-13
```

The following example prints the voltage of `node2`, the current flowing into `VIN`, and the voltage ratio of `OUT` and `IN`; then it assigns the voltage to the label `gain`.

```
.OPTION XA_CMD="enable_print_statement -switch yes"
.PRINT V(node2) I(VIN) gain = "V(OUT)/V(IN) "
```

Here is an example of the output ASCII file:

time	v(node2)	I(VIN)	gain
0.000000e+00	1.000000e+01	-3.030300e-03	1.000000e+00
5.000000e-01	1.403320e+00	-4.252479e-04	1.403320e-01
1.000000e+00	3.727833e-01	-1.129645e-04	3.727833e-02

Using the .LPROBE and .LPRINT Statements

The .LPRINT and .LPROBE statements output the logic waveform expressions in the specified waveform format file. The PrimeSim XA tool always treats all .LPRINT statements as if they were .LPROBE statements. There is no command to enable the ASCII output for .LPRINT statements.

Note:

These statements are not supported in the Eldo netlist format.

The syntax is:

```
.LPRINT|.LPROBE
[label1 =] expression1 [ ... [labeln =] expressionn] [subckt=subckt_name]
[except|filter=pattern]
[v1|lo1|low=v1th]
[vh|h1h|high=vhth]
```

Argument	Description
label1 ... labeln	Name given to the <i>expression</i>
expression1 ... expressionn	Waveform expression. It can be a node name or equation with one or more signal-access functions. See Table 35 .
subckt=subckt_name	Performs the operation on all specified nodes or instance of the specified subcircuit. If you specify this argument, the nodes or instances must be in the subcircuit instance hierarchy level. Wildcard characters are not supported in subcircuit names.
except filter=pattern	Performs the operation on all the nodes or instances, except those of the specified pattern. You can use wildcard characters to match the node names and instances to exclude from probing.
limit=limit_value	Specifies the absolute hierarchy level down to which the nodes or instances are specified. This option is only effective when you specify a wildcard character. The default is an infinite number of levels.
level=level_value	Specifies the relative hierarchy level down to which the nodes or instances are specified. This option is only effective when you specify a wildcard character. The default is an infinite number of levels.

Argument	Description
<code>vl loth low=vlth</code>	Specifies the value of the low threshold. If you do not specify this option, the PrimeSim XA tool uses the value defined in the <code>set_logic_threshold</code> command. The PrimeSim XA tool issues an error message if <code>vlth</code> is larger than <code>vhth</code> .
<code>vh hith high=vhth</code>	Specifies the value of the high threshold. If you do not specify this option, the PrimeSim XA tool uses the value defined in the <code>set_logic_threshold</code> command. The PrimeSim XA tool issues an error message if <code>vlth</code> is larger than <code>vhth</code> .

The following example probes the logic waveform expression of all nodes in the `amos` subcircuit; it sets the high threshold as 3.5 volts and lower threshold as 1.5 volts.

```
.LPROBE V(*) loth=1.5 hith=3.5 level=1 subckt=amos
```

Using the PrimeSim XA Probing Commands

The PrimeSim XA tool supports the `set_probe_option` command to control probing options and many commands to output the waveform expressions into the specified waveform format file. These commands are supported in all netlist formats. For a detailed description about each command, see *PrimeSim™ XA Command Reference*.

- `set_probe_option`
- `probe_waveform_current`
- `probe_waveform_ixba`
- `probe_waveform_logic`
- `probe_waveform_pcm`
- `probe_waveform_va`
- `probe_waveform_voltage`

See Also

- [Supported Waveform File Formats](#)

Measuring Statements and Commands

The `.MEASURE` statement evaluates information and post-processes results with user-defined functions for measurements. The PrimeSim XA tool prints the measurement results to the output `.meas` (default) or `.mt` file. Use the `set_meas_option -format hspice` command if you want to print the measurement results in PrimeSim HSPICE formatting in the `*.mt` file. The `.MEASURE` statement supports a wildcard expression.

There are some differences between the PrimeSim HSPICE and PrimeSim XA tools when using the `.MEASURE` statement:

- The PrimeSim XA tool supports all PrimeSim HSPICE measurement functions, with enhancements for improved ease-of-use.
- The PrimeSim XA `*.meas` or `*.mt` files are organized differently than the PrimeSim HSPICE `*.mt` file.
- If a measurement statement is not executed, the PrimeSim HSPICE tool writes 0 to the `.mt` file. The PrimeSim XA tool writes `FAILED` to the `.meas` or `.mt` file.

The `.MEASURE` statement supported in the PrimeSim XA tool is compatible with the PrimeSim HSPICE tool. For details, see the Specifying User-Defined Analysis (`.MEASURE`) topic in *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*. The statement also supports a feature to accommodate the PrimeSim XA application.

Following is an example of the output file for the measurement statement.

```
*
* PrimeSim XA linux64 <version> <time> <date>
* build id: 7331626
* trantime = 1ns

test_find_when = 1.2827454e-10 at = 1.3332047e-09
test_avg = 2.7297435e+00 from = 5.0000000e-08 to = 1.0000000e-07
temper = 2.5000000e+01
alter# = 1.0000000e+00
```

Supported Functions

Here are the supported functions to specify with the `.MEASURE` statement in PrimeSim XA:

- [PARAM Functions](#)
- [AVG, MAX, MIN, PP, RMS and INTEG Functions](#)
- [FIND, DERIVATIVE, and AT Functions](#)
- [FIND, DERIVATIVE, and WHEN Functions](#)
- [WHEN Function](#)

- [TRIG-TARG Function](#)
- [TRAN_CONT Function](#)

PARAM Functions

The syntax for the `PARAM` function is as follows:

```
.MEAS[URE] [TR[AN]] mname [PARAM[=]] expression
```

Argument	Description
<i>mname</i>	Reference name for the measurement result
<i>expression</i>	Scalar expression

The following example measures the average value of parameters A, B, and C. The measurement result is assigned to `AVERAGE_VALUE`.

```
.PARAM A=5 B=3 C=10
.MEASURE TRAN AVERAGE_VALUE PARAM="(A+B+C)/3"
```

AVG, MAX, MIN, PP, RMS and INTEG Functions

This function reports the average (`AVG`), maximum (`MAX`), minimum (`MIN`), peak-to-peak (`PP`), root-mean-squared (`RMS`), or integral (`INTEG`) value of the specified waveform expression. The syntax is:

```
.MEAS[URE] [TR[AN]] mname FUNCTION w+ [FROM[=] ta] [TO[=] tb]
```

Argument	Description
<i>mname</i>	Reference name for the measurement result
<i>FUNCTION</i>	Can be <code>AVG</code> , <code>MAX</code> , <code>MIN</code> , <code>PP</code> , <code>RMS</code> , or <code>INTEG</code>
<i>w</i>	Waveform expression at which the measurement is taken
<code>FROM=ta</code>	Starting transient time of the measurement evaluation
<code>TO=tb</code>	Ending transient time of the measurement evaluation

The following example measures the maximum value of $V(a) - V(b)$ during the time period of 20 ns and 100 ns. The measurement result is assigned to `MAXVAL`.


```
.MEASURE TRAN MAXVAL MAX V(a,b) FROM=20ns TO=100ns
```

FIND, DERIVATIVE, and AT Functions

This function reports the value or derivative of the specified waveform variable at a specific time. The syntax is:

```
.MEAS[URE] [TR[AN]] mname FUNCTION w AT[=] t
```

where

Argument	Description
<i>mname</i>	Reference name for the measurement result
<i>FUNCTION</i>	Can be FIND or DERIVATIVE
<i>w</i>	Waveform expression at which the measurement is taken
AT= <i>t</i>	Transient time of the measurement evaluation

The following example measures the derivative value of `V(out)` at 25 ns. The measurement result is assigned to `SLEW_RATE`.

```
.MEASURE TRAN SLEW_RATE DERIVATIVE V(out) AT=25ns
```

FIND, DERIVATIVE, and WHEN Functions

This function reports the value or derivative of the specified waveform expression when a condition is met. This syntax is used when the right-hand side of the conditional statement is equal to a constant value:

```
.MEAS[URE] [TR[AN]] mname FUNCTION wa+ WHEN wb [=] val [TD[=] td]  
[EVENT[=] cnt]
```

This syntax is used when the right-hand side of the conditional statement is equal to a waveform expression:

```
.MEAS[URE] [TR[AN]] mname FUNCTION wa + WHEN wb[=] wc [TD[=] td]  
[EVENT=cnt]
```

where

Argument	Description
<i>mname</i>	Reference name for the measurement result
<i>FUNCTION</i>	Can be <code>FIND</code> or <code>DERIVATIVE</code>
<i>wa</i>	Waveform expression at which the measurement is taken
<i>wb=wc</i>	Conditional statement when the right-hand side is equal to a waveform expression when the measurement is taken
<i>wb=val</i>	Conditional statement when the right-hand side is equal to a constant value when the measurement is taken
<i>TD=td</i>	Elapsed time before the measurement evaluation
<i>EVENT=cnt</i>	Applies to conditional statement. The <i>EVENT</i> can be <code>RISE</code> , <code>FALL</code> , or <code>CROSS</code> keyword. The <i>cnt</i> is the number of occurrences, or you can use the keyword <code>LAST</code> or <code>0</code> to specify the last occurrence.

The following example measures the derivative value of $v(A)$ when $v(B)$ is equal to 5 volts at the third rising edge. The measurement result is assigned to `DERVA`.

```
.MEASURE TRAN Derva DERIVATIVE v(A) when v(B)=5 RISE=3
```

WHEN Function

This function reports the time when a condition is met. This syntax is used when the right-hand side of the conditional statement is equal to a constant value:

```
.MEAS[URE] [TR[AN]] mname+WHEN wb[=] val [TD[=] td]
+[EVENT[=] cnt|LAST]
```

The syntax is used when the right-hand side of the conditional statement is equal to a waveform expression:

```
.MEAS[URE] [TR[AN]] mname+ WHEN wb[=] wc [TD[=] td] [EVENT[=] cnt]
```

where

Argument	Description
<i>mname</i>	Reference name for the measurement result

Argument	Description
<i>wa</i>	Waveform expression at which the measurement is taken
<i>wb=wc</i>	Conditional statement when the right-hand side is equal to a waveform expression when the measurement is taken
<i>wb=val</i>	Conditional statement when the right-hand side is equal to a constant value when the measurement is taken
<i>TD=td</i>	Elapsed time before the measurement evaluation
<i>EVENT=cnt</i>	Applies to conditional statement. The <i>EVENT</i> can be <i>RISE</i> , <i>FALL</i> , or <i>CROSS</i> keyword. The <i>cnt</i> is the number of occurrences, or you can use the keyword <i>LAST</i> or 0 to specify the last occurrence.

This first statement measures the time at which $v(A)$ is maximum during the time period of 0 and 100 ns and assigned to *MAXVA*. The second statement then measures the time when $V(A)=MAXVA$ and assigned it to *TIME_AT_MAXVA*.

```
.MEASURE TRAN MAXVA MAX v(A) FROM=0ns TO=100ns
.MEASURE TRAN TIME_AT_MAXVA WHEN v(A)=MAXVA
```

TRIG-TARG Function

This function reports the time difference between the triggered point and the target point. The syntax is:

```
.MEAS[SURE] [TR[AN]] mname TRIG specA TARG specB
```

There are two syntaxes that specify *specA* and *specB*. The syntax for *specA* is:

```
AT[=] ta
```

Or

```
wa [VAL[=] vala] [TD[=] tda] [VLG[=] vlga] [VHG[=] vhga]  
+[EVENTa[=] cnta]
```

The syntax for *specB* is:

```
AT[=] tb
```

Or

```
wb [VAL [=] vala] [TD [=] tdb] [VLG [=] vlgb] [VHG [=] vhgb]
+ [EVENTb [=] cntb]
```

where

Argument	Description
<i>mname</i>	Reference name for the measurement result
<i>spec</i>	Specification for the <code>TRIG</code> measurement
<i>specb</i>	Specification for the <code>TARG</code> measurement
<code>AT=ta</code>	Definite time at which the <code>TRIG</code> measurement is taken
<code>AT=tb</code>	Definite time at which the <code>TARG</code> measurement is taken
<i>wa</i>	Waveform expression at which the <code>TRIG</code> measurement is taken
<i>wb</i>	Waveform expression at which the <code>TARG</code> measurement is taken
<code>VAL=vala</code>	Triggered threshold for <i>wa</i> .
<code>VAL=valb</code>	Triggered threshold for <i>wb</i> .
<code>TD=tda</code>	Elapsed time before the <code>TRIG</code> measurement evaluation
<code>TD=tdb</code>	Elapsed time before the <code>TARG</code> measurement evaluation
<code>VLG=vlga</code>	Low voltage threshold for rising and falling edges to validate the <code>RISE</code> and <code>FALL</code> events respectively for the <code>TRIG</code> measurement. The <i>vlga</i> has to be below <i>vala</i> . This argument is not supported in Eldo netlist format. See Using VAL, VLG and VHG to Ignore Glitch Measurement section for details.
<code>VLG=vlgb</code>	Low voltage threshold for rising and falling edges to validate the <code>RISE</code> and <code>FALL</code> events respectively for the <code>TARG</code> measurement. The <i>vlgb</i> has to be above <i>valb</i> . This argument is not supported in Eldo netlist format. See the Using VAL, VLG and VHG to Ignore Glitch Measurement section for details.
<code>VHG=vhga</code>	High voltage threshold for rising and falling edges to validate the <code>RISE</code> and <code>FALL</code> events respectively for the <code>TRIG</code> measurement. The <i>vhga</i> has to be above <i>vala</i> . This argument is not supported in Eldo netlist format. See the Using VAL, VLG and VHG to Ignore Glitch Measurement section for details.

Argument	Description
<code>VHG=vhgb</code>	High voltage threshold for rising and falling edges to validate the <code>RISE</code> and <code>FALL</code> events respectively for the <code>TRIG</code> measurement. The <code>vhgb</code> has to be above <code>valb</code> . This argument is not supported in Eldo netlist format. See the Using VAL, VLG and VHG to Ignore Glitch Measurement section for details.
<code>EVENTa=cnta</code>	Applies to <code>wa</code> for the <code>TRIG</code> measurement. The <code>EVENT</code> can be <code>RISE</code> , <code>FALL</code> , or <code>CROSS</code> keyword. The <code>cnta</code> is the number of occurrences, or you can use the keyword <code>LAST</code> or <code>0</code> to specify the last occurrence.
<code>EVENTb=cntb</code>	Applies to <code>wb</code> for the <code>TRIG</code> measurement. The <code>EVENT</code> can be <code>RISE</code> , <code>FALL</code> , or <code>CROSS</code> keyword. The <code>cntb</code> is the number of occurrences, or you can use the keyword <code>LAST</code> or <code>0</code> to specify the last occurrence.

The following example measures the time delay between `nodes1` and `node2`. The `TRIG` measurement is taken at the second rising edge of `node1` after 10 ns when its value is equal to `supply/2`. The `TARG` measurement is taken at the second falling edge of `node2` after 10 ns when its value is equal to `supply/2`. The measurement result is assigned to `TDELAY`.

```
.MEASURE TRAN TDELAY TRIG V(node1) VAL='supply/2' TD=10n RISE=2
+ TARG V(node2) VAL='supply/2' TD=10n FALL=2
```

Using VAL, VLG and VHG to Ignore Glitch Measurement

The PrimeSim XA tool enhances the `.MEASURE` statement to validate the presence of glitch in the waveforms. The arguments are `VAL`, `VLG` and `VHG`.

Note:

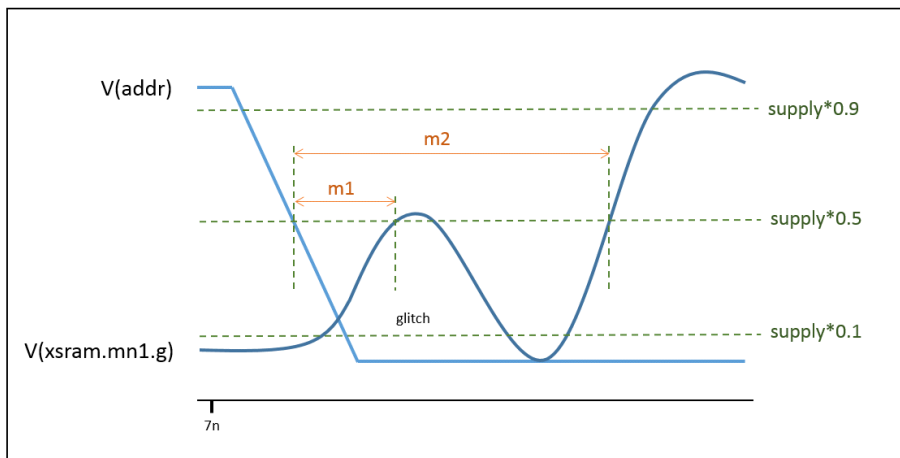
The `VLG` and `VHG` arguments are not supported in Eldo netlist format.

The PrimeSim XA tool uses the combination of `VAL`, `VLG` and `VHG` to validate rising and falling edges as follows:

- For a rising edge, the signal has to go through `VLG -> VAL -> VHG` to be considered as a valid `RISE` event.
- For a falling edge, the signal has to go through `VHG -> VAL -> VLG` to be considered as a valid `FALL` event.

[Figure 7](#) shows a graphical representation on how you can use the `VAL`, `VLG`, and `VHG` arguments.

Figure 7 VAL, VLG, and VHG Arguments Example



If you specify only the `VAL` argument as follows, the measurement result is the value of $m1$, which is a glitch to the signal.

```
.MEASURE TRAN setup_rise_4_data_delay
+ TRIG v(addr) VAL='0.5*supply' TD='7n' FALL=1
+ TARG v(xsram.mn1.g) VAL='0.5*supply' TD='7n' RISE=1
```

To correctly capture the measurement result in the presence of a glitch, use the `VAL`, `VLG` and `VHG` arguments as follows. The measurement result is the value of $m2$.

```
.MEASURE TRAN setup_rise_4_data_delay
+ TRIG v(addr) VAL='0.5*supply' TD='7n' FALL=1
+ TARG v(xsram.mn1.g) VAL='0.5*supply'
+ VHG='0.9*supply' VLG='0.1*supply' TD='7n' RISE=1
```

TRAN_CONT Function

The continuous measurement is a `.MEASURE` statement where `TRAN_CONT` replaces the `TRAN` keyword. When the PrimeSim XA tool runs continuous measurement, the measurement is performed continuously until the end of the simulation. All measurement values are reported.

Note:

The `TRAN_CONT` type is not supported in the Eldo netlist format.

The continuous measurement is limited to the `TRIG-TARG`, `WHEN`, `DERIVATIVE-WHEN` and `FIND-WHEN` types of the `.MEASURE` statement. All other `.MEASURE` statements are not supported with the continuous measurement. When a normal measurement depends on a continuous measurement, the last value of the continuous measurement result is used for normal measurement reference.

```
.MEAS[SURE] TRAN_CONT TRIG-TARG_spec|WHEN_spec|FIND-WHEN_spec
```

where

Argument	Description
<i>TRIG-TARG_spec</i>	See the TRIG-TARG Function section for details.
<i>WHEN_spec</i>	See the WHEN Function section for details.
<i>FIND-WHEN_spec</i> or <i>DERIVATIVE-WHEN_spec</i>	See the FIND, DERIVATIVE, and WHEN Functions section for details.

Output Files

The PrimeSim XA tool writes the continuous measurement results to a file suffixed with *_mname.meas or *_mname.mt. Both files have the same format.

The following is an example of an output file for the measure statement:

```
crossing@1 = 0.
crossing@2 = 2.271034737e-07
crossing@3 = 6.814809009e-07
...
vt2@1 = 1.475989329e-07
vt2@2 = 1.147681340e-06
vt2@3 = 2.147720241e-06
...
period@1 = 1.000082408e-06 trig = 1.475989329e-07 targ = 1.147681340e-06
period@2 = 1.000038900e-06 trig = 1.147681340e-06 targ = 2.147720241e-06
period@3 = 9.999610996e-07 trig = 2.147720241e-06 targ = 3.147681340e-06
...
temp = 25.0000000000
alter# = 1.0000000000
```

Example

The following example performs a continuous measurement to find all values for $v(d[1])$ when $v(b[3])$ is equal to 1.5V.

```
.MEASURE TRAN_CONT vd1 find v(d[1]) when v(b[3])=1.5
```

Following is the output measurement output file:

```
1. vd1 =2.999419212341e+00 at=9.005000000000e-08
2. vd1 =2.999318838120e+00 at=1.700500000000e-07
3. vd1 =2.999920822592e+00 at=2.500500000000e-07
4. vd1 =2.999974223151e+00 at=3.300500000000e-07
5. vd1 =9.570756927408e-05 at=4.100500000000e-07
```

The following example performs a continuous measurement to report every time delay between $v(\text{clk})=1.25\text{V}$ rising edge and $v(q)=1.25\text{V}$ falling edge.

```
.MEASURE TRAN CONT tdel1 TRIG v(clk) VAL=1.25 RISE=1
+ TARG v(q) VAL=1.25 FALL=1
```

The output file includes the following information:

- The first `tdel1` is the delay between the first rising edge of `clk` and the first falling edge of `q`.
- The second `tdel1` is the delay between the second rising edge of `clk` and the second falling edge of `q`.
- The third `tdel1` is the delay between the third rising edge of `clk` and the third falling edge of `q`.

PrimeSim XA Enhancements to the .MEASURE Syntax

In addition to the PrimeSim HSPICE measure functions, the PrimeSim XA tool also supports the enhanced built-in functions listed in [Table 36](#). The syntax to call the built-in functions with the `.MEASURE` statement is:

```
.MEAS[SURE] [TR[AN]] mname built-in-function
```

Table 36 *Enhanced Functions for the .MEASURE Statement*

Built-in Expression	Description
<code>AVG(<i>expression</i>, <i>ta</i>, <i>tb</i>)</code>	Returns the average value of <i>expression</i> from <i>ta</i> to <i>tb</i>
<code>MAX(<i>expression</i>, <i>ta</i>, <i>tb</i>)</code>	Returns the maximum value of <i>expression</i> from <i>ta</i> to <i>tb</i> .
<code>MIN(<i>expression</i>, <i>ta</i>, <i>tb</i>)</code>	Returns the minimum value of <i>expression</i> from <i>ta</i> to <i>tb</i> .
<code>pp(<i>expression</i>, <i>ta</i>, <i>tb</i>)</code>	Returns the peak-to-peak value of <i>expression</i> from <i>ta</i> to <i>tb</i>
<code>RMS(<i>expression</i>, <i>ta</i>, <i>tb</i>)</code>	Returns the root-mean-squared value of <i>expression</i> from <i>ta</i> to <i>tb</i> .
<code>INTEG(<i>expression</i>, <i>ta</i>, <i>tb</i>)</code>	Returns the integral of <i>expression</i> with respect to time from <i>ta</i> to <i>tb</i> .
<code>FIND(<i>expression</i>, <i>t</i>)</code>	Returns the value of <i>expression</i> at time <i>t</i>

Table 36 Enhanced Functions for the .MEASURE Statement (Continued)

Built-in Expression	Description
<code>DERIVATIVE(expression, t)</code>	Return the derivative of <i>expression</i> with respect to time at time <i>t</i> .
<code>CROSS(expression, val, td, cnt)</code>	Returns the time at which <i>expression</i> is equal to <i>val</i> for the <i>cnt</i> occurrence after time <i>td</i> , if <i>cnt</i> is greater than zero. If <i>cnt</i> is equal to zero or keyword <code>LAST</code> , it returns the time where <i>w</i> is equal to <i>val</i> at the last occurrence.
<code>RISE(expression, val, td, cnt)</code>	Returns the time at which <i>expression</i> is equal to <i>val</i> at the <i>cnt</i> rising edge after time <i>td</i> , if <i>cnt</i> is greater than zero. If <i>cnt</i> is equal to zero or keyword <code>LAST</code> , it returns the time at the last occurrence.
<code>FALL(expression, val, td, cnt)</code>	Returns the time at which <i>expression</i> is equal to <i>val</i> at the <i>cnt</i> falling edge after time <i>td</i> , if <i>cnt</i> is greater than zero. If <i>cnt</i> is equal to zero or keyword <code>LAST</code> , it returns the time at the last occurrence.

Note:

The `CROSS()`, `RISE()` and `FALL()` functions do not support the `VLG` and `VHG` arguments to ignore glitch measurement.

The following example designates the time where `V(1)` is equal to `V(2)` at the first falling edge, and assign it to `STIME`.

```
.MEASURE TRAN STIME FALL(V(1)-V(2),0,0,1)
```

The following example designates the voltage of `V(3)` when `V(1)` is equal to `V(2)` at the last rising edge, and assign the result to `TRT`.

```
.MEASURE TRAN TRT FIND(V(3),RISE(V(1)-V(2),0,0,LAST))
```

External Table Parameter File Interface

You can use the `table_param()` built-in function to specify a file that contains a parameter values table and the PrimeSim XA tool returns the linear interpolated value from that table.

Syntax

```
table_param(str('table_file_name'), disc_num, nd1, nd2,  
..., cont_num, nc1, nc2, ..., value_col)
```

Arguments

Argument	Description
<code>str('table_file_name')</code>	Name of file containing a parameter values table. Note: Specify the file name using the <code>str()</code> function.
<code>disc_num</code>	Total number of discrete index columns. Note: If the specified <code>disc_num</code> value is out of range of the given table, an error message will be issued.
<code>nd1, nd2, ...</code>	Discrete index column numbers.
<code>cont_num</code>	Total number of continuous value columns.
<code>nc1, nc2, ...</code>	Continuous values.
<code>value_col</code>	Continuous column number used in linear interpolation.

Note:

To comment out a line in a table, insert a pound sign (#) at the beginning of the line.

Examples

The following example uses `cap.table` and `res.table`, defined as follows:

The `cap.table` file:

```
#corner layer    width    space    Cbot      Ctop      Ccpl
1         3      0.0450  0.030   0.078901  0.06789  0.03456
1         3      0.0450  0.032   0.089012  0.05678  0.02109
1         3      0.0450  0.037   0.090123  0.04567  0.01098
...
```

The `res.table` file:

```
#corner layer    width    space    Rtotal
1         3      0.0450  0.030   78.90123
1         3      0.0450  0.032   67.89012
1         3      0.0450  0.037   56.78901
```

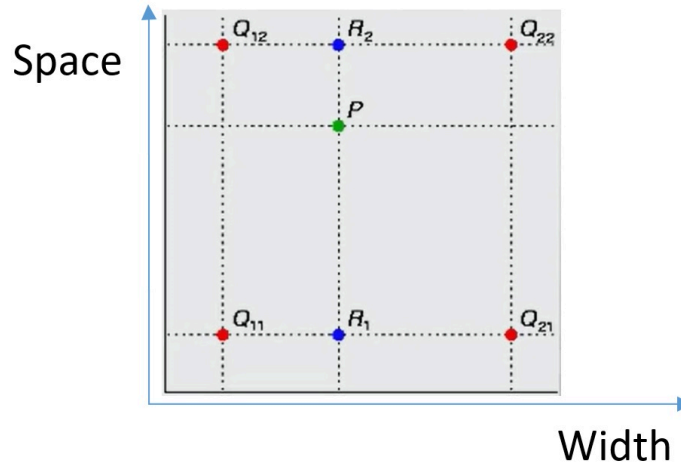
```
.param cbot = table_param('./cap.table',2,corner,layer,2,width,space,1)
.param ctop = table_param('./cap.table',2,corner,layer,2,width,space,2)
.param rnom = table_param('./res.table',2,corner,layer,2,width,space,1)
.param cbot = table_param('./cap.table',2,1,1,2,0.002,0.0252,1)
```

In this example, the `disc_num` function is 2. Corner and layer are of integer index (that is, do not interpolate), and are considered as discrete number; the input values are 1 and 3.

The `cont_num` function is 2. Therefore, next two columns (width and space) must be considered for interpolation, and user's values are 0.0002 and 0.0252, which is two-dimensional. The `value_col` is 1. It denotes the fifth column (`Cbot` column) is the final value column.

An example of return bilinear interpolation is shown in [Figure 8](#).

Figure 8 *Return Bilinear Interpolation*



6

Verilog-A Support

This chapter provides an overview of how the PrimeSim XA tool supports the use of Verilog-A language descriptions for systems and components.

Verilog-A derives from the IEEE 1364 Verilog Hardware Description Language (HDL) specification for describing behavior in analog systems. The Verilog-A language that PrimeSim HSPICE supports is compliant with Verilog-AMS Language Reference Manual, Version 2.3.1.

The Verilog-A implementation in PrimeSim XA supports a mixed design of Verilog-A descriptions and transistor-level SPICE netlists with a simple use model. Verilog-A supports most analysis features available in PrimeSim XA for Verilog-A based devices, including AC, DC, transient analysis, statistical analysis, and optimization.

For more information on Verilog-A support, see the Using Verilog-A topic in the *PrimeSim Continuum Overview*.

This chapter contains the following topics:

- [Verilog-A Language Support](#)
- [Netlist Syntax for Verilog-A in the PrimeSim XA Tool](#)
- [Generating Verilog-A Libraries for Reuse in PrimeSim XA Simulation](#)
- [Module- and Instance-Based Partitioning: Switching Between Verilog-A and SPICE Definitions](#)
- [Port Mapping](#)
- [Passing Parameters](#)
- [Passing the M-Factor](#)
- [Defining Verilog-A Macros With -va,define Command Line Option](#)
- [Verilog-A Output in the PrimeSim XA Tool](#)
- [Verilog-A Features Not Supported in the PrimeSim XA Tool](#)

Verilog-A Language Support

The Verilog-A language supports analog behavioral descriptions that encapsulate high-level behavioral and structural descriptions of systems and components. The behavior of each model, or module, can be described mathematically in terms of its ports and parameters, as applied to an instance of the module.

A module can be defined at the level of abstraction appropriate for the model and analysis, including architectural design and verification. Verilog-A supports both top-down design and bottom-up verification methodologies.

Netlist Syntax for Verilog-A in the PrimeSim XA Tool

Each supported SPICE language has its own syntax for including Verilog-A module descriptions. The following sections describe the basics of each syntax.

- [Using Verilog-A With the PrimeSim HSPICE Netlists](#)
- [Using Verilog-A With the Spectre Netlists in the PrimeSim XA Tool](#)
- [Using Verilog-A With Eldo Netlists in the PrimeSim XA Tool](#)

Further sections in this chapter describe details about [Port Mapping](#) and [Passing Parameters](#).

Using Verilog-A With the PrimeSim HSPICE Netlists

Verilog-A modules in PrimeSim SPICE use the following conventions:

- Verilog-A modules are loaded into the simulator with the `.hdl` statement in the SPICE netlist file.
- Modules are instantiated in the same manner as PrimeSim HSPICE subcircuits. The first character for the name of the instance should be X.
- You can modify instance and model parameters in the same way as other PrimeSim HSPICE instances.

Following is a simple Verilog-A example for an PrimeSim HSPICE netlist:

```
.hdl res.va
V0 net1 0 1
R1 net1 net2 10
x1 net2 net3 res r=20
r2 net3 0 10
.end
```

The `.hdl res.va` statement references this module definition, used for the `x1` instance:

```
module res (a, b);  
  electrical a, b;  
  parameter real R = 1.0;  
  analog begin  
    V(a,b) <+R * I(a,b);  
  end  
endmodule
```

The PrimeSim HSPICE tool is case-insensitive, but Verilog-A modules are case-sensitive. From within Verilog-A modules, a mixed-case name matches a mixed-case name. If there are no exact matches, Verilog-A matches the same name regardless of case. If two references exist in the Verilog-A definitions that are case-sensitive (for example, `mymodul1 inst1` and `mymodul1 Inst1`) and they are referenced by case-insensitive statements in the PrimeSim HSPICE tool, the PrimeSim XA tool displays a warning for any ambiguous references.

If both an PrimeSim HSPICE subcircuit and a module definition exist with the same name, the subcircuit definition is used by default. Use `set_va_view` to switch between definitions. See the [Module- and Instance-Based Partitioning: Switching Between Verilog-A and SPICE Definitions](#) section for more details.

Generating Verilog-A Libraries for Reuse in PrimeSim XA Simulation

Use the `pvalib` command-line option to manually generate a Verilog-A library (`.pvalib`) for reuse in PrimeSim XA simulation.

Note:

You can generate a `.pvalib` library using either PrimeSim HSPICE, PrimeSim or PrimeSim XA simulator, and then reuse it across all three simulators, except for complex libraries which might lead to accuracy issues or crashes. In particular, multi-threading operations should be avoided because different simulators use different methods for multi-threading operation.

General pvalib Command-Line Syntax

```
pvalib {options} file1 file2 ... fileN -o <outdir>
```

Where

- `*pvalib`: Indicates scripts to generate the `.pvalib` file on the 64-bit platform.
- `options`: Controls how the `.pvalib` file is generated. The supported options are:
 - `Ddefine{=<val>}`: Adds the specified define statement to the pre-processor table.
 - `-Lpath`: Specifies the location of input Verilog-A files.
 - `-Ipath`: Adds the specified path to search path for includes.
 - `-f <filename>`: Specifies a list of input Verilog-A files.
 - `-p <product-name>`: Specifies the product name to be `finesim` (FineSim) or `hspice` (PrimeSim HSPICE).
 - `-v <name>.pvalib`: Prints the `pvaInfo` (module information) of the `.pvalib`.
- `-o <outdir>`: Specifies the directory where the output files reside. pVA creates the `out.pvadir` directory in which the intermediate files reside. The default directory is `pvadir`.

Example

The following example shows how to generate and reuse the `.pvalib` file on the `.hdl` command line.

1. Generate pre-compiled pVA libraries using the `pvalib` command.

```
pvalib resistor.va -o reslib
```

The `reslib.pvadir/reslib_8003361L3.pvalib` file is generated in the `reslib/` directory.

```
pvalib capacitor.va -o caplib
```

The `caplib.pvadir/caplib_8003361L3.pvalib` file is generated in the `caplib/` directory.

2. Load the pre-compiled pVA library using the `.hdl` command-line in the netlist file.

```
.hdl reslib.pvadir/reslib_8003361L3.pvalib
```

or

```
.hdl reslib_8003361L3.pvalib
```

3. Run PrimeSim XA simulation.

In the .valog file, the tool reports that the Verilog-A module is successfully loaded from the pre-compiled pVA library. For example,

```
> Loading pVA library 'reslib.pvadir/reslib_8003361L3.pvalib' ...
  Loaded Verilog-A 'resistor' module ...
> Successfully loaded total 1 Verilog-A module(s)
```

If the loading is not completed successfully, the tool writes the error message in the .valog file:

```
.hdl testlib_8003361L3.pvalib
> Loading pVA library 'testlib_8003361L3.pvalib'...
*ERR* File 'testlib_8003361L3.pvalib' not found.
```

Checking Module Information in the Pre-Compiled pVA Library

You can use the `-v` option to check the module information in the pre-compiled pVA library.

For example,

```
pvalib -v reslib_8003361L3.pvalib

> Loading pVA library 'reslib_8003361L3.pvalib' ...
  Loaded Verilog-A 'resistor' module ...
  libepva built by pvamgr synmake_pva_build on <time>
  =====
  HVA Verilog-A Modules (filename:line#)
  =====
  N resistor (/rmnt/test/WORKs/pvalib/resistor.va:3)
  =====
  Total 1 module(s) loaded.
  =====
> Successfully loaded total 1 Verilog-A module(s)
```

See Also

- [Reusing Compiled Verilog-A Files](#)

Module- and Instance-Based Partitioning: Switching Between Verilog-A and SPICE Definitions

If your design has both SPICE subcircuit definitions and Verilog-A definitions, you can switch between SPICE and Verilog-A depending on the type of analysis you want to perform. This lets you achieve trade-offs between the speed and accuracy of the simulation for particular definitions.

Use the `set_va_view` command to perform module- and instance-based partitioning of the circuit definitions. You can switch to a Verilog-A module description for a single SPICE subcircuit or a set of subcircuits within a netlist.

If multiple definitions or “views” of a subcircuit or module exist, by default the PrimeSim XA tool uses the view of the parent netlist.

See the *PrimeSim XA Command Reference* for more information about the `set_va_view` command.

Port Mapping

When a module is instantiated from SPICE, the SPICE rules for instantiation and port connections apply. The ports are connected by position. If the number of connections does not match, the PrimeSim XA tool displays an error.

If the Verilog-A module has a bus in its port list and the bust is instantiated in a SPICE netlist, the signals are assigned by position.

SPICE netlist:

```
X1 a b c d e foo
```

Verilog-A definition:

```
module foo (bus, sig1, sig2);  
electrical bus [2:0];  
electrical sig1, sig2;
```

Bus assignment:

```
a -> bus[2]  
b -> bus[1]  
c -> bus[0]  
d -> sig1  
e -> sig2
```

If the bus definitions are reversed to `electrical bus [0:2]`, the bus assignments are also reversed:

```
a -> bus[0]  
b -> bus[1]  
c -> bus[2]
```

When a subcircuit is instantiated by Verilog-A, designate the port either by position or by the port name. When port-mapping by name, the port name must begin with a period (.), and must be identical to the SPICE port name. The case must be identical.

When mapping ports by position in a Verilog-A instantiation, make sure the positions are correct in the Verilog instance and the SPICE subcircuit. The port count must also be identical. If there is a port count mismatch, the PrimeSim XA tool returns an error.

In the following example, the `zn` port of the `nor1` SPICE subcircuit is connected to the `out1` port in the `nor1` Verilog-A module instances.

Verilog-A module:

```
//position mapping
nor1 il (out1, in1, in2);
```

SPICE subcircuit:

```
.subckt nor1 zn a b
...
.ends
```

Note that if a Verilog-A module instantiates a SPICE subcircuit that contains power net pins, the PrimeSim XA tool displays a port map error.

You can map ports between Verilog-A and SPICE on a scalar-net basis or a vector-port basis. If the sequentially numbered port names in the SPICE subcircuit are identical to the Verilog-A port names, you can use the vector port to instantiate Verilog modules. In the following example, port names `a[3]` `a[2]` `a[1]` `a[0]` in the SPICE subcircuit will map to a 4-bit vector port name in Verilog-A.

Verilog-A module:

```
addr i3 (.a(ai[3:0]), .b(bi[3:0]), .cin(ci), .s(su[3:0]), .cout(co));
```

SPICE subcircuit:

```
.subckt addr a[3] a[2] a[1] a[0]
+ b[3] b[2] b[1] cin
+ s[3] s[2] s[1] +s[0] cout
.ends
```

Passing Parameters

When SPICE instantiates a Verilog-A module, the instance parameters are passed as named parameters in SPICE fashion.

Verilog-A module with parameters:

```
module foo (a b);
electrical a,b;
parameter moduleparam=10;
```

SPICE subcircuit:

```
X1 a b foo moduleparam=7
```

The case-sensitivity issues that are described for each netlister in the [Netlist Syntax for Verilog-A in the PrimeSim XA Tool](#) section apply to parameter passing.

Caution:

If a Verilog-A module has a parameter of m or M , it cannot be passed using the SPICE instantiation, since m is used as the multiplicity factor. If a SPICE instantiation uses m as a parameter, its value is used to set the system `$mfactor` variable.

Passing the M-Factor

pVA (Unified Verilog-A) handles the issue of passing between the M-factor in a SPICE netlist and the Verilog-A `$mfactor`, as shown in the following example:

```
x0 1 3 test_mfactor m=10

module test_mfactor(a, c);
  parameter real m=0.1;
  real rs;
  analog begin
    rs = $mfactor * 100.0;
    $strobe ("m = %g, $mfactor = %g, rs = %g", m, $mfactor, rs);
  end
endmodule
```

During the pVA compilation of Verilog-A, a warning message appears in the .valog file:

```
*pvaW* parameter m is not treated as a mfactor (ddx.va:2)
```

Following is the result from `$strobe`:

```
m = 0.1, $mfactor = 10, rs = 1000
m = 0.1, $mfactor = 10, rs = 1000
```

Defining Verilog-A Macros With -va,define Command Line Option

In a situation where a constant value is repetitively used throughout a description, a define name might be useful in that only one place in the source description needs to be altered if you need to modify the value of the constant. PrimeSim XA supports the `-va,define` command line option to define Verilog-A macro names with or without a value. The `-va,define` command line option is similar to the HDL Compiler ``define` directive.

To define a Verilog-A macro with or without a value, run the `-va,define` command line option in the following syntax:

```
-va,define define_name
-v,define define_name=value
```

When a macro has been defined, you can use it anywhere in the source description. For example, to define a macro R-VAL with a value of 2.0, run the following command:

```
% xa testbench.sp -o outx -va,define R_VAL=2.0
```

Then include the defined name in your description, like

```
resistor.va
analog begin
  `ifdef R_VAL
    I(p,n) <+ V(p,n) / `R_VAL;
    $strobe("XYZ value is %g", `R_VAL);
  `else
    I(p,n) <+ V(p,n) / 1.0;
  `endif
end
```

Verilog-A Output in the PrimeSim XA Tool

The PrimeSim XA tool places messages from the `$strobe` command and other user messages from Verilog-A modules in the following places:

- The UNIX console (stdout)
- A Verilog-A log file, named *.valog
- The main PrimeSim XA log file, named *.log

Messages written to the console and the Verilog-A log file are dumped in their original format. Messages written to the PrimeSim XA log file are dumped according to PrimeSim XA log file rules. PrimeSim XA log files are formatted to be 80 columns wide.

Reusing Compiled Verilog-A Files

After you run a PrimeSim XA simulation that uses Verilog-A modules, you can save time in subsequent simulations by reusing the compiled Verilog-A files in the .pvadir output directory. To reuse this directory, use the `PVA_MPDIR` environment variable and specify a central location. For example:

```
setenv PVA_MPDIR $absolute|relative_path
```

The `PVA_MPDIR` environment variable creates a new runtime library path, shares the existing path, or reuses the precompiled Verilog-A code in the existing runtime library.

If you do not want to reuse the pva.so files compiled from the previous run, specify the `PVA_RMRTL` environment variable and pva will always compile the Verilog-A modules during the run. Setting the `PVA_RMRTL` environment to 1 clears all intermediate files in the .pvadir directory before executing the next command.

For example:

```
setenv PVA_RMRTL 1
```

To reuse the compiled Verilog-A files in the .pvadir directory, run the following steps:

1. Run the initial simulation. For example:

```
xa test.sp -o XA_results/test
```

The XA_results/test.pvadir output directory is created.

2. After the initial simulation runs, check for an existing .pvadir directory and, if necessary, replace it with the .pvadir directory created by the initial simulation. For example:

```
if ( $status != 0 ) exit 999
if ( -d ./XYZ.pvadir ) rm -r -f XYZ.pvadir
cp -r -p XA_results/test.pvadir ./XYZ.pvadir
```

3. Specify the PVA_MPDIR environment variable:

```
setenv PVA_MPDIR ./XYZ.pvadir | ./XYZ
```

4. Run the additional simulations that reuse the compiled Verilog-A files from the initial simulation:

```
xa test1.sp -o XA/final_results1 &
xa test2.sp -o XA/final_results2 &
xa test3.sp -o XA/final_results3 &
xa test4.sp -o XA/final_results4 &
xa test5.sp -o XA/final_results5 &
```

Note:

If a Verilog-A model changes or the PrimeSim XA version changes, you need to rerun steps 1 through 4.

Verilog-A Features Not Supported in the PrimeSim XA Tool

The following Verilog-A features are not supported in the PrimeSim XA tool.

- \$monitor
- Variable-width bus
- Ordered parameters lists in hierarchical instantiation
- Enforcement of input, output, and inout
- Defparam

Chapter 6: Verilog-A Support

Verilog-A Features Not Supported in the PrimeSim XA Tool

- Hierarchical reference and out-of-module references
- Timescale directive

7

Postlayout Simulation Flows

This chapter describes how to use the postlayout simulation features.

The chapter contains the following topics:

- [Back-Annotation Flow](#)
- [Extended Back-Annotation Flow](#)
- [Selective Net Back-Annotation Flow](#)
- [Selective Net Extraction Flow](#)

Back-Annotation Flow

The PrimeSim XA tool supports a back-annotation flow that works with the StarRC tool (Synopsys parasitic extraction tool) and IC Validator LVS tool to simulate a post-layout design.

Using the post-layout flows provides you with several benefits:

- Controlling the design hierarchically, such as setting different netlist options in different blocks in the design to achieve the best accuracy/speed tradeoff
- Analyzing the design hierarchically, such as determining the power consumption of each block in the design
- Increasing the design cycle throughput, such as reusing the same pre-layout netlist options and analysis statements for the post-layout simulation
- Speeding up the simulation run time and reducing the memory usage by taking advantage of the hierarchical database

[Table 37](#) explains the terminology used for the post-layout flow in this chapter.

Table 37 Back-Annotation Terminology

Terminology	Description
Ideal schematic netlist	Schematic netlist file used as an input file for IC Validator or other LVS tools containing the original schematic-based hierarchy.
Ideal layout netlist	Schematic netlist generated by StarRC or other parasitic extraction tools containing the layout-based hierarchy.
Ideal netlist	Both the ideal schematic netlist and ideal layout netlist.
Parasitic netlist	PrimeSim XA-supported parasitic netlist file generated by StarRC or other parasitic extraction tools (see the Understanding the PrimeSim XA-Supported Parasitic Netlist Format section).

Understanding the Back-Annotation Flow

Back-annotation has two types of flows:

- Schematic hierarchy-based that uses an ideal schematic netlist
- Layout hierarchy-based that uses an ideal layout netlist

Each flow has its advantages and limitations. Choose the flow that matches your design needs.

[Figure 9](#) shows the schematic hierarchy-based back-annotation flow. The advantages of using the schematic hierarchy-based back-annotation flow are:

- The ideal schematic netlist, which has the original design structure (schematic-based hierarchy), enables ease of debugging and tracing the circuit.
- You can reuse the identical pre-layout simulation netlist options and analysis statements in the post-layout simulation.

The main limitation of using the schematic hierarchy-based back-annotation flow is:

- The PrimeSim XA tool currently does not support the merging of the excessive layout devices to the schematic devices in the schematic hierarchy-based back-annotation flow, but you can use the layout hierarchy-based back-annotation flow to overcome this limitation.

Figure 9 Schematic-Based Back-Annotation Flow

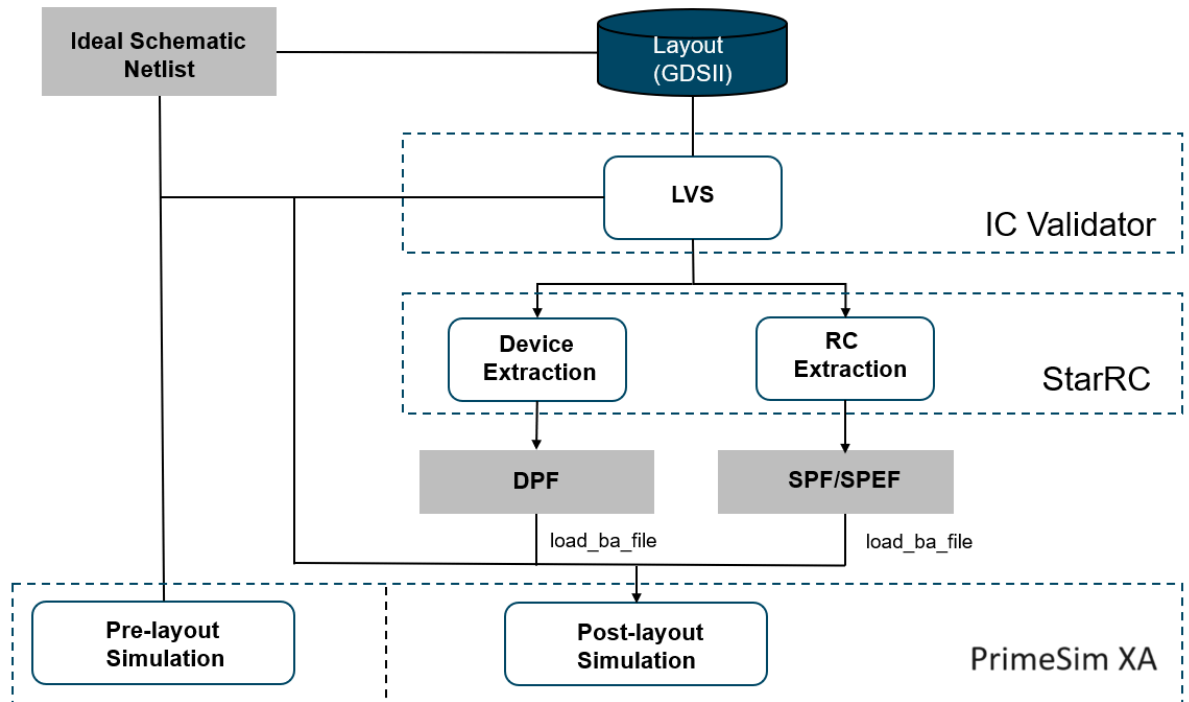


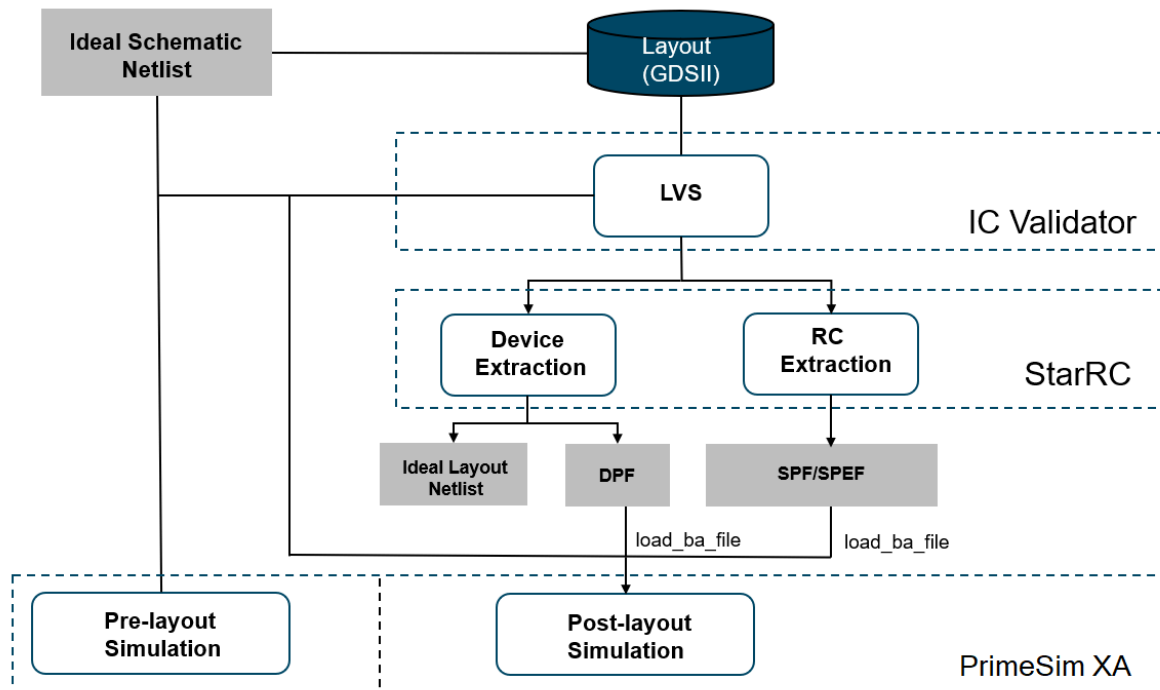
Figure 10 shows the layout hierarchy-based back-annotation flow. The advantage of using the layout hierarchy-based back-annotation flow is:

- There is always a 1:1 matching of the devices between the ideal layout netlist and the parasitic netlist. Therefore, no mismatches occur during the back-annotation flow.

The limitations of using the layout hierarchy-based back-annotation flow are:

- The ideal layout netlist has a layout-based hierarchy. This hierarchy may differ from the original design structure (schematic-based hierarchy), depending on the layout process. Therefore, the processing of debugging and tracing the circuit might be more difficult.
- Minor modifications may be required for the netlist options and analysis statements (before being reused) from the ideal schematic netlist to map to the layout-based hierarchy of the ideal layout netlist file.

Figure 10 Layout Hierarchy-Based Back-Annotation Flow



Understanding the PrimeSim XA-Supported Parasitic Netlist Format

The parasitic netlist describes the interconnect delay and load due to parasitic resistance and capacitance. Parasitics can be represented on a net-by-net basis, from a simple lumped capacitance to a fully distributed resistance-capacitance tree.

The PrimeSim XA tool supports the following netlist formats for back-annotation flow:

- [Standard Parasitic Format \(SPF\)](#)
- [Standard Parasitic Exchange Format \(SPEF\)](#)
- [Device Property Format \(DPF\)](#)

Note:

A Detailed Standard Parasitic Format (DSPF) is a specific type of SPF. The terms SPF and DSPF are used interchangeably.

Standard Parasitic Format (SPF)

An SPF file contains three sections:

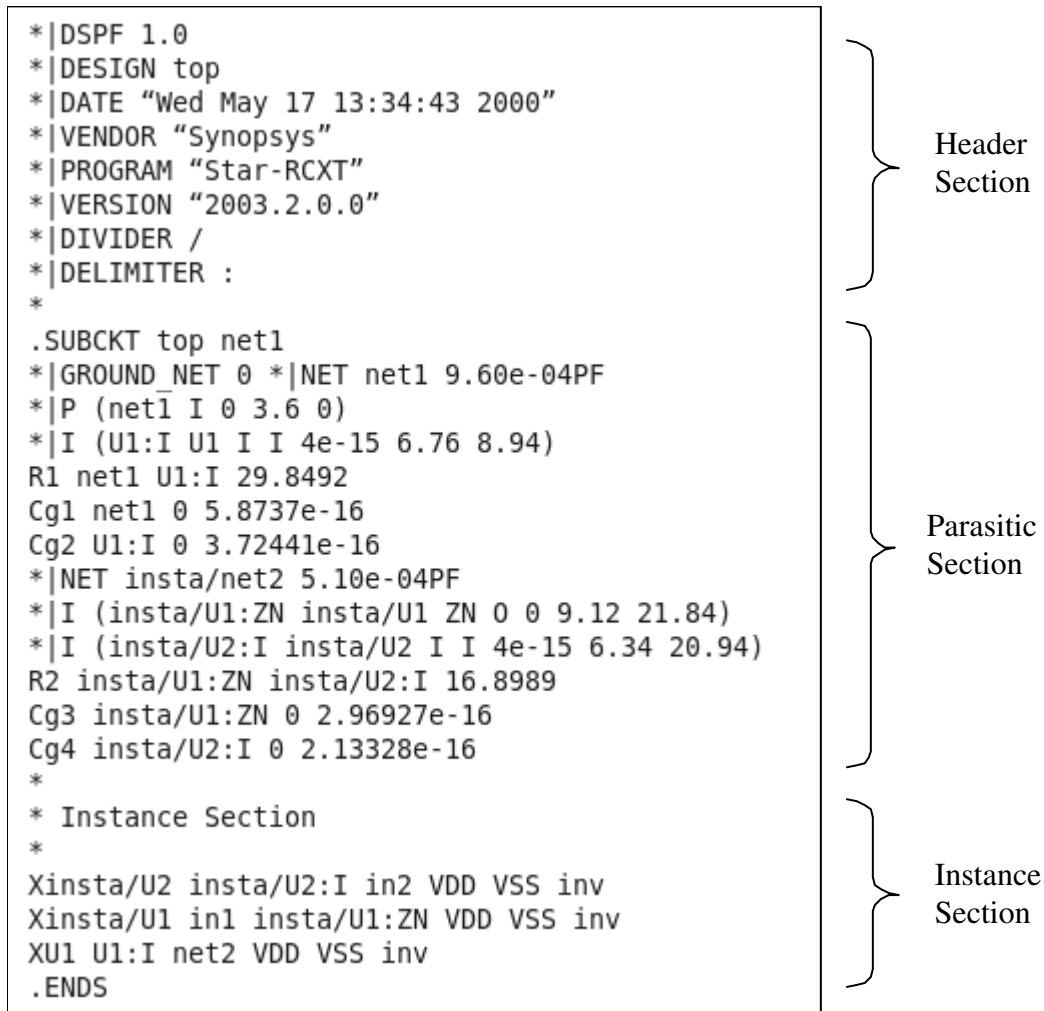
- Header section that provides information about the content of the SPF file
- Parasitic section that describes the interconnect delay, loading, and actual parasitic values for the nets
- Instance section that describes the actual layout topology, which also contains the device layout properties

Note:

You can specify the `NETLIST_FORMAT:SPF` command in the StarRC tool to generate an SPF file.

[Figure 11](#) shows an example of an SPF file.

Figure 11 SPF File Format



Standard Parasitic Exchange Format (SPEF)

A SPEF file consists of four definition sections:

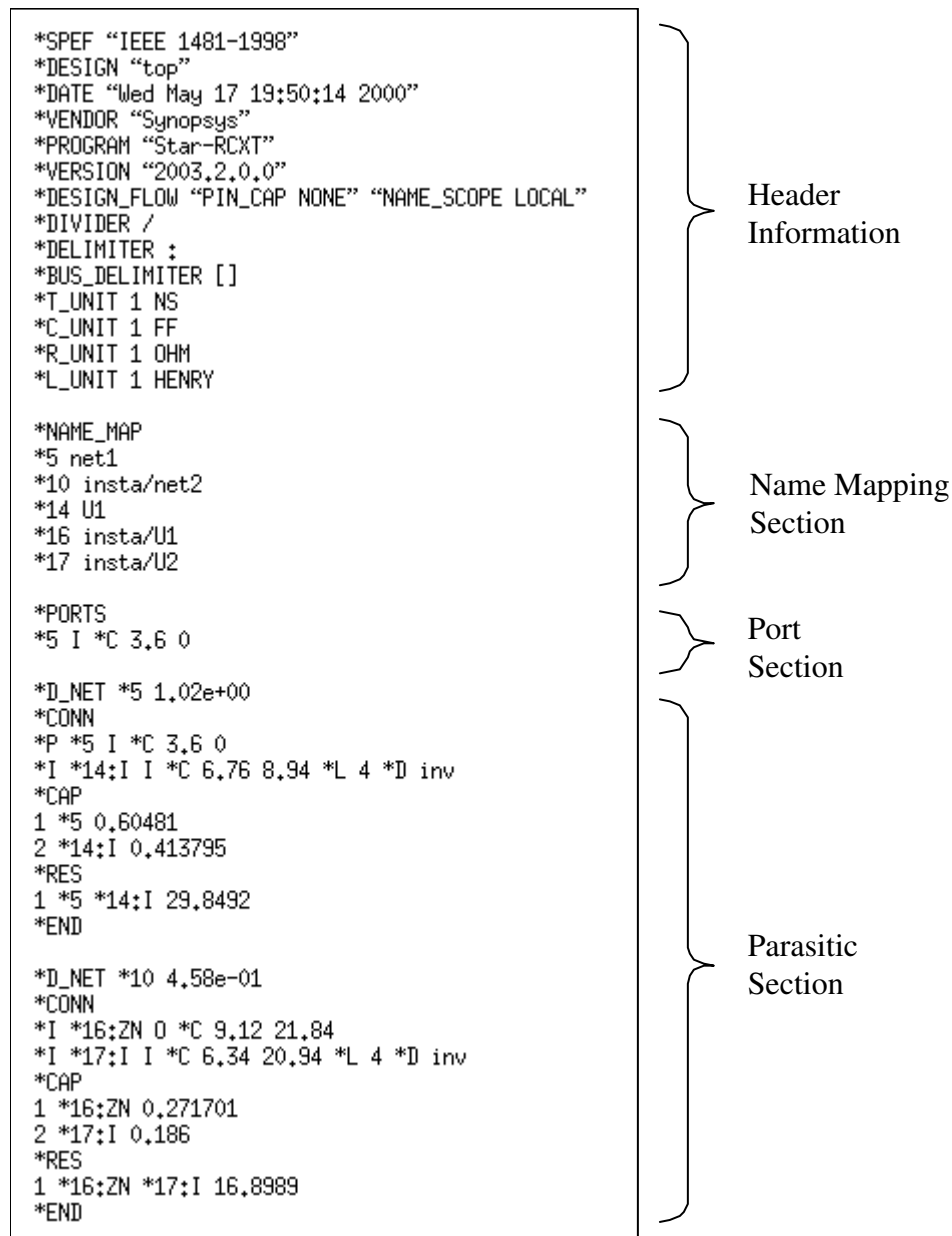
- Header information that provides information about the content of the SPEF file
- Name mapping that maps a name that might be used multiple times to a shorter name to reduce the file size
- Port section that defines a list of top-level ports in a design
- Parasitic section that contains the parasitics representation of the nets

Note:

You can specify the `NETLIST_FORMAT:SPEF` command in the StarRC tool to generate a SPEF file.

Figure 12 shows an example of a SPEF file.

Figure 12 SPEF File Format



Device Property Format (DPF)

A DPF netlist contains the device layout properties for each of the devices. An SPF file generally contains an instance section, which contains the DPF information.

If back-annotating with an SPF file, the PrimeSim XA tool back-annotates the SPF information from the instance section automatically. If an SPF file does not contain an instance section, or when back-annotating with a SPEF file, you can use the StarRC tool to generate a separate DPF netlist, which is identical to the instance section of the SPF file.

The following commands are required for the StarRC tool to generate DPF netlist that the PrimeSim XA tool recognizes.

- The `NETLIST_IDEAL_SPICE_FILE:filename` command specifies the name of the DPF netlist.
- The `NETLIST_IDEAL_SPICE_HIER:NO` command instructs the StarRC tool to generate a flattened DPF netlist. The PrimeSim XA tool does not currently support a hierarchical DPF netlist for DPF back-annotation.

Running the Back-Annotation Flow

The `load_ba_file` command enables back-annotation of the specified parasitic netlist. More than one parasitic netlist can be back-annotated by specifying multiple `load_ba_file` commands. It can be a mixture of both SPEF and SPF netlists. [Table 38](#) lists the commands that are associated with the back-annotation flow.

Table 38 Back-Annotation Commands

Command	Description
<code>map_ba_terminal</code>	Overwrites the terminal name mapping between parasitic netlist and the terminal names recognized by the PrimeSim XA tool.
<code>set_ba_option</code>	Controls many aspects of the back-annotation flow.

When running the back-annotation flow, a pre-layout node might be expanded into several different nodes in the post-layout simulation due to added parasitic elements. It is necessary to understand the exact location of the net when there is a mismatch between the ideal netlist and the parasitic netlist, or when probing a net.

Rules for Selecting a Pin for Each Net

The PrimeSim XA tool identifies one node as the pin in each net. The terminals of any instances that are in the ideal netlist that are not specified in the parasitic netlist are

connected to the pins of the nets for which the terminals belong. See the [Mismatch Warning in Back-Annotation Flow](#) section for more information. This is usually seen in the instances that provide power and stimulus to the netlist, or when the parasitic netlist contains the information only for a portion of the design.

The rules of precedence for selecting a pin in the parasitic netlist are:

1. If the `*|P (nodename ...)` statement exists in SPF netlist or the `*P nodename ...` statement exists in the SPEF netlist, the node with `nodename` of the first `*|P` statement in SPF netlist or the first `*P` statement in the SPEF netlist is the pin of the net.
2. Otherwise, the pin of the net is chosen by using the `set_ba_option -select_ipin_method value` command.

Mismatch Warning in Back-Annotation Flow

When there is a mismatch between the ideal netlist and the parasitic netlist, a problem with simulation accuracy can occur. The PrimeSim XA tool issues explicit warning and error messages when it encounters a mismatch between the ideal netlist and the parasitic netlist, as shown in [Figure 13](#).

Figure 13 Mismatch Warnings Example

#	Description of reason	Number of warnings
1	net does not exist in the design netlist or skipped by user	
2	instance is missing for the non-bulk net	
3	instance is missing for the bulk net	
4	instance does not exist in the design netlist	
5	instance does not connect to this net in design netlist	
6	terminal name is not recognizable	
7	terminal does not connect to this net in design netlist	
8	wrong node name used	
9	dc disconnect in net expansion	

All the following examples are based on this section of the SPF file.

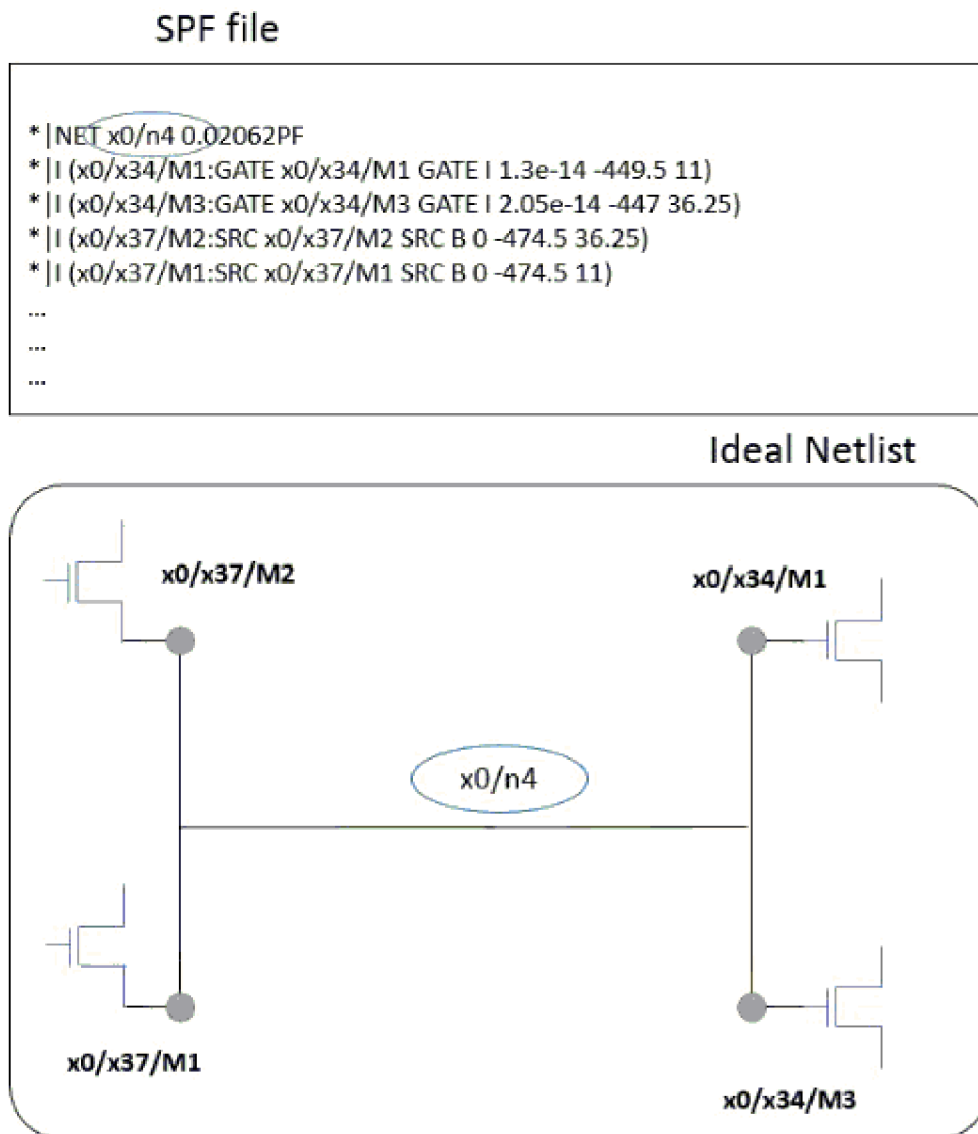
```
*|NET x0/n4 0.02062PF
*|I (x0/x34/M1:GATE x0/x34/M1 GATE I 1.3e-14 -449.5 11)
*|I (x0/x34/M3:GATE x0/x34/M3 GATE I 2.05e-14 -447 36.25)
*|I (x0/x37/M2:SRC x0/x37/M2 SRC B 0 -474.5 36.25)
*|I (x0/x37/M1:SRC x0/x37/M1 SRC B 0 -474.5 11)
...
```

This SPF file shows a net named `x0/n4`. This net is connected to the following terminal of instances:

- Gate of `x0/x34/M1`
- Gate of `x0/x34/M3`
- Source of `x0/x37/M2`
- Source of `x0/x37/M1`

If the ideal netlist matches with the connection of the SPF file as shown in [Figure 14](#), there are no warning messages.

Figure 14 Ideal Netlist Matches With the Connection of the SPF File



These are the most common mismatch issues in the back-annotation flow.

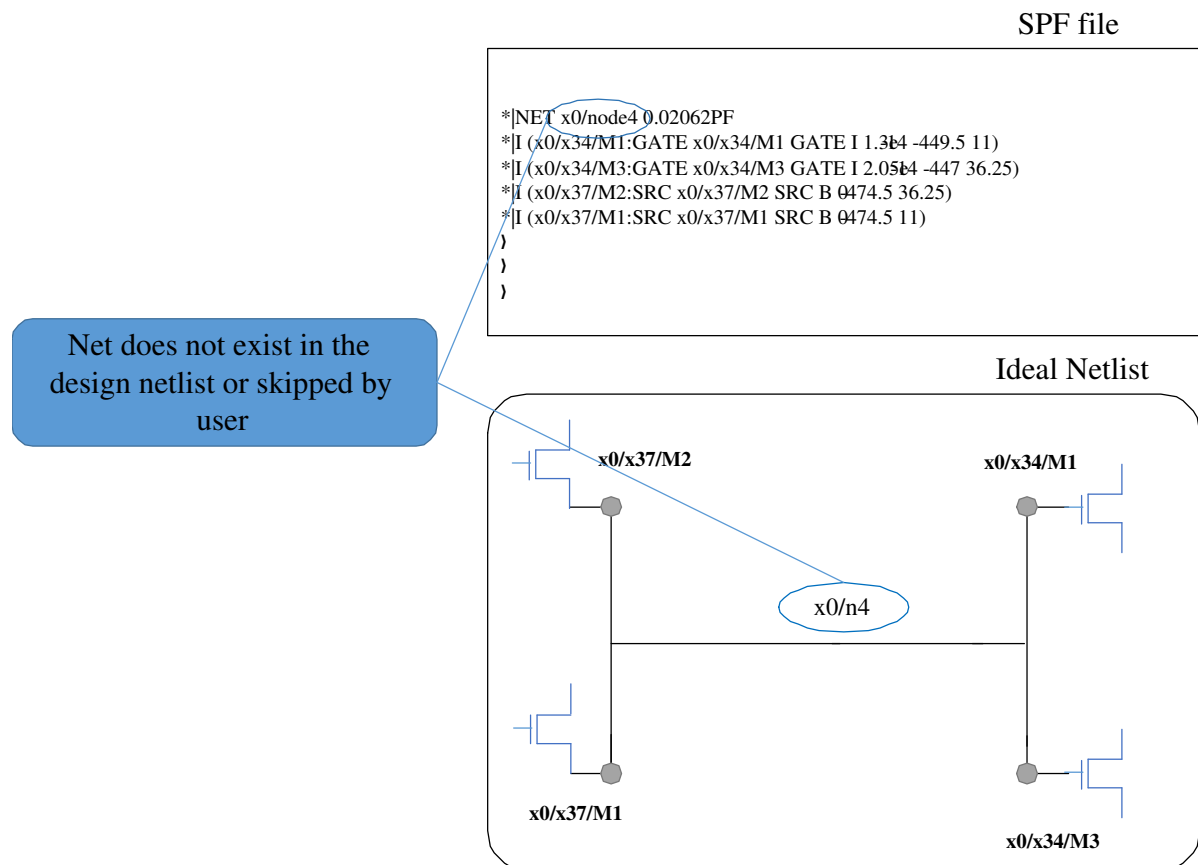
- Net does not exist in the design netlist or is skipped.
- Instance is missing for the non-bulk net.
- Instance does not exist in the design netlist.

- Instance does not connect to this net in the design netlist.
- Terminal name is not recognized.

Net Does Not Exist in the Design Netlist or Is Skipped

Figure 15 shows that the net `x0/node4` in the SPF does not match the node `x0/n4` in the ideal netlist.

Figure 15 Ideal Netlist Does Not Match the Connection in the SPF File



The following message is printed in the log file. The PrimeSim XA tool skips the back-annotation of this `* | NET` statement.

#	Description of reason	Number of warnings
1	net does not exist in the design netlist or skipped by user	1
2	instance is missing for the non-bulk net	
3	instance is missing for the bulk net	
4	instance does not exist in the design netlist	
5	instance does not connect to this net in design netlist	
6	terminal name is not recognizable	
7	terminal does not connect to this net in design netlist	
8	wrong node name used	
9	dc disconnect in net expansion	

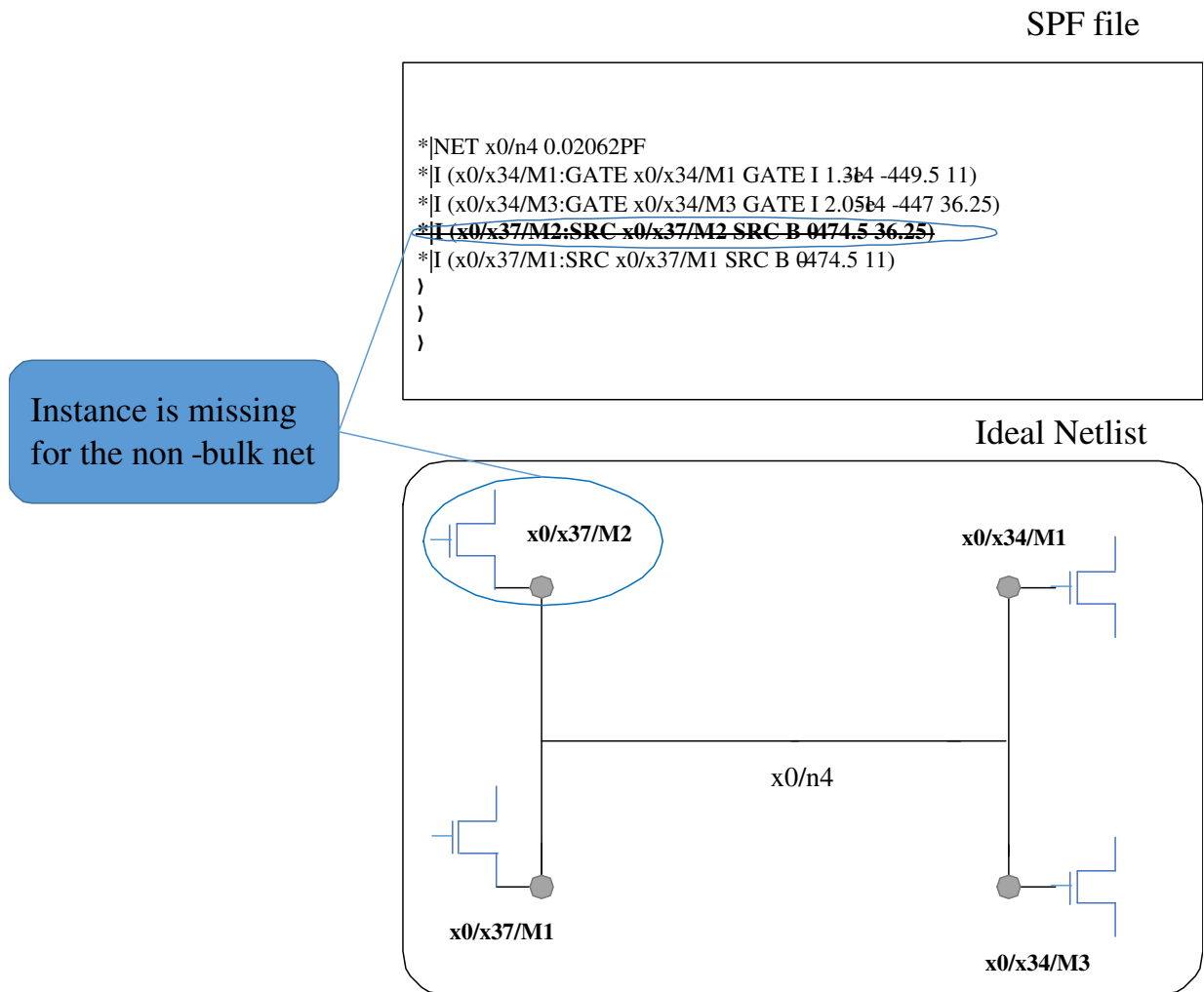
test.spf

line	net	instance	reason/action
18	x0/node4		1/skip net

Missing Instance for the Non-Bulk Net

Figure 16 shows the missing instance in the SPF file. If the line *|I (x0/x37/M2:SRC x0/x37/M2 SRC B 0 -474.5 36.25) is removed from the SPF file, the connectivity in the SPF file does not match the ideal netlist due to a missing instance.

Figure 16 Missing Instance for the Non-Bulk Net



The following message is printed in the log file. The PrimeSim XA tool reconnects the instance to the pin of the net, `x0/x34/M1:GATE`. See the [Rules for Selecting a Pin for Each Net](#) section for details.

#	Description of reason	Number of warnings
1	net does not exist in the design netlist or skipped by user	1
2	instance is missing for the non-bulk net	
3	instance is missing for the bulk net	
4	instance does not exist in the design netlist	
5	instance does not connect to this net in design netlist	
6	terminal name is not recognizable	
7	terminal does not connect to this net in design netlist	
8	wrong node name used	
9	dc disconnect in net expansion	

test.spf

line	net	instance	reason/action
18	X0/N4	x0.x37.m2:3	2/connect to pin
		-> X0/X34/M1:GATE	

Instance Does Not Exist in the Design Netlist

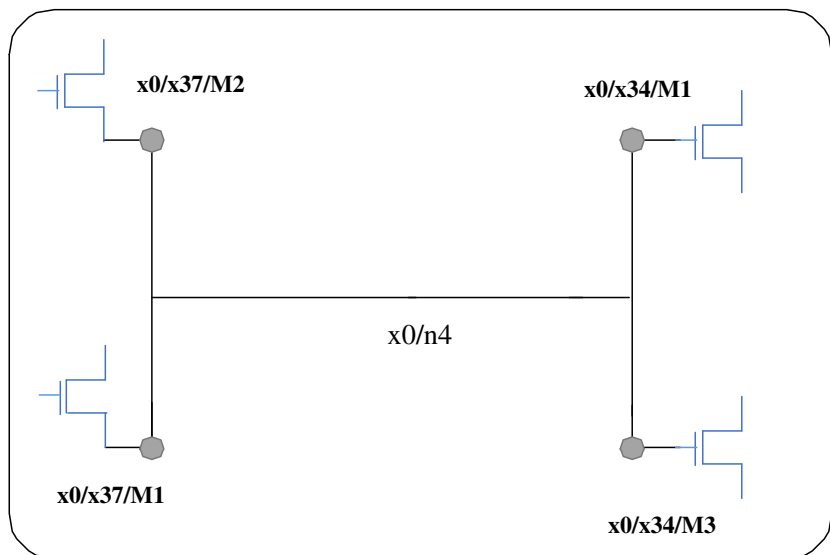
Figure 17 shows the additional instance in the SPF file. If the line *|I (x0/x37/M3:SRC x0/x37/M3 SRC B 0 -474.5 11) is added to the SPF, but this instance does not exist in the ideal netlist, the tool shows the mismatch between the ideal netlist and the SPF file.

Figure 17 Instance Not Exist in the Design Netlist

Instance does not exist in the design netlist

```
*|NET x0/n4 0.02062PF
*I (x0/x34/M1:GATE x0/x34/M1 GATE I 1.3e4 -449.5 11)
*I (x0/x34/M3:GATE x0/x34/M3 GATE I 2.05e4 -447 36.25)
*I (x0/x37/M2:SRC x0/x37/M2 SRC B 0474.5 36.25)
*I (x0/x37/M1:SRC x0/x37/M1 SRC B 0474.5 11)
*I (x0/x37/M3:SRC x0/x37/M3 SRC B 0474.5 11)
)
)
)
```

Ideal Netlist



The following message is printed in the log file. The PrimeSim XA tool skips the back-annotation of this `*| I` statement.

#	Description of reason	Number of warnings
1	net does not exist in the design netlist or skipped by user	
2	instance is missing for the non-bulk net	
3	instance is missing for the bulk net	
4	instance does not exist in the design netlist	1
5	instance does not connect to this net in design netlist	
6	terminal name is not recognizable	
7	terminal does not connect to this net in design netlist	
8	wrong node name used	
9	dc disconnect in net expansion	

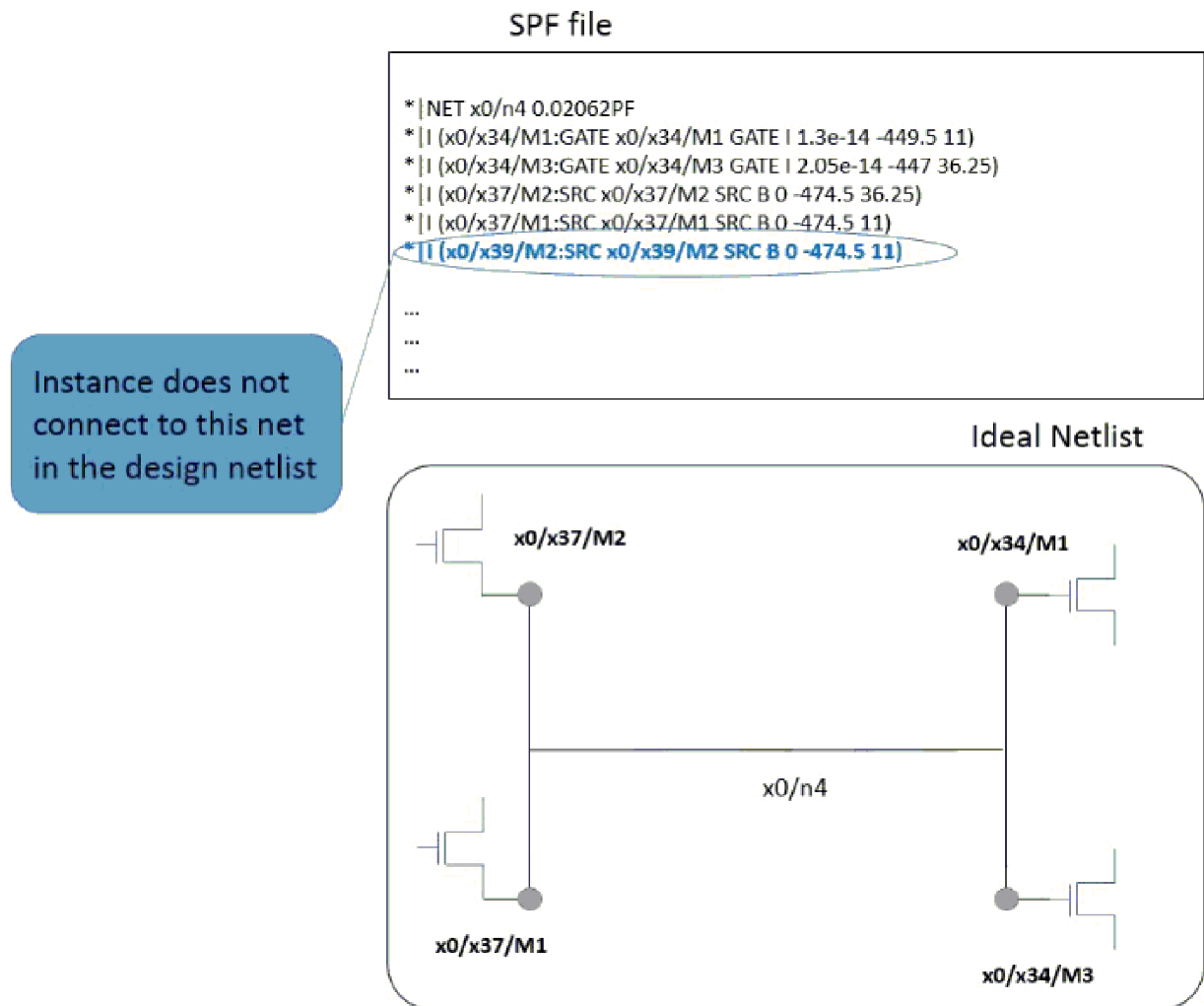
test.spf

line	net	instance	reason/action
23	X0/N4	x0/x37/M3	4/skip instance

Instance Does Not Connect to This Net in the Design Netlist

Figure 18 shows the additional instance in the SPF file. If the line `*|I (x0/x39/M2:SRC x0/x39/M2 SRC B 0 -474.5 11)` is added to the SPF and this instance exists in the ideal netlist, but it does not connect to net `x0/n4`, the tool shows the mismatch between the ideal netlist and the SPF file.

Figure 18 Instance Does Not Connect to Net in the Design Netlist



The following message is printed in the log file. The PrimeSim XA tool skips the back-annotation of this *| I statement.

#	Description of reason	Number of warnings
1	net does not exist in the design netlist or skipped by user	1
2	instance is missing for the non-bulk net	
3	instance is missing for the bulk net	
4	instance does not exist in the design netlist	
5	instance does not connect to this net in design netlist	
6	terminal name is not recognizable	
7	terminal does not connect to this net in design netlist	
8	wrong node name used	
9	dc disconnect in net expansion	

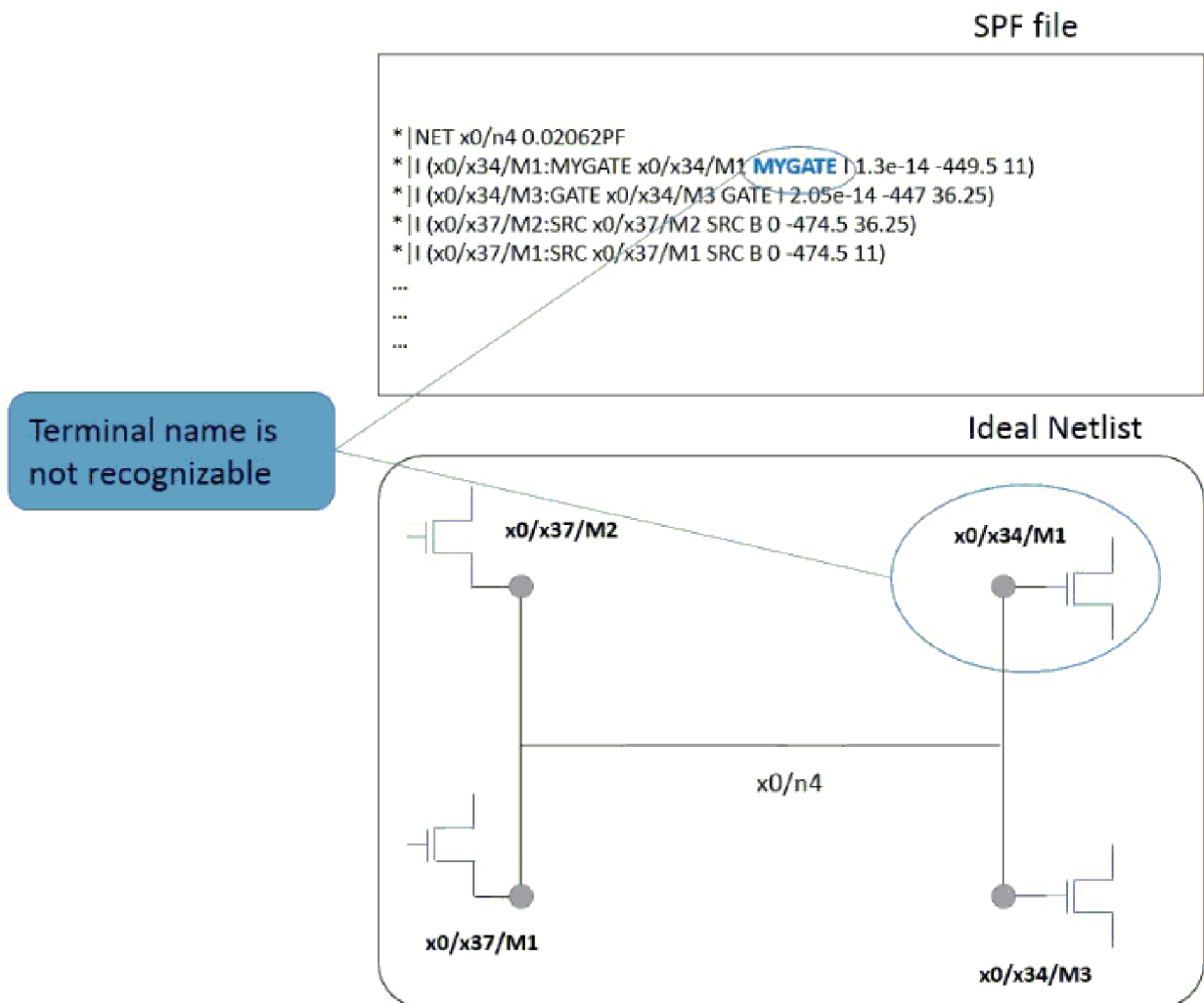
test.spf

line	net	instance	reason/action
23	X0/N4	x0/x39/M2:SRC	5/skip instance

Terminal Name Is Not Recognizable

Figure 19 shows the unrecognized terminal name. See the map_ba_terminal command for the supported terminal names for each device. The terminal name, MYGATE, is not a supported terminal name for MOSFET.

Figure 19 Terminal Name Not Recognizable



The following message is printed in the log file. The PrimeSim XA tool skips the back-annotation of this `*|I` statement, and the gate terminal of `x0.x34.m1` is reconnected to the pin of the net, `x0/X34/M3:GATE`. See the [Rules for Selecting a Pin for Each Net](#) section. You can use the `map_ba_terminal` command to map `MYGATE` to the supported terminal name.

#	Description of reason	Number of warnings
1	net does not exist in the design netlist or skipped by user	
2	instance is missing for the non-bulk net	1
3	instance is missing for the bulk net	
4	instance does not exist in the design netlist	
5	instance does not connect to this net in design netlist	
6	terminal name is not recognizable	1
7	terminal does not connect to this net in design netlist	
8	wrong node name used	
9	dc disconnect in net expansion	

test.spf

line	net	instance	reason/action
19	X0/N4	x0/x34/M1:MYGATE	6/skip instance
18	X0/N4	x0.x34.m1:2	2/connect to pin
		-> X0/X34/M3:GATE	

Using the Analysis Statement in the Post-Layout Flow

The back-annotation flow supports all analysis statements. You can reuse the analysis statements from the pre-layout simulation in the post-layout simulation using the back-annotation flow. Probing of terminal currents and voltages works identically to those in the ideal netlist. Probed nodes might be different, because nodes in the pre-layout simulation might be expanded into several different nodes in the post-layout simulation in the back-annotation flow due to added parasitic elements. Therefore, it is important to write analysis statements that use voltage probes as specifically as possible so you can predict the exact nodes for analysis in the back-annotation flow.

Probing of a node, which is specified in the `*|NET` statement, probes the pin node of the nets. The pin node is specified using the precedence rules described in the [Rules for Selecting a Pin for Each Net](#) section. Probing of a node, which is not specified in the `*|NET` statement, probes the node of the first `*|I` statement that has the same hierarchy instance.

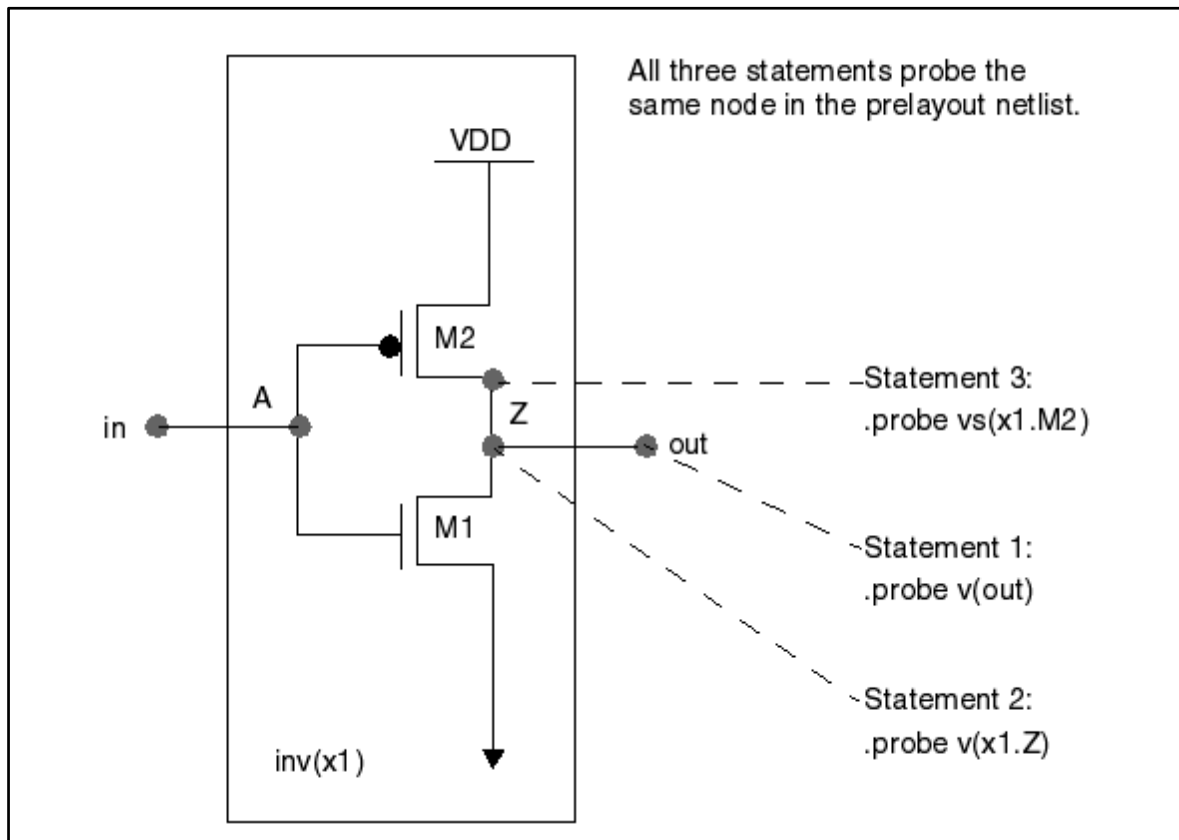
The following example shows how different analysis statements probe different nodes in the back-annotation flow.

```
.SUBCKT inv A Z
m1 GND A Z GND n ...
m2 VDD A Z VDD p ...
.ENDS inv
x1 in out inv* Analysis Statement
.PROBE v(out) $ statement1
```

```
.PROBE v(x1.Z) $ statement2
.PROBE vs(x1.m2) $statement3
```

A graphical representation of the inverter schematic is shown in [Figure 20](#). In the prelayout simulation, all three analysis statements probe the same node.

Figure 20 Probe Statements Example

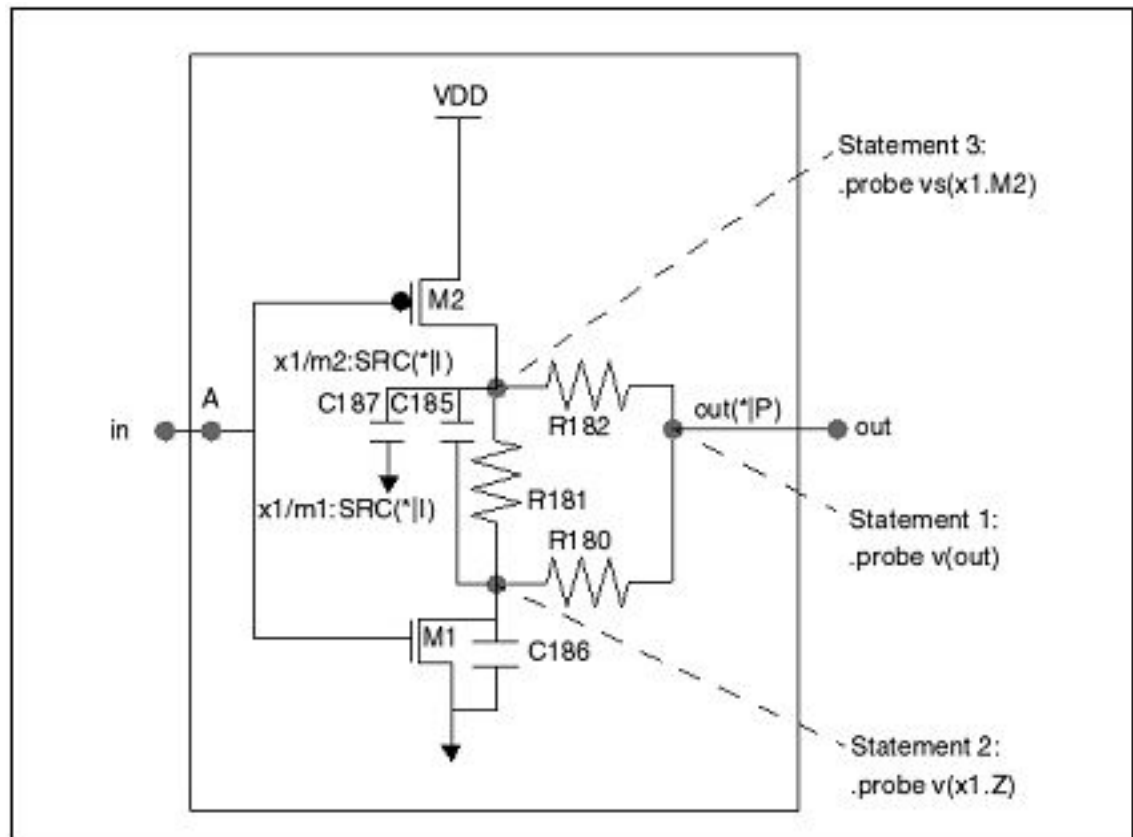


This is the SPF description of net out.

```
*|NET out ...
*|I (x1/m1:SRC x1/m1 SRC ...)
*|I (x1/m2:SRC x1/m2 SRC ...)
*|P (out ...)
C185 x1/m1:SRC x1/m2:SRC ...
C186 x1/m1:SRC 0 ...
C187 x1/m2:SRC 0 ...
C188 b 0 ...
R180 x1/m1:SRC out ...
R181 x1/m1:SRC x1/m2:SRC ...
R182 x1/m2:SRC out...
```

Figure 21 shows a graphical representation of the parasitic interconnects after the back-annotation of `net out`.

Figure 21 Representation of `net.out`



In statement 1, `.PROBE v(out)`, `net out` exists in an SPF parasitic netlist; therefore, the PrimeSim XA tool follows the rules of precedence described in the [Rules for Selecting a Pin for Each Net](#) section to probe the pin of the `net out`.

In statement 2, `.PROBE v(x1.Z)`, `x1.Z` is not specified by the `*|NET` statement; therefore, the PrimeSim XA tool probes the node of the first `*|I` statement that has the same hierarchy instance as `x1.Z` (`x1/m1:SRC`).

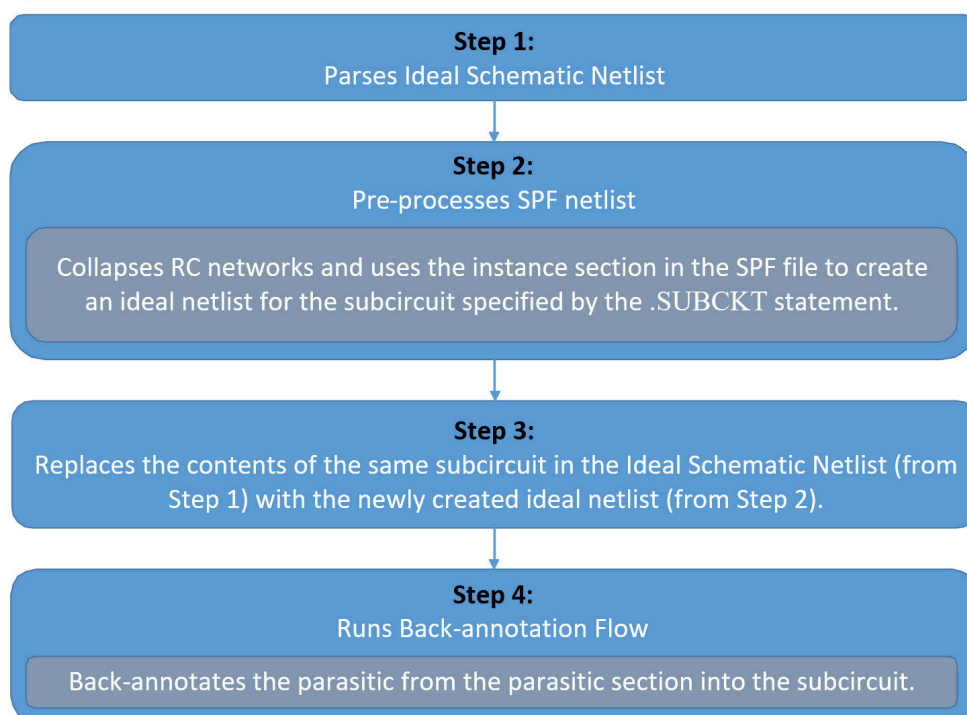
In statement 3, `.PROBE vs(x1.m2)` is an instance-based waveform-naming statement; therefore, the PrimeSim XA tool probes the source terminal of transistor `x1.m2`.

Extended Back-Annotation Flow

The traditional back-annotation flow is based on device-to-device name matching between the pre-layout and post-layout netlists. In some cases, this approach does not provide 100% matching due to stacking device configurations, finger nets, and other discrepancies.

The extended back-annotation flow provides another solution to overcome this issue, by replacing the entire contents of the pre-layout subcircuit with the post-layout contents of the same subcircuit in the SPF file. It guarantees a clean (100% device and parasitic) back-annotation and incorporates existing RC back-annotation functionality. It only works with the SPF file, and SPEF file is not supported. The SPF file must contain subcircuit definitions with the `.SUBCKT` statement and the instance section. [Figure 22](#) describes the steps in the PrimeSim XA tool when enabling the extended back-annotation flow.

Figure 22 *Extended Back-Annotation Flow*



When the contents of the subcircuit in the ideal schematic netlist is replaced by the contents of the newly created ideal netlist, the device names and the node names from the ideal schematic netlist are no longer available in the database to the analysis statements.

The following new information is saved in the database based on the newly created ideal netlist:

- The device and instance names in the instance section of the SPF file.
- The node names and instance names specified by the `*|NET` statement in the SPF file. The slash ("/") character in the node names and instance names are replaced by the period (".") character.

Running the Extended Back-Annotation Flow

The extended back-annotation flow follows the same rules as the back-annotation flow. To run extended back-annotation flow, you need to enable the `-xba` argument in the `load_ba_file` command.

The following example loads in the `addr8.spf` file and enables extended back-annotation flow for the subcircuit `addr8`. The device names and node names in the subcircuit are replaced by the device names and node names in the SPF file.

```
load_ba_file -file addr8.spf -xba 1
```

Limitations of the Extended Back-Annotation Flow

The extended back-annotation flow has the following limitations:

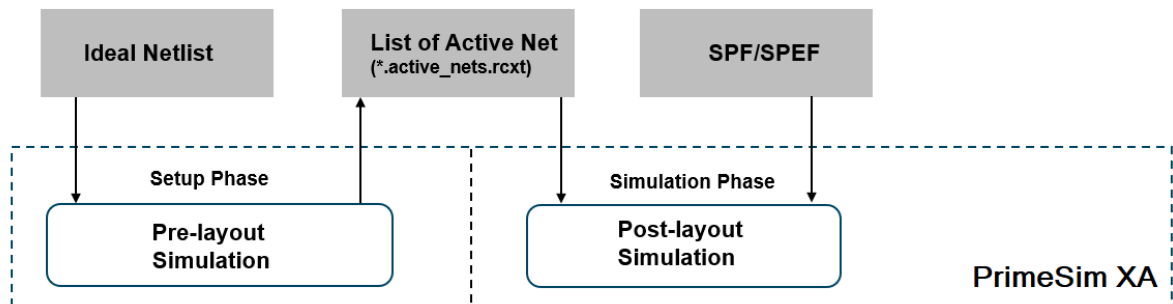
- Subcircuit and instance-based simulation command control is not supported if the subcircuit or instance is nested inside the subcircuit when the extended back-annotation flow is enabled.
- The `probe_waveform_current [-isub|-x subckt_instance_name.port]` command is not supported if the subcircuit or instance is nested inside the subcircuit when the extended back-annotation flow is enabled. You can use the `probe_waveform_ixba` command to provide similar functionality.

Selective Net Back-Annotation Flow

When you run a post-layout simulation, you often have a large number of nets with parasitic resistors and capacitors. Usually the simulation result is only impacted by parasitics of the nets that are active during the transient simulation. The PrimeSim XA tool supports the selective net back-annotation flow, which is ideal to run post-layout simulation with high latency. This flow reduces the back-annotated parasitics and therefore improves the simulation run time.

The PrimeSim XA tool runs a one-step simulation with two phases in the selective net back-annotation flow, as shown in Figure 23. The first step is the setup phase, when the PrimeSim XA tool runs a pre-layout simulation, and detects and reports the active nets. The second is a simulation phase where the PrimeSim XA tool back-annotates only the active nets from the parasitic netlist and runs a post-layout simulation. When setup phase completes, the PrimeSim XA tool automatically starts the simulation phase.

Figure 23 Selective Net Back-Annotation Flow



Running the Selective Net Back-Annotation Flow

Selective net back-annotation flow follows the same rules as the back-annotation flow. To enable selective net back-annotation flow, you need to specify both the `load_ba_file` and `set_active_net_flow` commands.

To detect the active nets, the PrimeSim XA tool runs a pre-layout simulation in the setup phase, ignoring the `load_ba_file` commands in the command file. The tool checks the peak-to-peak voltages of each net. If the peak-to-peak voltage is more than the tolerance specified in `-vtol` argument in the `set_active_net_flow` command (default is 100mV), then the net is considered active and therefore is printed into a file with a `*.active_nets.rcxt` suffix.

Once the pre-layout simulation is finished, the simulation phase starts automatically. The PrimeSim XA tool back-annotates the active nets based on the `*.active_nets.rcxt` file with the full RC description from the parasitic netlist. It also back-annotates the non-active nets with the total ground capacitor specified in `*|NET` statement from the parasitic netlist.

The following example invokes selective net back-annotation. A net is considered active if its peak-to-peak voltage is greater than 20 mV at any time during the transient simulation. When the PrimeSim XA tool has detected all active nets, it runs a post-layout simulation and back-annotates the RC description of those nets from the DSPF.spf file.

```
set_active_net_flow -switch 1 -vtol 20m
load_ba_file -file DSPF.spf
```


Note:

If the `.active_nets.rcxt` file already exists and you rerun a simulation with the same command line arguments with same output name, the first step is omitted. If you have changed some settings, such as the voltage tolerance of active net, it may not be taken into account in the PrimeSim XA simulation. Always check the PrimeSim XA log file for details.

Reusing the *.active_nets.rcxt File

You can reuse the `*.active_nets.rcxt` file to only run the simulation phase using the `set_ba_option -active_net_file fileName` command. The file name is the name of the active net file. The following example shows that you only run the simulation phase of selective net back-annotation flow with an active net file, named `sram.active_nets.rcxt`.

```
load_ba_file -file DSPF.spf
set_ba_option -active_net_file sram.active_nets.rcxt
```

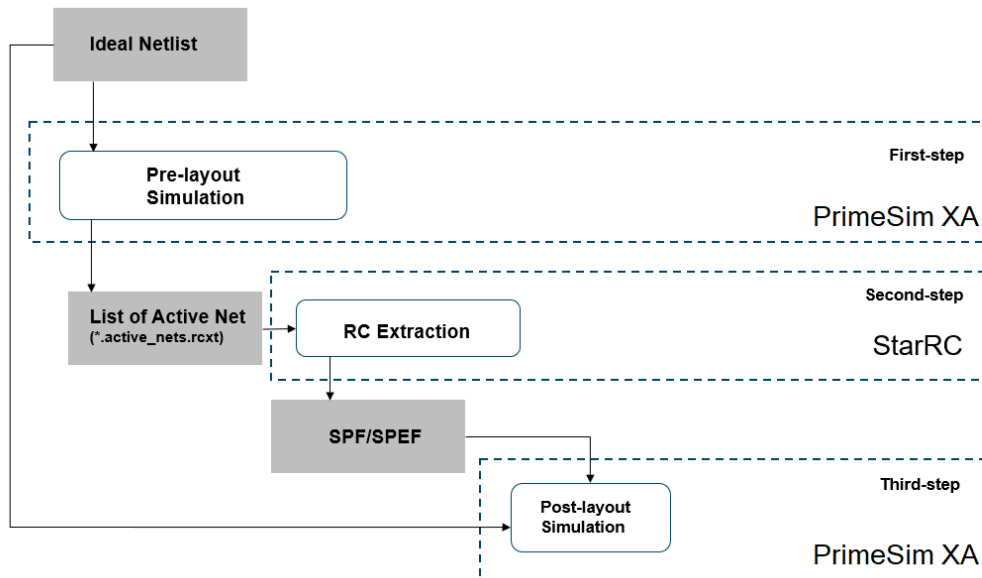
Selective Net Extraction Flow

Large designs often require long extraction time in the StarRC tool and long simulation time in the PrimeSim XA tool due to the large amount of parasitics. You can use the selective net extraction flow in the large post-layout designs with high latency to significantly reduce the extraction and simulation run time. To run this flow, you must own and be familiar with both the StarRC and PrimeSim XA tools.

This flow is useful when extraction generates a large parasitic netlist detecting the active nets and extracting only those nets, which leads to a smaller and reduced parasitic netlist. The PrimeSim XA tool uses less memory and runs faster in the post-layout simulation. Even though it is a three-step process, the total time is still faster than extracting and simulating the design with complete parasitics.

The selective net extraction flow is shown in [Figure 24](#).

Figure 24 Selective Net Extraction Flow



Running the Selective Net Extraction Flow

To run the selective net extraction flow, do the following steps.

1. Generate an active net file from a pre-layout simulation using the following command:

```
set_ba_active_file [-file] filename [-vtol tolerance_value]
```

A net is considered active if its peak-to-peak voltage is more than the *tolerance_value*. Refer to the *PrimeSim™ XA Command Reference* for details about this command. It generates a file with a suffix of **.rcxt*, which contains a list of active nets.

The following example generates a file which lists the nets whose peak-to-peak voltage is greater than 10 mV.

```
set_ba_active_file active_nets -vtol 10m
```

2. Load the active net file generated in Step 1 in the StarRC tool to generate a smaller and reduced parasitic netlist. The command to load the active net file in the StarRC tool is:

```
NET_FILES: filename
```

The *filename* is the name of the active net file generated at Step 1.

Note:

You might need to comment out the `NETS: *` statement in the StarRC command file. This command forces extraction of all nets.

3. Load the smaller and reduced parasitic netlist into the PrimeSim XA tool for post-layout simulation. The command to load the parasitic file is:

```
load_ba_file -file parasitic_filename
```

8

Sigma Amplification

This chapter describes how to run the sigma amplification simulation in PrimeSim XA.

The PrimeSim XA tool supports sigma amplification simulation in Monte Carlo data mining post-processing flow. In PrimeSim XA, you run sigma amplification simulation to validate the robustness of your circuit design and generate design-specific variation corners for circuit debugging and failure diagnosis.

When simulation is complete, the simulation results are post-processed by performing datamining on all measures defined in the netlist. The log file and output files generated during simulation and post-processing stages are fully HSPICE-compatible.

For a detailed description about sigma amplification in the advanced variability analysis flow, see the Sigma Amplification chapter in *PrimeSim™ AVA User Guide*.

This section contains the following topics:

- [Running Sigma Amplification](#)
- [Configuring Inputs for Sigma Amplification](#)
- [Variation Analysis With Design-Specific Variation Corners](#)

Running Sigma Amplification

To run sigma amplification, invoke the advanced variability analysis flow using the following syntax:

```
% xa -hspice netlist.sp -o outdir/ -adv my.json
```

The my.json file contains the sigma amplification setup parameters to execute the advanced variability analysis flow. If the .json file is not available, the Advanced Variation Analysis dialog box is invoked for entering the sigma amplification setup parameters to create the configuration file (.json). For more information, see [Configuring Inputs for Sigma Amplification](#).

Following is a sample .json file:

```
{
  "Active_Application": "Sigma_Amplification",
  "Sigma_Amplification": {
    "System_Sigma": <target sigma scale> ,
    "Component_Count": 1,
    "Average_Relative_Uncertainty": <uncertainty scale>,
    "N_Afford_Samples": <target sample run bandwidth>
  }
}
```

Output Files

Table 39 lists the output files that are generated when sigma amplification simulation is completed.

Table 39 *Output Files Generated by Sigma Amplification Simulation*

File	Description
out.pmpp0	Report file with moments and percentiles
out.pdf, out.wv_plt, and out.qq.csv	Q-Q plot files in the PDF (*.pdf file), the WaveView (*.wv_plt file), and CSV (out.qq.csv file) formats
out.plog	Log file
out.ava. dsvc.json	The file that contains design-specific variation block information

For more information about running sigma amplification simulation, see the Sigma Amplification chapter in *PrimeSim™ AVA User Guide*.

Note:

The following PrimeSim XA configuration command is the legacy Monte Carlo datamining process with the PrimeSim HSPICE tool. When specified in the advanced variability analysis flow, the tool issues a warning message and ignores the command.

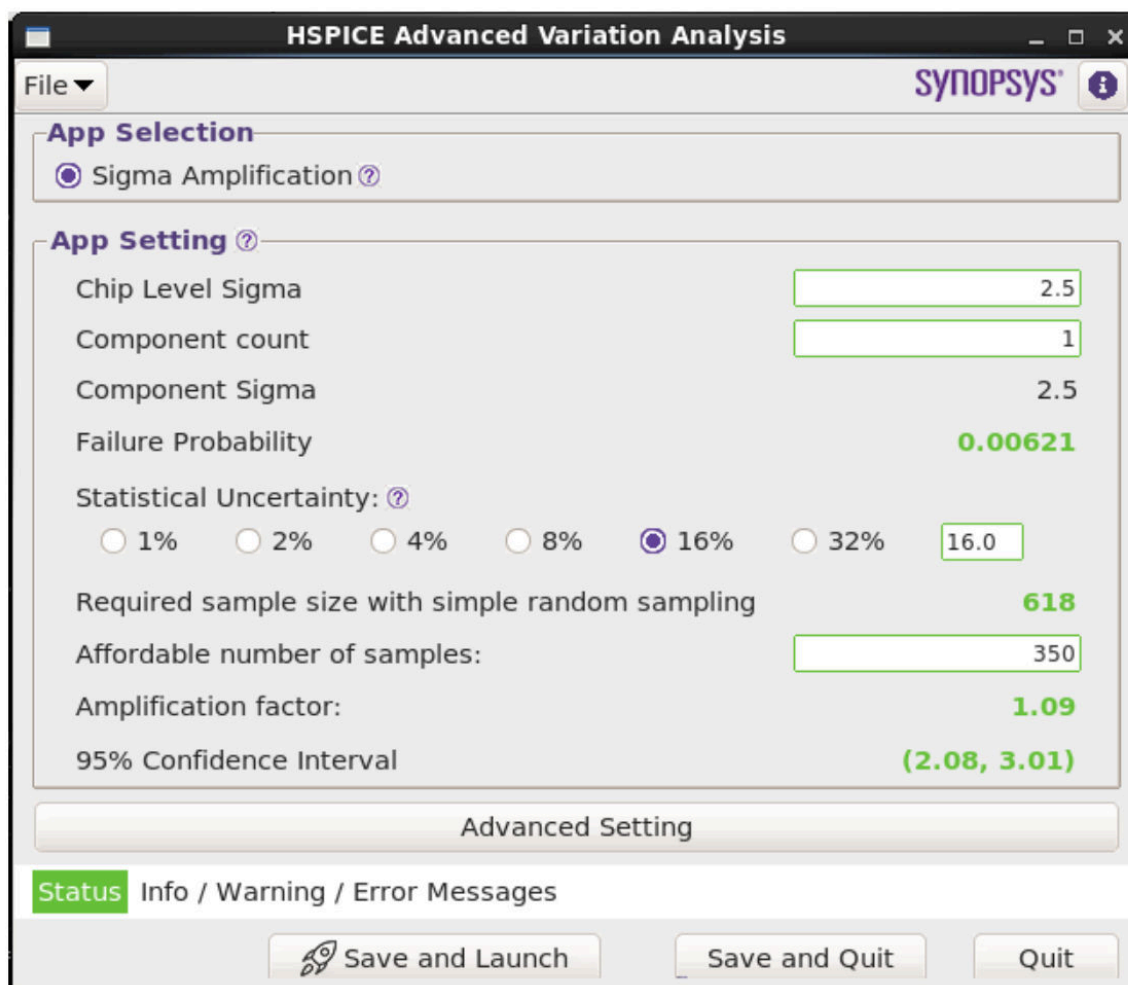
```
set_monte_carlo_option -datamining on
```

The advanced variability analysis platform only deploys the HSPICE compliant measurement (.mt) format. When `.option measform=0|1|2` is specified in the advanced variability analysis flow, the tool issues a warning message and overwrites the setting with the defaults in advanced variability analysis flow.

Configuring Inputs for Sigma Amplification

Figure 25 shows the Advanced Variability Analysis dialog box in which you configure inputs for running sigma amplification simulation.

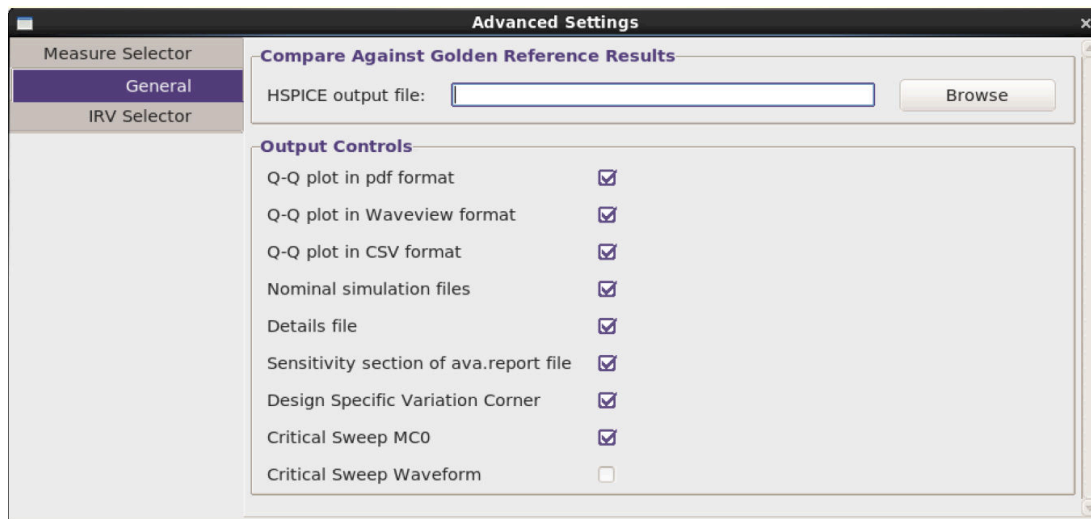
Figure 25 Advanced Variability Analysis: Sigma Amplification Dialog Box



In the Advanced Variation Analysis dialog box,

1. Configure basic settings to create a .json file. The tool computes and auto-populates the **Component Sigma**, **Failure Probability**, **Required sample size with simple random sampling**, and **Amplification factor** fields based on the values of these basic settings. For more information, see the following [Sigma Amplification](#) section.
2. Click the **Advanced Settings** button to configure advanced settings in the Advanced Settings dialog box that opens (see [Figure 26](#)).

Figure 26 The Measure Selection Dialog Box



3. In the Advanced Settings dialog box that opens,
 - Click the **Measure Selector** tab to select measures for analysis.
 - Click the **General** tab to specify additional advanced settings.

In the Output Controls field, choose the output files and formats to generate after simulation is completed. By default, all outputs are generated. Deselect some of these to reduce storage requirements, especially for script-based computations on a large number of circuits.

Check the Design Specific Variation Corner box to output the IRVs of tail samples for each measure in the netlist, based on the selected tails. You can select a specific block for which the IRV information is required using the DSVC Selector tab. By default, the outputs are saved in the out.ava.dsvc.json file.

Check the Critical Sweep MC0 box to write critical corner sweep indices and corresponding IRVs in MC0 file format. Critical corners include minimum and maximum measure values or failed measures and other user-specified quantiles. By default, the mc0 file is named out.ava.critical.mc0.

Check the Critical Waveform file box to simulate the critical samples that are in the above out.ava.critical.mc0 file. The waveform file is generated only if the netlist contains probe statements. This option is not enabled by default.

- Click the **IRV Selector** tab to limit the variation to certain parts (such as models, subcircuits and instance) only.

When finish configuring the advanced settings, close the Advanced Settings dialog box.

4. In the Advanced Variation Analysis dialog box, click the **Save and Launch Simulation** button to save the newly created .json file and run sigma simulation. To save the .json file without launching the simulation, click the **Save and Quit** button.

Determining the Amplification Factor for Sigma Amplification

The following factors influence the choice of the amplification factor for Sigma Amplification:

- How many samples can be simulated (computational budget).
- The required/target accuracy in terms of percentiles or sigmas.

The following table summarizes the links between the basic settings in the Sigma Amplification dialog box and the amplification factor for Sigma Amplification.

Table 40 *Sigma Amplification Basic Settings and Their Significance*

Setting	Significance
Chip Level Sigma	The higher this value, the higher is the amplification factor.
Component count	The higher this number, the higher is the Component Sigma and the amplification factor.
Statistical Uncertainty	The lesser this value, the narrower is the confidence interval and higher is the amplification factor.
Affordable number of samples	The lower this value, the higher is the amplification factor.

Depending on the configured values of these basic settings, the Sigma Amplification engine automatically calculates the amplification factor for the Sigma Amplification simulation.

Variation Analysis With Design-Specific Variation Corners

Sigma amplification simulation utilizes Monte Carlo design variation analysis to calculate design-specific variation corners for circuit debugging and failure diagnosis. By specifying different measures and target sigma values for simulation, you can identify what the critical corners are in your design and then re-run simulation on these corners to find out what causes the performance failure, thus improving robustness of the circuit.

The design-specific variation corners are saved in the output `.ava.dsvc.json` file.

To generate design-specific variation corners during sigma amplification analysis and run variation analysis with the generated design-specific variation corners,

1. Follow the steps in [Running Sigma Amplification](#) to run sigma amplification analysis on a bit cell or a sense amplifier. If you do not have a configuration file, follow the procedures in [Configuring Inputs for Sigma Amplification](#) to configure necessary settings to create a configuration file, such as specifying the measures and target sigma value for calculating variation corners and output file formats.
2. When sigma amplification simulation is finished, examine the output `*.ava.dsvc.json` file, report file (`*.amp_mpp`) and Q_Q plot files (`*.pdf` or `*.wv_plt`) to identify the critical corners in your circuit design.
3. Rerun sigma amplification on the instances to investigate with the `*.ava.dsvc.json` file generated at Step1.

For a detailed description about sigma amplification simulation, see the related sections in *PrimeSim™ AVA User Guide*.

9

Monte Carlo Analysis

This chapter describes the Monte Carlo analysis in PrimeSim XA.

Monte Carlo analysis is a common method used to understand the effect of random process variations on circuit performance. During a Monte Carlo analysis, the simulation is run a pre-determined number of times with small random variations in design and process variables. The simulation results are combined and correlated to determine key factors that might affect performance.

The PrimeSim XA tool supports traditional Monte Carlo analysis for transient simulations with the PrimeSim HSPICE netlist format. Traditional Monte Carlo defines a random variable with a distribution function. You can assign a random variable just as in PrimeSim HSPICE. The PrimeSim XA tool supports the same distribution functions as PrimeSim HSPICE:

- UNIF(nominal_val, rel_variation)
- AUNIF(nominal_val, abs_variation)
- GAUSS(nominal_val, rel_variation, num_sigmas)
- AGAUSS(nominal_val, abs_variation, num_sigmas)
- LIMIT(nominal_val, abs_variation)

Whenever a parameter defined by one of these functions is assigned, a new unique random variable is generated. For more information about distribution functions, see the .PARAM Distribution Function section in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

Monte Carlo simulation indexing in the PrimeSim XA tool is compliant to the PrimeSim HSPICE tool. The sample of index 1 is the nominal simulation, and the sample of index 2 is the first random simulation. For more information about sample indexing, see the Monte Carlo Analysis—Flow and Outputs section in *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

PrimeSim XA provides the `set_monte_carlo_option` command to specify options for running traditional Monte Carlo analysis in PrimeSim XA. See the [Configuring Options for Monte Carlo Analysis](#) section for details.

For more information about traditional Monte Carlo simulation, see the Traditional Monte Carlo Simulations section in *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

This chapter includes the following topics:

- [Configuring Options for Monte Carlo Analysis](#)
- [Running Monte Carlo Analysis in PrimeSim XA](#)
- [Output Files From Monte Carlo Analysis](#)
- [Running Incremental Monte Carlo Simulations](#)
- [Setting Instance-Based Seeds](#)
- [External Sampling During Monte Carlo Analysis](#)
- [Random Variables](#)
- [Data Mining Flow](#)
- [Auto-Replacing Monte Carlo Samples When Measurements Fail](#)
- [Controlling the Read-in of an External File](#)
- [Running Monte Carlo Analysis With 3DIC](#)
- [Variation Blocks in Monte Carlo Flow](#)

Configuring Options for Monte Carlo Analysis

You can run the `set_monte_carlo_option` command to specify options for running traditional Monte Carlo analysis in PrimeSim XA, such as output files, data mining, and so on.

The following table lists the available options of the `set_monte_carlo_option` command. For a detailed description about the command, see the `set_monte_carlo_option` command page in *PrimeSim XA Command Reference Manual*.

Table 41 Options for Monte Carlo Analysis

<code>-enable</code>	Enables Monte Carlo samples. The default is ON.
<code>-sample_output</code>	Specifies the Monte Carlo sample index in which the output files of measurement result files and the waveform files are kept in the output directory.

Table 41 Options for Monte Carlo Analysis (Continued)

<code>-mc0_file</code>	Creates a .mc0 file which lists all of the random variable values used in the simulation, and is equivalent to the PrimeSim HSPICE *.mc0 file.
<code>-parameter_file</code>	Creates a .mc_params parameter file.
<code>-mc0_header</code>	Prints the header information in the output .mc0 file.
<code>-dump_waveform</code>	Writes out waveforms when simulation is complete.
<code>-datamining</code>	Enables Monte Carlo data mining post-processing flow.
<code>-auto_replace</code>	Automatically replaces a failed simulation sample in DP by a new one when running Monte Carlo analysis.
<code>-compress_file</code>	Enables file compression of the specified <i>file_ext</i> files.
<code>-dp_parallel_post_process</code>	Enables DP parallel post-processing feature.

Example

```
.tran 1n 100n sweep Monte=2
.opt xa_cmd="set_monte_carlo_option -sample_output all"
```

Running Monte Carlo Analysis in PrimeSim XA

You can run Monte Carlo analysis in the PrimeSim XA environment with the following syntax:

```
.TRAN step start stop [SWEEP monte=mc_cmd]
```

Argument	Description
<code>monte=mc_cmd</code>	<i>mc_cmd</i> can be one of the following: <code>val</code>: Specifies the number of random samples to produce. <code>val firstrun=num</code>: Specifies the sample number on which the simulation starts. <code>list num</code>: Specifies the sample number to execute. <code>list(num1:num2 num3 num4:num5)</code>: Samples from num1 to num2, sample num3, and samples from num4 to num5 are executed. Using parentheses is optional.

For a detailed description about the HSPICE `.TRAN` command, see the `.TRAN / TR` command page in *PrimeSim Continuum Reference Manual: Commands and Options*.

Note:

- There is only one FE Summary for a single worker, no matter how many Monte Carlo samples are executed by a given worker.
- For the global variation dependent parameter dump, the PrimeSim XA tool now only writes out the dependent global parameter without its hierarchical name, which complies with PrimeSim HSPICE syntax.
- To save disk space, once the `-datamining` on command is triggered, only the *.mc0, *.mt and *.log files are kept and compressed in .gzip format. To disable file compression, use the `set_monte_carlo_option -compress_file none` command.
- If the `-gz` command line option is used with the following `set_monte_carlo_option` command:

```
set_monte_carlo_option ... [-compress_file none | \  
    "file_ext1{,file_ext2, ...}" ]"
```

The `set_monte_carlo_option` command takes precedence over the `-gz` command line option on the specified file extension. A warning is printed in the log file as follows:

```
Warning: set_monte_carlo_option command takes precedence over  
-gz command line option for file compression.
```

Example 1

The following example enables external sampling during Monte Carlo analysis and writes out waveform results when analysis is complete.

```
% xa monte_carlo_netlist -c cmd -o XA_RESULT/xa -gz
```

The `cmd` file contains the following Monte Carlo command:

```
set_monte_carlo_option -sample_output all -dump_waveform 1
```

And the following files are compressed:

- Waveform files: `xa.*.fsdb.gz`
- Measurement result files: `xa.*.meas.gz`

Example 2

The following example enables external sampling and data mining during Monte Carlo analysis and writes out waveform results when analysis is complete.

```
% xa monte_carlo_netlist -c cmd -o XA_RESULT/xa -gz
```

The cmd file contains the following Monte Carlo commands:

```
set_monte_carlo_option -sample_output all -dump_waveform 1  
set_monte_carlo_option -datamining on
```

And the following files are compressed:

- Waveform files: xa.*.fsdb.gz (triggered by the `-gz` command line option)
- Measurement result files: xa.*.meas.gz (triggered by the `-gz` command line option)
- MC file: xa.mc0.gz (triggered by `set_monte_carlo_option -datamining on`)
- HSPICE .mt file: xa.mt.gz (triggered by `set_monte_carlo_option -datamining on`)
- Log file: xa.log.gz (triggered by `set_monte_carlo_option -datamining on`)

Example 3

The following example uses the `-gz` command line option to enable gzip file compression and disables file compression for Monte Carlo output result files.

```
% xa monte_carlo_netlist -c cmd -o XA_RESULT/xa -gz
```

The cmd file contains the following Monte Carlo commands:

```
set_monte_carlo_option -sample_output all -dump_waveform 1  
set_monte_carlo_option -datamining on  
set_monte_carlo_option -compress_file none
```

Because the setting of the `set_monte_carlo_option -compress_file none` command overrides the `-gz` command line option, no file is compressed.

Example 4

The following example uses the `-gz` command line option to enable gzip file compression and enables file compression only for Monte Carlo .mc and .meas output files.

```
% xa monte_carlo_netlist -c cmd -o XA_RESULT/xa -gz
```

The cmd file contains the following Monte Carlo commands:

```
set_monte_carlo_option -sample_output all -dump_waveform 1  
set_monte_carlo_option -compress_file "mc,meas"
```

Because the `set_monte_carlo_option -compress_file "mc,meas"` command overrides the `-gz` command line option, the following files are compressed:

- Measurement result files: xa.*.meas.gz
- MC file: xa.mc.gz

Supported HSPICE Options for Monte Carlo Simulation

The PrimeSim XA tool also supports the following PrimeSim HSPICE options for Monte Carlo simulation. For more information about these options, see the related topics in *PrimeSim™ Continuum Reference Manual: Commands and Control Options*.

- `.OPTION MODMONTE=0|1`: Controls how random values are assigned to parameters with Monte Carlo definitions.
- `.OPTION MONTECON=0|1`: Continues a Monte Carlo analysis by retrieving the next random value, even if non-convergence occurs.
- `.OPTION SEED=value`: Specifies the starting seed for the random-number generator in Monte Carlo analysis.

Output Files From Monte Carlo Analysis

The following tables list the output files that are generated from the traditional Monte Carlo simulation:

- [Table 42](#): Lists the output files from Monte Carlo simulation
- [Table 43](#): Lists the output waveform files when the `-dump_waveform` option is specified with the `set_monte_carlo_option` command
- [Table 44](#): Lists the output files when the `-sample_output` option is specified with the `set_monte_carlo_option` command

Table 42 Monte Carlo Output Files

File Name	Description
<code>.mc0</code>	The random number sample file (<code>.mc0</code>) saves the changes in all parameter values subject to variation. For a detailed description of <code>.mc0</code> file, see the Monte Carlo Analysis—Flow and Outputs section in <i>PrimeSim HSPICE User Guide: Basic Simulation and Analysis</i> .
<code>.mc</code>	Contains the Monte Carlo measurement summary of all measurements and an ASCII histogram of each measurement. Monte Carlo analysis produces this file when a <code>.meas</code> command is used in the netlist. This file is not generated if there is no measurement. See the .mc File section for details.
<code>.mc.csv</code>	Contains all of the data in the <code>.mc</code> file, except the histogram. You can read this <code>.csv</code> file in a spreadsheet program.

Table 42 Monte Carlo Output Files (Continued)

File Name	Description
<code>.meas</code> and <code>.mt</code>	Contains all the Monte Carlo sample measurement results. Monte Carlo analysis always produces this file. It contains only the measure values, not the Monte Carlo parameter values. This file uses standard PrimeSim XA measure formatting and has a <code>.meas</code> extension. If you specify HSPICE formatting in the <code>set_meas_option</code> command, it has a <code>.mt</code> extension. When you specify the <code>set_meas_option -format hspice</code> command, the PrimeSim XA tool must create a measure file that is syntactically compatible with PrimeSim HSPICE so that a WaveView histogram can be viewed. When you choose the HSPICE measure format, the per sample and summary measure file has a <code>.mt</code> extension rather than the standard <code>.meas</code> extension.
<code>.mc_params</code>	Generated when you specify the <code>set_monte_carlo_option -parameter_file 1</code> command. The file lists all the random variable values used in the simulation. The default file format is the PrimeSim XA measurement file format. You can choose a PrimeSim HSPICE measurement file format with the <code>set_meas_option -format hspice</code> command.

.mc File

The `.mc` file contains a measurement summary of the Monte Carlo simulation. It is the primary output file and reports a summary of the statistical data for each measurement:

- nominal
- mean
- variance
- stddev
- avgdev
- min
- max
- median
- sample number of min
- sample number of max

- sample number of median
- 10 bin horizontal ASCII histogram:
 - First column of the histogram is the bin center point.
 - Second column, nb, reports the number of samples in the bin.
 - Third column, freq, reports the percentage of samples in the bin.

Table 43 *Generated Waveform Files*

-sample_output setting	-dump_waveform setting	Output File Format	Output Waveform File
none	0	Any	No waveform file
none	1	Any	No waveform file
all	0	Any	No waveform file
all	1	Any	<code>xa.m0.format</code> <code>xa.m1.format</code> <code>xa.m2.format ...</code> <code>xa.mn.format</code>
1,10	0	Any	No waveform file
1,10	1	Any	<code>xa.m1.format</code> <code>xa.m10.format</code>

Table 44 *Output files generated when the -sample_output option is specified with the set_monte_carlo_option command*

File Name	Description
<code>.m#.wdf/fsdb/out</code>	Waveform output file
<code>.m#.meas/mt</code>	Measure file with a <code>.meas</code> extension when you specify the standard PrimeSim XA measure format, or a <code>.mt</code> extension if you specify the HSPICE measure format with the <code>set_meas_option</code> command.
<code>.m#.ic</code>	Initial conditions file (if requested)

Running Incremental Monte Carlo Simulations

PrimeSim XA supports the HSPICE incremental distributed processing (DP) feature to re-run Monte Carlo simulation many times on unfinished samples or new samples. PrimeSim

XA then combines the results from the original and the new runs to provide a complete statistical Monte Carlo analysis report.

To run an incremental DP Monte Carlo simulation, use the following syntax:

```
% xa input_netlist -dp n -dpconfig [dp_config_file] \  
-dpincremental original_directory -o output_dir
```

where *original_directory* is the path to the original Monte Carlo run directory.

When the `-dpincremental` option is used, PrimeSim XA parses the netlist and checks the sweep value in the `.tran` line.

For more information about how to use the incremental distributed processing (DP) feature to re-run Monte Carlo simulation, see the Automatic Incremental DP Monte Carlo section in *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

- PrimeSim XA supports incremental Monte Carlo simulations with list based Monte Carlo samples.
- If the sweep value is greater than the number of samples in *original_directory* results, PrimeSim XA starts running Monte Carlo from the last sample in the first run.
- If the sweep value is less than or equal to the number of samples in *original_directory* results, PrimeSim XA issues the following error message:

```
Error: the number of MC samples is not greater than the first run
```

- The same seed must be used in the original and incremental Monte Carlo runs.

Note:

- During incremental Monte Carlo simulations in DP mode, only `.mt`, `.mc0` and `fsdb.grp` files will be merged.
- It is not recommended to change the accuracy settings in the PrimeSim XA configuration file. Otherwise, an error message will be issued.
- You must use the `-dpincremental` option with the `-dp` option. The `-dpincremental` option works only with distributed processing.
- You cannot use the same prefix for the output directories in the original and incremental runs. When the same prefix is used, PrimeSim XA renames the directory with the suffix `_inc`, like *prefix_inc*.

Setting Instance-Based Seeds

This section describes the supported Monte Carlo syntax for instance-based seed setting.

- The list of 3DIC instances and corresponding seeds are only allowed in the top level:

```
.mts_seed module_label::inst_name=seed_val \  
module_label::inst_name=seed_val ...
```

The designated die instance uses the seed value set by `.mts_seed` for variation and overrides the seed setting defined in module. The other die instances from the same module without `.mts_seed` setting use the seed setting defined in corresponding module as the default for variation.

- `.mc0` file generation:

For each die instance with a `.mts_seed` setting, the tool generates a separate `.mc0` file. The format of the `mc0` file is:

```
inst_name.module_name.mc0
```

For other die instances from the same module and without a `.mts_seed` setting, the PrimeSim XA tool generates a single `mc0` file. The format of the `.mc0` file is `module_name.mc0`.

External Sampling During Monte Carlo Analysis

Use the external sampling method to overload the internal random number generators and pass the PrimeSim XA sample values that are generated from other statistical tools.

PrimeSim XA supports external sampling in the same way as the PrimeSim HSPICE tool. For a detailed description about the external sampling method, see the Sampling Options section in *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

Note:

External sampling in PrimeSim XA does not support encrypted or protected model libraries.

To add the custom-generated samples to the PrimeSim XA Monte Carlo flow,

1. Execute PrimeSim HSPICE or PrimeSim XA with a standard simulation command (`.TRAN`) and `monte=1` to generate a `.mc0` file which lists all the independent variables.

For details, see the Parameter File section in *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

2. Create a data block outside the PrimeSim XA tool with the desired perturbations on the independent variables for global and local variations.
3. Run a PrimeSim XA simulation with externally generated data block content.
4. Repeat steps 2 and 3, depending on the outcome of the previous experiments.

See Also

- [External Sampling During Monte Carlo Analysis](#)

Random Variables

Traditional Monte Carlo simulation defines a random variable with a distribution function. You can assign a random variable just as in the PrimeSim HSPICE tool.

When using random variables in a `.data` block, the data block syntax is the same as for the regular PrimeSim HSPICE data block from `.Data` to `.EndData` block format. For practical usage and parsing performance, the `.Data` block must be in an external file, not embedded in the netlist itself.

The PrimeSim XA tool supports the same distribution functions as the PrimeSim HSPICE tool:

- UNIF (nominal_val, rel_variation)
- AUNIF (nominal_val, abs_variation)
- GAUSS (nominal_val, rel_variation, num_sigmas)
- AGAUSS (nominal_val, abs_variation, num_sigmas)
- LIMIT (nominal_val, abs_variation)

Whenever a parameter defined by one of these functions is assigned, a new unique random variable is generated. For more information on distribution functions, see the `.PARAM` Distribution Function section in *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

Data Mining Flow

Running Monte Carlo simulation can be lengthy and generates a large number of output files. Sometimes the raw output does not provide sufficient insight into the circuit behavior or guidance on how the circuit could be improved. PrimeSim XA provides the Monte Carlo data mining feature to generate comprehensive reports for you to analyze Monte Carlo results more efficiently.

To enable Monte Carlo post-processing flow in PrimeSim XA, specify the `-datamining` on option of the `set_monte_carlo_option` command using the following syntax:

```
set_monte_carlo_option ... -datamining on
```

The tool first collects information from the existing measure files and IRV files, and then performs post-processing to create the following output files: `*.ava.annotate`,

`*.ava.report`, and `*.ava.qqt0.wv_plt`. The output reports contain extensive data including statistical summary, correlated contribution of `.mc0` on elements and design environment back-annotation information.

For more information about advanced Monte Carlo data mining support, see the Postprocessing section in *PrimeSim AVA User Guide*.

Auto-Replacing Monte Carlo Samples When Measurements Fail

Use the `check_measure` command to check for failed measurements when running Monte Carlo simulation in DP mode. To automatically replace the sample of a failed DP task, use the `-auto_replace` option of the `set_monte_carlo_option` command.

To set the maximum number of auto-replacing indexes, use the `-auto_replace_max_attempt` option of the `set_monte_carlo_option` command. When having the expected successful Monte Carlo samples, PrimeSim XA stops the simulation and skips the remaining attempts.

When the `-auto_replace` option is specified, running Monte Carlo data mining generates reports based on the final successful Monte Carlo sample runs only.

For a detailed description about the command options, see the `set_monte_carlo_option` and `check_measure` command pages in *PrimeSim XA Command Reference*.

Examples 1

In the following example, when running Monte Carlo simulation in DP mode, a DP task that runs through completion is additionally checked for failed measurements. If there is any failure, PrimeSim XA considers the task a failed task and proceeds with auto-replacing, depending on the settings of the `-auto_replace` option of the `set_monte_carlo_option` command.

```
check_measure -label *
```

Examples 2

Same as Example 1, the following example enables the `-auto_replace` option and sets the number of maximum attempts for auto-replacing samples as 5. For example, when the last sample index specified in the `.tran` statement is 30, PrimeSim XA will try to simulate up to the Monte Carlo index of 35.

```
check_measure -label *
set_monte_carlo_option -auto_replace 1 \
    -auto_replace_max_attempt 5
```

Examples 3

Same as example 2, but the following example checks for failed measurements for measures `trise1` and `tfall1`. If other measures fail, they are not considered.

```
check_measure -label trise1 tfall1  
set_monte_carlo_option -auto_replace 1 \  
-auto_replace_max_attempt 5
```

Examples 4

Same as example 2, but the following example checks for failed measurements for all measures except trise2 and tfall2.

```
check_measure -except trise2 tfall2  
set_monte_carlo_option -auto_replace 1 \  
-auto_replace_max_attempt 5
```

Controlling the Read-in of an External File

Use the `Option External_File=filename` option command to read in an external data block line-by-line during the simulation stage.

For example:

```
Option Sampling_Method=External Block_Name=extern_data  
+ External_File=extern.mc0
```

The content of the `extern.mc0` file:

```
$DATA1 SOURCE='PrimeSim HSPICE' VERSION='M-2017.03 linux64'  
$option ignore_globn = no  
$option seed = 1  
[.....]  
** lines begin with '$' are ignored **  
.Data extern_data  
index a1:@:IGN res1:@:IGN r1:@:ILN r2:@:@:ILN r3:@:res2:@:a2:@:ILN  
1 0. 0. 0. 0. 0.  
2 -0.8014 -0.8294 -0.2677 0.2150 1.6757  
3 0.8946 0.2014 1.0447 -1.7987 2.7742  
4 0.6560 -1.2719 -0.4937 0.4482 0.3209  
5 1.4379 -0.5189 -0.3780 0.5518 0.3415  
.Enddata
```

NOTE:

- The PrimeSim XA tool does not check the range of values in the supplied external file against the value of `option Normal_Limit`.
- Independent random variables which are not specified in the data block are assigned with new random values generated by a new seed.

Running Monte Carlo Analysis With 3DIC

This section describes the supported Monte Carlo syntax with 3DIC.

- Module-based `.option seed`

```
.module mod_demo
.option seed=value
...
.endmodule
```

The module-based seed value affects only the variation defined in the module scope. The top-level seed setting does not affect the module-based seed value or the variation inside module. The default value is 1.

- Module-based `.option modmonte`

```
.module mod_demo
.option modmonte=0|1
...
.endmodule
```

The module-based `modmonte` affects only the models that are defined inside a current module. The top-level setting does not affect the module-based `modmonte` setup or model variations inside a module. The default value is 0.

- Module-based variation control for subcircuits and instances:

```
.module mod_demo
.variation
option do_not_vary subckts=subckt_list | instances=instance_list
option vary_only subckts=subckt_list | instances=instance_list
.end_variation
...
.endmodule
```

The module-based variation control options can only affect the subcircuit and instance defined in the same module. The top-level variation control only affects the top-level instance and subcircuit, and does not affect the module subcircuit instance.

Variation Blocks in Monte Carlo Flow

This section describes how to use the PrimeSim HSPICE Variation Block features for Monte Carlo analysis in the PrimeSim XA tool. It supports the following Variation Block commands:

```
.Variation
.Global_variation
.Local_variation
```

```
.Element_variation
.End_Element_variation
.End_Local_variation
.End_Global_variation
.End_variation
```

Table 45 shows the Variation Block features that the PrimeSim XA tool supports.

Table 45 Supported Variation Block Features

Variation Block Feature	Description
Variation type	Absolute variation, Relative variation, Model parameter variation, Subcircuit parameter variation, Top=level parameter variation
Distribution type	U(), N(), Limit(), Perturb()
Access function	Get_E(), Get_P(), Get_M()
Random generator	Default MCG
Sampling method	Default SRS

Table 46 shows options inside the Variation Block that the PrimeSim XA tool supports.

Table 46 Supported Variation Block Options

Option Name	Default
Do_not_vary instances=InstList	
Do_Not_Vary Subckts=SubcktList	
Do_Not_Print Subckts=SubcktList	
Ignore_Variation_Block	No
Ignore_Local_Variation	No
Ignore_Global_Variation	No
Modified_Distribution=scale()	
Modified_Distribution subckts = scale(value, subckt_name)	
Modified_Distribution instances = scale(value, inst_name)	
Normal_limit	+/- 20

Table 46 *Supported Variation Block Options (Continued)*

Option Name	Default
Print_Only Subckts=SubcktList	
Seed	'random' is not supported
Sampling_method= SRS, LHS	SRS
Vary_only instances=InstList	
Vary_Only Subckts=SubcktList	

The following PrimeSim HSPICE option is supported:

```
.option MODMONTE
```

Note:

You can use the Variation Block options inside the Variation Block, or as a usual PrimeSim HSPICE `.option` statement. For example, the following format is valid:

```
.option Vary_Only subckts=dff
```

Support for the Variation Block Within a Conditional Statement

The PrimeSim XA tool supports the Variation Block in a conditional statement (after the `If/Else` commands). For example:

```
If (control == 1)
.variation
.local_variation
.Element_Variation
R r= 50 %
End_Element_Variation
.end_local_variation
.End_Variation
Else
.variation
.local_variation
.Element_Variation
R r= 20 %
.End_Element_Variation
.end_local_variation
.End_Variation
endif
```

Output Files

The output files are the same as for the traditional Monte Carlo flow. The suffix is different from the PrimeSim HSPICE tool. The generation of the output files can be monitored by the `set_monte_carlo_option` PrimeSim XA command.

If you do not use the `set_monte_carlo_option` command, no waveform file is generated. If you want to generate the output file, you need to use `set_monte_carlo_option -dump_waveform 1 -sample_output value`. For example:

```
set_monte_carlo_option -dump_waveform 1 -sample_output all
```

Monte Carlo simulation indexing in the PrimeSim XA tool is compliant to the PrimeSim HSPICE tool. The sample of index 1 is a nominal simulation, and the sample of index 2 is the first random simulation. For more information about sample indexing, see the Monte Carlo Analysis—Flow and Outputs in *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

Differences Between PrimeSim XA and PrimeSim HSPICE

The PrimeSim HSPICE .mc0 file contains the independent parameter values while the PrimeSim XA .mc file contains the Monte Carlo statistics. The independent parameter values are dumped into the .mc_params file when the `set_monte_carlo_option -parameter_file 1` command is used.

The PrimeSim XA tool generates one output file (.meas/.mt and .fsdb) per sample and one global .meas/.mt file that comprises all measurement results.

These differences are consistent with the traditional Monte Carlo flow with the PrimeSim XA tool. They are not specific to the Variation Block support.

Distributed Processing

Distributed processing is supported in the same way as for the traditional Monte Carlo flow. When you run distributed processing, if a PrimeSim HSPICE license is available and the PrimeSim HSPICE format is used for measurement (`set_meas_format -format hspice`), then a .mpp0 file is generated.

When distributed processing is used, all waveforms are grouped into a grouped file with the .fsdb@mc.grp extension.

Using Relative Paths in Distributed Processing

If the SPICE files use relative paths, the paths might be considered incorrect when running with distributed processing. The following example might work when you run the PrimeSim XA tool without distributed processing, but does not work with distributed processing.

Suppose you have a netlist in your working directory called `netlist.sp` and you also have SPICE files in the `./models` directory, and the `netlist.sp` file has:

```
.lib models/include.sp nom
```

And the `models/include.sp` file has:

```
.lib nom      .include ./models/nom  .endl
```

This relative path does not work with distributed processing. You must use a path relative to the current directory, such as:

```
.lib nom      .include ./nom  .endl
```

This is valid for both traditional Monte Carlo and Variation Block usage.

Mixed-Signal Simulation

The Variation Block can be used in a VCS AMS simulation in the same manner as the traditional Monte Carlo flow. The usage is exactly the same. Distributed processing is supported with the VCS AMS tool.

Running ICSWEEP

When running Monte Carlo analysis, you can reuse the nominal `.IC` values with the `set_dc_icsweep_option` command.

Licensing to Run Monte Carlo Analysis

Each Monte Carlo iteration checks out a PrimeSim XA license. This is valid for both sequential runs and distributed processing. The license check is the same as for a usual PrimeSim XA simulation. If the license becomes unavailable during the Monte Carlo simulation, then the simulation aborts. If you have set `XA_WAIT_LICENSE` to 1, the simulation waits for the time defined in `XA_WAIT_LICENSE_TIMEOUT` and:

- Resubmits the failed iterations as soon as a license becomes available.
- Aborts if no license is made available during `XA_WAIT_LICENSE_TIMEOUT` time.

Limitations

The current limitations are:

- Only the options described in this section are supported. All other PrimeSim HSPICE options are not supported.
- The Variation Block cannot be in a conditional statement.

- Only the default random generator is supported.
- Only two sampling methods (SRS and LHS) are supported.
- Back-annotation flows (SPEF and DSPF) are not supported.
- The Group Operator {...} and Subexpressions are not supported.
- The user-defined cumulative distribution function: CDF is not supported.
- Models with model binning are not supported
- The PrimeSim HSPICE tool can support the repetition of the model name in the Variation Block. For example:

```
nmos nm vth0=perturb('vth0_vary')  
nmos nm tox='get_p(tox_param)' %
```

This syntax is not supported in the PrimeSim XA tool. You must use the '+' continuation sign:

```
nmos nm vth0=perturb('vth0_vary')  
+ nm tox='get_p(tox_param)' %
```

- You cannot use two different option lines with `vary_only` and/or `do_not_vary`. Only one option line is allowed. For example, if you specify the following two lines:

```
.option vary_only instances=x1  
.option do_not_vary instances=x1.x8
```

Only the last line is taken. The first one (`vary_only instances=x1`) is ignored. The following rules also apply:

- You can use multiple instances or subcircuits in `do_not_vary` and `vary_only`.
- You cannot mix `vary_only` and `do_not_vary` in the same option line.
- You cannot mix `vary_only`, subcircuits and instances in the same option line.
- You can mix `do_not_vary`, subcircuits and instances in the same option line:

```
.option do_not_vary subckts= instances=
```

10

PrimeSim GPU and Multicore Simulation

This chapter describes what you need to consider when you run a multicore simulation or PrimeSim GPU simulation in PrimeSim XA.

The chapter contains the following topics:

- [PrimeSim GPU Simulation](#)
- [Multicore Simulation](#)

PrimeSim GPU Simulation

PrimeSim XA supports the PrimeSim powerful GPU solver and efficient process management flow, which provides significant performance improvement on large, post-layout simulations by maximizing the parallel computing capability of GPU hardware.

Note:

The PrimeSim GPU support is enabled in PrimeSim XA only when you invoke PrimeSim XA with the `PRIMESIM=2` license mode.

To enable PrimeSim GPU support in PrimeSim XA, use the `-ngpu` command-line option in the following syntax:

```
xa -ngpu ...
```

Use the `primesim_gpu_mode` option command to control GPU operation mode.

For a detailed description about how to run GPU simulation, see the related topics in the PrimeSim GPU section in *PrimeSim User Guide: Pro and SPICE*.

See Also

- [Configuring the PrimeSim XA License](#)
- [PrimeSim GPU](#)

Multicore Simulation

The section contains the following topics:

- [Introducing Multicore Simulation](#)
- [Running a Multicore Simulation](#)
- [Determining the Multicore Benefit](#)
- [Factors that Affect a Multicore Simulation](#)

Introducing Multicore Simulation

The PrimeSim XA tool supports a multicore capability to speed up simulation performance. Multicore is also called multithreading, or parallel processing, which is the ability to divide one task into multiple parallel subtasks for execution in a single machine with different processors.

The PrimeSim XA tool uses specialized partitioning and simulation algorithms that maximize the amount of parallel work to be done to speed up simulation performance. The simulation results do not depend on the number of cores used, and the results remain consistent no matter how many cores you use.

The multicore partitioning algorithms often result in a performance improvement even with one core. Not all designs have the same performance benefits. Some design speed up more than others. The multicore capability gets better performance improvement and scalability on designs where traditional partitioning algorithms do not break the design into many small blocks for simulation, such as analog designs and large post-layout designs.

The multicore capability is recommended for all designs, regardless of the number of cores used. With the complexity of the Fin-Shaped Field Effect Transistor (FinFET) technology and smaller process nodes, the multicore capability in the PrimeSim XA tool has shown to have a significant performance advantage.

Running a Multicore Simulation

Use the following command to run multicore simulation in the PrimeSim XA tool from the command line:

```
xa ... -mt
```

Or you can run the `set_multi_core` configuration command. The `-mt` command line option overrides the `set_multi_core` command if both are specified.

Whenever the `-mt` command line option or `set_multi_core` command is specified, the PrimeSim XA tool enables multicore partitioning and simulation algorithms. This

is true even when only one core is used. Using the `-mt 1` command line option or `set_multi_core -core 1` command is different from not using it.

Some commands, such as `set_sram_characterization`, might have enabled the multicore capability by default.

Determining the Multicore Benefit

This multicore capability is recommended for all designs, but not all designs have a significant performance benefit. You should do a preliminary run to check how threading affects your simulation. The `set_multi_core` command supports options to check the threadability of a simulation.

Based on certain heuristics, the PrimeSim XA tool prints out one of the following messages to indicate whether multicore benefits the simulation when the `-check_netlist` option is used:

```
Multi-core check: this netlist is likely to benefit from a multicore simulation.
```

```
Multi-core check: this netlist can possibly benefit from a multicore simulation.
```

```
Multi-core check: this netlist is not likely to benefit from a multicore simulation.
```

Most of the new models are threadable when running a simulation, but some old models may not be threadable. When you use the `-check_model` option, the PrimeSim XA tool also reports the percentage of a non-thread safe models.

The following message shows that only 2% of the models are non-threadable.

```
Warning: 40000 (2%) non-thread-safe instances are used.
```

Factors that Affect a Multicore Simulation

This section describes the factors that affect multicore simulation performance.

System Loading

It is important to note that system loading can significantly affect simulation performance. It is even more significant when enabling the multicore capability when more than one processor is used. It is recommended that when you run a multicore simulation you request an appropriate number of processors through the top-level job queuing system, such as GRD or LSF. GRD, and LSF, which can be set up such that they do not overload the number of processors in one machine.

There are many ways to check system load. You can use the following commands to check the load of each processor of a multi-processor system and determine which

process is consuming the processors. This is the most common command to check the processor activity.

```
$> top
```

The following command displays the activity of each available processor.

```
$> mpstat -P ALL
```

The following command determines the top four tasks consuming the processors.

```
$> ps -eo pcpu,pid,user,args | sort -r -k1 | head -5
```

When you are running a multicore simulation, you see the %CPU to be larger than 100%. In this example, user1 runs a /home/snps_xa/bin/xa process with 12 cores, which consumes 1159% of the CPU.

```
%CPU PID USER COMMAND
1159 12116 user1 /home/snps_xa/bin/xa deck.sp -c -cmd -o xa_result/xa1
-mt 12
1.0 33380 user1 ps -eo pcpu,pid,user,args
0.4 48013 user2 /global/apps/cx_version/platforms/linux64/bin/wv
0.4 19300 user2 /global/apps/cx_version/platforms/linux64/bin/wv
```

CPU Time Versus Wall Time

Central Processing Unit (CPU) time is often used to measure the performance of a simulation, but this does not apply to a multicore simulation. CPU time is the amount of time an application uses to execute CPU instructions. When the multicore capability is enabled, the reported total CPU time in the PrimeSim XA tool is the sum of the time by all the CPUs.

Wall clock time is a better measure of how much real time elapses from the start of a process to the end. It is a true measurement of the simulation performance from start to finish in a multicore simulation.

"Play Nice" Feature

The PrimeSim XA tool has a "play nice" feature enabled when running multicore simulation. This feature continually monitors the system load and puts some cores "to sleep" when the system load is too high. It prevents the system from slowing down due to too many processes. The advantage of this feature is that system stalls can be averted, and all jobs on the system can continue. However, it is important to note that an individual job might not completely utilize all the requested number of processors and might seem slow compared to run times when the system is fully loaded.

The simulation runtime of multicore simulation is reported at the end of the log file, such as:

```
Total Wall Time = 169331 sec (1day 23hr 2min 11sec)
```


The elapsed simulation time with 12 cores is 169331 second.

SMP Versus NUMA

It is important to know if you are running the multicore simulation on a Symmetric Multiprocessing (SMP) or Non-Uniform Memory Access (NUMA) system. An SMP system is a symmetric multiprocessor system where two or more processors are connected and have equal access to system resources, such as memory, IO ports, and so on. An SMP system is a tightly-coupled system that shares everything with multiple processors working under one operating system, accessing each other's memory over a common bus path. It can get overloaded when there are too many processors competing for resources. Best performance on SMP comes from using all available cores.

A NUMA system is configured such that a small number of processors are interconnected on a local bus to access its local shared memory. The unit is called a NUMA node. This unit can be added to other similar units to form an SMP. The memory access time is faster than an SMP when a processor accesses its local shared memory within a NUMA node. It might take longer access time to access remote memory if the processor uses more memory than the assigned NUMA node. Best performance on NUMA is to tie the CPU and memory to the same NUMA node and use all available cores on that node. Performance degrades if you specify more cores than exist on one node.

You can use the following command to check for NUMA system and its available memory:

```
$> numactl --hardware
```

or

```
$> numastat
```

11

Analog Mixed-Signal Simulation

Use the PrimeSim XA multicore technology for analog mixed-signal simulation to achieve higher performance and accuracy for functional blocks and analog components.

The PrimeSim XA tool provides the capability to further adjust the level of accuracy for foundry library model simulations and FinFET transceivers simulations, as described in the following topics:

- [Simulation Accuracy and Speed for Analog Mixed-Signal Designs](#)
- [Foundry Model Library Simulation](#)
- [FinFET Transceiver Simulations](#)

Simulation Accuracy and Speed for Analog Mixed-Signal Designs

When running a simulation on analog mixed-signal designs, the PrimeSim XA tool provides the capability to control simulation performance for better simulation speed and accuracy and model complexity tradeoffs with the help of the following commands:

- `set_sim_level`

Specifies tradeoffs between speed and model complexity.

- `set_model_level`

Overrides the choice of table models used for each `set_sim_level` command. This command can be applied globally, or locally using the `instance_spec`. If not specified, the default is to use the tables defined by the `set_sim_level` command.

- `set_speed_scale`

Dynamically scales the transient simulation engine tolerance to help tune performance and accuracy. If not specified, the default is to use the engine tolerances defined by the `set_sim_level` command.

- `set_sim_mode`

Enables high performance simulation algorithms for analog mixed signal designs. It is recommended that you use the `set_sim_mode` command along with the

`set_sim_level` command. Alternatively, you can enable the high performance simulation algorithms by specifying the `-sim_mode` command line option.

By default, running the simulation with the `set_sim_mode 1` command uses 2 cores. You can adjust the number of cores by specifying the `-mt` command line option or the `set_multi_core` command.

The `set_sim_mode 1` command optimizes 0v voltage sources. As a result, wild card probes for 0v voltage sources may be missing in the waveform file. If you want to include these voltage sources in your simulation, add the `set_rc_option -short_0vsrc 0` command to your XA settings. However, keeping a large amount of 0v voltage sources for simulation could adversely impact your simulation runtime.

For SRAM simulations, the `set_sram_characterization` command should be used. The `set_sim_mode` and `set_sram_characterization` commands are mutually exclusive. When used together, the simulation run stops with an error message.

- Specifies the number of cores used to run a multi-core simulation.

Use the `set_sim_level` command to control simulation speed, accuracy, and model complexity tradeoffs. The higher the level, the more accurate the simulation result, but with longer simulation time. This command can be issued multiple times to specify the simulation level desired for the entire netlist and for subcircuits or instances in the design.

You can also use the `set_sim_mode 1` command along with the `set_sim_level` command to provide more simulation throughput (faster runtime). The `set_sim_mode 1` command turns on high performance algorithms for analog and mixed signal simulations.

The `set_sim_level`, `set_model_level`, `set_sim_mode` and `set_speed_scale` commands have the greatest impact on overall simulation performance and accuracy tradeoffs. The `set_multi_core` command can also be used to increase your simulation performance by running the simulations using more CPU cores. These settings are a place to start, but need to be evaluated in the context of your design.

For a detailed description about how to run the commands, see the command pages in *PrimeSim™ XA Command Reference*.

Examples

Following is an example of simulation setups for a mixed signal design.

```
set_sim_level 4
set_model_level 5
set_sim_level 6 -subckt block_name1
set_multi_core -core 8
```

Following is an example of simulation setups for a mixed-signal design with the `set_sim_mode 1` command. The `set_sim_mode 1` command turns on the PrimeSim XA high performance simulation algorithms for mixed-signal simulation.

```
set_sim_level 4
set_model_level 5
set_sim_level 6 -subckt block_name1
set_multi_core -core 8
set_sim_mode 1
```

The following example sets the `set_speed_scale` value of 1.5 to scale the engine tolerance by 1.5X to increase the simulation performance.

```
set_sim_level 4
set_model_level 5
set_sim_level 6 -subckt block_name1
set_sim_mode 1
set_multi_core -core 8
set_sim_mode 1
set_speed_scale 0 1.5
```

In the following example, the simulation runs with tighter engine tolerances. The `set_speed_scale` command scales the engine tolerance by 0.5.

```
set_sim_level 4
set_model_level 5
set_sim_level 6 -subckt block_name1
set_sim_mode 1
set_multi_core -core 8
set_speed_scale 0 0.5
```

Setting Simulation Levels for Analog and Mixed Signal Simulation

For analog and mixed-signal designs, it is recommended that you set the simulation level to 4, 5 or 6 by running the `set_sim_level` command with the `-level` option.

When the `-level` option is set to

- 4: More aggressive settings are set for fundamental FastSPICE features, such as partitioning, modeling, synchronization, and so on. With these aggressive settings, running the simulation with the `-level 4` option generates more functional simulation results with less detailed timing, voltage, and current data.
- 5: The simulation results contain a higher level of detail for timing, voltage, and current data but with more conservative settings, which might result in some performance penalty.
- 6: The simulation results has a SPICE-like accuracy for timing and power simulation of all circuits, and cell characterization.

In general, for cases with excessively long runtime, use `-level 4`. For other cases or those with higher accuracy requirements, use `-level 5`. For instance, for a transceiver simulation, the setting of `set_sim_option -level 4` is usually specified globally, and the `set_sim_option -level 5` setting is used locally for blocks that require higher accuracy.

Most analog and mixed-signal simulations achieve acceptable results with the `set_sim_level -level 5 -acc 3` setting. Use this setting as a good starting point if you are not yet familiar with the `ssl2` simulation. If you cannot decide which option to use, run simulations separately with the `-level 4` and `-level 5` options. If the `-level 4` simulation is significantly faster than the `-level 5` simulation, use the results from the `-level 4` simulation to quickly identify problems with simulation, testbench or circuit. For example, if the testbench failed to enable a clock at the 90% point of the transient, the `-level 4` simulation results can identify this more quickly and the corrected `-level 5` simulation can be restarted sooner than if you only use `-level 5`.

Foundry Model Library Simulation

Simulating foundry's model libraries requires detailed correlation to achieve the highest accuracy for tool certification or foundry's model library development. In PrimeSim XA, you run the `set_sim_level -level 7` command for a fully synchronous simulation with equation device models. In cases where time step control needs to be turned off or a fixed global time step is required for extremely detailed correlation, use the `-tstep_control` or `-tstep` option of the `set_certification_option` command. To enable MOS model checking for accurate model validation, use the `-mos_check` option.

Examples

The following example turns off time step control and uses the time step from the transient command in the netlist for transient simulation.

```
set_certification_option -enable 1 -tstep_control 0
```

The following example turns off time step control and sets a fixed time step value based on the process node.

```
set_certification_option -enable 1 -tstep_control 0 -tstep 1e-14
```

FinFET Transceiver Simulations

High performance Fin Field-Effect Transistor (FinFET) transceiver simulations are challenging due to the high frequencies of operation and the level of voltage and current precision needed to verify the functionality of these designs. Adding to the challenge is the length of the simulation time needed to power-up simulation and to verify the different operational modes for these designs.

To address these challenges, the PrimeSim XA tool provides the `set_circuit_transceiver` command to configure the basic setup for FinFET transceiver simulations. This command is used with the `set_sim_level` and `set_rf_option` commands to control the simulation performance, accuracy, and model complexity trade-offs both globally and at the instance/sub-circuit level.

This section describes the steps in the transceiver simulation flow, as discussed in the following topics:

- [Setting Up the FinFET Transceiver Simulation](#)
- [Setting RF Options](#)
- [Setting Simulation Levels](#)
- [Setting Model Levels](#)
- [Setting Cores for Multicore Simulation](#)
- [Setting DC Options](#)
- [Examples](#)

Setting Up the FinFET Transceiver Simulation

Use the `set_circuit_transceiver` command to configure setups for FinFET transceiver simulations, in the following syntax:

```
set_circuit_transceiver -enable 1 -process 0|1 [-powerup 0|1]
```

Set the `-enable` option to 1 to enable FinFET transceiver simulation. Use the `-process` option to specify if simulating designs for advanced process technology. For example, to run FinFET simulation for 16nm designs and above, run the following command:

```
set_circuit_transceiver -enable 1 -process 0
```

For process 7nm and below, set the `-process` option to 1 due to the smaller size of MOSFET devices. For example,

```
set_circuit_transceiver -enable 1 -process 1
```

Note:

You do not need to run the `set_circuit_transceiver` command for planar bulk process transceivers.

For a detailed description on the command options, see the `set_circuit_transceiver` command page in *PrimeSim XA Command Reference Manual*.

Setting RF Options

Use the `set_rf_option` command to control if the critical RF components and blocks in a transceiver circuit are simulated more conservatively, including tighter circuit partitioning rules and simulator tolerances for the specified sub-circuit and instances. By default, the command scales the tolerances by 0.2 for the specified sub-circuit or instances.

The `set_rf_option` command should be used with the `set_circuit_transceiver` command.

The syntax is:

```
set_rf_option [-enable 0|1] -process 0|1 [-scale] [-subckt] [-inst]
```

For a detailed description on the command options, see the `set_rf_option` command page in *PrimeSim XA Command Reference Manual*.

Setting Simulation Levels

Use the `set_sim_level` command to control the speed, accuracy and model complexity tradeoffs for the simulation. For transceiver simulations, use `set_sim_level=4` to apply the settings to the entire circuit. To set the simulation level for blocks that require higher accuracy, use `set_sim_level=5`.

For more information, see the [Setting Simulation Levels](#) section or the `set_sim_level` command page in *PrimeSim XA Command Reference Manual*.

Setting Model Levels

Use the `set_model_level instance_spec` command if you want to override the choice of table models used for each of the `set_sim_level` commands.

This command can be applied globally or locally using `instance_spec`. If not specified, the default is to use the tables defined by the `set_sim_level` command.

For more information about the command options, see the `set_model_level` command page in *PrimeSim XA Command Reference Manual*.

Setting Cores for Multicore Simulation

Use the `set_multi_core` command to specify the number of cores used to run a multicore simulation, in the following syntax:

```
set_multi_core -core num_cores
```

For more information on the command options, see the `set_multi_core` command page in *PrimeSim XA Command Reference Manual*.

Alternatively, you can specify the number of cores for multicore simulation using the `-mt` command line option. For example,

```
% xa -mt num_cores
```

For more information, see the [Multicore Simulation](#) section.

Setting DC Options

Use the `set_dc_option -analog_level 3` command for simulations that are starting from a power-up state with clocks and oscillators on. Since there are multiple operational modes in the power-up state, the starting state is defined by the IC's set for the simulation, and the command is used for DC to converge in the presence of running clocks and oscillators

For power-up simulations, running the `set_dc_option -analog_level 3` command is not necessary. Use the `set_circuit_transceiver -powerup 1` command to inform the simulator that this simulation starts mostly from 0 at time 0, and a power-up sequence analysis will be run during transient analysis to ramp up all of the supplies and internal powers.

Examples

Following are the examples for running FinFET transceiver simulations.

16nm Power-Up Simulation

The following example configure setups for a 16 nm simulation.

```
set_circuit_transceiver -enable 1 -process 0
set_sim_level 4
set_sim_level 5 -subckt block_name1
set_sim_level 5 -subckt block_name2
set_multi_core -core 8
report_operating_point -time t0 -report all -type ic
```

16nm Power-Up Simulation Using Prior State From Example 1

The following example configure setups for a 16nm simulation using prior state from Example 1.

```
set_circuit_transceiver -enable 1 -process 0
set_sim_level 4
set_sim_level 5 -subckt block_name1
set_sim_level 5 -subckt block_name2
set_multi_core -core 8
set_dc_option -analog_level 3
load_operating_point -file prior_state.ic -type ic
```

7nm Transceiver Top-level Simulation Using Default -scale Value

The following example uses the default scale value of 0.2 for the subcircuit and instances. This simulation runs with a tighter tolerance for the specified subcircuit and instances.

```
set_circuit_transceiver -enable 1 -process 1
set_sim_level 4
set_model_level 5
```



```
set_rf_option -enable 1 -process 1 -subckt block_name1
set_rf_option -enable 1 -process 1 -inst inst_name1
set_multi_core -core 8
```

7nm Transceiver Top-level Simulation Using Non-default -scale Value

The following example loosens the simulator engine tolerances for the specified subcircuit and instances. It overrides the default scale value of 0.2 and sets it to 0.4 for the subcircuit and instances.

```
set_circuit_transceiver -enable 1 -process 1
set_sim_level 4
set_model_level 5
set_rf_option -enable 1 -process 1 -scale 0.4 -subckt block_name1
set_rf_option -enable 1 -process 1 -scale 0.4 -inst inst_name1
set_multi_core -core 8
```

7nm Transceiver Top-level Power Up and Leakage Simulation

In the following example, the `-app leakage` option is specified to inform the simulator that this is a leakage simulation.

```
set_circuit_transceiver -enable 1 -process 1 -powerup 1 -app leakage
set_sim_level 4
set_model_level 6
set_rf_option -enable 1 -process 1 -scale 0.4 -subckt block_name1
set_rf_option -enable 1 -process 1 -scale 0.4 -inst inst_name1
set_multi_core -core 8
```

12

SRAM Design Simulation

This chapter describes how to simulate Static Random Access Memory (SRAM) designs efficiently with a single, easy-to-use command.

This chapter contains the following topics:

- [SRAM Characterization Flow](#)
- [Debugging Tips](#)

SRAM Characterization Flow

The SRAM characterization flow is especially important to memory designs because they require high performance and precision for hundreds of simulations to complete the characterization. Accurate characterization of an SRAM is crucial to enable precise chip verification and signoff quality. One method used is instance-based memory characterization, which treats the entire memory as a single black box and characterizes the whole instance with parasitic elements. The advantage of this method is that it ensures accurate timing power and leakage modeling.

Thousands of simulations to support a wide range of process, voltage, and temperature corners make analyzing SRAM simulations a daunting and time-consuming task. These many independent simulations can be distributed across a number of machines to improve the turnaround time.

To address these challenges, the PrimeSim XA tool features a single command, `set_sram_characterization`, to efficiently simulate SRAM designs and enable accurate and fast performance. When you specify this command, the PrimeSim XA tool enables technologies specialized in simulating SRAM designs efficiently. It provides the best performance and accuracy tradeoff with a sliding-scale command, with tuning based on the application and accuracy requirements.

Accuracy and Performance Settings

To follow the basic guidelines using the `set_sram_characterization` command, you must be familiar with the following:

- The process node of the SRAM design to set the `-version` argument.

For SRAM designs that use FinFET process nodes of 10nm and below, or for SRAM designs that have extracted parasitic elements on the power supply, use the default setting, `-version 1`. You should also use `-version 1` for SRAM designs that use Internet of Things (IOT) process nodes. Note that `-version 1` also enables two cores for simulation. You can adjust the number of cores with the `set_multi_core` command or `-mt` command-line option. It is recommended to use `-version 1` if you plan to run multicore simulation with `set_sram_characterization`.

Other designs that can be simulated with `-version 0`.

- The simulation type to set the `-application` argument.

The common simulation types are timing and power. Simulation to check for timing is often faster than simulation to check for power or current. If you are only concerned about the timing of your SRAM design, specify the `-application timing` option. If you are concerned about power or current accuracy, set `-application power`. The `-application power` setting is more conservative compared to `-application timing` because it uses a more detailed modeling technique.

- The accuracy requirement to set the `-accuracy` argument.

The default for the `-accuracy` argument is 3, which provides the best performance and accuracy tradeoff for most SRAM designs. You can obtain more accuracy simulation results by setting it to 4 or 5; or you can use 1 or 2 to get faster performance if accuracy requirements have been satisfied.

The command settings are heavily based on the extraction commands from extraction tool, the foundry and corner, and the SRAM design itself. It is recommended to do a simulation sweep on all accuracy levels on a small subset of SRAM designs of a specific process node to find one that meets your specific accuracy and performance requirements before running thousands of simulations.

Note:

To address the complexity of different technology nodes from different foundries, you can use other commands in addition to the `set_sram_characterization` command. It is recommended *not* to use the `set_sram_characterization` command with any of the performance and accuracy commands listed in [Table 47](#). For example, do not use the `set_sim_level` and `set_sram_characterization` commands in the same simulation.

Table 47 *Commands Not to Use With set_sram_characterization*

Commands Not to Use With set_sram_characterization

set_sim_level

set_model_level

set_model_option

set_synchronization_level

set_synchronization_option

Commands to Replace for an SRAM Simulation

If you used the following commands for your simulation:

```
set_sim_level 6
set_model_level 5
set_tolerance_option 100
set_message_limit 100
load_ba_file cell.spf
set_duplicate_rule -select_subckt last
```

You convert it to the following setup as a starting point to simulate SRAM designs.

```
set_sram_characterization 1 -application timing -accuracy 4
set_message_limit 100
load_ba_file cell.spf
set_duplicate_rule -select_subckt last
```

Timing Methodology and Simulation Settings

Timing characterization can be done with a full-chip netlist. One method to improve performance, while maintaining timing accuracy, is to "divide and conquer" by breaking the SRAM into a set of critical paths and characterizing each of these paths separately. The critical paths are created by the designer with a path tracing and cutting tool.

By default, the `set_sram_characterization` command lumps all the coupling capacitors with values less than 1e-18 to ground. This value often gives the best performance and accuracy tradeoff. If your accuracy requirement has been met and you want to further improve the performance, you can increase the value to lump more coupling capacitors to ground with the `set_ccap_option` command.

[Table 48](#) provides an example of timing error percentage versus speed up when using the `set_ccap_option` command to lump different values of coupling capacitors. You can

only try to speed up the simulation performance by lumping more coupling capacitors if the base setting has met your accuracy requirement.

The number of coupling capacitors and their range of values are heavily based on the design, process node, and foundry. The PrimeSim XA tool generates a histogram of the coupling capacitors in the log file. Based on your accuracy and performance requirements, the rule of thumb is to lump 30% to 40% of the coupling capacitors in the design.

Table 48 *Timing Error Percentages*

Lumped Capacitor Value	1e-18	1e-17
Timing Error %	Base	+0.5%
Performance Speed Up	1X	1.4X

Dynamic Power Methodology and Simulation Settings

Accurate power measurement has become increasingly important, mainly due to the popularity of embedded systems like cell phones and other devices with limited hardware resources and battery supply. SRAM is usually a significant contributor to power dissipation in these embedded systems. Lower power dissipation means longer battery life and a better user experience.

Power characterization is done on a full-chip netlist. Running a full-chip netlist takes a much longer simulation run time than running a critical path netlist recommended for a timing simulation. Because the accuracy requirement of power simulation is often looser than the accuracy requirement of a timing simulation, there are methods that could help to improve the performance of the power simulation.

By default, the `set_sram_characterization` command shorts all the resistors with values less than 0.1. This value often gives the best performance and accuracy tradeoff. However, parasitic resistors are not the main contributor to power dissipation. You can further improve performance by increasing the value to short more resistors if the accuracy requirement has been met. You can do that with the `set_resistor_option` command.

[Table 49](#) provides power error percentage versus speed up when using the `set_resistor_option` command to short different resistor values. The number of resistors and their range of values are heavily based on the design, process node, and foundry. The PrimeSim XA tool generates a histogram of the resistor of the design in the log file. Based on your accuracy and performance requirement, the rule of thumb is to lump 30% to 40% of the resistor in the design.

Table 49 Power Error Percentages

Shorted Resistor Value	0.1	10	50	100
Power Error %	Base	+0.2%	+0.4%	+0.6%
Performance Speed Up	1X	1.4X	1.7X	1.9X

Static and Leakage Power Methodology and Simulation Settings

Power dissipation is rapidly becoming a significant design constraint. Dynamic power has been a predominant source of power dissipation. Static power dissipation is also becoming a significant fraction of the total power. Static power is the power dissipation in the absence of any switching activity and it is referred to as leakage power.

Leakage power simulation is often done on a netlist with a functional block of an SRAM (a group of leaf cells). The netlist size is much smaller than what is typically used for timing and dynamic power simulation. Due to the size of the netlist and high accuracy requirement, you can use very accurate settings to run leakage power simulation.

Debugging Tips

You should review the waveforms for the signals of interest in a waveform viewer if you notice any issues. Here are some debugging tips:

- [Initializing Latch Circuitry](#)
- [Identifying Floating Nodes](#)
- [Probing Sub-Nodes in Post-Layout Simulation](#)

Initializing Latch Circuitry

If you do not specify initial conditions for latch circuitry in an SRAM design, the simulator could initialize the latch nodes to be 1/0, 0/1 or a meta-stable state; all of which can be mathematically valid. Missing initial conditions for latches could cause your timing and power measurements to be off from your accuracy requirement. You should specify initial conditions for all the latches in your netlist before running the simulation to ensure more predictable and robust results. You can set initial conditions using full hierarchical node names or with wildcard patterns as follows.

To specify the initial conditions using full hierarchical node names:

```
.ic v(xtop.xarray_right.xblk.xdff.rt) 0
.ic v(xtop.xarray_right.xblk.xdff.rb) HIGH
```

To specify the initial conditions with wildcards:

```
.ic v(xtop.xarray_*.x*.xdff.rt) 0  
.ic v(xtop.xarray_*.x*.xdff.rb) HIGH
```

You should always check the PrimeSim XA log file to make sure that the initial conditions you specified were used for the simulation.

Identifying Floating Nodes

Floating nodes are another source of simulation inaccuracy. You should examine the PrimeSim XA log file to make sure there are no warning messages for floating nodes in your netlist. The PrimeSim XA tool considers a node to be floating if it does not have a DC path to ground, and it touches at least one of the following:

- The gate of a MOSFET
- A current source
- The controlling input node of a controlled source

If your netlist has floating nodes, the PrimeSim XA tool keeps the nodes floating during the simulation. The floating nodes can take on any value, so the results may be unpredictable. Floating gate and bulk nodes can also cause performance and accuracy issues. You should fix the netlist file to remove floating nodes. If you cannot modify your netlist, you can use the `set_floating_node` command to tie down the floating nodes.

Probing Sub-Nodes in Post-Layout Simulation

In a flat post-layout simulation, the sub-nodes are usually not of interest, and suppressing thousands of these nodes can improve performance. The post-layout sub-nodes are the node names with colon character, ":". The `set_sram_characterization` command enables the `set_probe_option -skip_flat_pl_node 1` command to exclude the probing of sub-nodes when a wildcard is used. If you want to probe the sub-nodes, there are a couple of different ways to achieve this:

- Add the `set_probe_option -skip_flat_pl_node 0` command after the `set_sram_characterization` command. This command instructs the PrimeSim XA tool to save all nodes (including the post-layout sub-nodes) in the design. Note that printing all of the sub-nodes in the design adversely impacts your simulation performance.
- Explicitly probe the signal name of interest as follow:

```
.probe v(x1.net17:f7154)
```

or

```
probe_waveform_voltage -v x1.net17:f7154
```


13

Flash Core Cell Models

This chapter provides data about the PrimeSim XA tool support of flash core cells, a non-volatile memory technology based on a floating gate-device. This model facilitates the simulation of NOR flash design styles.

This chapter contains the following topics:

- [Modeling Flash Core Cells](#)
- [Using Flash Level 1](#)
- [Using Flash Level 3](#)

Modeling Flash Core Cells

The PrimeSim XA tools supports the use of a compact model for flash core cells. Flash is a nonvolatile memory technology based on a floating-gate device. This modeling capability lets you verify all the peripheral circuitry and its routing to the core cell array simultaneously without compromising on accuracy or simulation speed. This capability is available at the prelayout as well as the postlayout simulation stage.

This compact modeling provides a solution that is more efficient and flexible than traditional means of macro-modeling like analog HDL or complex subcircuits with many elements. The flash core cell model allows for high capacity simulation that can take full advantage of PrimeSim XA optimizations.

The floating gate modeling is appended to a traditional MOS model. Each flash cell can be initialized via a netlist instance parameter or configuration command. An additional fifth terminal is available to represent the Nwell for twin-well processes.

This feature is based upon the legacy HSIM flash core cell model. It is available in Eldo and HSPICE netlist formats and models.

Using Flash Level 1

The `Flashlevel =1` parameter is used for simple models that target NOR flash technology. All cell programming and erasing behavior is controlled by built-in equations. The flash model definition is a regular SPICE model statement whose parameters define the program and erase conditions as well as the rate of voltage change, minimum, and maximum voltages. The flash model is then appended to a traditional MOS model with the `.appendmodel` statement.

This section includes the following topics:

- [Supported MOS Models](#)
- [Defining Flash Model Parameters](#)
- [Programming and Erasing Conditions](#)
- [Vth Change Behavior](#)
- [Instantiating Flash Core Cells](#)
- [Initializing Flash Core Cells](#)
- [Probing Flash Core Cells](#)
- [Flash Core Cell Level 1 Example](#)

Supported MOS Models

The following base MOS models can be appended with a flash model:

- Level 49 – BSIM3v3
- Level 53 – BSIM 3V3
- Level 54 – BSIM4
- Eldo Level 1 – Berkeley Spice Model
- Eldo Level 53 – BSIM3v3
- Eldo Level 59 – MOS9

Defining Flash Model Parameters

The flash model parameters are defined in a `.model` card with the `flashcell` keyword, the `flashlevel=1` parameter and other flash cell parameters.

For example:

```
.model flash_model_name flashcell flashlevel=1 \
    model_parameter { model_parameter }
```

The flash model is then appended to a regular MOS model with the `.appendmodel` statement:

```
.appendmodel flash_model_name flashcell mos_model_name mos_type
```

The append model statement searches for the flash model and MOS model definitions within the current subcircuit scope or at a hierarchical level. It is recommended that the flash model, MOS model, and append model all be placed at the same scope. The following is a correct example:

```
* Top hierarchical netlist level or within the same .subckt.
.model myflash flashcell flashlevel=1
.model mymos nmos level=1
.appendmodel myflash flashcell mymos nmos
```

The following example is incorrect:

```
* The following is illegal
.subckt mysub
.model myflash flashcell flashlevel=1
.ends
.subckt mysub2
.model mymos nmos level=1
.ends
.appendmodel myflash flashcell mymos nmos
* fails because appendmodel is at top level and mos and/or flashcell are
  defined at a deeper hierarchal depth.
```

The following table lists the flash model parameters and their default values.

Parameter	Default Value	Description
VDPGMMIN	3.0	Program min drain voltage
VGPGMMIN	8.0	Program min gate voltage
VSPGMAX	1.0	Program max source voltage
VPWPGMMAX	1.0	Program max bulk voltage
VNWPGMMAX	2.0	Program max bulk2 voltage
VNWPGMMIN	No default, active when set	Program min bulk2 voltage
TPGMSTEP	1n	Program unit time

Parameter	Default Value	Description
KPGM	1m	Program constant
VTPGMSAT	0.0	Program saturation
VTPGMMAXSHIFT	5.0	Program max vt shift
VTHIGH	7.0	Program max vt
VDERSMIN	7.0	Erase min drain voltage
VGERSMAX	-8.0	Erase max gate voltage
VSESRMIN	-8.0	Erase min source voltage
VPWERSMIN	7.0	Erase min bulk voltage
VNWERSMIN	7.0	Erase min bulk2 voltage
VNWERSMAX	No default, active when set	Erase max bulk2 voltage
TERSSTEP	1n	Erase unit time
KERS	1m	Erase constant
VTERSAT	0.0	Erase saturation
VTERSMAXSHIFT	5.0	Erase max vt shift
VTLOW	0.0	Erase min vt

Programming and Erasing Conditions

A flash cell enters a programming or erasing event when the cell terminal voltages satisfy all conditions. The event ends when any condition is no longer satisfied. V(S) refers to source voltage, which is the lowest voltage on terminals 1 or 3. V(D) refers to the drain voltage, which is the highest voltage on terminals 1 or 3. The source and drain terminals are swapped dynamically during simulation so that this definition remains true.

A programming event occurs when all of the following conditions are met:

$V(D) > VDPGMMIN$

$V(G) > VGPGMMIN$

$V(S) < VSPGMMAX$

$$V(B) < VPWPGMMAX$$

If B2 is defined:

- If no $VNWPGM^*$ parameter is set or $VNPGMMAX$ is set, $V(B2) < VNWPGMAX$.
- If $VNWPGMMIN$ only is set, $V(B2) > VNWPGMMIN$.
- If $VNWPGMAX$ and $VNWPGMMIN$ are both set, $V(B2) > VNWPGMMIN$.

An erasing event occurs when all of the following conditions are met:

$$V(D) > VDERSMIN$$

$$V(G) < VGERSMAX$$

$$V(S) > VSERSMIN$$

$$V(B) > VPWERSMIN$$

If B2 is defined:

- If no $VNWERS^*$ parameter is set or $VNWERSMIN$ is set, $V(B2) > VNWERSMIN$.
- If $VNWERSMAX$ only is set, $V(B2) < VNWERSMAX$.
- If both $VNWERSMIN$ and $VNWERSMAX$ are set, $V(B2) < VNWERSMAX$.

Vth Change Behavior

A programming event corresponds to an increase in the cell threshold voltage. An erasing event corresponds to a decrease in the cells threshold voltage. During a programming event the Vth change per TPGMSTEP is given by:

$$KPGM \cdot (V(G) - VTPGMSAT - VTH) \cdot (V(D) - VDPGMMIN)$$

The maximum value of VTH is determined by the minimum of:

- $V(G)$
- $VTH_init + \text{abs}(VTPGMMAXSHIFT)$
- $VTHIGH - VTPGMSAT$

During an erasing event the VTH change per TERSTEP us given by:

$$KERS \cdot (VTH - V(G) - VTERSSAT) \cdot (V(S) - VSERSMIN)$$

The minimum value for VTH is the maximum of:

- $V(G)$
- $VTH_init - \text{abs}(VTERS\text{MAXSHIFT})$
- $VTLOW + VTERSSAT$

Instantiating Flash Core Cells

Flash core cells are instantiated by specifying a MOS instance and using the flashCell model name. A second bulk connection for a twin-well process and an initial programming voltage can also be defined on the instance line. The syntax is:

```
Mos d g s b flash_model_name w=wval l=lval \  
    [ b2=b2node_name ] [ delvto=pval ]
```

where `b2node_name` is the name of the second bulk connection. The `delvto` parameter is the initial change in the threshold voltage.

The following example instantiates a cell in a single well process:

```
M1 d g s b flashmodel w=1u l=1u
```

The following example instantiates a cell in a twin-well process:

```
M1 d g s b flashmodel w=1u l=1u b2=bulk2
```

Initializing Flash Core Cells

The flash core cell can be initialized by an instance parameter in the netlist or a PrimeSim XA configuration command. To initialize a cell in the netlist, use the `delvto` instance parameter. This parameter applies an initial change in threshold voltage. It does not set the actual threshold voltage. For example:

```
M1 d g s b flashcell w=1u l=1u delvto=2
```

In the previous example the cell has been initialized with +2V of VTH change.

Cells can also be initialized with the `set_flash_option` command. This command overrides any `delvto` instance parameter setting in the netlist. For example:

```
M1 d g s b flash_cell_model w=1u l=1u b2=bulk2 delvto=-2  
.opt xa_cmd="set_flash_option -delvto 2 -inst M1"  
* M1 is initialized with a threshold voltage change of +2V
```

Probing Flash Core Cells

The threshold voltage of a cell can be plotted using the lv9() signal access function. The change in threshold voltage due to programming or erasing can be plotted using the lv0() signal access function.

Flash Core Cell Level 1 Example

```
*signal printing section
.print lv9(m*) lv0(m*) v(*) i(*)

*nmos model definition section
.model mos1 nmos level=1 vto=2.5 kp=25u gamma=0 lambda=0.1 phi=0

*flash model definition section
.model flash flashcell flashlevel=1
+ *programming parameters
+ VTHIGH=20
+ KPGM=0.1m
+ VTPGMMAXSHIFT=10
+ VGPGMMIN=2.5
+ VDPGMMIN=2
+ VSPGMMAX=2
+ VPWPGMMAX=10
+ VNWPMMAX=10
+ *erasing parameters
+ VTLOW=-20
+ KERS=0.1m
+ VTERSMAxSHIFT=10
+ VGERsMAX=-3
+ VDERsMIN=4
+ VSERsMIN=-10
+ VPWERSMIN=-10
+ VNWERSMIN=-10

.appendmodel flash flashcell mos1 nmos

*device instantiation section
m1 nd ng ns nb mos1 L=1u W=1u B2=nb2

*stimuli section
Vng ng 0
+pwl(0.5u 5.5 0.6u 9 1u 9 1.1u 5.5 1.5u 5.5 1.6u -9 2u -9 2.1u 5.5)
Vnd nd 0
+pwl(0.5u 0.9 0.6u 4.5 1u 4.5 1.1u 0.9 1.5u 0.9 1.6u -6 2u -6
2.1u 0.9)
Vns ns 0 pwl(1u 0 1.1u 0 1.5u 0 1.6u 8 2u 8 2.1u 0)
Vnb nb 0 pwl(0.5u -8 0.6u 1 1u 1 1.1u -8 1.5u -8 1.6u 8 2u 8 2.1u -8)
Vnb2 nb2 0 10
```

```
*simulation setup section
.tran 10n 2.5u
.end
```

Using Flash Level 3

The `Flashlevel=3` parameter is used for simple models that target split-gate flash technology. All cell programming and erasing behavior is controlled by built-in equations. The flash model definition is a regular SPICE model statement whose parameters define the program and erase conditions, as well as the rate of voltage change, minimum, and maximum voltages. The flash model is then appended to a traditional MOS model with the `.appendmodel` statement.

This section includes the following topics:

- [Supported MOS Models](#)
- [Defining Flash Model Parameters](#)
- [Programming and Erasing Conditions](#)
- [Vth Change Behavior](#)
- [Instantiating Flash Core Cells](#)
- [Initializing Flash Core Cells](#)
- [Probing Flash Core Cells](#)
- [Flash Core Cell Level 3 Example](#)

Supported MOS Models

The following base MOS models can be appended with a flash model having `flashlevel=3`:

- Level 49 – BSIM3v3
- Level 53 – BSIM4

Defining Flash Model Parameters

The flash model parameters are defined in a `.model` card with the `flashcell` keyword, the `flashlevel=3` parameter and other flash cell parameters. For example:

```
.model flash_model_name flashcell flashlevel=3 model_parameter \
{ model_parameter }
```


The flash model is then appended to a regular MOS model with the `.appendmodel` statement:

```
.appendmodel flash_model_name flashcell mos_model_name mos_type
```

The append model statement searches for the flash model and MOS model definitions within the current subcircuit scope or at a hierarchical level. It is recommended that the flash model, MOS model, and append model all be placed at the same scope. The following example is correct:

```
* Top hierarchical netlist level or within the same .subckt.
.model myflash flashcell flashlevel=3
.model mymos nmos level=3
.appendmodel myflash flashcell mymos nmos
```

The next example is incorrect:

```
* The following is illegal
.subckt mysub
.model myflash flashcell flashlevel=3
.ends
.subckt mysub2
.model mymos nmos level=3
.ends
.appendmodel myflash flashcell mymos nmos
* fails because appendmodel is at top level and mos and/or flashcell are
  defined at a deeper hierarchal depth.
```

The following table lists the flash model parameters and their default values.

Parameter	Default Value	Description
VDPGMMAX	1.0	Program max drain voltage
VGPGMMIN	1.0	Program min gate voltage
VSPGMMIN	5.0	Program min source voltage
VPWPGMMAX	1.0	Program max bulk voltage
TPGMSTEP	1n	Program unit time
KPGM	1m	Program constant
VTPGMSAT	0.0	Program saturation
VTPGMMAXSHIFT	5.0	Program max vt shift
VTHIGH	7.0	Program max vt
VDERSMAX	1.0	Erase max drain voltage

Parameter	Default Value	Description
VGERMIN	10.0	Erase max gate voltage
VSERSMAX	1.0	Erase max source voltage
VPWERSMAX	1.0	Erase max bulk voltage
TERSSTEP	1n	Erase unit time
KERS	1m	Erase constant
VTERSSAT	0.0	Erase saturation
VTERSMAXSHIFT	5.0	Erase max vt shift
VTLOW	0.0	Erase min vt

Programming and Erasing Conditions

A flash cell enters a programming or erasing event when the cell terminal voltages satisfy all conditions. The event ends when any condition is no longer satisfied. In contrast to NOR- and NAND flash, a split-gate flash core cell is asymmetric, so drain (D) and source (S) terminals are not swappable. This means program or erase events do not happen if the conditions below are satisfied when the terminals are swapped.

A programming event occurs when all of the following conditions are met:

Drain: $V(D) < VDPGMMAX$

Gate: $V(G) > VGPGMMIN$

Source: $V(S) > VSPGMMIN$

Bulk: $V(B) < VPWPGMMAX$

An erasing event occurs when all of the following conditions are met:

Drain: $V(D) < VDERSMAX$

Gate: $V(G) > VGERSMIN$

Source: $V(S) < VSERSMAX$

Bulk: $V(B) < VPWERSMAX$

Vth Change Behavior

A programming event corresponds to an increase in the cell threshold voltage. An erasing event corresponds to a decrease in the cells threshold voltage. During a programming event the Vth change per TPGMSTEP is given by:

$$KPGM \cdot (V(G) - VTPGMSAT - VTH) \cdot (V(S) - VSPGMMIN)$$

This VTH increase is prorated to the time step taken by the simulator. The maximum value of VTH is determined by the minimum of:

- $V(S)$
- $VTH_init + \text{abs}(VTPGMMAXSHIFT)$
- $VTHIGH - VTPGMSAT$

During an erasing event the VTH change per TERSTEP is given by:

$$KERS \cdot (VTH + V(G) - VTERSSAT) \cdot (VSERSMAX - V(S))$$

This VTH decrease is prorated to the time step taken by the simulator. The minimum value for VTH is the maximum of:

- $V(S)$
- $VTH_init - \text{abs}(VTERSMAXSHIFT)$
- $VTLOW + VTERSSAT$

Instantiating Flash Core Cells

Flash core cells are instantiated by specifying a MOS instance and using the flashcell model name. A second bulk connection for a twin-well process and an initial programming voltage can also be defined on the instance line. The syntax is:

```
Mos d g s b flash_model_name w=wval l=lval \
    [ b2=b2node_name ] [ delvto=pval ]
```

where `b2node_name` is the name of the second bulk connection. The `delvto` parameter is the initial change in the threshold voltage.

The following example instantiates a cell in a single well process:

```
M1 d g s b flashmodel w=1u l=1u
```

The following example instantiates a cell in twin-well process:

```
M1 d g s b flashmodel w=1u l=1u b2=bulk2
```

Initializing Flash Core Cells

The flash core cell can be initialized by an instance parameter in the netlist or a PrimeSim XA configuration command. To initialize a cell in the netlist, use the `delvto` instance parameter. This parameter applies an initial change in threshold voltage. It does not set the actual threshold voltage. For example:

```
M1 d g s b flashcell w=1u l=1u delvto=2
```

In the previous example the cell has been initialized with +2V of VTH change.

Cells can also be initialized with the `set_flash_option` command. This command overrides any `delvto` instance parameter setting in the netlist. For example:

```
M1 d g s b flash_cell_model w=1u l=1u b2=bulk2 delvto=-2
.opt xa_cmd="set_flash_option -delvto 2 -inst M1"
* M1 is initialized with a threshold voltage change of +2V
```

Probing Flash Core Cells

The threshold voltage of a cell can be plotted using the `lv9()` signal access function. The change in threshold voltage due to programming or erasing can be plotted using the `lv0()` signal access function.

Flash Core Cell Level 3 Example

```
*signal printing section
.print lv9(m*) lv0(m*) v(*) i(*)

*nmos model definition section
.model mos1 nmos level=3 vto=2.5 kp=25u gamma=0 lambda=0.1 phi=0

*flash model definition section
.model flash flashcell flashlevel=3
+ *programming parameters
+ VTHIGH=20
+ KPGM=0.1m
+ VTPGMMAXSHIFT=10
+ VGPGMMIN=2.5
+ VDPGMMIN=2
+ VSPGMMAX=2
+ VPWPGMMAX=10
+ VNWPMMAX=10
+ *erasing parameters
+ VTLOW=-20
+ KERS=0.1m
+ VTERSMAXSHIFT=10
+ VGERSMAX=-3
+ VDERSMIN=4
```

```
+ VSERSMIN=-10
+ VPWERSMIN=-10
+ VNWERSMIN=-10

.appendmodel flash flashcell mos1 nmos

*device instantiation section
m1 nd ng ns nb mos1 L=1u W=1u B2=nb2

*stimuli section
Vng ng 0
+pw1(0.5u 5.5 0.6u 9 1u 9 1.1u 5.5 1.5u 5.5 1.6u -9 2u -9 2.1u 5.5)
Vnd nd 0
+pw1(0.5u 0.9 0.6u 4.5 1u 4.5 1.1u 0.9 1.5u 0.9 1.6u -6 2u -6
2.1u 0.9)
Vns ns 0 pw1(1u 0 1.1u 0 1.5u 0 1.6u 8 2u 8 2.1u 0)
Vnb nb 0 pw1(0.5u -8 0.6u 1 1u 1 1.1u -8 1.5u -8 1.6u 8 2u 8 2.1u -8)
Vnb2 nb2 0 10

*simulation setup section
.tran 10n 2.5u
.end
```

14

MRAM Core Cell Models

This chapter provides information about the PrimeSim XA tool support of MRAM core cells, a nonvolatile memory technology based on magnetic states. These models facilitate the simulation of circuits with MRAM devices.

This chapter contains the following topics:

- [Modeling MRAM Core Cells](#)
- [Spin-Torque-Transfer \(STT\) MRAM Core Cell Model \(MRES0\)](#)
- [Dual-Active Layer \(DAL\) MRAM \(MRES1\)](#)
- [Toggle MRAM Core Cell Model \(MRES2\)](#)

Modeling MRAM Core Cells

MRAM architectures integrate magnetic devices within standard CMOS microelectronics. Unlike other memory technologies, MRAM data is stored as a magnetic state, rather than electrical charge. The elements are formed from two ferromagnetic plates, each of which can hold a magnetic field, separated by a thin insulating layer. The read (or "sensing") of the magnetic state is accomplished by measuring the electrical resistance of the cell. Due to the magnetic tunnel effect, the electrical resistance of the cell changes due to the orientation of the fields in the two magnetic plates.

By measuring the current flowing through the cell, the resistance inside a cell can be determined. Typically, if two magnetic plates have the same polarity (parallel), this is considered a "0" state (or R_{MIN}); if two plates are of opposite polarity (anti-parallel), the resistance is higher (R_{MAX}) which is considered a "1" state.

Due to the various different MRAM architectures, MRAM core cells differ in structure, functionality and write/read cycles. There are several types of MRAM core cells supported in the PrimeSim XA tool. You can choose one of the supported MRAM core cell models to properly define the correct functionality of their MRAM designs.

Currently, the PrimeSim XA tool supports three MRAM core cell architectures with 0, 1, and 2 word lines:

- Spin-Torque-Transfer MRAM: MRES0
 - Bidirectional symmetrical write
 - Write driver has bidirectional current flow (behaves as a current source AND current sink)
 - Parallelizing-direction read
- Dual 'Active' Layer MRAM: MRES1
 - Contains two 'active' magnetic layers whose polarity can be switched
 - Symmetrical writing current and time between "0" and "1"
 - Two-cycle read
- Toggle MRAM: MRES2
 - Same pulse sequence used to write "0" to "1" or "1" to "0"
 - Toggle current magnetic state to the opposite state with each execution

Spin-Torque-Transfer (STT) MRAM Core Cell Model (MRES0)

Spin-torque-transfer (STT) MRAM uses spin-aligned ("polarized") electrons to directly torque the magnetic domains. Specifically, if the electrons flowing into a layer have to change their spin, this develops a torque that is transferred to the nearby layer.

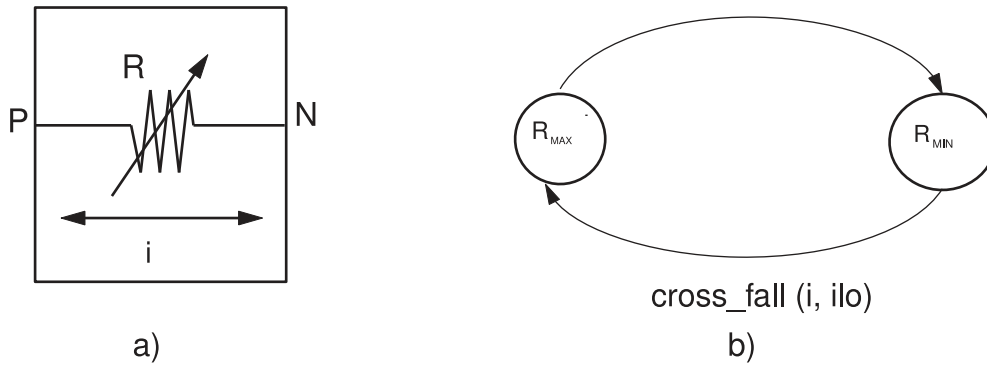
A stream of conducting electrons moving through the fixed magnetic layer are spin polarized (that is, most of the electrons' spins become aligned to that of the fixed layer). When these spin-polarized electrons pass through the free layer, they become re-polarized. In re-polarizing, the free-layer magnet experiences a torque associated with the change in angular momentum resulting from the rotation of the spins. This torque pumps enough energy to reverse the orientation of the free-layer magnet.

This spin-alignment of the electrons is ultimately achieved by changing the direction of the write current requiring a bidirectional and symmetrical write current.

The magnetic tunnel junction (MTJ) or magnetic device within the STT MRAM core cell consists of two ferromagnetic layers, one free to change polarity (sense layer) and the other fixed to some predetermined polarity (reference layer).

Graphically, the STT MRAM core cell can be represented as a 2-terminal device, consisting of a bidirectional bit line (BL) shown in [Figure 27](#).

Figure 27 STT MRAM 2-Terminal Device (MRES0) and STT MRAM State Sequence and Current Requirements



MRES0: STT MRAM Core Cell Definition

Gxx p n MRES0

- + rmin=<val>
- + rmax=<val>
- + ihi=<val>
- + ilo=<val>
- + [ic=<val>]
- + [iwin=<val>]

MRES0: Supported Parameter Descriptions

- rmin - Minimum resistance of 'parallel' state of the MRAM cell dipoles
- rmax - Maximum resistance of 'anti-parallel' state of the MRAM cell dipole
- ihi - i upper threshold
- ilo - i lower threshold
- ic - Initial state of MRAM cell; default=0 <Boolean> (0 ~ rmin; 1 ~ rmax)
- iwin - ime duration window which current must be above the threshold (ihi | ilo); default=0

Note:

This current threshold condition must always be followed: $i_{hi} > i_{lo}$.

iwin - Check when current crosses threshold, the current remains above (or below) that threshold for a certain period of time (specified by iwin) in order to cause a switch. If the current falls before meeting the iwin criteria, the exam time data is reset.

MRES0: Instantiation Example

```
G0 p n MRES0 rmin=1k rmax=1.2k ihi=1m ilo=-1m
```

MRES0: STT MRAM Core Cell Functionality

```
if (cross_rise (i, ihi)) {  
    state = rmin ;  
} elsif (cross_fall (i, ilo)) {  
    state = rmax ;  
}
```

Limitations and Assumptions for the MRES0 Core Model

Current thresholds (ihi and ilo)

- i is bi-directional current, which can flow into or out of the MRAM cell
- The bi-directional functionality determines the state to which the cell resolves
- The current flowing may either be 'positive' (flowing in one direction) or 'negative' (flowing in the opposite direction). Therefore, ihi and ilo are typically close in magnitude but opposite in sign: ihi >> ilo.

Dual-Active Layer (DAL) MRAM (MRES1)

Similar to the Toggle and STT MRAM architectures, the MTJ within the Dual-Active Layer (DAL) MRAM core cell also consists of two ferromagnetic layers. However, within the DAL MRAM, both magnetic layers are free to change polarity. For naming differentiation from the other architectures, they are referred to here as soft (S) and hard (H) layers.

The word line (WL) is bidirectional, so the direction and magnitude of the WL current will determine the resulting polarity of the two dipoles. The S layer dipole is controlled via the WL current $i_l > i_{lshi}$ || $i_l < i_{lslo}$. At this current value (i_{lshi} || i_{lslo}), the H layer dipole is not affected, as it is thicker. The H layer dipole is controlled via $i_l > i_{lshi}$ || $i_l < i_{lhlo}$. $i_{lshi} < i_{lhhi}$ & $i_{lslo} > i_{lhlo}$, so changing the H layer dipole may also cause a change in the S layer dipole.

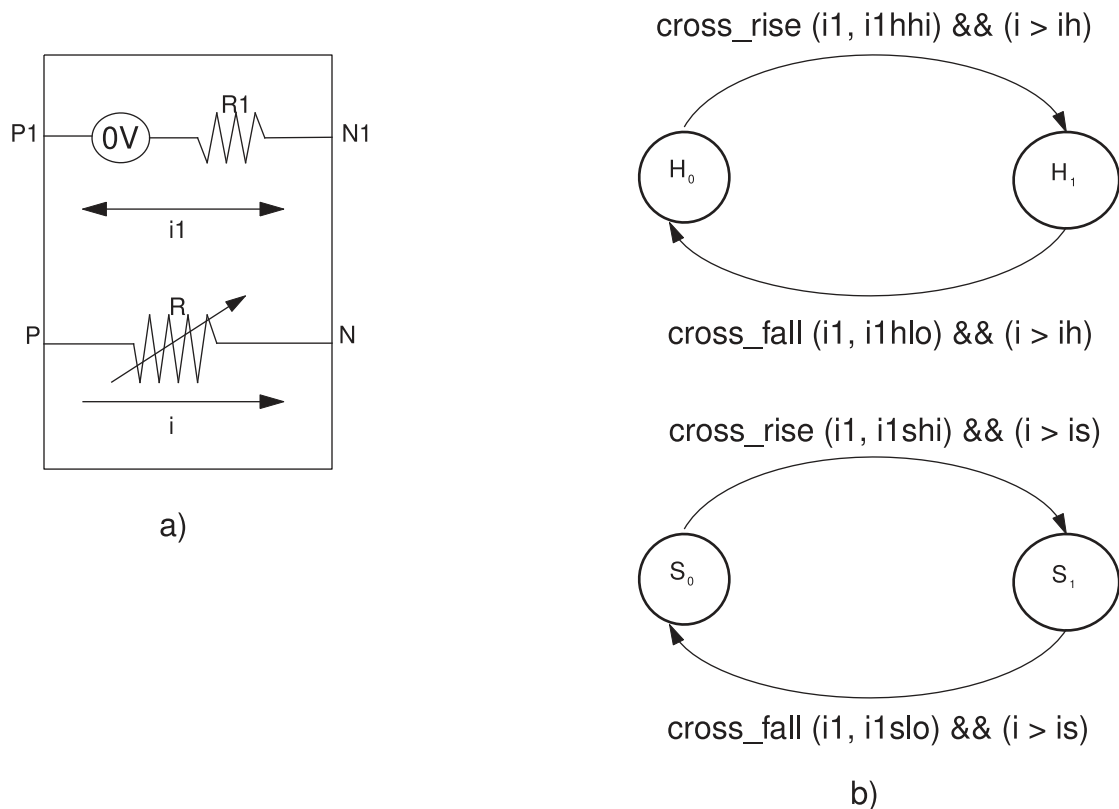
In order to determine the state of the cell or orientation of the H layer, a two-cycle read operation is required. The H layer dipole direction is determined by flipping the S layer

only ($i_{lshi} < i_l < i_{lhhi}$) first in one direction (shaping the S layer dipole in a fixed orientation) and then reversing the direction if the i_l ($i_{lhlo} < i_l < i_{lslo}$) current (shaping the S layer dipole in the opposite orientation). The difference in resistance measured between the two read cycles determines the orientation of the H layer and thus the state of the cell. If the S and H layer dipoles are parallel (or aligned), the measured resistance will be lower (RMIN) then if the dipoles of the S and H layers were in an anti-parallel (or unaligned) position (RMAX).

The model also requires a sense line (SL) running in between the two magnetic layers used to help change the polarity of the magnetic layers as well as help determine resistance shifts.

Graphically, the DAL MRAM core cell can be represented as a 4-terminal device, consisting of a bidirectional word line (WL) and a unidirectional sense line (SL) and a two-state structure (one for each magnetic layer) shown in [Figure 28](#).

Figure 28 DAL MRAM 4-Terminal Device (MRES1) and DAL MRAM State Sequence and Current Requirements



MRES1: DAL MRAM Core Cell Definition

Gxx p n MRES1 p1 n1

+ rmin=<val>

+ rmax=<val>

+ ih=<val>

+ is=<val>

+ i1hhi=<val>

+ i1hlo=<val>

+ i1shi=<val>

+ i1slo=<val>

+ [r1=<val>]

+ [ich=<val>]

+ [ics=<val>]

MRES1: Supported Parameter Description

- rmin - Minimal resistance of 'parallel' state of the MRAM cell dipoles
- rmax - Maximal resistance of 'anti-parallel' state of the MRAM cell dipoles
- ih - i threshold for magnetic hard (H) layer
- is - i threshold for magnetic soft (S) layer
- i1hhi - i1 upper threshold for magnetic hard (H) layer
- i1hlo - i1 lower threshold for magnetic hard (H) layer
- i1shi - i1 upper threshold for magnetic soft (S) layer
- i1slo - i1 lower threshold for magnetic soft (S) layer
- r1 - Additional internal line resistance (optional and defaults to 10mΩ ('10m'))
- ich - Initial state of MRAM hard layer; default=0 <Boolean>
- ics - Initial state of MRAM soft layer; default=0 <Boolean>

Note:

These current threshold conditions must always be followed:

$$i_h > i_s$$

$$i_{1hhi} > i_{1shi} > i_{1slo} > i_{1hlo}$$

MRES1: Instantiation Example

```
G1 p n MRES1 p1 n1 rmin=1k rmax=1.2k  
+ ih=3m is=2m i1hhi=15m i1hlo=-15m i1shi=10m i1slo=-10
```

MRES1: DAL MRAM Core Cell Functionality

```
if (cross_rise (i1, i1hhi)) {  
    if (i > ih) {  
        stateH = 1 ;  
    }  
} elseif (cross_fall (i1, i1hlo)) {  
    if (i > ih) {  
        stateH = 0 ;  
    }  
} if (cross_rise (i1, i1shi)) {  
    if (i > is) {  
        stateS = 1 ;  
    }  
} elseif (cross_fall (i1, i1slo)) {  
    if (i > is) {  
        stateS = 0 ;  
    }  
}  
if (stateH == stateS) {  
    R = rmin ;  
} elseif (stateH != stateS) {  
    R = rmax ;  
}
```

Limitations and Assumptions for the MRES1 Core Model

Current thresholds for i1:

- i1 is bi-direction current which flow into or out of the MRAM cell
- The bi-directional functionality determines the state to which the cell resolves

- The current flowing may either be 'positive' (flowing in one direction) or 'negative' (flowing in the opposite direction). Therefore, i_{hi} and i_{lo} are typically close in magnitude but opposite in sign:

$$i_{1hhi} \gg i_{1hlo} \ \&\& \ i_{1shi} \gg i_{1slo}$$

$$i_h > i_s$$

$$i_{1hhi} > i_{1shi} > i_{1slo} > i_{1hlo}$$

Toggle MRAM Core Cell Model (MRES2)

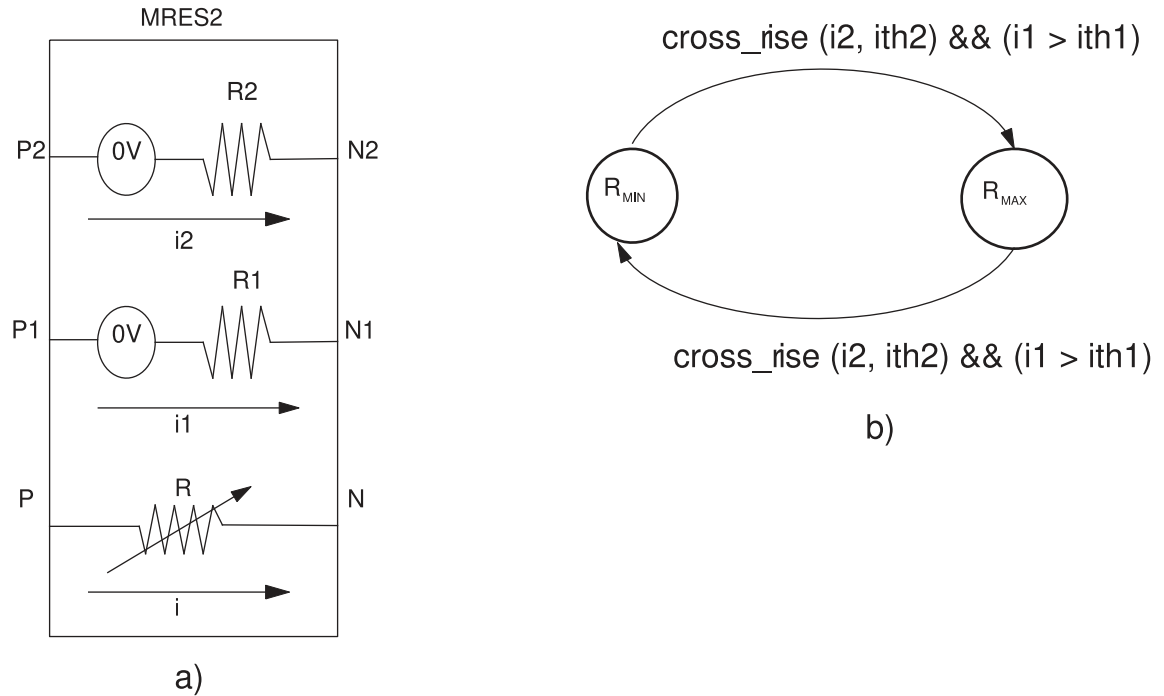
The Toggle MRAM architecture obtains its name from its use of the same pulse sequence when writing a "0" to "1" or "1" to "0" state. Each time the write current sequence is executed, the magnetic device changes from its current magnetic state to the opposite magnetic state.

Similar to the STT MRAM architecture, the MTJ within the Toggle MRAM core cell also consists of two ferromagnetic layers, one free to change polarity and the other fixed to some predetermined polarity.

By applying a current pulse sequence through two independent write lines, a rotating magnetic field is generated which moves the free, or sense magnetic layer, from one state to the other. Since the write sequence toggles the bit to its opposite state regardless of its existing state, a pre-read must be performed to determine if a write is required.

Graphically, the toggle MRAM core cell can be represented as a 6-terminal device, consisting of unidirectional write-through bit line (BL) and word lines (WL1 and WL2) as shown in [Figure 29](#).

Figure 29 Toggle MRAM 6-Terminal Device (MRES2) and Toggle MRAM State Sequence and Current Requirements



MRES2: Toggle MRAM Core Cell Definition

Gxx p n MRES2 p1 n1 p2 n2

+ rmin=<val>

+ rmax=<val>

+ ith1=<val>

+ ith2<val>

+ [r1=<val> r2=<val>]

+ [ic=<val>]

MRES2: Supported Parameter Description

- rmin - Minimal resistance of “parallel” state of the MRAM cell dipoles
- rmax - Maximal resistance of 'anti-parallel' state of the MRAM cell dipole

- ith1 - i1 threshold value of current flowing through word line 1 (WL1)
- ith2 - i2 threshold value of current flowing through word line 2 (WL2)
- r1 - Additional internal word line (WL1) resistance (optional and defaults to 10mΩ ('10m'))
- r2 - Additional internal word line (WL2) resistance (optional and defaults to 10mΩ ('10m'))
- ic - Initial state of MRAM cell; default=0 <Boolean> (0 ~ rmin; 1 ~rmax)

MRES2: Instantiation Example

```
G2 p n MRES2 p1 n1 p2 n2 rmin=1k rmax=1.2k ith1=3m ith2=2m
```

MRES2: Toggle MRAM Core Cell Functionality

```
if (cross_rise (i2, ith2) && (i1 > ith1)) {  
    state = ~state;  
}
```

15

Phase Change Memory (PCM) Models

This chapter describes the built-in PCM model and how to simulate circuits with this type of non-volatile memory.

This chapter contains the following topics:

- [PCM Model](#)
- [Programming Conditions](#)
- [Probing PCM Cells](#)
- [Printing States of PCM Cells](#)

PCM Model

PCM is a type of non-volatile memory with data storage based on reversible changes of resistivity. It offers random access, quick read and write times, high endurance, and ease of integration with standard CMOS processes. The PrimeSim XA tool supports a built-in model of PCM cells, which offer better ease-of-use and performance compared to more traditional approaches, such as Verilog-A and macro modeling in full-chip simulation.

PCM is made from a special alloy, which exhibits reversible transformation between a crystalline, highly conductive state, called SET, and an amorphous, highly resistive state, called RESET. The resistance difference between these two states can be as large as two order of magnitude, allowing binary information to be associated with the states (phases) of the material. The phase transition can be electrically induced by applying current pulses with sufficient amplitude and duration to melt the material. If the falling edge of pulse is short, the material is cooled quickly resulting in the high resistance RESET state. If the falling edge is long enough for gradual cooling, the material is brought to the low resistance SET state. If conditions for neither RESET or SET states are met, the material is in one of the intermediate states, allowing for multi-level storage capability.

In the PrimeSim XA tool, the built-in PCM cell is implemented as a nonlinear resistor model from ST with level=10 and associated parameters to allow for customization of the PCM cell. This model has the state parameter to keep track of the PCM cell state, which

can change during simulation after a programming cycle. The built-in model parameters are shown in [Table 50](#).

Table 50 *PCM Model Parameters and Default Values*

Parameter	Default Value	Description
Rs	7K	SET resistance
ISth	500uA	Current level of the SET pulse
TSth	50ns	Minimum duration of the SET pulse
RR	200K	RESET resistance
IRth	1000uA	Current level of the RESET pulse
TRth	8ns	Minimum duration of the RESET pulse
TmaxSL	10ns	Maximum duration of the falling edge of the RESET pulse (*)
TminSL	100n	Minimum duration of the falling edge of the SET pulse (*)
RI	50K	INTERMEDIATE resistance
RM	500	MELTING resistance
VH	0.45	Holding voltage
Vth	0.8	Threshold voltage for electronic switching
State	1	State of PCM cell

Model Customization

A built-in model can be customized using the following syntax.

```
.MODEL R level=10 [param=value]
```

For example:

```
.MODEL pcm1 R level=10 Rs=5k Rr=500K
```

Instantiation of PCM Cells

You can instantiate a PCM cell with two methods using:

- An instance parameter. For example:

```
Rxxx n1 n2 pcm_model_name state=value
```

For example:

```
Rc n1 n2 pcm_md1 state=1
```

- The `set_parameter_value` command to initialize many PCM cells. This command overrides instance parameter state set in the netlist. For example:

```
set_parameter_value -name state -value 0 -model pcmmod -inst x1.x2.*
```

This command initializes the states of all PCM cells inside `x1.x2` to 0.

Programming Conditions

A PCM cell may change its phase and resistance when a programming current pulse flows through it.

RESET Conditions

The state of the PCM cell is reset to 0, where it has the high resistance RR , after the following conditions of the current pulse are satisfied:

- $I > IR_{th}$: Level of current pulse must be larger than IR_{th}
- $T > TR_{th}$: Duration of current pulse must be larger than TR_{th}
- $TSL < T_{maxSL}$: Duration of falling edge of current pulse must be less than T_{maxSL}
- Current falls below IM

SET Conditions

The state of the PCM cell is set to 1, where it has the high resistance Rs , after the following conditions of the current pulse are satisfied:

- $I > IS_{th}$: Level of current pulse must be larger than IS_{th}
- $T > TS_{th}$: Duration of current pulse must be larger than TS_{th}

- $TSL < T_{minSL}$: Duration of falling edge of current pulse must be less than T_{minSL}
 - Current falls below I_M
-

INTERMEDIATE Conditions

The state of the PCM cell is set to -1, where it has the intermediate resistance R_I , after the following conditions of the current pulse are satisfied:

- Not all conditions are met for SET state
 - Not all conditions are met for RESET state
 - Current falls below I_M
-

MELTING Conditions

The state of the PCM cell is set to 2, where it has the melting resistance R_M , if following condition is met:

- Current is above I_M
-

Probing PCM Cells

The state and resistance of PCM cells can be probed and output to an FSDB waveform file by the following two methods:

- [Probing Using the .probe Statement](#)
 - [Using the probe_waveform_pcm Command](#)
-

Probing Using the .probe Statement

- To probe the states of PCM cells:

```
.probe lvs(inst_name)
```

Where you can use a wildcard in *inst_name*. For example:

```
.probe lvs(x1.x2.*)
```

- To probe the resistances of PCM cells:

```
.probe lvs(inst_name)
```

Where you can use a wildcard in *inst_name*. For example:

```
.probe lvr(x1.x2.*)
```

Using the `probe_waveform_pcm` Command

You can use the `probe_waveform_pcm` command to probe the states and resistances of selected PCM cells.

Printing States of PCM Cells

You can use the `print_pcm_state` command to print the states of PCM cells at a given time to an ASCII file for inspection, or to save for initialization in a later simulation.

Printing States of PCM Cells in Interactive Mode

In interactive mode, you can use the `iprint_pcm_state` command can be used to print the states of PCM cells.

16

Three-Dimensional Integrated Circuit (3DIC) Modularization

This chapter describes the PrimeSim XA solution to modularize IC chips inside a 3DIC design to be simulated as a single circuit.

This chapter contains the following topics:

- [Overview of 3DIC Simulation Netlist](#)
- [3DIC Netlist Construct and Usage](#)
- [Module-Based PrimeSim XA Commands](#)
- [3DIC Back-Annotation Support](#)
- [3DIC Support for TMI and Custom CMI Models](#)
- [External Sampling for 3DIC Netlists](#)

Overview of 3DIC Simulation Netlist

A three-dimensional integrated circuit is a single chip that integrates two or more layers of active electronic components into a single circuit. All components on the layers communicate use on-chip signaling, either vertically or horizontally. The layers in a 3DIC are IC designs that might be fabricated using different technology processes and operate at different sets of parameters, including temperature and geometry scaling.

The PrimeSim XA solution modularizes these IC designs and enables the simulation of a 3DIC as a single circuit. The PrimeSim XA tool supports PrimeSim HSPICE syntax for 3DIC netlists, as described in the Multi-Technology Simulation of 3D Integrated Circuit section in *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

3DIC Netlist Construct and Usage

The 3DIC netlist construct and usage, and the transient analysis and `.ALTER` simulation feature in the PrimeSim XA tool follow the same rules as in the PrimeSim HSPICE tool.

For more details, see the Multi-Technology Simulation of 3D Integrated Circuit section in *PrimeSim Continuum Overview*.

The current 3DIC solution also supports the Monte Carlo analysis flow.

This section contains the following topics:

- [Instantiating Hierarchical Modules](#)
- [Full Circuit Example](#)

Instantiating Hierarchical Modules

The hierarchical configuration in 3DIC netlists supports hierarchical module instantiation, module scope specification, and wildcard matching for PrimeSim XA commands and PrimeSim HSPICE probing syntax. You can instantiate the top-level subcircuit of a module inside the subcircuits of another module. Such a situation could arise in the vertical 3D integration of integrated circuits where an integrated circuit is used as a block in another integrated circuit with a different technology.

In the following example, a CMOS image sensor is integrated into a camera control chip that is manufactured in a different technology.

```
*Top-level netlist
xcam ... cammod::camtop

...
*Camera control modules
.module cammod
.subckt camtop ...
xcis ... cismod::cistop

...
.ends
.endmodule cammod

*CMOS image sensor module
.module cismod
.subckt cistop ...
...
.ends
.endmodule cismod
```

Full Circuit Example

The following full circuit example includes these characteristics:

- This 3DIC consists of multiple single dies with the same design and technology node.
- This 3DIC has die-based simulation corners or circuit properties.

The example shows how you can use a single IC module netlist for a single IC set of memory simulation properties from the file `memory.lib`.

```
.lib TT
    .param ... * parameters for the simulation corner TT.
    ...
.endl TT
.lib FF
    .param ... * parameters for the simulation corner FF
    ...
.endl FF
.lib SS
    .param ... * parameters for the simulation corner SS
    ...
.endl SS
.lib models
    .models ... * model cards for the memory IC.
    .subckt nch_mac ... * macro-models for the memory IC.
    ...
    .ends nch_mac
.endl models
```

The netlist then draws on single IC memory circuit definitions from the file `memory.sp`:

```
* Top level circuit from single memory IC module
.subckt 1G_mem_top ...
    .temp 100
    xbank1 ... bank
    ...
.ends 1G_mem_top
* Other subcircuit definitions.
.subckt bank ...
    ...
.ends bank
```

The 3DIC Memory netlist draws from the `3D_mem.sp` file:

```
* global control and parameters for 3D IC simulation.
.temp -40
* 1st IC memory module (fast corner)
xmem1 .... 1GMem::1G_mem_top
* 2nd IC memory module (slow corner)
xmem2 .... 1GMem::1G_mem_top
* 3rd IC memory module (typical - default)
xmem3 .... 1GMem::1G_mem_top
* 4th IC memory module
xmem4 .... 1G_mem_top
    * top level control logic block.
x5 .... memory_control
.subckt memory_control
    ...
.ends memory_control
* Netlist definitions from the original single IC circuit.
```

```
.include "memory.sp"

.module lGMem                                * Default control and parameters
    .lib "memory.lib" TT
    * Default single IC memory properties
    .temp 25
    ...
    * Models for the circuit elaborations in the memory circuit.
    .lib "memory.lib" models
    * Netlist definitions from the original single IC circuit.
    .include "memory.sp  "
.endmodule lGMem
```

Module-Based PrimeSim XA Commands

In a regular netlist, the PrimeSim XA tool supports many local commands at the subcircuit level. With the introduction of modules in a 3DIC netlist, the PrimeSim XA tool extends the support of selected local commands to the module level. See [Table 51](#) for the list of commands with 3DIC module support.

Table 51 *PrimeSim XA Commands with 3DIC Module Support*

```
check_node_excess_rf
check_node_quick_rf
check_node_zstate
force_node_voltage
probe_waveform_current
probe_waveform_logic
probe_waveform_va
probe_waveform_voltage
release_node_voltage
report_operating_point
report_power
set_va_view
set_ccap_level
```


Table 51 *PrimeSim XA Commands with 3DIC Module Support
(Continued)*

```

set_duplicate_rule
set_flash_option
set_model_level
set_model_option
set_oscillator
set_partition_option
set_sim_level
set_synchronization_level
set_synchronization_option
set_tolerance_level
set_tolerance_option
skip_circuit_block

```

These commands can be set for a subcircuit within a module using the `-subckt` argument as follows:

```
command_name -subckt module_name:subckt_name
```

Examples

Following is an example of the top-level netlist, named `top_3d.sp`.

```

*** Top level netlist with inverter chains inside two 3DIC modules
.global vdd gnd
.module mod1
.inc './defcir.sp'
.endmodule
.module mod2
.inc './defcir.sp'
.endmodule
v1 vdd gnd 1.8
vg n0 gnd pulse 0 1.8 5n 1n 1n 20n 40n
xc1 n0 n1 mod1::chain
xc2 n0 n2 mod2::chain
.tran 1n 100n
.print v(*)

```

```
.lprint v(*)  
.end
```

Following is an example of the netlist file that describes circuit definition of invert chain, named defcir.sp.

```
*** Circuit definition of inverter chain  
.subckt inv in out  
mp out in vdd vdd pch l=1u w=3u  
mn out in gnd gnd nch l=1u w=2u  
.ends  
.subckt chain in out  
x1 in a inv  
x2 a b inv  
x3 b out inv  
.ends
```

Following is an example of configuration file, named xa.cfg.

```
# PrimeSim XA configuration file  
set_model_level 3 -subckt mod1::chain  
set_model_level 6 -subckt mod2::chain
```

In this example, the top level netlist has two inverter chains inside a couple of 3DIC modules. The commands in the configuration file set the model level to 3 for the first module, and to 6 for the second module. So the inverter chain in the second module is simulated with better accuracy.

3DIC Back-Annotation Support

The PrimeSim XA tool supports the following DSPF back-annotation flows for a 3DIC netlist:

- [Flat IC Module DSPFs With a Separate DSPF for the Silicon Interposer](#)
- [Full 3DIC Flat Extracted DSPF](#)

Flat IC Module DSPFs With a Separate DSPF for the Silicon Interposer

This flow lets you reuse the extracted DSPFs of individual IC modules. For 3DIC integration, IC modules are extracted and verified individually before assembly using a

silicon interposer that is extracted separately. The DSPF back-annotation steps in this flow are:

- Extract each IC module with a separate DSPF file. Each DSPF file must contain the IC module top subcircuit definition.
- Extract the silicon interposer layer as a flat DSPF file.
- Set the DSPF file reference for the corresponding module in the following syntax:

```
load_ba_file -file filename [-icmodule module_name]
```

Following is an example of a top netlist named top.sp.

```
*simulation setup
...
*silicon interposer netlist
X1 ... IC_1::top1
X2 ... IC_2::top2
...
*IC modules
.module IC_1
.subckt top1 ...
...
.ends
.endmodule IC_1
.module IC_2
.subckt top2 ...
...
.ends
.endmodule IC_2
DSPF file for interposer - silicon_interposer.dspf:
*Extracted RC nets
*|NET ...
...
```

Following is an example of the DSPF file for the module “IC_1”, named IC_1_flat.dspf.

```
.subckt top1
*Extracted RC nets
*|NET ...
...
*INSTANCE
M1... w=0.3u l=0.2u ...
...
.ends
DSPF file for module "IC_2" - IC_2_flat.dpsf:
.subckt top2
*Extracted RC nets
*|NET ...
...
*INSTANCE
M1... w=0.3u l=0.2u ...
```

```
...  
.ends
```

Following is an example of the Tcl command file, named xa.cfg.

```
load_ba_file -file silicon_interposer.dspf  
load_ba_file -file IC_1_flat.dspf -icmodule IC_1  
load_ba_file -file IC_2_flat.dspf -icmodule IC_2
```

DSPF Back-Annotation

The DSPF back-annotation supports the following different scenarios:

- DSPF File Has Matching Subcircuits

If the flat DSPF file has one or more subcircuits that match the IC module top-level subcircuits, the DSPF file is back-annotated to those subcircuits.

- DSPF File Has No Subcircuit

If the flat DSPF file does not have a subcircuit definition, the PrimeSim XA tool automatically detects it and back-annotates the DSPF file as follows:

- Get the module name from the `load_ba_file` command.
- Find the instances in the netlist with a matching module name.
- Back-annotate the DSPF file to the corresponding subcircuit of these instances.

This automatic detection works only if the module has only one subcircuit referenced at the top level of the netlist. If more than one subcircuit of the module is referenced, the PrimeSim XA tool does not back-annotate the DSPF file and issues a warning.

- DSPF File Without Matching Subcircuit

If the flat DSPF file has one or more subcircuits but none of them matches the IC module top-level subcircuit, the PrimeSim XA tool does not back-annotate the DSPF file and issues a warning on the unmatched subcircuit.

- Netlist Without Matching Module

If the module specified in the `load_ba_file` command has no matching module in the netlist, the PrimeSim XA tool does not back-annotate the DSPF file and issues a warning on the unmatched module.

Full 3DIC Flat Extracted DSPF

The DSPF back-annotation flow for a full 3DIC netlist is the same as the one for the flat IC module, except that you must provide the fully extracted flat DSPF files.

Following is an example of the top-level netlist, named top.sp.

```
*simulation setup
...
*silicon interposer netlist
X1 ... IC_1::top
...
*IC modules
.module IC_1
.subckt top
...
.ends
.endmodule IC_1
...
DPF file - full_MTS_flat.dpsf:
*Extracted RC nets
*|NET ...
...
*INSTANCE
MX1.M1... w=0.3u l=0.2u ...
MX1.M2... w=0.3u l=0.2u ...
...
```

Following is an example of the Tcl command file, named xa.cfg.

```
load_ba_file -file full_MTS_flat.dspf
```

3DIC Support for TMI and Custom CMI Models

The PrimeSim XA tools supports TMI (TSMC Model Interface) and Custom CMI (Common Model Interface) models in 3DIC netlists. Multiple technology TMI or Custom CMI model libraries can be packaged to the same netlist through `.module`. You can set the `tmiPath` and `cmiPath` options inside `.module`, and use them to specify the respective paths to `TMI.so` and `CMI.so`.

For more information, see the 3DIC Support for TMI, Custom CMI, and MOSRA section in the *PrimeSim Continuum Overview*.

External Sampling for 3DIC Netlists

The PrimeSim XA tool supports the external sampling method for 3DIC designs. The PrimeSim XA tool generates different `.mc0` files for the corresponding 3DIC hierarchy in the netlist: top-level instance, module-based instances and module-based instances with specified seeds. The generated `.mc0` files are named *prefix.mc0*, where *prefix* is the given output prefix in the command line option `-o`. The relationship of these `.mc0` files is defined in the names of the `.mc0` files.

When restoring parameters for 3DIC with external sampling, ensure

- The relationship of the .mc0 files is correctly maintained.
- The prefix of the generated .mc0 files is consistent.
- All the file extensions are .mc0.

Otherwise, the tool cannot reconstruct the hierarchy for external sampling and an error message is issued.

Example

The following netlist describes a 3DIC design:

```
xins1 ... tmod1::top
xins21 ... tmod2::top
xins22 ... tmod2::top
xins23 ... tmod2::top

.mts_seed tmod2::xins23=5
.module tmod1
.subckt top
...
.ends
.endmodule

.module tmod2
.subckt top
.option seed=2
...
.ends
.endmodule
```

PrimeSim XA generates the following four .mc0 files for the above 3DIC netlist:

- *output.mc0*: The .mc0 file for the top-level
- *output.tmod1.mc0*: The .mc0 file for the tmod1 module
- *output.tmod2.mc0*: The .mc0 file for the tmod2 module
- *output.xins23.tmod2.mc0*: The .mc0 file for the xins23 instance in the tmod2 module when the instance seed (5) is specified.

For external sampling, list these 3DIC .mc0 files in the statement of `option Sampling_Method`:

```
.option Sampling_Method=External External_File="output.mc0, \
output.tmod1.mc0, output.tmod2.mc0, output.xins23.tmod2.mc0"
```

Note that when reading a group of .mc0 files,

- If a listed file is missing, an error message will be issued.
- If a file name does not match any 3DIC module or module instance in the netlist, the content of that file will be ignored.
- If an existing module does not have its corresponding module-based .mc0 file listed in the statement of `.option Sampling_Method`, a warning message will be issued. The value of IRV will be set to random or zero based on the `.option Set_Missing_Values`.

For more information about the external sampling method, see the Sampling Options section in *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

17

PrimeSim MOSRA Support in PrimeSim XA

This chapter describes the PrimeSim MOSFET reliability (MOSRA) support in PrimeSim XA.

This chapter contains the following topics:

- [Supported PrimeSim MOSRA Features](#)
- [Unsupported PrimeSim MOSRA Features](#)
- [Running PrimeSim MOSRA with Monte Carlo Analysis](#)
- [Supported Eldo Features in MOSRA Analysis](#)

Supported PrimeSim MOSRA Features

The PrimeSim XA tool supports all features as documented in *PrimeSim™ MOSRA User Guide*, but only for:

- MOSRA models: PrimeSim HSPICE Built-in HCI and BTI level 1, 3 models, MOSRA API models
- Core models: BSIM3v3, BSIM4, PSP, BSIM-CMG, BSIM-IMG, UTSOI, HiSIMHV, HiSIM2, BSIMBULK, EKV3.0, Custom CMI models, MOS Varactor
- The PrimeSim MOSRA feature in both equation and table simulation mode

MOSRA Gradual Aging Flow

The PrimeSim XA tool supports the MOSRA gradual aging flow to dynamically detect if gradual aging is required during simulation and if the tool needs to exit early when the circuit is not sensitive to gradual aging effects or when gradual aging simulation does not provide additional benefits. This feature applies to both `simmode=2` and `simmode=3`.

To keep or skip unnecessary post-age simulation steps, run the following command:

```
.MOSRA postAgeSim = 0 | 1
```

where

- 0 (default): Enables the `simmode=3` enhanced flow and skips the redundant simulation steps
- 1: Enables the `simmode=3` full flow and runs all aging simulations in each `.alter` command

To control whether the gradual aging flow needs to detect early-exit in the aging model, run the following command:

```
.MOSRA earlyExitEnabled = 0 (default) | 1
```

where

- 0: Do not detect early exit in the aging model
- 1: Detect early exit in the aging model

Unsupported PrimeSim MOSRA Features

The PrimeSim XA tool does not support the following PrimeSim MOSRA command arguments:

- AGINGPERIOD
- AGINGWIDTH
- FREQUENCY
- INTEGMOD
- XPOLATEMOD
- TSAMPLE1
- TSAMPLE2

The PrimeSim XA tool does not support the following `.OPTION` controls:

- MRAPAGED

The PrimeSim XA tool does not support the following MOSRA commands:

- `.MOSRAPRINT`
- `.MOSRA_SUBCKT_PIN_VOLT`

Running PrimeSim MOSRA with Monte Carlo Analysis

You can run MOSRA analysis with Monte Carlo analysis. The existing MOSRA and Monte Carlo commands are the same and PrimeSim HSPICE datamining is supported, but with the following limitations:

- The Monte Carlo Variation Block feature is not supported, and only traditional Monte Carlo analysis is supported

The steps to run MOSRA with Monte Carlo analysis are:

1. [Running MOSRA in a Fresh Simulation](#)
2. [Running MOSRA in a Post-Stress Simulation](#)

Running MOSRA in a Fresh Simulation

To run a fresh MOSRA simulation with Monte Carlo analysis, do the following steps.

1. Before you run the fresh simulation and generate the .radeg file, include the .mosra `simmode=0` netlist option. For example:

```
*Netlist content
.mosra reltotaltime =1e8
+ simmode=0
.tran step stop sweep monte=10
circuit description
models with aging model card
```

2. Run the fresh simulation.

```
xa netlist -o res/out
```

The fresh simulation generates the following output files:

- The out.meas file is the Monte Carlo measurement file.
- The out.mc file is the Monte Carlo statistic file (in PrimeSim XA format).
- There is one .radeg file per Monte Carlo sample, which means there are 10 .radeg file in this example.
- When the PrimeSim HSPICE tool is loaded with datamining enabled, the usual PrimeSim HSPICE datamining files are generated: .mt, .mc0, .mpp0, .qqt0.csv, and so on.

Running MOSRA in a Post-Stress Simulation

To run a post-stress MOSRA simulation with Monte Carlo analysis, run the following steps.

1. Before you run the post-stress simulation, include the `.mosra simmode=1` and `.option radeqfile netlist` options. For example:

```
*Netlist content
.mosra reltotaltime =1e8
+ simmode=1
.option radeqfile = 'res/out.m*.radeq'
.tran step stop sweep monte=10
circuit description
models with aging model card
```

2. Run the post-stress simulation.

```
xa netlist -o res/out
```

For each Monte Carlo run with the same index, the random variable value used for `simmode=1` is the same as for `simmode=0`.

The post-stress simulation generates the following output files:

- For each age time (1e8 in this example):
 - One measure file per Monte Carlo sample
 - A merged `.meas` file with all measurements
 - A merged `.mc` file with all statistics in PrimeSim XA format
- When the PrimeSim HSPICE tool is loaded with datamining enabled, the PrimeSim HSPICE datamining files are generated per age time.

A

Spectre Netlist Compatibility

The PrimeSim XA tool is compatible with most of the documented Spectre syntaxes and features. This chapter describes the supported features in Spectre netlist format.

This appendix contains the following topics:

- [Running PrimeSim XA With Spectre Netlist](#)
- [Supported Spectre Simulator Elements](#)
- [Supported Spectre Simulator Device Models](#)
- [Supported Spectre Simulator Statements](#)
- [Supported Spectre Simulator Functions](#)
- [Supported Spectre Simulator Option Statement Parameters](#)
- [Supported Spectre Simulator Tran Statement Parameters](#)
- [Running Monte Carlo Analysis With Spectre Netlists](#)
- [Using Verilog-A With the Spectre Netlists in the PrimeSim XA Tool](#)
- [Using Spectre Syntax in VCD Files](#)
- [Using SPECTRE Syntax in 3DIC Netlists](#)
- [Specifying Initial Conditions](#)

Running PrimeSim XA With Spectre Netlist

To run the PrimeSim XA tool with Spectre netlists, run the following command at the UNIX command prompt:

```
$ xa -spectre netlist_file [command_line_options]
```

See the [Running the Simulator](#) section for details about the command line options.

Supported Spectre Simulator Elements

The syntax for each element is compatible with the Spectre simulator. [Table 52](#) lists the supported Spectre elements in the PrimeSim XA tool.

Table 52 *Supported Spectre Simulator Elements*

Instance Keyword	Element
<code>bsource</code>	Behavioral Source
<code>capacitor</code>	Two Terminal Capacitor
<code>cccs</code>	Linear Current Controlled Current Source
<code>ccvs</code>	Linear Current Controlled Voltage Source
<code>inductor</code>	Two Terminal Inductor
<code>iprobe</code>	Current Probe
<code>mutual_inductor</code>	Mutual Inductor
<code>nport</code>	Linear N Port
<code>paramtest</code>	Parameter Value Tester
<code>pcccs</code>	Polynomial Current Controlled Current Source
<code>pccvs</code>	Polynomial Current Controlled Voltage Source
<code>phy_res</code>	Physical Resistor
<code>pvccs</code>	Polynomial Voltage Controlled Current Source
<code>pvcvs</code>	Polynomial Voltage Controlled Voltage Source
<code>rdiff</code>	Diffusion Resistor Model
<code>resistor</code>	Two Terminal Resistor
<code>switch</code>	Ideal Switch
<code>transformer</code>	Linear Two Winding Ideal Transformer
<code>vccs</code>	Linear Voltage Controlled Current Source
<code>vcvs</code>	Linear Voltage Controlled Voltage Source
<code>vsource</code>	Independent Voltage Source
<code>isource</code>	Independent Current Source

Supported Spectre Simulator Device Models

The PrimeSim XA tool supports the following device models when running :

- Diode Device Model
- BJT Models
- MOSFET Device Models
- JFET Device Models
- Resistor Models

[Table 53](#) lists the supported Spectre simulator diode models in the PrimeSim XA tool.

Table 53 *Supported Spectre Diode Simulator Device Models*

Instance Keyword	Model Description
diode	Diode Level-1 and Level-2 Models
diode_cmc	DIODE_CMC Model
dio500	Diode Level-500 Model
juncap200	JUNCAP2 Model, JUNCAP2 Express Model

[Table 54](#) lists the supported Spectre simulator BJT models in the PrimeSim XA tool.

Table 54 *Supported Spectre Simulator BJT Device Models*

Instance Keyword	Model Description
bjt	BJT Model
bht	HICUM L2 Model
bht0	HICUM Level-0 Model
bjt503	MEXTRAM 503 Model
bjt504	MEXTRAM 504 Model
bjt505	MEXTRAM 505 Model
vbic	VBIC Model

[Table 55](#) lists the supported Spectre simulator MOSFET models in the PrimeSim XA tool.

Table 55 *Supported Spectre Simulator MOSFET Device Models*

Instance Keyword	Model Description
bsim3v3	BSIM3v3 Model
bsim4	BSIM4 Model
bsimsoi	BSIMSOI4 Model
bsimsoi_s	BSIMSOI100 Model
bsimcmg	BSIM-CMG Model
bsimimg	BSIM-IMG Model
bsim6	BSIM6 Model
bsimbulk	BSIM-BULK Model
ekv	EKV Model
ekv3, ekv3_rf	EKV3 Model
mos1	MOS Level-1 Model
mos903	MOS Model 9, Level 903
mos1102	MOS Model 11, Level 1102
mos2002	MOS Model 20, Level 2002
psp102	PSP102 Model
psp103	PSP103 Model
psp104	PSP104 Model
psitft	TFT Polysilicon Model
mosvar	CMC MOS Varactor Model
utsoi	UTSOI1 Model
utsoi2	UTSOI2 Model
hisim_hv	HiSIM_HV Model
hisim2	HISIM2 Model

[Table 56](#) lists the supported Spectre simulator JFET models in the PrimeSim XA tool.

Table 56 *Supported Spectre Simulator JFET Device Models*

Instance Keyword	Description
jfet	JFET Level 1, 3 Model

[Table 57](#) lists the supported Spectre resistor models.

Table 57 *Supported Spectre Simulator Resistor Models*

Instance Keyword	Description
r2	CMC R2 Model
r3	CMC R3 Model
rdiff	Diffusion Resistor Model

Supported Spectre Simulator Statements

Following are the Spectre simulator statements supported in PrimeSim XA.

- analogmodel
- global
- include
- ic
- if
- library
- nodeset
- options
- parameters
- save
- simulator
- subckt
- tran

See [Supported Spectre Simulator Tran Statement Parameters](#).

- veriloga

Supported Spectre Simulator Functions

The PrimeSim XA tools supports the Spectre simulator functions shown in [Table 58](#). You can use these function in the Spectre tool for the calculation of parameters, such as device parameter, model parameters and so on.

Table 58 *PrimeSim XA Supported Spectre Simulator Functions*

ABS()	ACOS()
ASIN()	ATAN()
COS()	COSH()
FLOOR()	INT()
LOG()	LOG10()
MAX()	MIN()
POW()	SIN()
SINH()	SORT()
TAN()	TANH()

The PrimeSim XA tool supports user-defined functions in Spectre netlist format. For details about user-defined functions in Spectre format, see the related Spectre documents.

Supported Spectre Simulator Option Statement Parameters

Following are the Spectre option parameters supported in PrimeSim XA. If an option parameter is not supported, a warning message is issued in the log file.

- currents
- gmin
- nestlvl
- redundant_currents

Appendix A: Spectre Netlist Compatibility

Supported Spectre Simulator Tran Statement Parameters

- `rforce`
- `save`
- `scale`
- `scalm`
- `subcktprobelev`
- `temp`
- `title`
- `tnon`
- `useterms`

Supported Spectre Simulator Tran Statement Parameters

The PrimeSim XA tool supports the following Spectre `tran` parameters that have impact on the simulation results. If a `tran` parameter is not supported, a warning message is issued in the log file.

- `autostop`
- `cmin`
- `ic`
- `nestlvl`
- `oppoint`
- `readic`
- `readns`
- `save`
- `skipdc`
- `stop`
- `write`
- `writefinal`

Running Monte Carlo Analysis With Spectre Netlists

When running Monte Carlo analysis with Spectre netlists in PrimeSim XA, specify the measurements in SPICE netlist section starting with `simulator lang=spice`. For example:

```
simulator lang=spice
.meas tran value_vg_n18_0 find v(vg_n18_0) at=10u
.meas tran value_vg_p18_0 find v(vg_p18_0) at=10u
simulator lang=spectre
```

This section includes information on the commands and features that are supported during Monte Carlo simulation with Spectre netlists, as described in the following topics:

- [Enabling Monte Carlo Simulation With Spectre Netlists](#)
- [Specifying Statistical Variation](#)
- [Specifying Distribution](#)
- [Using .ALTER Statements in Spectre Monte Carlo Simulation](#)

Enabling Monte Carlo Simulation With Spectre Netlists

In Spectre syntax, Monte Carlo analysis is enabled with the `montecarlo` command in the netlist:

```
name montecarlo parameter=value...
```

You specify this analysis in the `montecarlo` line. Note that only transient analysis is supported. For example:

```
mcl montecarlo variations=process seed=1 numruns=5
    {tran1 tran start=0 stop=100u
    }
```

Note:

PrimeSim HSPICE datamining is not supported. The Monte Carlo output data conform to the PrimeSim XA standards.

Supported Parameters

[Table 59](#) lists the supported parameters of the Spectre `montecarlo` command in the PrimeSim XA tool.

Table 59 Supported Parameters

Parameter	Description
donominal	Whether to run nominal run. Can be yes or no. Default is yes.
dut	Variation applied to specified subcircuit instances.
firststrun	Start iteration number. Default=1.
ignore	No variation applied to specified subcircuit instances.
numruns	Number of Monte Carlo iterations (does not include the nominal run). Default=100.
sampling	Level of statistical sampling to apply. Can be: Standard (default), LHS, Orthogonal, and LDS. PrimeSim XA provides standard support for this parameter.
seed	Starting seed (no default).
variations	Level of statistical variation to apply. Can be: Process (default), Mismatch, and All.

Specifying Statistical Variation

In Spectre syntax, the statistical variation is defined in a `statistics` block. The global variations are specified within the process block and the local variation in the `mismatch` block. For example:

```
statistics {
    process { //process generates random number once per MC run
        // process variation specifications
        ...
    }
    mismatch { //mismatch generates a random number per instance
        //mismatch variation specifications
        ...
    }
}
```

The `statistics` block is supported in the PrimeSim XA tool, as well as both `process` and `mismatch`.

Specifying Distribution

The distribution is specified inside the `process{}` and `mismatch{}` blocks using the `vary` syntax:

```
vary param_name dist=<type> {std=value | N =value} {percent=yes|no}
```

The `vary` command and its parameters are supported in the PrimeSim XA tool. However not all types of the parameters are supported. See [Table 60](#).

Table 60 Supported Spectre Parameters

Vary param_name	Description	PrimeSim XA Support
dist	Distribution type. Can be: Gaussian, Uniform, and lognormal.	Gaussian: Yes Uniform: Yes lognormal: No
N	Used for the uniform distribution	Yes
percent	Can be yes or no.	Yes
std	Used for Gaussian distribution.	Yes

Using .ALTER Statements in Spectre Monte Carlo Simulation

The `.ALTER` commands are supported in Monte Carlo simulation in the PrimeSim XA tool. When you have sections for corners in the netlist and each of the corners has statistical mismatch parameters, use the `.ALTER` statements to rerun simulations using different parameters and data.

When using the `.ALTER` command in Monte Carlo simulation, all Monte Carlo features are supported.

- Datamining Analysis

Running data mining analysis for each `.ALTER` block is independent; related reports are generated separately for each `.ALTER` block. You must separate the data mining process and report by `.ALTER` index. Otherwise, a warning message will be issued.

- Sampling Method

Each `.ALTER` block can define its own sampling method. If there is no sampling method setting for the current `.ALTER` block, PrimeSim XA picks the previous sampling method setting for the current `.ALTER` block. External sampling is defined through `Option External_File=filename`. If external sampling is triggered and there is no external file path given in the current `.ALTER` block, the simulator picks the previous external file path for the current `.ALTER` block.

- Variation Block

Variation block that is defined outside the `.ALTER` block is supported. However, having accumulative definition, redefinition or overriding a variation block through the `.ALTER` block is not allowed. An error message will be issued if a variation block is defined inside the `.ALTER` block.

- Distributed Processing (DP)

When running one or multiple specific `.ALTER` blocks, by default the simulator groups the tasks by ALTER index. Monte Carlo analysis for each `.ALTER` block is independent from each other.

If the machine count defined by the `-dp` option is less than the total task count, the simulator distributes tasks which carry the same ALTER index as many as possible.

For example:

```
-dp 100
.alter count: 5
Monte sweep: 100 for each alter block
```

The task distribution sequence is:

```
Alter#1 MC#1-#100
Alter#2 MC#1-#100
...
Alter#5 MC#1-#100
```

Monte Carlo analysis will be triggered independently after all simulation runs of a particular ALTER index are completed.

Use the `-dptask cfg_task` option to customize the task list without modifying the netlist. The task definition is line-based; each line will be resolved to single or multiple tasks.

For example:

```
$feature_tasks          // Setup DP task mode: feature_task/batch_tasks
AL= #1 MC=#6            // AL #1: Alter index 1; MC: Monte Carlo
  index 6.              // Defined a task with alter index1 and mc
                        // index 6.
AL=#(1, 3, 5) MC=#(10-49) // This line can be resolved to 120 tasks
  (3*(49-10+1)).        // The comma sign as delimiter for discrete
task index              // definition. Dash sign can be used to
define continuous       // arrange tasks. In this example, A total
of 121 tasks            // will be simulated.
```

- Spectre Netlist Compatibility

Using `.ALTER` statements in Monte Carlo with Spectre netlists is supported. See [Using .ALTER Statements in Spectre Monte Carlo Simulation](#) for details.

Output Files

All the output files have the suffix `.a#`, along with usual Monte Carlo extensions. For example:

- `xa.a0.log`
- `xa.a0.fsdb.grp`
- `xa.a0.meas`
- `xa.a0.mc`
- `xa.a0.mc0.gz`
- `xa.a0.datamining.mpp0`
- `xa.a0.datamining.qqt0.csv`
- ...
- `xa.a1.log`
- `xa.a1.fsdb.grp`
- `xa.a1.meas`
- `xa.a1.mc`
- `xa.a1.mc0.gz`
- `xa.a1.datamining.mpp0`
- `xa.a1.datamining.qqt0.csv`

Example

Following is an example of a netlist which includes multiple `.ALTER` blocks for Monte Carlo simulation.

```
<test.sp>
four resistors

simulator lang=spectre
global 0
simulator lang=spice

.param bias=1m
.param globw=agauss(1u,0.1u,3)
```

Appendix A: Spectre Netlist Compatibility

Using Verilog-A With the Spectre Netlists in the PrimeSim XA Tool

```
.option sampling_method = default
.option use_agauss_format = yes
.param globwidth=globw

.param locwidth = agauss(1u,0.3u,4)

i1 0 1 bias
i2 0 2 bias
i3 0 3 bias
i4 0 4 bias
r1 1 0 resistor w=globwidth l=10u
r2 2 0 resistor w='globwidth + locwidth' l=10u
r3 3 0 resistor w=locwidth l=10u
r4 4 0 resistor w=globwidth l=10u
.model resistor R w=3u l=3u rsh=100
.op
.tr 10n 300n sweep monte=8

.probe v(1) v(2) v(3) v(4)
.measure tran v1 find v(1) at=200n
.measure tran v2 find v(2) at=200n
.measure tran v3 find v(3) at=200n
.measure tran v4 find v(4) at=200n

.alter "2nd"
.param locwidth = agauss(2u,0.3u,4)

.alter "3nd"
.param locwidth = agauss(3u,0.5u,4)

.end
```

Following is an example of configuring settings for Monte Carlo simulation with the `set_monte_carlo_option` command.

```
set_monte_carlo_option -enable 1 -sample_output all -parameter_file 1 \
    -mc0_file 1 -mode_ffmc 1 -dump_waveform 1
```

Using Verilog-A With the Spectre Netlists in the PrimeSim XA Tool

Verilog-A modules in Spectre netlists use the following conventions:

- Verilog-A modules are loaded into the simulator with the `ahdl_include` statement in the Spectre netlist file.
- Modules are instantiated in the same manner as Spectre subcircuits.

- Instance and model parameters can be modified in the same way as other Spectre instances.
- Spectre is case-sensitive. For example, “MyModule” is not equal to “mymodule” in Spectre or in Verilog-A.

Following is a simple Verilog-A example for Spectre:

```
ahdl_include "res.va"
V0 (net1 0) vsource dc=1.0
R0 (net1 net2) resistor r=10
I1 (net2 net3) res R=20
R1 (net3 0) resistor r=10
```

The `ahdl_include "res.va"` statement references the following module definition, used for the `I1` instance:

```
module res (a, b);
  electrical a, b;
  parameter real R = 1.0;
  analog begin
    V(a,b) <+R * I(a,b);
  end
endmodule
```

If both a Spectre subcircuit and a module definition exist with the same name, the subcircuit definition will be used by default. Use `set_va_view` to switch between definitions. See [Module- and Instance-Based Partitioning: Switching Between Verilog-A and SPICE Definitions](#) for more details.

Using Spectre Syntax in VCD Files

The PrimeSim XA tool supports VCD files in the Spectre syntax to read in vector information when you perform a transistor-level simulation for the inverter schematic and observe its transient behavior.

[Table 61](#) lists the supported features for Spectre VCD control and data files.

Table 61 Spectre VCD Control and Data File Support

Feature	VCD Control or Data File	Spectre Format	PrimeSim HSPICE Format
Command key character	Control file	.	#
Comment line symbol	Control file	* or \$	!

Table 61 Spectre VCD Control and Data File Support (Continued)

Feature	VCD Control or Data File	Spectre Format	PrimeSim HSPICE Format
Bus signal name format	Control and data file	<code>str[num:num]</code>	<code>str[[num:num]]</code>
Netlist hookup of VCD bus signals	Control file	<code>.alias *[*] *<*></code>	<code>.format %<#></code>
<code>.trise</code> and <code>.tfall</code> automatically be scaled by <code>\$timescale</code>	Control file	By default, time unit is set to fs. Example: <code>.trise 100</code> becomes <code>.trise 100fs</code> after parsing.	Time unit is required before parsing. Example: <code>.trise 100fs</code>
Auto-detect Spectre VCD file	Control file	Spectre VCD files included in the <code>lang=spice</code> section are accepted	N/A
First VCD data point	Data file	The first VCD data point is converted to <code>time=0</code> data point, regardless of what is in the VCD file	N/A
Auto-detect Spectre VCD format, even if embedded in <code>simulator=lang</code> section with <code>.vcd</code>	Control file	Default behavior	N/A

Supporting Sweeping Parameters During Transient Analysis

When reading in a vector file in Spectre format for transient simulation, PrimeSim XA converts time-dependent parameters to PWL voltage sources and behavioral devices through the design hierarchy.

The `start=tr_start=` command in `tran` statement is equivalent to the HSPICE `simstart=` command. Use the `start=tr_start=` command in `tran` statement to define time values for time-dependent behavior resistors. SUBCKT elements are also supported.

The first node name in the global statement that appears in the list is taken to be the name of the ground node. If no global statement is specified, by default the tool uses 0 as the name of the ground node.

Support of the Spectre Save Statement

The Spectre `save` statement is used to specify the values of specific nodes or signals that must be saved in the output file. The PrimeSim XA tool supports the `save` statement with the following features:

- Pattern matching characters (`?` and `*`)
- SUBCKT names quoted by the brackets symbols (`{}`)
- `[sub_inst]:currents` for isub models
- Mixed usage of `:currents` and `sigtype=all`
- The `subcktiprobes` parameter

Using SPECTRE Syntax in 3DIC Netlists

In the PrimeSim XA tool, 3DIC constructs support both PrimeSim HSPICE and SPECTRE syntaxes and features. When both PrimeSim HSPICE and SPECTRE statements exist in a 3DIC netlist, the 3DIC must be constructed by PrimeSim HSPICE 3DIC commands, such as `.module`, `.endmodule`, `.modulevar` and `.endmodulevar`. These PrimeSim HSPICE 3DIC commands must be explicitly specified in the PrimeSim HSPICE format when constructing a 3DIC in a SPECTRE netlist.

The temperature settings for a 3DIC module are effective either in PrimeSim HSPICE or SPECTRE language block, with the corresponding compliant temperature option:

- In the PrimeSim HSPICE language block, use the `.temp` command to set a temperature value. For example,

```
.temp 60
```
- In the SPECTRE language block, use the `options` statement to set a temperature value. For example,

```
option_name options temp=60
```
- If there is no temperature definition either in the PrimeSim HSPICE or SPECTRE language block, the temperature is referred to the setting in the top module.
- The `temper` variable is reserved in PrimeSim HSPICE and SPECTRE and cannot be modified in both of the PrimeSim HSPICE and SPECTRE simulators.

All PrimeSim HSPICE 3DIC-equivalent functionality, like `.subckt`, `.model` and `.param`, refer to the entities defined in the 3DIC module as

```
3DIC_label\:\:netlist_entity_name
```

To comply with the SPECTRE identifier naming convention, use escape characters with any non-alpha-numeric characters in the naming.

For more information about PrimeSim HSPICE 3DIC syntaxes and features, see the related topic in *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

In the *simulator lang=spice* block of a SPECTRE netlist, use the following commands to construct a 3DIC module:

- `.module 3DIC_label`
- `.endmodule 3DIC_label`
- `.modulevar 3DIC_label`
- `.endmodulevar 3DIC_label`

For a detailed description about PrimeSim HSPICE and SPECTRE netlist keywords, see *PrimeSim HSPICE User Guide: Basic Simulation and Analysis* and *Spectre Circuit Simulator Reference*.

Example 1

The following netlist uses both the SPECTRE and PrimeSim HSPICE 3DIC syntaxes to defines a 3DIC module. Instance instantiation is compliant to the PrimeSim HSPICE-equivalent syntax but requires to be fully compliant to the SPECTRE identifier naming rules.

test.scs:

```
// 3DIC cell instantiation
top_inst ... tmod\:\:topcell ...
// 3DIC scope definition with unique labeling
simulator lang=spice
.module tmod
// Native SPECTRE netlist circuit definitions
simulator lang=spectre
include "../cell.scs" section=top
// 3DIC construct block end
simulator lang=spice
.endmodule tmod
```

cell.scs:

```
subckt topcell 1 2 3 4
model nm bsim1 vfb0=-0.5 ...
m1 (1 2 3 4) nm l=5u w=10u ...
res1 (1 3) resistor r=100
ends topcell
```

Example 2

The `.temp` statement the default nominal temperature parameter in both the PrimeSim HSPICE and SPECTRE syntax format. The following example uses the `.temp` statement to obtain the temperature of the current module.

The `.option scale` statement in PrimeSim HSPICE format scales only the devices defined within the “tmod” 3DIC module.

The `options` statements in SPECTRE format are also applicable for module-based options, such as `temp`, `scale` and `tnom`, and are placed inside the SPECTRE language block. The values of `temp` and `tnom` are also accessible through parameters.

Netlist (test.scs):

```
// 3DIC cell instantiation
top_inst ... tmod\:\:topcell ...

// 3DIC scope definition with unique labeling
simulator lang=spice
.module tmod
.temp 60
.option scale=0.9

// Native SPECTRE netlist circuit definitions
simulator lang=spectre
tnomopt options tnom=25
subckt topcell 1 2 3 4
model nm bsim1 vfb0=-0.5 ...
m1 (1 2 3 4) nm l=5u w=10u ...
r1 (1 3) resistor r="100+temp-tnom"
ends topcell

// 3DIC construct block end
simulator lang=spice
.endmodule tmod
```

Specifying Initial Conditions

You can specify initial condition in multiple ways in Spectre format netlist. These rules are applied with running the PrimeSim XA tool with Spectre netlist format:

- You can set initial conditions for nodes with the `ic` and `nodeset` statements.
- You can set initial conditions for devices with a device parameter (`IC=value`) only for capacitors and inductors. The `IC=value` parameter is not supported for other devices.
- You can set initial conditions to be read in from a file specified with the `readic` or `readns` parameter in the `tran` statement.

- The DC solver can be skipped if the `skipdc=yes` parameter is specified in the `tran` statement.
- The `ic` parameter in the `tran` statement controls the interaction of various methods of setting the initial conditions. See [Table 62](#).

Table 62 *IC Parameter Settings*

IC Parameter Setting	Description
<code>ic=dc</code>	Initial conditions specifiers are ignored, and the existing DC solution is used.
<code>ic=node</code>	The <code>ic</code> statements are used, and the <code>ic</code> parameter settings on the capacitors and inductors are ignored.
<code>ic=dev</code>	The <code>ic</code> parameter settings on the capacitors and inductors are used, and the <code>ic</code> statements are ignored.
<code>ic=all</code>	Both the <code>ic</code> statements and the <code>ic</code> parameters are used. If specifications conflict, <code>ic</code> parameters override <code>ic</code> statements.

B

Eldo Netlist Compatibility

The PrimeSim XA tool is compatible with most of the documented Eldo netlist format syntaxes and features. This chapter describes the supported features. Behavior variations are also noted.

This appendix contains the following topics:

- [Running PrimeSim XA With Eldo Netlist Format](#)
- [Supported Eldo Simulator Commands](#)
- [Supported Eldo Simulator Built-In Functions](#)
- [Supported Eldo Simulator Options](#)
- [Supported Eldo Simulator Elements](#)
- [Supported Eldo Simulator Device Models](#)
- [Running Monte Carlo Analysis With Eldo Netlists](#)
- [Using Verilog-A With Eldo Netlists in the PrimeSim XA Tool](#)
- [Supported Eldo Features in MOSRA Analysis](#)
- [Using SPECTRE Syntax in 3DIC Netlists](#)
- [Eldo Syntax and Behavior Variations](#)

Running PrimeSim XA With Eldo Netlist Format

You run the PrimeSim XA tool with the following command at the UNIX command prompt:

```
$ xa -eldo netlist_file [command_line_options]
```

See the [Running the Simulator](#) section for details about the command line options.

Supported Eldo Simulator Commands

The PrimeSim XA tool supports the following Eldo simulator commands shown in [Table 63](#). Commands that have behavioral differences with the Eldo simulator are described in [Table 64](#).

Table 63 *Supported Eldo Commands*

<code>.ALTER</code>	<code>.INIT</code>
<code>.chrent</code>	<code>.LIB... .ENDL</code>
<code>.COMCHAR</code>	<code>.MSELECT</code>
<code>.CONNECT</code>	<code>.NODESET</code>
<code>.DATA</code>	<code>.OP</code>
<code>.DEFMAC</code>	<code>.OPTION</code>
<code>.DEFWAV</code>	<code>.PARAM</code>
<code>.DEL LIB</code>	<code>.SETBUS</code>
<code>.DSPF_INCLUDE</code>	<code>.SETSOA</code>
<code>.END</code>	<code>.SIGBUS</code>
<code>.GLOBAL</code>	<code>.SUBCKT... .ENDS</code>
<code>.HIER</code>	<code>.TEMP</code>
<code>.IC</code>	<code>.TITLE</code>
<code>.INC</code>	<code>.TRAN</code>

Table 64 *Differences in PrimeSim XA-Supported Eldo Commands*

Eldo Command	Description
<code>.CHECKSOA</code>	Performs a safe operating area check. The <code>tstart</code> , <code>tstop</code> , <code>tmin</code> , <code>twindow</code> , <code>file</code> , <code>runmsg</code> , <code>subckt</code> , and <code>inst</code> arguments are supported. There is a limitation for the <code>inst</code> and <code>subckt</code> options: the <code>.SETSOA</code> command must be in the subcircuit, and not in a <code>.LIB</code> command. And <code>inst</code> argument matches both device element and <code>subckt</code> instances.

Table 64 Differences in PrimeSim XA-Supported Eldo Commands (Continued)

Eldo Command	Description
.EXTRACT	Extracts waveform information by combining arithmetical expressions or predefined functions. See The .EXTRACT Command section for the supported features.
.PLOT or .PROBE	Defines a signal to be output in the binary output file. See the Probing Statements and Commands section. The PrimeSim XA tool supports the <code>vdss()</code> function, which accesses the <code>VDSAT</code> value. The <code>vdss()</code> function is only supported with <code>set_sim_level 7</code> .
.PRINT	Defines a signal to be output in ASCII format. The PrimeSim XA tool treats the <code>.PRINT</code> statement as a <code>.PROBE</code> statement by default, unless the <code>enable_print_statement</code> command is used. See the Using .PRINT Statements section.
.MEASURE	Defines a measurement. See Measuring Statements and Commands section.
.STEP	Performs several simulations while sweeping one circuit parameter. Only the <code>.STEP</code> , <code>TEMP</code> , and <code>.STEP PARAM</code> features are supported.
.MODEL	Defines a model, and also lets you rename a Verilog-A model.

Supported Eldo Simulator Built-In Functions

[Table 65](#) lists the Eldo simulator functions supported by the PrimeSim XA tool. You can use these functions in Eldo for the calculation of parameters, such as device parameter, model parameters, and so on.

Table 65 Supported Eldo Built-In Functions

ABS ()	MAX ()
ACOS ()	MIN ()
ASIN ()	POW ()
ATAN ()	PWR ()
CEIL ()	ROUND ()
COS ()	SGN ()

Table 65 Supported Eldo Built-In Functions (Continued)

COSH ()	SIN ()
DB ()	SMABS ()
EVECT ()	SORT ()
EXP ()	SIGN ()
FLOOR ()	SINH ()
INT ()	TANH ()
LOG ()	TAN ()
LIMIT ()	WINTeg ()
LOG10 ()	

Supported Eldo Simulator Options

Table 66 lists the options that the PrimeSim XA tool supports in Eldo format.

Table 66 Supported Eldo Options

ASPEC	DEFAD
DEFAS	DEFL
DEFNRD	DEFNRS
DEFPD	DEFPS
DEFW	SCALE
SCALM	SEARCH
STVER	TNOM

Supported Eldo Simulator Elements

The syntax for each element is compatible with the Eldo simulator. Table 67 lists the supported Eldo elements in the PrimeSim XA tool.

Table 67 *Supported Eldo Simulator Elements*

Instantiation Character	Element
C	Capacitor
D	Diode
E	Voltage-Controlled Voltage Source (VCVS)
F	Current-Controlled Current Source (CCCS)
G	Voltage-Controlled Current Source (VCCS)
H	Current-Controlled Voltage Source (CCVS)
I	Current Source. See Table 69 for the supported voltage and current source functions.
J	JFET or MESFET
K	Mutual Inductor
L	Linear Inductor
M	MOSFET
Q	BJT
R	Resistor
S	Switch
T	Transmission Line
V	Voltage Source, See Table 69 for the supported voltage and current source functions.
X	Subcircuit Instance

The PrimeSim XA tools supports the macro models from the Eldo simulator listed in [Table 68](#).

Table 68 *Supported Eldo Macro Models*

Instantiation Character	Macro Model	Supported Parameters
ADC or Y	Analog to Digital Converter (ADC)	VSUP, VINFIN, VHI, VLO, TPD, VTH, TCOM

Table 68 *Supported Eldo Macro Models (Continued)*

Instantiation Character	Macro Model	Supported Parameters
AND or Y	2-input AND gate	VHI, VLO, TPD, VTH
AND3 or Y	3-input AND gate	VHI, VLO, TPD, VTH
COMP or Y	Single output comparator	VHI, VLO, VOFF, VDEF, TCOM, TPD
COMPD or Y	Differential Output Comparator	VHI, VLO, VOFF, VDEF, TCOM, TPD
DEL	Delay	Val
INV	Inverter	VHI, VLO, TPD, VTH
NAND	2-input NAND gate	VHI, VLO, TPD, VTH
NAND3	3-input NAND gate	VHI, VLO, TPD, VTH
NOR	2-input NOR gate	VHI, VLO, TPD, VTH
OR	2-input OR gate	VHI, VLO, TPD, VTH
Yxx VSWITCH	Voltage Controlled SWITCH	LEVEL, VON, VOFF

The PrimeSim XA tool supports the voltage and current source functions listed in [Table 69](#):

Table 69 *Supported Voltage and Current Source Functions*

Source	Type
Pulse Source	PULSE (or) PU (or) PUL
Piecewise Linear Source	PWL (or) PL
Sinusoidal Source	SIN
Exponential Source	EXP
Single-Frequency FM Transient Source	SFFM
Amplitude Modulation (AM) Source	AM

Supported Eldo Simulator Device Models

[Table 70](#) lists the Eldo simulator diode models that are supported in the PrimeSim XA tool.

Table 70 Supported Eldo Diodes

Level	Description
1	Berkeley Level 1
2	Modified Berkeley Level 1
3	Fowler-Nordheim Model
4	Eldo-ST Diode Level 1
5	Eldo-ST Diode Level 2
6	Eldo-ST Diode Level 3
8	JUNCAP Model, JUNCAP 2 Model JUNCAP2 Express Model
9	Diode Level 500 Model
27	DIODE_CMC Model

[Table 71](#) lists the Eldo simulator BJT models that are supported in the PrimeSim XA tool.

Table 71 Supported Eldo BJT Models

Level	Description
4	Philips Mextram 503.2 Model
5	Improved Berkeley Model
8	VBIC Model
9	HICUM L2 Model
22	MEXTRAM 504 Model
24	HICUM L0 Model

[Table 72](#) lists the Eldo simulator MOSFET models that are supported in the PrimeSim XA tool.

Table 72 *Supported Eldo MOSFET Models*

Level	Description
19	Eldo-ST MOSFET Level 3
53	Berkeley BSIM3v3 model
59	MOS Model 9, Level 903
60	BSIM4 Model
61	EKV3 Model
62	TFT Polysilicon Model
66	HiSIM2 Model
69	MOS Model 11, Level 1102
70	PSP102 Model
72	BSIMSOI4.0 to 4.3 Model
73	HISIM_HV v1.x Model
74	CMC MOS Varactor
75	PSP103 and PSP104 Models
77	BSIM-CMG Model
79	BSIMSOI4.4 to 4.6 Model, BSIMSOI100 Model
80	UTSOI1 Model
81	BSIM-IMG Model
82	BSIM6 Model, BSIMBULK Model
83	HISIM_HV v2.x Model
84	UTSOI2 Model

[Table 73](#) lists the JFET models that are supported in Eldo format in the PrimeSim XA tool.

Table 73 *Supported JFET Models*

Level	Description
1	JFET Level 1 Model
2	JFET Level 2 Model

[Table 74](#) lists the resistor models that are supported in Eldo format in the PrimeSim XA tool.

Table 74 *Supported Resistor Models*

Level	Description
23	CMC R3 Model
25	CMC R2 Model

Running Monte Carlo Analysis With Eldo Netlists

The PrimeSim XA tool supports Monte Carlo analysis with the Eldo format netlists.

To run Eldo Monte Carlo simulation in the PrimeSim XA environment, run the following command:

```
.MC NRUN_MAX [OV] [Parameters]
```

Use the .MC statement to define control parameters for Monte Carlo simulation. The following sections list the supported parameters and commands in Eldo Monte Carlo simulation in PrimeSim XA.

Supported .MC Parameters

Following are the .MC parameters that are supported in Eldo Monte Carlo simulation:

- NRUN_MAX
- IRUN
- NONOM
- SEED
- VARY

- `SAMPLING=RAND`. Only `RAND` is supported. Other sampling methods are not supported.
- `ALL`

Supported Eldo Variation Parameters

Following are the Eldo variation parameters supported in Eldo Monte Carlo:

- `UNIF (NOM_VALUE, RANGE_VALUE, MULT)`
- `AUNIF (NOM_VALUE, RANGE_VALUE, MULT)`
- `GAUSS (NOM_VALUE, STD_VALUE, SIGMA_COEF, MULT)`
- `AGAUSS (NOM_VALUE, STD_VALUE, SIGMA_COEF, MULT)`
- `LIMIT (NOM_VALUE, STD_VALUE, SIGMA_COEF, MULT)`

Supported Eldo Parameters for Variation Definition

Following are the supported Eldo parameters for variation definition in Monte Carlo:

- `LOT/[UNIF/GAUSS]=MCVAL[%]` for `Global_Variation`
- `DEV/[UNIF/GAUSS]=MCVAL[%]` for `Local_Variation`
- `LOTGROUP`
- `DEVX`

Supported Eldo Commands

Following are the supported Eldo commands in Monte Carlo simulation in the PrimeSim XA tool:

- `DEV[UNIF/GAUSS]` and `LOT[UNIF/GAUSS]` in the `.model` card
- `DEV={E(*, PAR)}` in the `.model` card
- `.MCMOD MODEL_NAME [(LIST_OF_INSTANCES)] PARAM1_NAME LOT|DEV=VALUE [PARAM2_NAME LOT|DEV=VALUE ...]`
- `.MODEL ModelName ModelType PAR=VAL DEV={E(*, PAR)}`
- `.option MODMONTE =0|1`
- `.option CARLO_GAUSS`
- Variation definition for parameter reference shared by model bins

Note:

PrimeSim XA does not support the `PAR1=AUNIF(nom_val, val1)` parameter for variation distribution Inheritance.

Sigma Amplification in Eldo Monte Carlo

Sigma amplification is supported during Eldo Monte Carlo analysis with the following command:

```
.option modified_distribution=scale(value)
```

Sigma amplification is also supported in the PrimeSim advanced variability analysis (AVA) flow. To run sigma amplification simulation in the Eldo AVA flow in PrimeSim XA, use the following command:

```
%xa -eldo filename -adv file.json -o outdir/
```

For more information about how to run sigma amplification simulation in PrimeSim XA, see [Sigma Amplification](#).

For more details on the AVA flow, see the [Sigma Amplification](#) chapter in *PrimeSim™ AVA User Guide*.

Improving Sensitivity Results

Running Eldo Monte Carlo requires the minimum number of samples to be equal to the number of Independent Random Variables (IRVs). In some cases, the tool might write inaccurate sensitivity information in the output .mpp0 file when running Eldo Monte Carlo with smaller sample sizes (from hundreds to thousands). To improve the accuracy of the sensitivity calculation result, specify the `.option fast_sens_method=1` command in the netlist.

Using Verilog-A With Eldo Netlists in the PrimeSim XA Tool

Verilog-A modules in Eldo netlists use the following conventions:

- Verilog-A modules are loaded into the simulator with the `.verilog` and `.model` commands in the Eldo netlist file.
- Verilog instances are explicitly declared with a `y` element.

Following is a simple Verilog-A example for Eldo netlists:

```
.verilog res.va  
.model res macro lang=veriloga  
V0 net1 0 1  
r1 net1 net2 10  
yr3 res net2 net3 GENERIC: R=20  
r2 net3 0 10
```

The `.verilog res.va` and `.model res macro lang=verilog` statements reference the following Verilog-A definition, used for the `yr3` instance.

```
module res (a, b);  
  electrical a, b;  
  parameter real R = 1.0;  
  analog begin  
    V(a,b) <+R * I(a,b);  
  end  
endmodule
```

Eldo netlists are case-insensitive, but Verilog-A modules are case-sensitive. From within Verilog-A modules, a mixed-case name matches a mixed-case name. If there are no exact matches, Verilog-A matches the same name regardless of case. If two references exist in the Verilog-A definitions that are case-sensitive (for example, `mymodul1 inst1` and `mymodul1 Inst1`) and they are referenced by case-insensitive statements in Eldo, the PrimeSim XA tool displays a warning for any ambiguous references.

Note:

Instances that are instantiated in Eldo netlists as `y` elements cannot be overridden; the Verilog-A definition is always used.

Supported Eldo Features in MOSRA Analysis

In order to support the Eldo `.age` command, the PrimeSim XA tool mixes the Eldo format for netlists and models and the PrimeSim HSPICE format for aging models. The PrimeSim XA tool maps every supported option of the Eldo `.age` command to an equivalent option of the PrimeSim HSPICE `.mosra` command. Then the PrimeSim XA tool runs the simulation with the Eldo netlists and models and PrimeSim HSPICE MOSRA age library.

Specifying the Required Data

The following information is required to use the Eldo supported features:

- You need to use the Eldo format for your netlists and model libraries.
- The library file (sometimes called `age.lib`) that specifies the correspondence between the age models (`agemodel`) and the MOS models (`.model`) must also be in Eldo format.
- You need to have an age library (usually called `libmosapi.so`) compatible with PrimeSim HSPICE MOSRA. For that file, the only supported platform is `linux64`.

You also need to set the path to the age library (`libmosapi.so`) with the environment variable `eldo_mosra_models`:

```
setenv eldo_mosra_models ../FRESH_DEVICE/age/linux_64
```

Then you can invoke the PrimeSim XA tool with your Eldo netlist and run the MOSRA simulation.

Supported Options of the .age Command

The PrimeSim XA tool only supports the following commands and options:

```
.agemodel
.age
+ tage
+ tunit
+ nbrun(s)
+ lin/log
+ tstart
+ tstop
+ mode=save|load|ageload|sim
+ agelib
+ agedsim
+ circuit_report
+ trelax
+ area_scaling
+ hci
+ bti
+ tddeb
+ path_report
```

All other options are ignored.

Required Arguments and Defaults

The required arguments are:

```
.agemodel ...
.age
  tage= ...
  tunit= ...
  mode=sim or mode=save|load agelib= ...
```

The `agelib` option is a required argument if `mode=load` or `mode=save`. It is ignored if `mode=sim`.

Note:

The `agelib` option is ignored when `mode=sim`, whatever the `agedsim` value is. That means the `agelib` file is not generated when `mode=sim`, even if `agedsim=0|no`.

The `agedsim` option behavior is:

- If `agedsim=1`, and `mode=save` or `sim`, it's equivalent to PrimeSim HSPICE `.mosra simmode=2`.
- If `agedsim=0`, and `mode=save` or `sim`, it's equivalent to PrimeSim HSPICE `.mosra simmode=0`.
- If `agedsim=1`, and `mode=load`, it's equivalent to PrimeSim HSPICE `.mosra simmode=1`.
- If `agedsim=0`, and `mode=load`, no aging simulation is run.

The other default parameter values are:

- `nbrun 1`
- `lin` and `log` are enabled only if `nbrun` is greater than 1. If `nbrun` is greater than 1, and `lin` or `log` are missing, the PrimeSim XA tool errors out. If `nbrun=1`, `lin` and `log` are ignored, and the simulation is run with 1 time interval.
- `circuit_report 0`
- `trelax 0`
- `area_scaling 1`
- `hci 1`
- `bti 1`
- `tddb 1`

The default level for the age model is `level=500`.

Example

Suppose you have a `netlist.cir` netlist, like:

```
.age tage=10
+ tunit=y
+ nbrun=1
+ hci=1
+ bti=1
+ tddb=1
+ log
+ mode=sim agelib=age_testcase.lib ascii
+ tstart=6E-9
+ tstop=20E-9
+ ageall
+ compute_last=yes
+ plot=all
+ ageall=yes
```

```
+ agedsim=yes
+ trelax=0
+ circuit_report = 1
+ area_scaling=100
```

These commands are internally mapped to the PrimeSim HSPICE `.mosra` equivalent commands. When you run:

```
xa -eldo netlist.cir
```

Both fresh and post-stress MOSRA simulations are run (because `mode=sim`). A mapping file is generated with the extension `.mosra_map`:

Eldo command	Hspice command
<code>.age</code>	<code>.mosra</code>
<code>.agemodel model=lvtpfet</code>	<code>.model RELlvtpfet mosra</code>
	<code>.appendmodel RELlvtpfet mosra lvtpfet PMOS</code>
<code>.agemodel model=lvtnfet</code>	<code>.model RELlvtnfet mosra</code>
	<code>.appendmodel RELlvtnfet mosra lvtnfet NMOS</code>

Eldo <code>.age</code> parameter	HSPICE <code>.mosra</code> parameter
<code>hci=1</code>	<code>HCI=1</code>
<code>bti=1</code>	<code>BTI=1</code>
<code>tddb=1</code>	<code>TDDB=1</code>
<code>tstart=6E-9</code>	<code>AGINGSTART=6E-9</code>
<code>tstop=20E-9</code>	<code>AGINGSTOP=20E-9</code>
<code>trelax=0</code>	<code>TRELAX=0</code>
<code>circuit_report=1</code>	<code>CIRCUIT_REPORT=1</code>
<code>area_scaling=100</code>	<code>AREA_SCALING=100</code>
<code>tage=10 tunit=y</code>	<code>RELTOTALTIME=10yr</code>
<code>agedsim=yes mode=sim</code>	<code>SIMMODE=2</code>

Known Limitations

- Only UTSOI models with level 80 and level 84 are supported.
- Linux64 is the only supported platform.
- `.alter` is not supported. Any data within the `.alter` blocks is ignored.
- Only the specified options of the `.age` command are supported. All other options are not supported.
- When MOSRA is run with the `set_sram_characterization` command, aging is not applied to MOSCAP devices. To apply aging to MOSCAP devices, you must use `set_model_option -moscap 0`.

Using SPECTRE Syntax in 3DIC Netlists

In the PrimeSim XA tool, 3DIC constructs support both PrimeSim HSPICE and SPECTRE syntaxes and features. When both PrimeSim HSPICE and SPECTRE statements exist in a 3DIC netlist, the 3DIC must be constructed by PrimeSim HSPICE 3DIC commands, such as `.module`, `.endmodule`, `.modulevar` and `.endmodulevar`. These PrimeSim HSPICE 3DIC commands must be explicitly specified in the PrimeSim HSPICE format when constructing a 3DIC in a SPECTRE netlist.

The temperature settings for a 3DIC module are effective either in PrimeSim HSPICE or SPECTRE language block, with the corresponding compliant temperature option:

- In the PrimeSim HSPICE language block, use the `.temp` command to set a temperature value. For example,

```
.temp 60
```
- In the SPECTRE language block, use the `options` statement to set a temperature value. For example,

```
option_name options temp=60
```
- If there is no temperature definition either in the PrimeSim HSPICE or SPECTRE language block, the temperature is referred to the setting in the top module.
- The `temper` variable is reserved in PrimeSim HSPICE and SPECTRE and cannot be modified in both of the PrimeSim HSPICE and SPECTRE simulators.

All PrimeSim HSPICE 3DIC-equivalent functionality, like `.subckt`, `.model` and `.param`, refer to the entities defined in the 3DIC module as

```
3DIC_label\:\:netlist_entity_name
```

To comply with the SPECTRE identifier naming convention, use escape characters with any non-alpha-numeric characters in the naming.

For more information about PrimeSim HSPICE 3DIC syntaxes and features, see the related topic in *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

In the *simulator lang=spice* block of a SPECTRE netlist, use the following commands to construct a 3DIC module:

- `.module 3DIC_label`
- `.endmodule 3DIC_label`
- `.modulevar 3DIC_label`
- `.endmodulevar 3DIC_label`

For a detailed description about PrimeSim HSPICE and SPECTRE netlist keywords, see *PrimeSim HSPICE User Guide: Basic Simulation and Analysis* and *Spectre Circuit Simulator Reference*.

Example 1

The following netlist uses both the SPECTRE and PrimeSim HSPICE 3DIC syntaxes to defines a 3DIC module. Instance instantiation is compliant to the PrimeSim HSPICE-equivalent syntax but requires to be fully compliant to the SPECTRE identifier naming rules.

test.scs:

```
// 3DIC cell instantiation
top_inst ... tmod\:\:topcell ...
// 3DIC scope definition with unique labeling
simulator lang=spice
.module tmod
// Native SPECTRE netlist circuit definitions
simulator lang=spectre
include "./cell.scs" section=top
// 3DIC construct block end
simulator lang=spice
.endmodule tmod
```

cell.scs:

```
subckt topcell 1 2 3 4
model nm bsim1 vfb0=-0.5 ...
m1 (1 2 3 4) nm l=5u w=10u ...
res1 (1 3) resistor r=100
ends topcell
```

Example 2

The `.temp` statement the default nominal temperature parameter in both the PrimeSim HSPICE and SPECTRE syntax format. The following example uses the `.temp` statement to obtain the temperature of the current module.

The `.option scale` statement in PrimeSim HSPICE format scales only the devices defined within the “tmod” 3DIC module.

The `options` statements in SPECTRE format are also applicable for module-based options, such as `temp`, `scale` and `tnom`, and are placed inside the SPECTRE language block. The values of `temp` and `tnom` are also accessible through parameters.

Netlist (test.scs):

```
// 3DIC cell instantiation
top_inst ... tmod\:\:topcell ...

// 3DIC scope definition with unique labeling
```

```
simulator lang=spice
.module tmod
.temp 60
.option scale=0.9

// Native SPECTRE netlist circuit definitions
simulator lang=spectre
tnomopt options tnom=25
subckt topcell 1 2 3 4
model nm bsim1 vfb0=-0.5 ...
m1 (1 2 3 4) nm l=5u w=10u ...
r1 (1 3) resistor r="100+temp-tnom"
ends topcell

// 3DIC construct block end
simulator lang=spice
.endmodule tmod
```

Eldo Syntax and Behavior Variations

The Eldo netlist syntax and behavior in the PrimeSim XA tool are compatible with the Eldo simulator with some variations. This section describes some important syntax and behaviors supported in the PrimeSim XA tool when running simulation with Eldo netlist format.

Preprocessor Directives

The PrimeSim XA tool directly supports many of the Eldo simulator preprocessor directives. The following directives are processed even without the `-D` and `-U` command line options:

- `#define`
- `#ifdef`
- `#ifndef`
- `#if`
- `#else`
- `#endif`
- `#com`
- `#endcom`

If you use any other preprocessor directives, they must be invoked with the `-D` or `-U` command line options. In addition, you can use `undefined(VAR)` in an `#if` clause.

Expression Syntax

Mathematical expressions in Eldo format netlists must be enclosed by curly braces {} to ensure correct parsing.

The .EXTRACT Command

The .EXTRACT command extracts waveform information by combining arithmetical expressions or predefined functions. The PrimeSim XA tool only supports the .EXTRACT TRAN command for transient analysis only.

Table 75 lists the function parameters that are supported in the PrimeSim XA tool.

Table 75 Supported Function Parameters for .EXTRACT TRAN Command

BEFORE	VL
AFTER	VTH
OCCUR	VTHIN
LABEL	VTHOUT
VH	

Table 76 lists the functions that are supported in the PrimeSim XA tool.

Table 76 Supported Functions for .EXTRACT TRAN Command

Function	
AVERAGE ()	BUS ()
INTEG ()	MAX ()
MIN ()	RMS ()
SLOPE () with first occurrence only (n=1)	TPD ()
TPDUU ()	TPDUD ()
TPDDD ()	TPDDU ()
TRISE ()	TFALL ()
VALAT ()	WFREG ()

Table 76 Supported Functions for *.EXTRACT TRAN* Command (Continued)

Function	
XDOWN () with 1st occurrence only (n=1)	XUP () with first occurrence only (n=1)
XMAX ()	YVAL ()

Specifying Initial Conditions

The following rules are applied when running the PrimeSim XA tool with netlists in Eldo format:

- Initial condition for nodes can be set with the `.IC` and `.NODESET` statements.
- Initial condition for devices can be set with a device parameter (`IC=value`) for capacitors and inductors only. The `IC=value` parameter is not supported for other devices.
- Initial condition for devices can only be enabled if the `UIC` argument is set in the `.TRAN` statement.

C

Debugging Tools

This chapter describes the debugging tools that are available in the PrimeSim XA tool.

PrimeSim XA provides several commands and scripts to identify any issue in installation environment or differences between setups or releases, as listed in the following sections:

- [The -diff_log Command Line Option](#)
- [Debugging Scripts](#)

The -diff_log Command Line Option

Use the `-diff_log` command line option to compare two PrimeSim XA run log files and generates a comparison file for checking differences in two different runs. Use this script to identify if there are setup or configuration differences in the runs. The script checks for the following differences:

- Test case information
- PrimeSim XA commands and options
- Binary or file version
- Machine

Syntax

```
xa -diff_log log_file1 log_file2 <-o output.diff>
```

where

- The `log_file1` and `log_file2` files are the PrimeSim XA run log files to compare. If the files are not present in the current folder, specify them with the relative or full path along with the file names.
- The `output.diff` file is the output comparison result file. By default, the file is saved in the current folder with the suffix `.diff`. To save the result file to a different folder, specify the file name with the full path.

Examples

The following example compares two XA log files, f1.log and f2.log, and generates comparison results xa.diff in the same folder.

```
xa -diff_log f1.log f2.log
```

The following example compares two XA log files, f1.log and f2.log, and generates comparison results tmpa.diff in the same folder.

```
xa -diff_log f1.log f2.log -o tmpa
```

The following example compares two XA log files in different locations, u/abc/test/f1.log and /testb/f2.log, and generates the comparison results in a different location /testc/tmpb.diff.

```
xa -diff_log /u/abc/test/f1.log /testb/f2.log -o /testc/tmpb
```

Comparison Result File Examples

The following example shows the differences in command settings between the tda.log and tdb.log files.

```
=====
PrimeSim XA Log File Comparison Report
=====
Log files
-----
| #0          | #1          |
|-----+-----|
| tmp/tda.log | tmp/tdb.log |
|-----+-----|

=====
PrimeSim XA setup : items not matching
=====
Commands
-----
| keyword      | #0          | #1          |
|-----+-----+-----|
| set_sim_level | set_sim_level 3 | set_sim_level 4 |
|-----+-----+-----|
```

Co-simulation run is 2-3X slower than the one with equivalent standalone setup. The following example shows the temperature setting difference between these two runs.

Commands		
keyword	#0	#1
load_vector_file	load_vector_file -file ./vec/lane0_stimu lii.vec -format vecset_option xa_clist 1	N/A
set_capacitor_option	N/A	set_capacitor_option -rule 1 -min 1e-17
set_tolerance_level	N/A	set_tolerance_level 5
set_partition_option	N/A	set_partition_option -ap 3
set_sim_level	N/A	set_sim_level 5
set_multi_core	N/A	set_multi_core -core 8
set_waveform_option	N/A	set_waveform_option -flush 200n
set_waveform	N/A	set_waveform -format fsdb
set_duplicate_rule	N/A	set_duplicate_rule -select_subckt last - select_model lastset_message_option -lim it 1000
xa_clist	N/A	set_option xa_clist 1
XA_NON_SPICE_BREAK_HIZ	N/A	set_option XA_NON_SPICE_BREAK_HIZ 3
XA_NON_SPICE_SYNC_NEVER	N/A	set_option XA_NON_SPICE_SYNC_NEVER 1

Options		
keyword	#0	#1
AMS_ADD_FILES_IN_LIB	N/A	wor
AMS_DEFAULT_SPICE_LIB	N/A	VCS_SPICE_LIB
AMS_INST_LIB_BIND_FILE	N/A	simv.daidir/snps_...
AMS_SPICE_CHILD_CHANGE_MAP	N/A	simv.daidir/snps_...
XA_NON_SPICE_BREAK_HIZ	N/A	3
XA_NON_SPICE_SYNC_NEVER	N/A	on
TEMPC	25	125

Debugging Scripts

Table 77 lists the debugging scripts that are available in the PrimeSim XA binary.

Table 77 Debugging Scripts

Script	Description	Generated File
bin/pstack_file.sh	Creates periodic stack profile for taking runtime stack snapshots. This script works for optimized binary runs. The script writes the /proc/pid/limits.env information to check user environment for issues with batch environments.	Two files are generated: <i>output_file_prefix</i>.process-id Consists of encrypted stack function information that can be decrypted by Synopsys to identify the function stack where the application is slow, or the location of segmentation faults. <i>output_file_prefix</i>.process-id.proc Consists of environment information for debugging farm or job launch environment setups.

D

PrimeSim XA Tutorials

The tutorial in this chapter contains examples to run the PrimeSim XA analyses.

This chapter contains the following topics:

- [Running a Setup Timing Check](#)
- [Generating Power Reports](#)
- [Running the Back-Annotation Flow](#)
- [Using Interactive Mode](#)
- [Using a Vector File in a Simulation](#)
- [Using a VCD File in a Simulation](#)
- [Running a MOSRA Simulation](#)

Running a Setup Timing Check

This section describes how to run a setup timing check simulation in batch mode and uses two simple ideal clock sources. [Table 78](#) lists the files needed for this tutorial. These files are located in the *XA_installation_directory/doc/tutorials/timing* directory.

Table 78 Setup Timing Check Files

File Name	Description
setup.spi	SPICE netlist
run	Run script for this tutorial
cmd_setup_basic	Batch-run command file
cmd_setup_check_rise_fall_edge	Batch-run command file
cmd_setup_high_low_thresh	Batch-run command file
cmd_setup_error_file	Batch-run command file

Table 78 Setup Timing Check Files (Continued)

File Name	Description
cmd_setup_negative	Batch-run command file
cmd_setup_subckt	Batch-run command file
cmd_setup_combine	Batch-run command file

The `run` script contains the following lines:

```
xa -hspice setup.spi -c cmd_setup_basic -o setup_basic/xa
xa -hspice setup.spi -c cmd_setup_check_rise_fall_edge -o
  setup_check_rise_fall_edge/xa
xa -hspice setup.spi -c cmd_setup_high_low_thresh -o
  setup_high_low_thresh/xa
xa -hspice setup.spi -c cmd_setup_error_file -o setup_error_file/xa
xa -hspice setup.spi -c cmd_setup_negative -o setup_negative/xa
xa -hspice setup.spi -c cmd_setup_subckt -o setup_subckt/xa
xa -hspice setup.spi -c cmd_setup_combine -o setup_combine/xa
```

This script file specifies the SPICE netlist, named `setup.spi`, and the command files to run the different setup conditions.

From the UNIX command line, use `cat` to show the contents of the setup simulation command files:

1. `cat cmd_setup_basic`

```
# In this setup check command, the reference signal is clk and
# the data signal is data. The edge to check for is the rising
# edge and the setup time to check for is 6ns. If the data
# moves in the 6ns window, a setup violation is reported.

check_timing_setup -title setup -node data -data_edge_type rise \
-ref clk -ref_edge_type rise -setup_time 6n \
set_waveform_option -format wdf
```

2. `cat cmd_setup_check_rise_fall_edge`

```
# In this setup check command, the reference signal is clk and
# the data signal is data. The edge to check for is both the
# rising/falling edge, and the setup time to check for is
# 6ns. If the data moves in the 6ns window, a setup violation
# is reported.

check_timing_setup -title setup -node data -data_edge_type rf \
-ref clk -ref_edge_type rf -setup_time 6n set_waveform_option \
-format wdf
```

3. cat cmd_setup_high_low_thresh

```
# In this setup check command, the reference signal is clk and
# the data signal is data. The edge to check for is the rising
# edge and the setup time to check for is 6ns. The high voltage
# threshold is 0.7v, and the low voltage threshold is 0.3v.
# If the data moves in the 6ns window, a setup violation
# is reported.
```

```
check_timing_setup -title setup -node data -data_edge_type rise \
-ref_clk -ref_edge_type rise -hith 0.7 -loth 0.3 -setup_time 6n \
set_waveform_option -format wdf
```

4. cat cmd_setup_error_file

```
# In this setup check command, the reference signal is clk and
# the data signal is data. The edge to check for is the rising
# edge and the setup time to check for is 6ns. The output error
# data is sent to a separate file called setup_error_file.
```

```
check_timing_setup -title setup -node data -data_edge_type rise \
-ref_clk -ref_edge_type rise -setup_time 6n \
-error_file setup_error_file set_waveform_option -format wdf
```

5. cat cmd_setup_negative

```
# In this setup check command, the reference signal is clk and
# the data signal is data. The edge to check for is the rising
# edge and the setup time to check for is -1n. If the data moves
# in the -1ns window, a setup violation is reported.
```

```
check_timing_setup -title setup -node data -data_edge_type rf \
-ref_clk -ref_edge_type rise -setup_time -1n -window 3n \
set_waveform_option -format wdf
```

6. cat cmd_setup_subckt

```
# In this setup check command, the reference signal is clk and
# the data signal is data. The edge to check for is the rising
# edge and the setup time to check for is 6ns. The signal to
# check is inside the sub1 subcircuit. If the data moves in
# the 6ns window, a setup violation is reported.
```

```
check_timing_setup -title setup -node data -data_edge_type rise \
-ref_clk -ref_edge_type rise -setup_time 6n -subckt sub1 \
set_waveform_option -format wdf
```

7. cat cmd_setup_combine

```
# In this setup check command, the reference signal is clk and
# the data signal is data. The edge to check for is the rising
# edge and the setup time to check for is 6ns the high voltage
# threshold is 0.8v. The low voltage threshold is 0.2v and
```



```
# the output error data is send to a separate file called
# setup_error_file.

check_timing_setup -title setup -node data -data_edge_type rise \
-ref clk -ref_edge_type rise -hith 0.8 -loth 0.2 -setup_time 6n \
-error_file setup_error_file -subckt sub1 set_waveform_option \
-format wdf
```

Running the Setup Simulation

To run the setup simulation, do the following steps:

1. Run the simulation from the UNIX command line using the `source` command to source the relevant PrimeSim XA binary to run the PrimeSim XA tool.
2. Check the file permission to make sure that the `run` script is set to be executable.
3. Type `run` to run the simulation script.
4. When the simulation finishes, it contains the following directories and files:

```
./setup_basic (directory)
    xa.errt
    xa.log
    xa.wdf
./setup_check_rise_fall_edge (directory)
    xa.errt
    xa.log
    xa.wdf
./setup_high_low_thresh (directory)
    xa.errt
    xa.log
    xa.wdf
./setup_error_file (directory)
    setup_error_file.errt
    xa.log
    xa.wdf
./setup_negative (directory)
    xa.errt
    xa.log
    xa.wdf
./setup_subckt (directory)
    xa.errt
    xa.log
    xa.wdf
./setup_combine (directory)
    setup_error_file.errt
    xa.log
    xa.wdf
```

The *.errt files are the timing violation output files. The number 0 at the end indicates this is the first run. If there are multiple runs due to alter/data sweep statements, the number increases based on the number of these statements.

Running the Back-Annotation Flow

This section describes how to run the back-annotation flow in the PrimeSim XA tool. [Table 79](#) lists the files needed for this tutorial. These files are located in the `XA_installation_directory/doc/tutorials/ba_flow` directory.

Table 79 Back-Annotation Files

File Name	Description
top.spi	Top-level SPICE netlist
add4.sp	SPICE netlist
add4.spf	SPF file
mos.model	BSIM3 MOS model
cmd_ba	PrimeSim XA command file
cmd_ba_conly	PrimeSim XA command file
cmd_ba_min_res	PrimeSim XA command file
cmd_selective_extract	PrimeSim XA command file
cmd_ba_active_net	PrimeSim XA command file
run	Run script for this tutorial

The `run` script contains the following lines:

```
xa top.spi -c cmd_ba -o RESULT/XA_normal_ba_flow
xa top.spi -c cmd_selective_extract -o
  RESULT/XA_selective_extract_ba_flow
xa top.spi -c cmd_ba_active_net -o RESULT/XA_active_net_ba_flow
xa top.spi -c cmd_ba_conly -o RESULT/XA_ba_conly
xa top.spi -c cmd_ba_min_res -o RESULT/XA_ba_remove_min_res
```

This script specifies the SPICE netlist, named `top.spi`, and the command files to run the back-annotation flow.

From the UNIX command line, use `cat` to show the contents of the power reporting simulation command files.

1. `cat cmd_ba`

```
# Sets the simulation accuracy/performance control
set_sim_level 5

# Specifies the back-annotation command
load_ba_file -file add4.spf
```

2. `cat cmd_selective_extract`

```
# Sets the simulation accuracy/performance control
set_sim_level 5

# The default active net generation monitors all node activities
# from time=0 to the end of transient time.
set_ba_active_file -file RESULT/active_net -twindow 0ns 20ns
```

3. `cat cmd_ba_active_net`

```
# Sets the simulation accuracy/performance control.
set_sim_level 5

# Specifies the back-annotation command.
load_ba_file -file add4.spf

# Invokes the active net back-annotation flow.
set_active_net_flow 1 -vtol 150m
```

4. `cat cmd_ba_only`

```
# Sets the simulation accuracy/performance control.
set_sim_level 5

# Specifies the back-annotation command.
# The -cnet argument instructs XA to perform lump C #
# back-annotation only.
load_ba_file -file add4.spf -cnet *
```

5. `cat cmd_ba_min_res`

```
# Sets the simulation accuracy/performance control.

set_sim_level 5
# Specifies the back-annotation command.
# The -min_res/-min_cap instructs XA to short the # small resistors.
load_ba_file -file add4.spf -min_res 0.1
```

Running Back-Annotation

To run the back-annotation flow, do the following steps:

1. To run the simulation from the UNIX command line, use the `source` command to source the relevant PrimeSim XA binary to run the PrimeSim XA tool.
2. Check the file permission to make sure that the `run` script is set to be executable.
3. Type `run` to run the simulation script.
4. When back-annotation finishes, it contains the following directories and files:

```
./RESULT (directory)
  active_net.rcxt
  XA_active_net_ba_flow.active_nets.rcxt
  XA_active_net_ba_flow.fsdb
  XA_active_net_ba_flow.log
  XA_active_net_ba_flow.meas
  XA_ba_only.fsdb
  XA_ba_only.log
  XA_ba_only.meas
  XA_ba_remove_min_res.fsdb
  XA_ba_remove_min_res.log
  XA_ba_remove_min_res.meas
  XA_normal_ba_flow.fsdb
  XA_normal_ba_flow.log
  XA_normal_ba_flow.meas
  XA_selective_extract_ba_flow.fsdb
  XA_selective_extract_ba_flow.log
  XA_selective_extract_ba_flow.meas
```

If there are multiple runs due to alter/data sweep statements, the number increases based on the number of these statements.

Interpreting Back-Annotation Reports

To interpret the back-annotation reports, do the following steps.

1. Use a text editor to open the `xa_norma_ba_flow.log` file.

Search for the “Summary of parsing DSPF file” section, like:

```
Summary of parsing DSPF file "add4.spf"-----
| module name                               |           |
| # of instantiations                       | 1         |
| Lines parsed                             | 3877      |
| Nets parsed                              | 113       |
| Nets back-annotated without error         | 113 (100%) |
| Nets not back-annotated                   | 0 ( 0%)   |
| Empty nets                               | 0 ( 0%)   |
| Parasitic nodes added                     | 990       |
| Resistors back-annotated                  | 1891 (100%) |
```

```
| Grounded capacitors back-annotated          | 985 (100%) |
| Coupling capacitors back-annotated          | 0 (0%)     |
| Resistors skipped due to errors             | 0 (0%)     |
| Capacitors skipped due to errors            | 0 (0%)     |
| Devices updated (DPF) without error         | 208 (100%) |
| Devices not updated (DPF) due to error      | 0 (0%)     |
|-----|
This statistics shows the RC back-annotation as well as device
parameter back-annotation has been successfully completed.
```

The format of the back-annotation report is as follows:

- **module name** indicates whether the back-annotation is from the top-level or subcircuit base. If it is a subcircuit based back-annotation, the name of the subcircuit is placed under the module name section.
- **Lines parsed** is the total number of lines parsed in the SPF file.
- **Nets parsed** is the total number of SPF nets in the SPF file.
- **Nets back-annotated without error** is the total number of SPF nets successfully back-annotated.
- **Nets back-annotated with enable_ba_error_net** is the number of SPF nets that the PrimeSim XA tool successfully back-annotated if `set_ba_option -enable_error_net 1` is used.
- **Nets not back-annotated** is the number of SPF nets that the PrimeSim XA tool cannot back-annotate. The PrimeSim XA tool issues a warning message in the log file that explains why a specific SPF net cannot be back-annotated.
- **Empty nets** is the number of SPF nets with no RC information.
- **Parasitic nodes** is the number of nodes created due to the addition of parasitic resistors.
- **Resistors back-annotated** is the number of resistors back-annotated from the SPF file.
- **Grounded capacitors back-annotated** is the total number of grounded capacitors back-annotated from the SPF file.
- **Coupling capacitors back-annotated** is the total number of coupling capacitors back-annotated from the SPF file.
- **Resistors skipped due to error** is the number of resistors that could not be back-annotated.

- **Capacitors skipped due to error** is the number of capacitors that could not be back-annotated.
 - **Instance parameters updated (DPF)** and **Instances back-annotated** list the number of devices with device parameters updated.
2. Use a text editor to open the output .log file for the C-only back-annotation statistic and report and search for the “Summary of parsing DSPF file” section. like:

Summary of parsing DSPF file "add4.spf"

module name		
# of instantiations	1	
Lines parsed	3877	
Nets parsed	113	
Nets back-annotated without error	113 (100%)	
... Nets with lump C	113	
... Nets with distributed RC	0	
Nets not back-annotated	0 (0%)	
Empty nets	0 (0%)	
Parasitic nodes added	0	
Resistors back-annotated	0	
Grounded capacitors back-annotated	113 (100%)	
Coupling capacitors back-annotated	0 (0%)	
Resistors skipped due to errors	0	
Capacitors skipped due to errors	0 (0%)	
Devices updated (DPF) without error	208 (100%)	
Devices not updated (DPF) due to error	0 (0%)	

This statistics shows the RC back-annotation as well as device parameter back-annotation has been successfully completed.

This format of the back-annotation report is similar to the normal back-annotation report. The only difference is that the “Nets back-annotated” section is further broken down to two sub-categories. One is “Nets with lump C back-annotation” and the other is “Nets with distributed RC”. Because this run is C-only back-annotation, only the net capacitance is added for each SPF net.

3. Use a text editor to open the output .rcxt file for the active net information:

```
*
* PrimeSim XA RHEL64 M-2017.03
* build id
* Report active notes in StarRCXT format.
* rcxt_hierid = /
* twindow = 0 2e-08
* reuse_active_net = 0
* reuse_ic = 0
NETS: a0
NETS: a2
NETS: b0
```

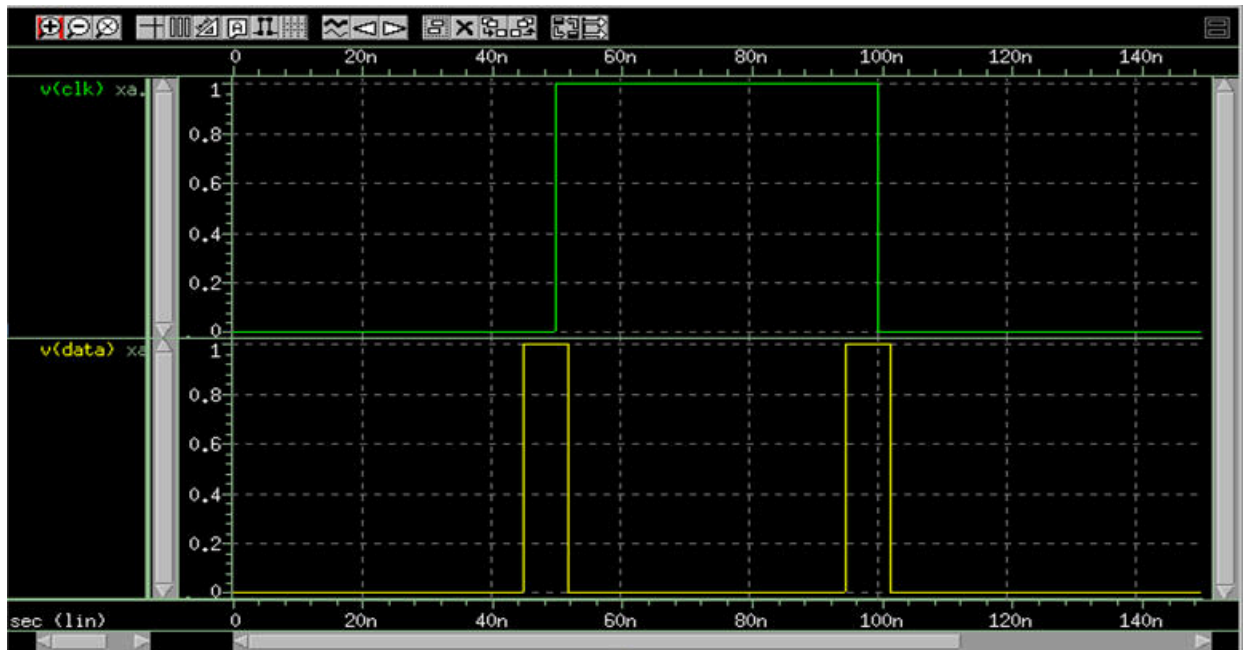
```
NETS: b1  
NETS: b3  
NETS: n6  
...
```

This file shows the active net information generated by the PrimeSim XA tool, which can be fed back to StarRC to generate the active net SPF or SPEF file.

Viewing the Simulation Output

Use WaveView to open the .wdf output waveform file and plot the v(clk) and v(data) signals, as shown in [Figure 30](#).

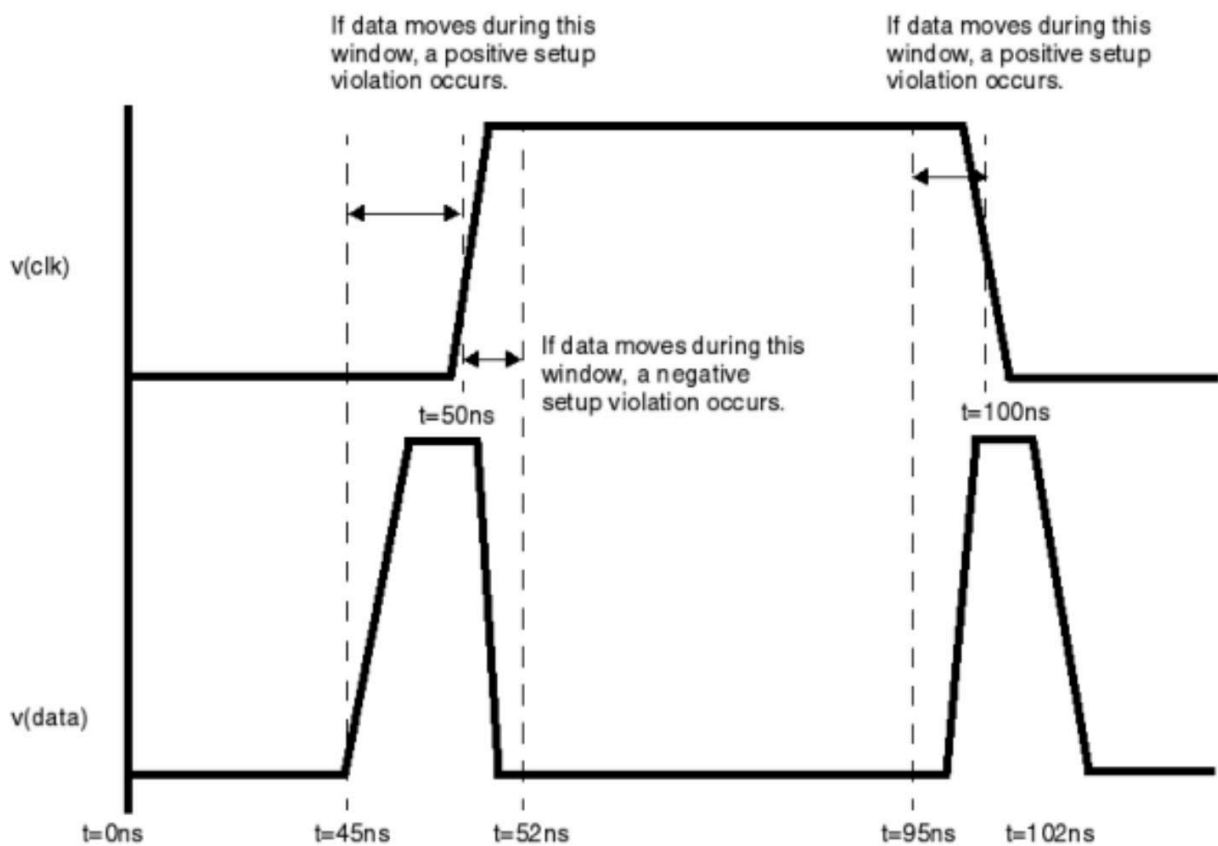
Figure 30 v(clk) and v(data) Signals



The first rise edge of the clock signal occurred at 50 ns and the first falling edge occurred at 100 ns. The slope of the rise/fall edge is 100 ps each.

The first rise edge of the data signal occurred at 49 ns and the first falling edge occurred at 52 ns. The second rise edge of the data signal occurred at 95 ns and the second falling edge occurred at 102 ns. The slope of the rise/fall edge is 100 ps each. For example:

Conditions for 'clk' and 'data'	Setup time condition	
For rising edge clk and rising edge data	If setup_time > 5ns	If setup_time < 5ns
$ t_{data} - t_{clk} = 5ns$	'positive' setup violation	No violation
For falling edge clk and rising edge data	If setup_time > 5ns	If setup_time < 5ns
$ t_{data} - t_{clk} = 5ns$	'positive' setup violation	No violation
For rising edge clk and falling edge data	No 'positive' setup violation	
For falling edge clk and falling edge data	No 'positive' setup violation	



Reviewing Violations

Review the set up violations of each command file and verify them against the waveform file to confirm the validity of the timing violations.

1. cat ./setup_basic/xa.errt

```
** XA xx-bit LINUX D-2010.03-SP1 (built xx:xx:xx xxx xx xxxx)

* build id: xxxxxxxx
*-----
* Timing Setup check
* title           =setup
* node            =data
* data_edge_type  =rise
* ref             =clk
* ref_edge_type   =rise
* setup_time      =6e-09
* window          =not specified
* subckt          =not specified
* loth            =default value
* hith            =default value
* error_file      =not specified
*-----
Violation#1: setup: Ref=V(CLK) Sig=V(DATA)  Tsig=4.507e-08
          Tref=5.007e-08 Tsig-Tref=-5e-09
```

2. cat ./setup_check_rise_fall_edge/xa.errt

```
** XA xx-bit LINUX D-2010.03-SP1 (built xx:xx:xx xxx xx xxxx)
* build id: xxxxxxxx
*-----* Timing
Setup check
* title           =setup
* node            =data
* data_edge_type  =rf
* ref             =clk
* ref_edge_type   =rf
* setup_time      =6e-09
* window          =not specified
* subckt          =not specified
* loth            =default value
* hith            =default value
* error_file      =not specified
*-----
Violation#1: setup: Ref=V(CLK) Sig=V(DATA)  Tsig=4.507e-08
          Tref=5.007e-08 Tsig-Tref=-5e-09
Violation#2: setup: Ref=V(CLK) Sig=V(DATA)  Tsig=9.507e-08
          Tref=1.0007e-07 Tsig-Tref=-5e-09
```

3. cat ./setup_high_low_thresh/xa.errt

```
*
* XA xx-bit LINUX D-2010.03-SP1 (built xx:xx:xx xxx xx xxxx)
* build id: xxxxxxxx
*-----
* Timing Setup check
* title           =setup
```

```
* node           =data
* data_edge_type =rise
* ref            =clk
* ref_edge_type  =rise
* setup_time     =6e-09
* window         =not specified
* subckt         =not specified
* loth           =0.3
* hith           =0.7
* error_file     =not specified
*-----
Violation#1: setup: Ref=V(CLK) Sig=V(DATA)  Tsig=4.507e-08
          Tref=5.007e-08 Tsig-Tref=-5e-09
```

4. cat ./setup_error_file/setup_error_file.errt

```
** XA xx-bit LINUX D-2010.03-SP1 (built xx:xx:xx xxx xx xxxx)
* build id: xxxxxxxx
*-----
* Timing Setup check
* title           =setup
* node            =data
* data_edge_type  =rise
* ref             =clk
* ref_edge_type   =rise
* setup_time      =6e-09
* window          =not specified
* subckt          =not specified
* loth            =default value
* hith            =default value
* error_file      =setup_error_file
*-----
Violation#1: setup: Ref=V(CLK) Sig=V(DATA)  Tsig=4.507e-08
          Tref=5.007e-08 Tsig-Tref=-5e-09
```

5. cat ./setup_negative/xa.errt

```
** XA xx-bit LINUX D-2010.03-SP1 (built xx:xx:xx xxx xx xxxx)
* build id: xxxxxxxx
*-----
* Timing Setup check
* title           =setup
* node            =data
* data_edge_type  =rise
* ref             =clk
* ref_edge_type   =rise
* setup_time      =-1e-09
* window          =3e-09
* subckt          =not specified
* loth            =default value
* hith            =default value
* error_file      =not specified
*-----
```

```
Violation#1: setup: Ref=V(CLK) Sig=V(DATA) Tsig=5.207e-08
Tref=5.007e-08 Tsig-Tref=2e-09
```

6. cat ./setup_subckt/xa.ertr

```
*
* XA xx-bit LINUX D-2010.03-SP1 (built xx:xx:xx xxx xx xxxx)
* build id: xxxxxxxx
* -----
* Timing Setup check
* title           =setup
* node            =data
* data_edge_type  =rise
* ref             =clk
* ref_edge_type   =rise
* setup_time      =6e-09
* window          =not specified
* subckt          =sub1
* loth            =default value
* hith            =default value
* error_file      =not specified
* -----
Violation#1: setup: Ref=V(CLK) Sig=V(DATA) Tsig=4.507e-08
Tref=5.007e-08 Tsig-Tref=-5e-09
```

7. cat ./setup_combine/setup_error_file.ertr

```
*
* XA xx-bit LINUX D-2010.03-SP1 (built xx:xx:xx xxx xx xxxx)
* build id: xxxxxxxx
* -----
* Timing Setup check
* title           =setup
* node            =data
* data_edge_type  =rise
* ref             =clk
* ref_edge_type   =rise
* setup_time      =6e-09
* window          =not specified
* subckt          =sub1
* loth            =0.8
* hith            =0.2
* error_file      =setup_error_file
* -----
Violation#1: setup: Ref=V(CLK) Sig=V(DATA) Tsig=4.507e-08
Tref=5.007e-08 Tsig-Tref=-5e-09
```

Generating Power Reports

This section describes how to generate power reports in the PrimeSim XA tool. [Table 80](#) lists the files needed for this tutorial. These files are located in the *XA_installation_directory/doc/tutorials/power* directory.

Table 80 *Power Reporting Files*

File Name	Description
top.spi	Top-level SPICE netlist
circuit.sp	SPICE netlist
cells.sp	SPICE cell subcircuit
bsim3.mod	BSIM3 MOS model
cmd_report_by_node	Batch-run command file
cmd_report_by_node_default	Batch-run command file
cmd_report_by_port	Batch-run command file
cmd_report_by_port_default	Batch-run command file
run	Run script for this tutorial

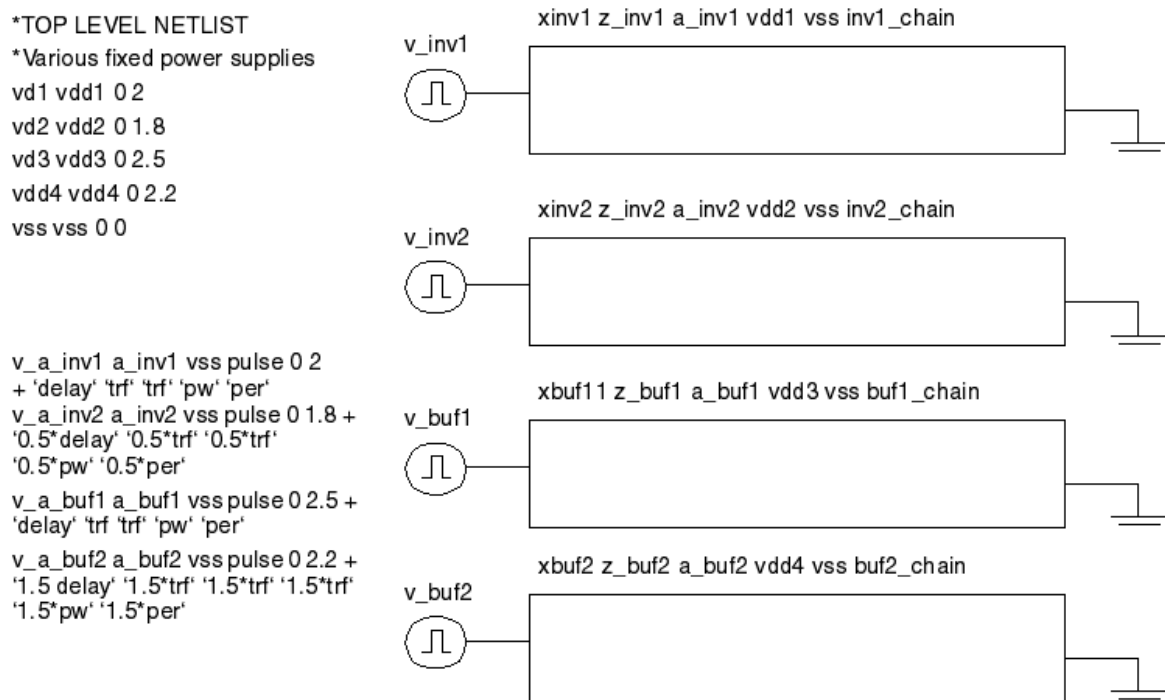
The `run` script contains the following lines:

```
xa top.spi -c cmd_report_by_port_default -o RESULT/ \
  xa_power_report_by_port_default
xa top.spi -c cmd_report_by_port -o RESULT/xa_power_report_by_port
xa top.spi -c cmd_report_by_node_default -o RESULT/ \
  xa_power_report_by_node_default
xa top.spi -c cmd_report_by_node -o RESULT/xa_power_report_by_node
```

This script file specifies the SPICE netlist, named `top.spi`, and the command files to generate different types of power reports.

[Figure 31](#) shows a high-level block diagram of the `circuit.sp` netlist.

Figure 31 Block Diagram of circuit.sp



From the UNIX command line, use `cat` to show the contents of the power reporting simulation command files.

```
1. cat cmd_report_by_port_default

# Standard commands for setting simulation and waveform
# probing.

set_sim_level -level 5
probe_waveform_voltage * -port 1

# Power/current reporting is based on the port name. By default
# power reporting is to print up to 3 levels of hierarchy,
# starting from the top-level (top-level is considered
# a hierarchy of 0).

# Use power reporting by port name when you want to analyze
# the power consumption of a particular block (subcircuit) and
# find the contribution from each port in relation to the total
# power consumption of that block (subcircuit).

# Default power reporting time window is
# from 0ns to the end_of_transient simulation.

# -label label_name <--(user defined label_name)
```

```
report_power -port xinv1.* -label xinv1_block_power
```

2. cat cmd_report_by_port

```
# Standard commands for setting simulation and waveform
# probing.

set_sim_level -level 5
probe_waveform_voltage * -port 1

# Power/current reporting is based on the port name. By default
# power reporting is to print up to 3 levels of hierarchy,
# starting from the top-level (top-level is considered
# a hierarchy of 0).

# Power/current reporting is based on the port name. By default
# power reporting is to print up to 3 levels of hierarchy,
# starting from the top-level (top-level is considered
# a hierarchy of 0).

# Use power reporting by port name when you want to analyze
# the power consumption of a particular block (subcircuit) and
# find the contribution from each port in relation to the total
# power consumption of that block (subcircuit).
# Default power reporting time window is
# from 0ns to the end_of_transient simulation.

# -label label_name <--(user defined label_name)
report_power -port xinv1.* -label xinv1_block_power_level_1 -limit 1
report_power -port xinv1.* -label xinv1_block_power_level_1_window \
  -limit 1 -from 0ns -to 32ns
```

3. cat cmd_report_by_node_default

```
# Standard commands for setting simulation and waveform
# probing.

set_sim_level -level 5
probe_waveform_voltage * -port 1

# Power/current reporting is based on the port name. By default
# power reporting is to print up to 3 levels of hierarchy,
# starting from the top-level (top-level is considered
# a hierarchy of 0).

# Power/current reporting is based on the port name. By default
# power reporting is to print up to 3 levels of hierarchy,
# starting from the top-level (top-level is considered
# a hierarchy of 0).

# Use power reporting by port name when you want to analyze
# the power consumption of a particular block (subcircuit) and
```

```
# find the contribution from each port in relation to the total
# power consumption of that block (subcircuit).
# Default power reporting time window is
# from 0ns to the end_of_transient simulation.

# -label label_name <--(user defined label_name)
report_power -by_node vdd1 -label xinv1_power_thru_vdd
report_power -by_node vdd2 -label xinv2_power_thru_vdd
report_power -by_node vdd3 -label xbuf1_power_thru_vdd
report_power -by_node vdd4 -label xbuf2_power_thru_vdd
```

4. cat cmd_report_by_node

```
# Standard commands for setting simulation and waveform
# probing.

set_sim_level -level 5
probe_waveform_voltage * -port 1

# Power/current reporting is based on the port name. By default
# power reporting is to print up to 3 levels of hierarchy,
# starting from the top-level (top-level is considered
# a hierarchy of 0).

# Power/current reporting is based on the port name. By default
# power reporting is to print up to 3 levels of hierarchy,
# starting from the top-level (top-level is considered
# a hierarchy of 0).

# Use power reporting by port name when you want to analyze
# the power consumption of a particular block (subcircuit) and
# find the contribution from each port in relation to the total
# power consumption of that block (subcircuit).
# Default power reporting time window is
# from 0ns to the end_of_transient simulation.

# -label label_name <--(user defined label_name)

report_power -by_node vdd1 -label xinv1_power_thru_vdd \
  -limit=1
report_power -by_node vdd2 -label xinv1_power_thru_vdd \
  -limit=1
report_power -by_node vdd3 -label xbuf1_power_thru_vdd \
  -limit=1
report_power -by_node vdd4 -label xbuf2_power_thru_vdd \
  -limit=1
```

Running the Power Reporting Simulation

To run the power reporting simulation, do the following steps:

1. Run the simulation from the UNIX command line using the `source` command to source the relevant PrimeSim XA binary to run the PrimeSim XA tool.
2. Check the file permission to make sure that the `run` script is set to be executable.
3. Type `run` to run the simulation script.
4. When the simulation finishes, it contains the following directories and files:

```
./RESULT (directory)
  xa_power_report_by_node_default.fsdb
  xa_power_report_by_node_default.log
  xa_power_report_by_node_default.power
  xa_power_report_by_node.fsdb
  xa_power_report_by_node.log
  xa_power_report_by_node.power
  xa_power_report_by_port_default.fsdb
  xa_power_report_by_port_default.log
  xa_power_report_by_port_default.power
  xa_power_report_by_port.fsdb
  xa_power_report_by_port.log
  xa_power_report_by_port.power
```

The *.power files are the timing violation output files. If there are multiple runs due to alter/data sweep statements, the number increases based on the number of these statements.

Interpreting Power Reports

To interpret the power reports, do the following steps.

1. Use a text editor to open the `xa_power_report_by_port.power` file.

Note that power reporting in this example is only up to the first level of hierarchy.

This file reports power by port name:

```
##### LABEL=xinv1_block_power_level_1 FROM=0 TO=6.4000000e-08 #####
```

```
Port Name: xinv1.0 (0)
Sub-circuit Definition: inv1_chain
Max(A)= 0.0000000e+00 at = 0.0000000e+00 Min(A)= 0.0000000e+00
at = 0.0000000e+00 Avg(A)= 0.0000000e+00 Rms(A)= 0.0000000e+00
Max(W)= 0.0000000e+00 at = 0.0000000e+00 Min(W)= 0.0000000e+00
at = 0.0000000e+00 Avg(W)= 0.0000000e+00 Rms(W)= 0.0000000e+00
```

```
Port Name: xinv1.a (a_inv1)
Sub-circuit Definition: inv1_chain
```



```
Max(A)= 4.4599692e-05 at = 1.1502626e-09 Min(A)= -4.0414113e-05
at = 3.2199998e-08 Avg(A)= 4.6925708e-08 Rms(A)= 1.5248058e-05
Max(W)= 7.0980168e-05 at = 1.1645074e-09 Min(W)= -4.3803784e-05
at = 3.2062570e-08 Avg(W)= 1.1317937e-06 Rms(W)= 1.6633663e-05
```

Port Name: xinv1.vdd (vdd1)

Sub-circuit Definition: inv1_chain

```
Max(A)= 2.7773310e-04 at = 1.5279325e-09 Min(A)= -2.2575315e-05
at = 1.0266944e-09 Avg(A)= 9.9222129e-05 Rms(A)= 1.4313898e-04
Max(W)= 5.5546620e-04 at = 1.5279325e-09 Min(W)= -4.5150630e-05
at = 1.0266944e-09 Avg(W)= 1.9844426e-04 Rms(W)= 2.8627797e-04
```

Port Name: xinv1.vss (vss)

Sub-circuit Definition: inv1_chain

```
Max(A)= 1.3556621e-05 at = 3.2041568e-08 Min(A)= -2.7773338e-04
at = 1.5279325e-09 Avg(A)= -9.9268782e-05 Rms(A)= 1.4339026e-04
Max(W)= 0.0000000e+00 at = 0.0000000e+00 Min(W)= 0.0000000e+00
at = 0.0000000e+00 Avg(W)= 0.0000000e+00 Rms(W)= 0.0000000e+00
```

Port Name: xinv1.z (z_inv1)

Sub-circuit Definition: inv1_chain

```
Max(A)= 3.5783160e-08 at = 2.5665368e-09 Min(A)= -4.0745363e-10
at = 1.5537400e-09 Avg(A)= 2.1743198e-10 Rms(A)= 1.6743959e-09
Max(W)= 5.5125580e-08 at = 5.4566536e-08 Min(W)= -4.1796546e-10
at = 2.5561720e-08 Avg(W)= 1.6895784e-10 Rms(W)= 2.5141304e-09
```

```
##### LABEL=xinv1_block_power_level_1_window FROM=0 TO=3.2000000e-08
#####
```

Port Name: xinv1.0 (0)

Sub-circuit Definition: inv1_chain

```
Max(A)= 0.0000000e+00 at = 0.0000000e+00 Min(A)= 0.0000000e+00
at = 0.0000000e+00 Avg(A)= 0.0000000e+00 Rms(A)= 0.0000000e+00
Max(W)= 0.0000000e+00 at = 0.0000000e+00 Min(W)= 0.0000000e+00
at = 0.0000000e+00 Avg(W)= 0.0000000e+00 Rms(W)= 0.0000000e+00
```

Port Name: xinv1.a (a_inv1)

Sub-circuit Definition: inv1_chain

```
Max(A)= 4.4599692e-05 at = 1.1502626e-09 Min(A)= -4.0413032e-05
at = 2.1999980e-09 Avg(A)= 1.6427514e-07 Rms(A)= 1.5137730e-05
Max(W)= 7.0980168e-05 at = 1.1645074e-09 Min(W)= -4.3648000e-05
at = 3.0063004e-08 Avg(W)= 1.2226605e-06 Rms(W)= 1.6538544e-05
```

Port Name: xinv1.vdd (vdd1)

Sub-circuit Definition: inv1_chain

```
Max(A)= 2.7773310e-04 at = 1.5279325e-09 Min(A)= -2.2575315e-05
at = 1.0266944e-09 Avg(A)= 9.7573063e-05 Rms(A)= 1.4195541e-04
Max(W)= 5.5546620e-04 at = 1.5279325e-09 Min(W)= -4.5150630e-05
at = 1.0266944e-09 Avg(W)= 1.9514613e-04 Rms(W)= 2.8391083e-04
```

Port Name: xinv1.vss (vss)

Sub-circuit Definition: inv1_chain

```
Max(A)= 1.3024580e-05 at = 2.0420020e-09 Min(A)= -2.7773338e-04
at = 1.5279325e-09 Avg(A)= -9.7737074e-05 Rms(A)= 1.4227725e-04
Max(W)= 0.0000000e+00 at = 0.0000000e+00 Min(W)= 0.0000000e+00
at = 0.0000000e+00 Avg(W)= 0.0000000e+00 Rms(W)= 0.0000000e+00
```

```
Port Name: xinv1.z (z_inv1)
Sub-circuit Definition: inv1_chain
Max(A)= 3.5783160e-08 at = 2.5665368e-09 Min(A)= -4.0745363e-10
at = 1.5537400e-09 Avg(A)= 2.1012442e-10 Rms(A)= 1.6472864e-09
Max(W)= 5.5125304e-08 at = 2.5665368e-09 Min(W)= -4.1796546e-10
at = 2.5561720e-08 Avg(W)= 1.6322400e-10 Rms(W)= 2.4734206e-09
```

The format of this file is as follows:

- The first line contains a user-defined name and time window in which power reporting is performed.
- The next line shows the port name and, in parenthesis, the node name associated with the port name.
- Following the port name is the subcircuit definition for the port. For each port name, the default power reporting includes the max/min/average/Rms. The power report also shows both current (A) and power (W).

In previous power report, there are two sections. The first section is defined by the `xinv1_block_power_level_1` label name, and the time window is from 0 ns to 64 ns. The second section is defined by the `xinv1_block_power_level_1_window` label name, and the time window is from 0 ns to 32 ns.

2. Use a text editor to open the `xa_power_report_by_node.power0` file.

Note that power reporting in this example is only up to the first level of hierarchy.

This file reports power by node name:

```
##### LABEL=xinv1_power_thru_vdd FROM=0 TO=6.4000000e-08 #####

Port Name: vd1 (_power_element_)
Sub-circuit Definition: _power_element_
Max(A)= 2.2583094e-05 at = 1.0269051e-09 Min(A)= -2.7794437e-04
at = 1.5281242e-09 Avg(A)= -9.9203008e-05 Rms(A)= 1.4313153e-04
Max(W)= 5.5588875e-04 at = 1.5281242e-09 Min(W)= -4.5166187e-05
at = 1.0269051e-09 Avg(W)= 1.9840602e-04 Rms(W)= 2.8626305e-04

Port Name: xinv1.vdd (vdd1)
Sub-circuit Definition: inv1_chain
Max(A)= 2.7794438e-04 at = 1.5281242e-09 Min(A)= -2.2583093e-05
at = 1.0269051e-09 Avg(A)= 9.9203098e-05 Rms(A)= 1.4313166e-04
Max(W)= 5.5588876e-04 at = 1.5281242e-09 Min(W)= -4.5166186e-05
at = 1.0269051e-09 Avg(W)= 1.9840620e-04 Rms(W)= 2.8626331e-04

##### LABEL=xinv2_power_thru_vdd FROM=0 TO=6.4000000e-08 #####
```

```
Port Name: vd2 (_power_element_)
Sub-circuit Definition: _power_element_
Max(A)= 7.7517325e-05 at = 5.1667363e-10 Min(A)= -4.9994624e-04
at = 1.6011893e-08 Avg(A)= -3.4633934e-04 Rms(A)= 3.5225006e-04
Max(W)= 8.9990321e-04 at = 1.6011893e-08 Min(W)= -1.3953118e-04
at = 5.1667363e-10 Avg(W)= 6.2341079e-04 Rms(W)= 6.3405009e-04
```

```
Port Name: xinv2.vdd (vdd2)
Sub-circuit Definition: inv2_chain
Max(A)= 4.9644897e-04 at = 5.1010837e-08 Min(A)= -7.7537325e-05
at = 5.1669013e-10 Avg(A)= 3.4358634e-04 Rms(A)= 3.4955419e-04
Max(W)= 8.9360811e-04 at = 5.1010837e-08 Min(W)= -1.3956718e-04
at = 5.1669013e-10 Avg(W)= 6.1845539e-04 Rms(W)= 6.2919752e-04
```

```
##### LABEL=xbuf1_power_thru_vdd FROM=0 TO=6.4000000e-08 #####
```

```
Port Name: vd3 (_power_element_)
Sub-circuit Definition: _power_element_
Max(A)= 3.8663398e-05 at = 5.5022903e-08 Min(A)= -4.4108575e-04
at = 5.5306383e-08 Avg(A)= -2.5448256e-04 Rms(A)= 2.8610026e-04
Max(W)= 1.1027144e-03 at = 5.5306383e-08 Min(W)= -9.6658496e-05
at = 5.5022903e-08 Avg(W)= 6.3620640e-04 Rms(W)= 7.1525065e-04
```

```
Port Name: xbuf1.vdd (vdd3)
Sub-circuit Definition: buf1_chain
Max(A)= 4.2530775e-04 at = 2.1307094e-08 Min(A)= -3.6422239e-05
at = 5.5020000e-08 Avg(A)= 2.5348048e-04 Rms(A)= 2.8490514e-04
Max(W)= 1.0632694e-03 at = 2.1307094e-08 Min(W)= -9.1055597e-05
at = 5.5020000e-08 Avg(W)= 6.3370120e-04 Rms(W)= 7.1226286e-04
```

```
##### LABEL=xbuf2_power_thru_vdd FROM=0 TO=6.4000000e-08 #####
```

```
Port Name: vd4 (_power_element_)
Sub-circuit Definition: _power_element_
Max(A)= 2.0524261e-05 at = 5.5535970e-08 Min(A)= -5.2076875e-04
at = 5.4580612e-08 Avg(A)= -2.2750674e-04 Rms(A)= 2.9057920e-04
Max(W)= 1.1456913e-03 at = 5.4580612e-08 Min(W)= -4.5153376e-05
at = 5.5535970e-08 Avg(W)= 5.0051485e-04 Rms(W)= 6.3927425e-04
```

```
Port Name: xbuf2.vdd (vdd4)
Sub-circuit Definition: buf2_chain
Max(A)= 5.1083847e-04 at = 5.4391364e-08 Min(A)= -1.8554571e-05
at = 1.6531365e-08 Avg(A)= 2.2710148e-04 Rms(A)= 2.8958801e-04
Max(W)= 1.1238446e-03 at = 5.4391364e-08 Min(W)= -4.0820056e-05
at = 1.6531365e-08 Avg(W)= 4.9962328e-04 Rms(W)= 6.3709363e-04
```

The format of this file is as follows:

- The first line contains a user-defined name and time window in which power reporting is performed.
- The next line shows the port name and, in parenthesis, the node name associated with the port name. If the node is connected to supply, it is tagged as (`_power_element_`).
- Following the port name is the subcircuit definition for the port. For each port name, the default power reporting includes the max/min/average/Rms. The power report also shows both current (A) and power (W).

In the previous power report, there are four sections:

- The first section is defined by the `xinv1_power_thru_vdd` label name. It prints the power reporting through the `v(vdd1)` node.
- The second section is defined by the `xinv2_power_thru_vdd` label name. It prints the power reporting through the `v(vdd2)` node.
- The third section is defined by the `xbuf1_power_thru_vdd`. It prints the power reporting through the `v(vdd3)` node.
- The fourth section is defined by the `xbuf2_power_thru_vdd` label name. It prints the power reporting through the `v(vdd4)` node.

Using Interactive Mode

This section describes how to run the PrimeSim XA tool in interactive mode. It uses the same circuit as in the [Generating Power Reports](#) section and shows you how to:

- Access interactive mode.
- Use interactive mode help.
- Use interactive mode commands.

[Table 81](#) lists the files needed for this tutorial. These files are located in the `XA_installation_directory/doc/tutorials/interactive` directory.

Table 81 *Interactive Mode Files*

File Name	Description
top.spi	Top-level SPICE netlist
circuit.sp	SPICE netlist

Table 81 Interactive Mode Files (Continued)

File Name	Description
cells.inc	SPICE netlist
bsim3.mod	BSIM3 MOS model
run	Run script for this tutorial

To run interactive mode, do the following steps:

1. From the UNIX command line, use the `run` script, which contains the `-intr` PrimeSim XA command line option to activate interactive mode.

At time zero, the simulation tops and switches to interactive mode. The `XA>` prompt is displayed.

2. To see a list of the interactive commands, type `help` at the `XA>` prompt.
3. Type `icontinue_sim` at the interactive prompt to restart the simulation.

Using Interactive Commands

This section of the tutorial uses the following interactive mode commands:

- `iopen_log`
- `iclose_log`
- `icontinue_sim`
- `iprint_help`
- `iprint_node_info`
- `iprint_elem_info`
- `iprint_connectivity`
- `iset_break_point`
- `ilist_break_point`

1. To run the interactive commands, type `run`.

The simulation stops at 0 ns.

2. To save the interactive mode session information, type `iopen_log interactive_file` at the interactive mode command prompt:

```
XA> iopen_log interactive_file
```

An `interactive_file` file is created to contain all interactive activities.

3. To get more detailed information about how each interactive mode command works, type `iprint_help` at the interactive prompt without any arguments.

A list of available interactive commands is printed.

4. To get more detailed information about how the `iopen_log` command works, type `iprint_help` at the interactive prompt:

```
XA> iprint_help iopen_log
```

A description of the `iopen_log` command is displayed.

5. To stop the simulation at multiple break points, type `iset_break_point`:

```
XA> iset_break_point -at 10ns
```

```
XA> iset_break_point -at 50ns
```

6. To list the break points, type `ilist_break_point`:

```
XA> ilist_break_point
```

```
1: break at time: 10ns
```

```
2: break at time: 50ns
```

7. To proceed to the first break point, type `icontinue_sim`:

```
XA> icontinue_sim
```

```
1: break at time: 10ns
```

8. To get more information about the `vddi` node, type `iprint_node_info`:

```
XA> iprint_node_info vdd1
```

```
Node=vdd1 (2)
```

```
V=2 V dV/dt=0 V/ns t=10 ns
```

```
Node capacitance=49.1543 fF
```

9. To obtain the connectivity information for the `vddi` node, type:

```
XA> iprint_connectivity vdd1
```

```
Node=vdd1 (2)
```

```
Channel-connected elements:
vd1 (0) Type=V N1=vdd1 (2) N2=0 (0)
xinv1.x2.mp (137) Type=PMOS D=xinv1.2 (62) G=xinv1.1 (61) S=vdd1 (2)
  B=vdd1 (2)
xinv1.x3.mp (139) Type=PMOS D=xinv1.3 (63) G=xinv1.2 (62) S=vdd1 (2)
  B=vdd1 (2)
xinv1.x4.mp (141) Type=PMOS D=xinv1.4 (64) G=xinv1.3 (63) S=vdd1 (2)
  B=vdd1 (2)
xinv1.x5.mp (143) Type=PMOS D=xinv1.5 (65) G=xinv1.4 (64) S=vdd1 (2)
  B=vdd1 (2)
xinv1.x6.mp (145) Type=PMOS D=xinv1.6 (66) G=xinv1.5 (65) S=vdd1 (2)
  B=vdd1 (2)
xinv1.x7.mp (147) Type=PMOS D=xinv1.7 (67) G=xinv1.6 (66) S=vdd1 (2)
  B=vdd1 (2)
xinv1.x8.mp (149) Type=PMOS D=xinv1.8 (68) G=xinv1.7 (67) S=vdd1 (2)
  B=vdd1 (2)
xinv1.x9.mp (151) Type=PMOS D=xinv1.9 (69) G=xinv1.8 (68) S=vdd1 (2)
  B=vdd1 (2)
xinv1.xa.mp (153) Type=PMOS D=xinv1.1 (61) G=a_inv1 (21) S=vdd1 (2)
  B=vdd1 (2)
xinv1.xz.mp (155) Type=PMOS D=z_inv1 (10) G=xinv1.9 (69) S=vdd1 (2)
  B=vdd1 (2)
```

Gate-connected elements:

10. To obtain the element information for the `xinv1.x2.mp` device connected to the `vdd1` node, type:

```
XA> iprint_elem_info xinv1.x2.mp

Elem=xinv1.x2.mp (137) Type=PMOS Model=pch
D=xinv1.2 (62) G=xinv1.1 (61) S=vdd1 (2) B=vdd1 (2) Vd=2
Vg=2.60755e-10 Vs=2 Vb=2
M=1
Weff=1.16305u Leff=0.16141u PD=3u PS=3u AD=2u^2 AS=2u^2 SA=0u SB=0u
Vt=-0.42012 ON
Ids=-5.4269e-05u
gds=0.000347309 gm=1.68119e-11
cgs=1.02213f cgd=1.16166f cgb=0.199658f cbs=3.56378f cbd=4.62321f
id=-1.84934e-05u ig=-6.34383e-06u is=5.00845e-05u ib=-2.52473e-05u
```

11. To proceed to the next break point, type:

```
XA> icontinue_sim

2: break at time: 50 ns
```

12. To close the interactive session, type:

```
XA> iclose_log
```

13. To review all of the interactive mode commands used in this section, type:

```
XA> history
  1  iopen_log interactive_file
  2  iprint_help iopen_log
  3  iset_break_point -at 10ns
  4  iset_break_point -at 50ns
  5  ilist_break_point
  6  icontinue_sim
  7  iprint_node_info vdd1
  8  iprint_connectivity vdd1
  9  iprint_elem_info xinv1.x2.mp
 10  icontinue_sim
 11  iclose_log
 12  history
```

14. To exit the interactive mode session, type:

```
XA> iquit_sim
```

Using a Vector File in a Simulation

This section shows you how to use a vector file in a PrimeSim XA simulation. The simulation use a simple adder circuit and demonstrates how to:

- Use the run script and PrimeSim XA command files.
- Run a simulation with .VEC inside an PrimeSim HSPICE netlist file.
- Run a simulation with a vector file from a PrimeSim XA command.
- Run a simulation with a vector file using `period` syntax.
- Run a simulation with vector file using `stop_at_error` syntax.

[Table 82](#) lists the files needed for this tutorial. These files are located in the `XA_installation_directory/doc/tutorials/vec` directory.

Table 82 *Vector Files*

File Name	Description
top.spi	Top-level SPICE netlist with a vector file
top_no_vec.sp1	Top-level SPICE netlist without a vector file
adder.vec	Vector file
adder_use_period.vec	Vector file
adder_stop_at_error.vec	Vector file

Table 82 Vector Files (Continued)

File Name	Description
cmd	PrimeSim XA command file
cmd_load_vec_file	PrimeSim XA command file
cmd_load_vec_file_use_period	PrimeSim XA command file
cmd_load_vec_file_stop_at_error	PrimeSim XA command file
run	PrimeSim XA run script

The `run` script contains the following lines:

```
xa top.spi -c cmd -o RESULT/xa_with_vector_file_in_netlist
xa top_no_vec.spi -c cmd_load_vec_file -o RESULT/xa_with_load_vector_file
xa top_no_vec.spi -c cmd_load_vec_file_use_period -o
  RESULT/xa_with_load_vector_file_use_period
xa top_no_vec.spi -c cmd_load_vec_file_stop_at_error -o RESULT/
  xa_with_load_vector_file_stop_at_error
```

The `run` script specifies the SPICE netlist files, named `top.spi` and `top_no_vec.spi`, and the command files for the different vector file conditions to run.

From the UNIX command line, use `cat` to show the contents of the vector file simulation command files.

1. `cat cmd`

Probes voltage signals
probe_waveform_voltage *
2. `cat cmd_load_vec_file`

Probes voltage signals
probe_waveform_voltage *

Loads the vector file
load_vector_file -file adder.vec
3. `cat cmd_load_vec_file_use_period`

Probes voltage signals
probe_waveform_voltage *

Loads the vector file
load_vector_file -file adder_use_period.vec
4. `cat cmd_load_vec_file_stop_at_error`

```
# Probes voltage signals
probe_waveform_voltage *

# Loads the vector file
load_vector_file -file adder_stop_at_error.vec
```

To run simulation with a vector file, run the following steps:

1. From the UNIX command line, use the `cat` command to open the `run` script.
2. Check the file permission to make sure the `run` script is set to be executable,
3. Run the simulation with the first line from the `run` script.

When the simulation is finished, list the directory contents. The following directories and files are created:

```
./RESULT (directory)

a_with_vector_file_in_netlist.err
xa_with_vector_file_in_netlist.fsdb
xa_with_vector_file_in_netlist.log
```

The `.err` file is the output checking result from the vector file.

4. Use the UNIX `cat` command to open the `xa_with_vector_file_in_netlist.err` file:

```
Expected Output Summary
-----| Node | Pass/ |
Expected | Simulated | Time      |
| name | Fail | state | state | (us)
|-----+-----+-----+-----+
| cout | P    |      |      | no error |
| s[0] | P    |      |      | no error |
| s[1] | P    |      |      | no error |
| s[2] | P    |      |      | no error |
| s[3] | P    |      |      | no error |
|-----+-----+-----+-----+
|
```

Running the Vector File Simulation With the PrimeSim XA Command

To run the vector file simulation with the PrimeSim XA command, do the following steps.

1. Use the UNIX `source` command to source the relevant PrimeSim XA binary to set up the path to run the PrimeSim XA tool.

The simulation stops at 0 ns.

2. Check the file permission to make sure the `run` script is set to be executable.

3. Run the simulation with the second line from the `run` script:

```
xa top_no_vec.spi -c cmd_load_vec_file -o RESULT/  
xa_with_load_vector_file
```

When the simulation is finished, list the directory contents. The following directories and files are created:

```
./RESULT (directory)  
xa_with_load_vector_file.err  
xa_with_load_vector_file.fsdb  
xa_with_load_vector_file.log
```

The `.err` file is the output checking result from the vector file.

4. Use the UNIX `cat` command to open the `xa_with_load_vector_file.err` file.

Expected Output Summary

----- Node Pass/				
Expected	Simulated	Time		
name	Fail	state	state	(us)
-----+-----+-----+-----+-----				
cout	P			no error
s[0]	P			no error
s[1]	P			no error
s[2]	P			no error
s[3]	P			no error
-----+-----+-----+-----+-----				

Running the Vector File Simulation Using the period Syntax

To run the vector file simulation using the `period` syntax, do the following steps:

1. Use the `source` command to source the relevant PrimeSim XA binary to run the PrimeSim XA tool.
2. Check the file permission to make sure that the `run` script is set to be executable.
3. Type `run` to run the simulation script.
4. Run the simulation with the third line from the `run` script:

```
xa top_no_vec.spi -c cmd_load_vec_file_use_period -o RESULT/  
xa_with_load_vector_file_use_period
```

5. When the simulation is finished, list the directories and files that are created:

```
./RESULT (directory)  
xa_with_load_vector_file_use_period.err  
xa_with_load_vector_file_use_period.fsdb  
xa_with_load_vector_file_use_period.log
```

- Use the UNIX `cat` command to open the `xa_with_load_vector_file_using_period.err` file.

Expected Output Summary

----- Node Pass/				
Expected	Simulated	Time		
name	Fail	state	state	(us)
-----+-----+-----+-----+-----				
cout	P			no error
s[0]	P			no error
s[1]	P			no error
s[2]	P			no error
s[3]	P			no error
-----+-----+-----+-----+-----				

Running the Vector File Simulation Using the `stop_at_error` Syntax

To run the vector file simulation using the `stop_at_error` syntax, do the following steps:

- Use the `source` command to source the relevant PrimeSim XA binary to run the PrimeSim XA tool.
- Check the file permission to make sure that the `run` script is set to be executable.
- Type `run` to run the simulation script.
- Run the simulation with the fourth line from the `run` script:

```
xa top_no_vec.spi -c cmd_load_vec_file_stop_at_error -o RESULT/
xa_with_load_vector_file_stop_at_error
```

- When the simulation finishes, list the directories and files that are created:

```
./RESULT (directory)
xa_with_load_vector_file_stop_at_error.err
xa_with_load_vector_file_stop_at_error.fsdb
xa_with_load_vector_file_stop_at_error.log
```

The `.err0` file is the output checking result from the vector file.

- The `stop_at_error` syntax inside the `adder_stop_at_error.vec` file instructs the PrimeSim XA tool to error out when an output violation occurs. The `xa_with_load_vector_file_stop_at_error.log` contains the following information:

```
Warning: output comparison error on 's[0]' at 0.462 us, L expected
Autostop ends the simulation at: xxx
```

7. Use the UNIX `cat` command to open the `xa_with_load_vector_file_using_period.err` file:

Expected Output Summary					
----- Node Pass/					
Expected	Simulated	Time			
name	Fail	state	state	(us)	
-----+-----+-----+-----+-----+-----					
cout	P			no error	
s[0]	F	L	1	0.462	
s[1]	P			no error	
s[2]	P			no error	
s[3]	P			no error	
-----+-----+-----+-----+-----+-----					

Using a VCD File in a Simulation

This section demonstrates using a value change dump (VCD) file with the PrimeSim XA tool. It uses a simple adder circuit.

[Table 83](#) lists the files needed for this tutorial. These files are located in the `XA_installation_directory/doc/tutorials/vcd` directory.

Table 83 VCD Files

File Name	Description
<code>top.spi</code>	Top-level SPICE netlist
<code>adder.vcd</code>	VCD file
<code>adder.sig</code>	Signal information file
<code>cmd</code>	PrimeSim XA command file
<code>run</code>	PrimeSim XA run script

To use a VCD file with the PrimeSim XA tool, do the following steps:

1. From the UNIX command line, use `cat` to show the contents `run` script:

```
xa top.spi -c cmd -o RESULT/xa
```

This script file specifies the SPICE netlist, named `top.spi`, and the command files to use a VCD file in the PrimeSim XA tool.

2. Use the UNIX `cat` command to display the content of the `cmd` command file:

```
# command to load in the VCD file
load_vector_file -file adder.vcd -format VCD -ctl adder.sig
```

3. To run the simulation from the UNIX command line, use the `source` command to source the relevant PrimeSim XA binary to run the PrimeSim XA tool.
4. Check the file permission to make sure that the `run` script is set to be executable.
5. Type `run` to run the simulation script.
6. When the simulation finishes, it contains the following directories and files:

```
./RESULT (directory)
    xa.err
    xa.fsdb
    xa.log
```

The `.err` file is the output checking result from the vector file.

7. Use the UNIX `cat` command to display the `xa.err` file.

Expected Output Summary

Node	Pass/	Expected	Simulated	Time
name	Fail	state	state	(us)
cout	P			no error
s[0]	P			no error
s[1]	P			no error
s[2]	P			no error
s[3]	P			no error

Running a MOSRA Simulation

This section demonstrates how to run a MOSRA simulation. There are two MOSRA simulation directories: Lab1 and Lab2:

```
XA_installation_directory/doc/tutorials/mosra/Lab1
XA_installation_directory/doc/tutorials/mosra/Lab2
```

Table 84 MOSRA Files for Lab 1

File Name	Description
test1.sp	Top-level MOSRA netlist
measure.meas	Measurements file
run	PrimeSim XA run script
README	Lab1 overview.

Table 85 MOSRA Files for Lab 2

File Name	Description
test1.sp	First top-level MOSRA netlist
test2.sp	Second top-level MOSRA netlist
measure.meas	Measurements file
run_step1	First PrimeSim XA run script
run_step2	Second PrimeSim XA run script
xa.cfg	PrimeSim XA configuration file
README	Lab2 overview

For more information about running the tutorials, see the README files in the Lab1 and Lab2 directories.

To run the tutorial in Lab1, type:

```
run
```

In Lab2, to run a fresh simulation and generate the .radeg file, type

```
run_step1
```

To use the .radeg file to run the age simulation, type

```
run_step2
```